RPGDEMO • 源码复盘 + CODE REVIEW + 面试追问 + 发散场景题

# RPGDemo 源码复盘
# 与游戏客户端面试

以 dxHWH/RPGDemo（Unity 2022.3 · 2D 横版动作 RPG）的真实源码与场景数据为唯一依据：玩家 13 个状态怎么切、动画事件怎么驱动命中、弹反窗口到底多宽、伤害公式里藏着哪些 off-by-one、骷髅为什么每秒自己切换到自己几十次、掉落和合成为什么能“刷”。每一处都往下追到 Unity 生命周期、FixedUpdate、GC、对象池、数据驱动设计，再发散成面试官最爱出的场景题。最后的方法论篇站在“面试官看不到 Demo”的前提下，把每个功能讲成“需求 → 我的做法 → 常见的坑 → 更好的实现 → 取舍”：状态判定、滑墙、Tilemap 关卡搭建、输入缓冲与手感、背包 / 装备 / 技能的系统设计。全书把每个功能分成两条线：**现状**是 Demo 里真实存在的做法和它的问题，**理想**是重构 / 上线级方案（大多只是设计，少数做了 C++ 模型验证）；每章开头一张“现状 vs 理想”对照表，附录 D 汇总成一张阶梯图。

21 章 + 附录 · 6 个部分（新增 Part VI 方法论篇）

全书区分 现状（Demo 822270c）→ 问题（分级）→ 理想（重构方案）

C++ 逐行移植实测 E1~E11 · F1~F5

问题总表 36 条（P0~P3）

每章：30 秒答案 · 四层追问 · 错误回答 · 场景题

CHAPTER 00

# 读前须知、项目一页纸与面试地图

先把三件事讲清楚：书里每句话的可信度怎么标；这个项目一句话、一分钟、三分钟分别怎么讲；面试官会从哪一行代码一路追问到哪里。

---

**本章目标**

- 认识 8 种事实标签，知道哪些结论是“读源码得到的”、哪些是“C++ 移植跑出来的”、哪些只是推断
- 分清两条线：**Demo 现在是什么样**（现状）和**理想方案应该什么样**（理想），以及讲的时候各用什么时态
- 拿到项目一页纸：规模、模块、场景里真正挂着的组件、关键序列化数值
- 背熟 1 分钟 / 3 分钟项目介绍，并知道每句话会把面试官引向哪一章

## 0.1 事实等级：每句话都要分清来源

| 标签 | 含义 | 例子 |
| --- | --- | --- |
| 源码 | 当前 commit（822270c）里逐行读过的真实行为，附文件名与行号 | `PlayerFallState.Update` 三个 if 之间没有 return（第 25~34 行） |
| 意图 | 从命名、注释推测的设计目标，不等于已实现 | 注释“冰冻状态下额外减少 15% 护甲值的伤害” |
| 问题 | 缺陷、边界漏洞、注释与实现不一致；附严重度 P0 P1 P2 P3 | 死亡后按 Shift 可以“复活” |
| 改进 | 修改方向与关键代码骨架（C#，未在 Unity 中编译）；属于“理想”这条线（0.2） | 状态机加“转移守卫 + 优先级” |
| 扩展 | 项目没涉及、但面试会顺着追下去的知识 | 输入缓冲、Coyote Time、命中停顿 |
| 实测 Ex | 把 C# 逻辑逐行移植成 C++（MSVC /O2）跑出的结果，数字全部来自 `demos/figdata.json` | E7：骷髅 10 s 内自转移 252 次 |
| 模型 | 依赖 Unity 运行时顺序的部分，按下文 R1~R4 规则建模 | 协程在 Update 之后恢复 |
| 推断 | 根据源码 + 资源配置推出、但需要 Play Mode 验证的结论；附录 B 给验证步骤 | 瞄准时按 Dash 仍会掷剑 |
| 现状 | 状态标记：Demo（822270c）里**真实存在**的做法，包括它的问题；能演示、有文件和行号 | “空中用一条腰部射线判墙” |
| 理想 | 状态标记：重构 / 上线级方案；**不在 Demo 里**，除非注明“模型验证”，否则没有任何代码实现 | “脚、腰两条射线按覆盖率判墙” |

**“实测”的边界与四条模型规则**

本机没有 Unity，C# 编译器被安全策略禁用，所以纯逻辑（状态转移、计时器、概率、合成扣料）逐行移植成 C++ 验证；依赖 Unity 运行时的部分写成模型规则，书里遇到时标 模型 ：

- **R1 帧顺序**：FixedUpdate × N（含物理步）→ 所有 Update → 协程 `WaitForSeconds` 恢复 → 动画更新（触发动画事件）→ LateUpdate → 渲染。
- **R2 动画事件**：进入状态那一帧记为片段 0 时刻；事件在“片段时间首次 ≥ 事件时间”那一帧的动画阶段触发。
- **R3 延迟**：`Destroy` 在帧末生效；两帧之间的按键在下一帧的 Update 里被读到。
- **R4 精度**：`Time.deltaTime` 与所有计时器都是 `float`，移植时同样用 float。

依赖 C# 运行时的结论（某次调用到底分配多少字节、协程对象的开销）只标 源码 或 扩展 ，**不冒充实测**——面试时说“我推断会分配，并在 Profiler 的 GC Alloc 列验证过”比报一个没测过的数字安全得多。

## 0.2 两条线：现状 vs 理想（全书读法）

这本书讲的每个功能都有两种状态，必须分开看：现状 是 Demo（commit 822270c）里**真实存在**的做法——包括它的问题，可以演示、能给出文件和行号；理想 是“如果重做 / 做成上线项目应该怎么做”的**设计方案**——它不在 Demo 里。两条线一旦混在一起，读者会把设计当成已实现，面试时也会把“我会怎么做”说成“我做过”。所以全书每讲一个功能，都按下图从左走到右。




现状 vs 理想 图 0-1　一个功能的两条线。左边只写 Demo 里真实存在的东西；中间把问题提炼成通用的坑和根因；右边才是理想做法与取舍。三段的标记颜色和讲述时态都不同（下半部分），读者和面试官都不会把设计当成已实现

| 标记 | 长什么样 | 出现在哪 | 怎么读 |
| --- | --- | --- | --- |
| 正文标签 | 现状 理想 | 段落开头、卡片、坑卡 | 标签后面那句话描述的是哪条线 |
| 代码块徽标 | 当前源码 • 822270c<br>理想方案 • 骨架 | 代码块标题栏（橙框 / 绿框） | 橙框 = 仓库原文摘录，行号可查；绿框 = 理想方案的 C# 骨架，**没有在 Unity 里编译过**，只用来说明结构 |
| 图注徽标 | 现状 理想方案<br>现状 vs 理想 | 图注最前面 | 现状图由源码 / 场景文件 / 实测数据画出；理想图是设计示意；对比图把两条线并列（橙 = 现状，绿 = 理想） |
| 章首对照表 | x.1 节的对照表（表头：现状 → 问题 → 理想） | 第 1~12、17~21 章 | 橙列 = Demo 现在怎么做；**红列 = 这样做存在的问题**；绿列 = 理想怎么做；绿格底部的小徽标说明这个理想做法**落地到了哪一步**（下表） |
| 现状问题条 | 现状问题 P1 … | 每个橙框代码块底部、每张现状图的图注下方 | 这段代码 / 这张图里的做法**错在哪、后果是什么、证据是什么**（附录 A 编号、实验编号或“推断”） |

“现状”只说 Demo 怎么做还不够，必须同时说清**这样做有什么问题**——否则读者分不清哪些现状是可以保留的、哪些是必须改的，理想方案也就失去了理由。全书的问题统一按下表分级：

| 问题分级 | 含义 | 例 |
| --- | --- | --- |
| P0 P1<br>P2 P3 | **缺陷**，严重度与附录 A 相同：P0 无法构建 / 核心约束被破坏，P1 明显错误的玩法结果，P2 边界与体验问题，P3 工程规范；括号里是附录 A 编号（#n）与实验编号 | 死亡后按冲刺“复活”（P0，E1-S3，#3） |
| 隐患 | Demo 里还没触发、或需要 Play Mode 验证，但条件一满足就会出错 | 弹反成功依赖同一时刻两个动画事件的排序 |
| 局限 | **不是 bug**：现在能用，但扩展、规模化、联网时会成为障碍 | 物品没有实例，做不了“+5 的剑” |
| ✓ 无问题 | 现状做法正确，理想方案沿用 | 玩家用零摩擦物理材质，避免贴墙被挂住 |

**读对照表的顺序：先看红列**

每一行先读“当前做法”，再读“存在的问题”，最后看“理想方案”是不是**正好解决了这个问题**——如果理想方案解决的不是红列里写的问题，它就是过度设计。面试时也是这个顺序：“我当时这样做 → 后来发现它会导致……（证据）→ 所以正确做法是……”。缺陷（P0~P3）要讲证据，局限要讲“在什么条件下才会成为问题”，不要把局限说成 bug。

| 理想格的落地徽标 | 含义 | 面试时能说到什么程度 |
| --- | --- | --- |
| 设计方案 • 未实现 | 只是方案：Demo 里没有，本书也没写成可运行代码 | “如果重做，我会……”——讲思路和取舍，不报数字 |
| 模型验证：Ex / Fx | 把理想规则写进了 C++ 移植模型，和现状对拍过（实验里的“修正版 / 理想版”就是它） | “我把修正规则放进移植模型对比过：现状 X，修正后 Y”——仍要说明是模型，不是 Unity 里改好的工程 |
| 保持 • 现状已正确 | 现状就是对的，理想方案沿用 | 可以当亮点主动讲（“这一处我是有意这样做的”） |
| 公开参考 | 理想参数来自公开资料（例如 Celeste 的强制离墙 0.16 s），不是本书测的 | 注明出处：“参考了某游戏公开的数值” |

**理想不等于“全都要做”：分三档**

同一个功能从现状走到理想，中间其实有三档：**修正**（在现有结构里只修错，例如冰冻破甲改成判断目标）→ **重构**（改结构，例如属性改成带来源的修正器栈；这是“重做这个 Demo 就该做到”的理想）→ **上线级**（规模化或联网才值得，例如服务器裁决、行为树、技能编辑器）。各章对照表的“理想重构方案”列以前两档为主，第三档出现在“代价”列和追问里。附录 D 用一张阶梯图把 12 个模块的四档并排画出来。

**讲的时候用不同的时态**

现状用**过去时 + 证据**：“我当时用一条腰部射线判墙，后来测到它在脚底上方 1.32 m，接近墙顶时会有一段死区”；理想用**条件句 + 理由**：“如果重做，我会用脚、腰两条射线按覆盖率判，因为……”。**不要把理想说成做过**——面试官追问一句“这个参数你在 Demo 里调的是多少？”就会露馅（口径见第 16 章 16.3 的三种时态）。

## 0.3 项目一页纸

| 项 | 事实 |
| --- | --- |
| 仓库 / 快照 | `github.com/dxHWH/RPGDemo`，main@`822270c`（4 次提交，最后一次 “IventorySystem” 加入背包 / 装备 / 合成 / 掉落） |
| 引擎 | Unity **2022.3.54f1c1**；包：2D Feature、Cinemachine（cinematic feature）、TextMeshPro、Visual Scripting、Timeline |
| 规模 | 自写 C# **57 个文件 / 3543 行**：player 18 个（880 行）、Enemy 12（577）、Skill 8（671）、Stats 4（468）、Items and Iventory 7（453）、UI 4（148）、根目录 4（346） |
| 场景 | 只有 `SampleScene`：1 个玩家 `player`、1 只骷髅 `Enenmy_Skeleton`、`PlayerManager`、`SkillManager`（挂 Dash / Clone / ThrowSword 三个技能）、`Inventory`、4 块 UI（背包 4 格、仓库 3 格、装备 4 格、合成 1 格）、Cinemachine 虚拟相机、两层视差背景 |
| 玩家数值 | 移速 7、跳跃 20（重力倍率 5 → 理论跳高 4.08 m）、冲刺 25 × 0.2 s、墙滑 2、攻击位移 [2, 1, 3]、反击持续 1 s；**damage 1000 / maxHealth 1000 / armor 10**（明显是测试值） |
| 骷髅数值 | 移速 2、发现距离 5、攻击距离 1.75、攻击冷却 1、战斗距离 10；maxHealth 100、**damage 0**、iceDamage 20（魔法伤害调用被注释，所以骷髅现在打不疼玩家） |
| 物品 | 5 个 ScriptableObject：Animal_Skin 84.8%、Iron 88.2%（材料）；Nice_Armor 35.4%、Stolen_Sword 32%、Wooden_Sword 21.3%（装备）；骷髅最多掉 3 件 |
| 物理 | Fixed Timestep 0.02、Maximum Allowed Timestep 0.333；玩家 / 骷髅刚体开了 Interpolate + Continuous；飞剑刚体 Discrete、无插值；`Queries Hit Triggers` 开启 |
| 脚本执行顺序 | meta 里设置了 CharacterStats 300、UI_HealthBar 400、ItemData 500、ItemDrop 600（第 2 章会讲其中两个其实没起作用） |

它的骨架来自一门流行的 Unity 2D RPG 教程（Entity → 状态机 → Skill → Stats → Inventory 的组织方式、命名和数值都高度一致），作者在此基础上做了自己的改动：弹反改成“受击时判定”、状态效果计时器重构成通用函数、魔法伤害改成“排序选最大元素”、掉落改成“先筛选再无放回抽样”。**面试时主动说明来源**，再重点讲“我改了什么、为什么改、改完发现了什么问题”——这比假装全部原创可信得多，也正是本书的组织方式。

## 0.4 项目介绍：一句话、一分钟、三分钟

**一句话**

“一个 Unity 2D 横版动作 RPG Demo：用有限状态机组织玩家和怪物行为，用动画事件驱动命中，做了冲刺分身、三段连招、弹反、可回收的飞剑技能，以及属性 / 异常状态 / 掉落 / 装备 / 合成这一套数值与物品系统。”

### 一分钟版（按“结构 → 亮点 → 反思”组织）

**结构** 所有角色继承 `Entity`（组件缓存、地面 / 墙检测、翻转、击退），行为由状态机驱动：玩家 13 个状态、骷髅 6 个状态，每个状态是一个普通 C# 类，Enter / Update / Exit 三段式，进入时把同名 Animator bool 置真。

**亮点** 命中、结束、掷剑这些关键帧都放在动画事件里，逻辑和美术对齐；弹反我改成了“受击瞬间若处于反击状态且怪物正在出手就眩晕它”；伤害走闪避 → 暴击 → 护甲三步，魔法伤害按最高元素施加点燃 / 冰冻 / 感电，感电二次命中会召唤追踪雷击。

**反思** 后来我用 C++ 把关键逻辑移植出来逐帧回放，发现了一批问题：状态转移缺守卫（死亡后还能冲刺“复活”）、怪物战斗状态每帧自己切换到自己、多个系统同时写移动速度、合成只扣一份材料。我知道怎么修，也知道这些问题在更大的项目里会变成什么。

### 三分钟版：在一分钟版基础上，每段多给一个“数字”

① 状态机：落地瞬间贴墙会在**一帧内连续切两次状态**（Fall→Idle→WallSlide，E1）；② 时间：冲刺在 Update 里计时、在 FixedUpdate 里位移，120 / 144 fps 下距离会在 4.5~5.0 m 之间跳，冲刺中卡一帧 250 ms 能冲出 7.0 m（E10）；③ 判定：弹反成功窗口实测 **1.01 s**，比原教程写法的 1.33 s 更严，但仍远宽于动作游戏常见的零点几秒，而且弹反成功时伤害照样结算（E2）；④ 数值：暴击判定用 `≤`，暴击率 0 也有 1% 暴击（E3）；⑤ AI：骷髅战斗时每秒执行约 26 次 `GameObject.Find`（E7）；⑥ 物品：合成 Nice_Armor 需要 2 张皮却只扣 1 张，10 皮 10 铁能做 9 件而不是 5 件（E6）。最后说重构方向：转移表 + 守卫、DamageInfo 管线、Modifier 栈、物品实例化、对象池。

**面试官看不到 Demo**

上面两个版本偏“复盘”；面向看不到 Demo 的面试官，第 16 章给出了**讲出画面的口播稿、术语对齐表、把问题讲成“坑与正解”的转换表**和方法论版的 1 分钟 / 3 分钟介绍，建议面试前以第 16 章为准。

**介绍项目时最常见的三个坑**

① **只报功能清单**：“我做了冲刺、连招、弹反、背包……”——面试官听不到你的判断力。② **夸大**：“我设计了一套完整的技能系统”——第一个追问“技能数据怎么配？”就会露馅（技能参数全在 MonoBehaviour 的序列化字段里）。③ **回避问题**：被问到“有什么不足”只说“美术不好看”。更好的做法是主动抛出 1~2 个你测过的问题，把追问引到你准备好的地方（第 13 章专门练这个）。

## 0.5 面试地图：每章会被问什么、会被追到哪里

第三列是“为什么面试官会从这个项目追到那个知识点”——**能说出这条因果链，发散题就不再是背题。**

| 章 | 面试官最可能问 | 会被追到哪里（发散） | 为什么会追过去 |
| --- | --- | --- | --- |
| Ch1 全景 | 介绍架构；为什么用继承 Entity；模块之间怎么通信 | 组合 vs 继承、ECS、依赖方向、事件总线 | 你说“架构”，面试官会验证你懂不懂分层和依赖 |
| Ch2 生命周期 | Awake / Start 分工；单例怎么写；Destroy 什么时候生效 | PlayerLoop、脚本执行顺序、fake null、DontDestroyOnLoad、场景切换 | 项目的单例写反了，组件在 Start 里取，执行顺序配在了父类上 |
| Ch3 状态机 | 状态机怎么设计；状态之间怎么切；怎么防止非法切换 | HFSM、转移表、守卫与优先级、输入缓冲、状态机 vs 行为树 | 13 个状态 + 全局冲刺 + 外部强制切换，正好是“状态爆炸”的入口 |
| Ch4 动画事件 | 命中帧怎么做；动画和逻辑谁驱动谁 | Animator 参数设计、动画事件可靠性、逻辑时间轴（技能编辑器） | 动画速度为 0、被打断、同时间点事件都会让逻辑出错 |
| Ch5 时间与帧率 | Update 和 FixedUpdate 区别；为什么移动要乘 deltaTime | 固定步长、插值、帧率无关、确定性、轨迹预测 | 冲刺距离、点燃跳数、旋转剑命中数都随帧率变 |
| Ch6 战斗判定 | 攻击判定怎么做；弹反怎么实现；为什么会多段命中 | Hitbox / Hurtbox、命中去重、无敌帧、顿帧、伤害事件化 | 弹反窗口、受击清速度、多碰撞体重复命中 |
| Ch7 数值 | 伤害公式；Buff 怎么做；属性修正怎么叠加 | Modifier 栈、减伤公式、伪随机分布、Buff 生命周期 | off-by-one、冰冻判定对象写反、等级复利、多个系统写同一字段 |
| Ch8 技能 | 技能系统怎么设计；冷却放在哪；飞剑四种模式怎么扩展 | 数据驱动、策略模式、GAS、技能打断 / 预输入、投射物 | “Can” 函数有副作用、bool 标志汤、分身空引用 |
| Ch9 AI | 怪物 AI 怎么写；为什么用状态机 | 行为树 / GOAP / Utility、感知系统、AI LOD、百怪性能 | 每帧自转移 + Find、战斗永不超时、追到平台下抽搐 |
| Ch10 物品 | 背包数据结构；装备怎么加属性；掉落概率怎 | 模板 vs 实例、唯一 ID、存档、掉落表 / 保底、事务性合成、防刷 | ScriptableObject 当字典键、合成少扣料、序列化默认对象 |

| | | | |
|---|---|---|---|
| | 么做 | | |
| Ch11 UI 与表现 | 血条怎么刷新；受击闪白怎么做 | MVC / MVVM、Canvas 重建、材质实例、MaterialPropertyBlock | 事件订阅不对称、`sr.material` 会复制材质 |
| Ch12 性能与工程 | 项目有哪些性能问题；怎么定位 | Profiler、GC、对象池、NonAlloc、程序集、版本管理 | 9 处 OverlapCircleAll、字符串反射调用、Editor 命名空间进了运行时代码 |
| Ch13~15 | 连续追问、场景题、速问速答 | 联网化、Boss、存档、技能编辑器、千怪同屏…… | 项目只是入口，面试官最终考的是“你会怎么做” |
| **Ch16 口径** | 看不到 Demo 时你怎么描述；怎么证明是你做的；有什么不足 | 表达、诚实边界、项目包装 | 面试官只能通过你的语言理解项目 |
| **Ch17 状态判定** | 怎么判断着地 / 贴墙；射线够不够；坡怎么办 | 形状投射、接触法线、感知层、宽限与滞回、单向 / 移动平台 | 单射线只伸出脚底 4 cm，坡度约 25.5° 就失效 |
| **Ch18 滑墙** | 滑墙怎么设计；为什么墙顶会掉；单墙能不能爬 | 转移完整性、墙覆盖检测、粘墙 / 墙宽限 / 强制离墙时间 | Demo 里单墙攀爬成败取决于手速 |
| **Ch19 Tilemap** | 关卡怎么搭；Tilemap 原理；为什么合并碰撞 | 稀疏存储、块渲染与图集、复合碰撞、规则瓦片、关卡度量 | 你说“用瓦片搭的”，就要讲清它是怎么工作的 |
| **Ch20 手感** | 没有输入缓冲怎么办；什么是土狼时间；跳跃为什么要可变 | 输入延迟链、宽容窗口、跳跃曲线、反馈导演 | “按了没反应”是可以量化的 |
| **Ch21 系统设计** | 背包 / 装备 / 技能为什么这样设计 | 四层分离、属性修正 / 效果 / 标签三通道、事务、数据驱动、取舍 | RPG 方向最常见的系统设计题 |

## 0.6 知识地图

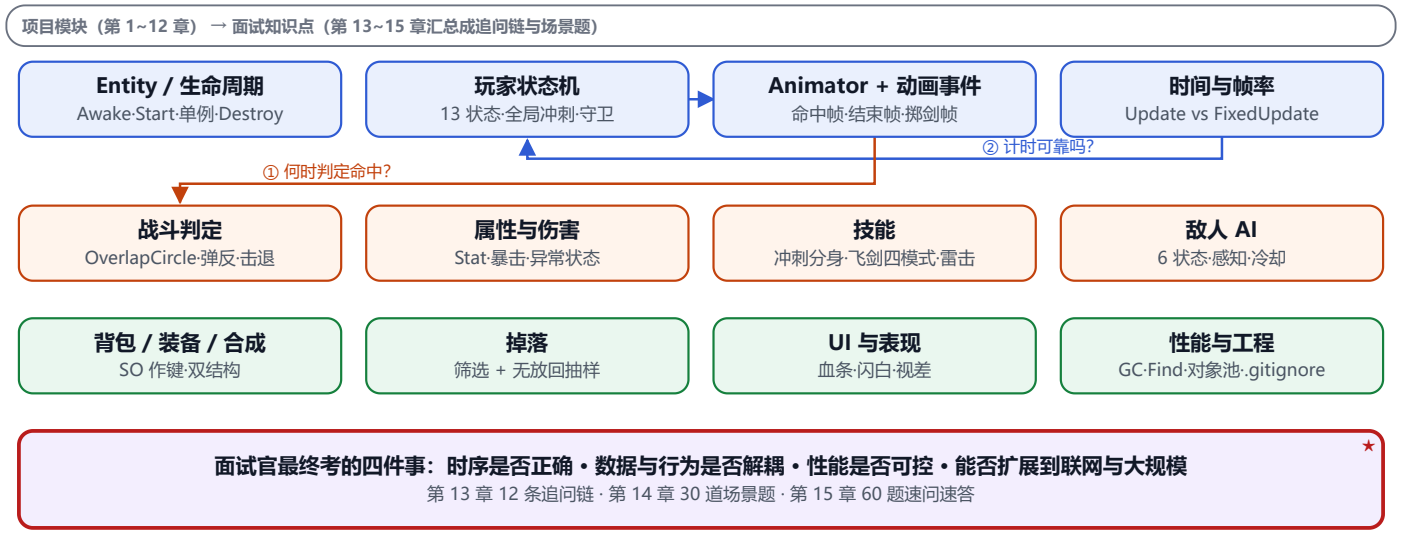



图 0-2　知识地图：上三行是项目模块（蓝 = 地基与手感，橙 = 战斗与行为，绿 = 物品 / UI / 工程），最后汇总成面试真正考察的四件事

图中两条箭头是全书最重要的两条因果链：① **动画事件决定战斗判定何时发生**（第 6 章的弹反窗口完全由骷髅攻击片段里事件的时间点决定）；② **时间与帧率决定状态机里的计时是否可靠**（第 5 章会看到冲刺距离随帧率跳动）。

## 0.7 本书实验清单

E1~E11 来自 `demos/rpg_verify.cpp`（MSVC 14.44 `/O2`，无参数运行输出 `figdata.json` 与 `run_output.txt`），F1~F5 来自 `demos/feel_verify.cpp`（输出 `feeldata.json` / `feel_output.txt`），关卡数据来自 `tools/scan_tilemap.py` 解析场景得到的 `tilemap_dump.json`。正文里的每个实测数字都由构建脚本从 json 读取，不手写。

| 编号 | 移植对象 | 回答的问题 | 对拍的两条线 | 章 |
| --- | --- | --- | --- | --- |
| E1 | 玩家 12 个状态类 + Player.Update | 一帧两次转移、墙到头还在滑、死亡后冲刺 / 接剑“复活”、取消攻击后连招计数 | 现状 只复现问题（理想的 Request + 守卫未建模） | 3 |
| E2 | CounterAttack + 骷髅攻击片段事件 | 弹反成功的按键时间窗多宽；与原教程写法对比；帧率影响 | 现状 本项目 vs 原教程两种现状写法；理想窗口为示意 | 6 |
| E3 | CharacterStats.DoDamage | 暴击 / 闪避真实概率；冰冻判定对象写反的后果；暴击倍率 | 现状 vs 修正（统一 Roll、判目标） | 7 |
| E4 | EnemyStats.Modify | 每级 +25% 是线性还是复利 | 现状 复利 vs 线性对照 | 7 |
| E5 | ItemDrop.GenerateDrop | 每件物品真实掉率、平均掉几件、掉落初速度偏向 | 现状 精确解与蒙特卡洛对拍 | 10 |
| E6 | Inventory.CanCraft | 合成少扣材料能多做几件 | 现状 vs 理想 按数量扣料 | 10 |
| E7 | 骷髅 4 个状态类 | 自转移次数、Find 调用频率、追到平台下会不会放弃 | 现状 vs 理想 拒绝自转移 / 计时修正 | 9 |
| E8 | FreezeTimeFor 协程 × 多段命中 | 重叠协程让“冻结”实际只覆盖多少时间 | 现状 vs 理想 冻结取最晚截止 | 7 |
| E9 | 瞄准虚线公式 vs Box2D 积分 | 剑为什么总比虚线低一点，低多少 | 现状 连续抛物线 vs 理想 离散闭式解 | 5 |
| E10 | PlayerLoop 模型 | 冲刺距离、点燃跳数、旋转剑命中数随帧率怎么变；卡顿时冲刺多远 | 现状 只复现帧率依赖 | 5 |

| | | | | |
|---|---|---|---|---|
| E11 | SkillController_Clone.FaceClosesttTarget | 分身朝向判断写在循环里，正确率随碰撞体数量怎么变 | 现状 只复现朝向问题 | 8 |
| F1 | 地面 / 墙探测点几何（feel_verify.cpp） | 单射线能识别多陡的坡、墙检测覆盖多高 | 现状 单射线 vs 理想 三射线 | 17 |
| F2 / F3 | 土狼时间 / 跳跃缓冲（帧相位模型） | 不同窗口长度下“按了没反应”的比例 | 现状 无窗口 vs 理想 不同窗口长度 | 20 |
| F4 | 跳跃曲线（Box2D 离散积分） | 可变跳高、下落加重、顶点悬停分别改变什么；关卡度量 | 现状 固定跳高 vs 理想 可变跳高 / 下落加重 | 19 · 20 |
| F5 | 下落 / 滑墙 / 蹬墙三状态 + 7 m 立柱 | 单墙攀爬是否可能、取决于什么；墙顶死区 | 现状 单腰部射线 vs 理想 加脚部射线 | 18 |

**本章速记**

- 标签：**源码 / 意图 / 问题 / 改进 / 扩展 / 实测 / 模型 / 推断**；面试时也按这个口径说话。
- 两条线：**现状** = Demo 里真实存在（过去时 + 证据）；**理想** = 设计方案，多数未实现（条件句 + 理由）；理想分修正 / 重构 / 上线级三档。
- 项目一句话：**状态机 + 动画事件 + 数值物品系统**的 2D 动作 RPG；来源于教程，主动说明改动点。
- 三分钟版的骨架：每个模块一个“测出来的数字” + 一个“我会怎么修”。

PART I

# 全景与地基

先看清项目的骨架：场景里到底挂了什么、类怎么继承、一帧里谁先谁后、谁依赖谁；再看 Unity 生命周期、单例与对象存亡这些“地基”问题——后面十几章的 bug，有一半根子在这里。

CHAPTER 01

# 项目全景：模块、对象图与一帧的调用顺序

“介绍一下你的项目架构”是所有项目面的第一问。答好它需要三张图：运行时对象图、类继承树、一帧调用顺序；再加一张依赖图，说明“谁知道谁”。

---

**本章目标**

- 画出场景里真实存在的对象和组件（不是“代码里有的类”）
- 说清继承结构的好处与代价：Entity 基类、状态类层次、Stats 与 Skill 的子类
- 按 Unity 的帧顺序复述一次“按下攻击键 → 怪物掉血”的完整调用链
- 知道全书 36 个问题分布在哪些模块，面试时挑哪几个主动说

## 1.1 30 秒面试版 • 现状 vs 理想

**30 秒回答**

项目按“实体 + 状态机 + 子系统单例”组织：玩家和骷髅继承 `Entity`，在 Start 里缓存刚体、动画机、特效、属性组件；行为由普通 C# 状态类驱动，状态只在 Enter 时设 Animator bool；命中、动画结束、掷剑这些时刻由动画事件回调到 `*AnimationTrigger` 组件；技能挂在 `SkillManager` 上，玩家数据通过 `PlayerManager` 单例被技能、装备、怪物访问；属性在 `CharacterStats`，物品在 `Inventory` 单例。优点是上手快、各系统边界直观；代价是依赖都汇聚到“玩家”这个单点、状态类直接读输入、数据写在组件字段里，扩展和测试都难。

现状 → 问题 → 理想　**架构总览**

| 维度 | 当前做法（Demo 现状） | 存在的问题 | 理想重构方案 | 收益 / 代价 |
|---|---|---|---|---|
| **角色组织** | 继承 `Entity`：组件缓存、检测、翻转、击退、受击都在基类；减速这种非共有行为也塞进基类 | 局限 基类越来越胖：减速这类非共有能力也被所有角色继承；加“会飞的弓箭手”只能在继承树上开岔 | 组合：检测、移动、击退、受击反馈做成独立组件；状态机通过接口（移动能力、可受伤）拿能力<br>设计方案 · 未实现 | 新角色按需组装，不在继承树上开岔；能力可单测 |
| **模块通信** | 3 个单例 + 直接引用 + public 委托，几乎所有依赖都指向 Player（图 1-5） | P0 两个单例写反（#4）；所有依赖指向唯一的 Player，无法单测，换成多人 / 分身 / AI 托管要全部改 | 组装处显式注入 + 事件总线；写操作收口到数据的拥有者（只有属性集能改属性）<br>设计方案 · 未实现 | 依赖显式；多人 / 托管 / 回放时不用替换“唯一玩家” |
| **数据位置** | 数值序列化在场景与组件字段（移速、技能参数、AI 参数） | 局限 策划改数要开场景；技能、AI 参数绑在组件上，第二种怪物 / 第二套技能无法复用 | 配置表 / ScriptableObject 定义 + 运行时实例；组件只存配置 id<br>设计方案 · 未实现 | 策划改数不开场景；可热更、可校验 |
| **时序驱动** | Update 驱动状态，动画事件驱动命中，协程 / Invoke 驱动计时 | P2 结果随帧率与卡顿变化（冲刺距离、点燃跳数，E10，#20 #21）；冻结让动画停住时命中事件也停 | 固定逻辑步驱动状态、计时与判定；动画只做表现（单机可保留动画事件命中，但语义要完整）<br>设计方案 · 未实现 | 帧率无关、可回放、可联网（第 4、5 章） |
| **调用方式** | 19 处按字符串反射调用（协程 / Invoke） | P1 改名不报错；眩晕闪红调用了不存在的方法，只在运行时打日志（#12 #33） | 直接调用、`nameof`、统一计时器<br>设计方案 · 未实现 | 改名时编译期报错 |
| **做对的部分** | 状态是对象而不是 switch；关键帧交给动画时间轴；物品用 ScriptableObject；Stat 有修正列表 | ✓ 无问题 方向正确；问题在于没有在其上补齐分层 | 保持这些方向，在其上补齐分层<br>保持 · 现状方向正确 | |

现状问题：P0 ×1　P1 ×1　P2 ×1　局限 ×2　✓ 无问题 ×1　（P0~P3 编号见附录 A，分级说明见 0.2）

图 1-5 是现状架构，图 1-6（1.7 节末）是理想架构；全书总表见附录 D。

## 1.2 目录与模块职责

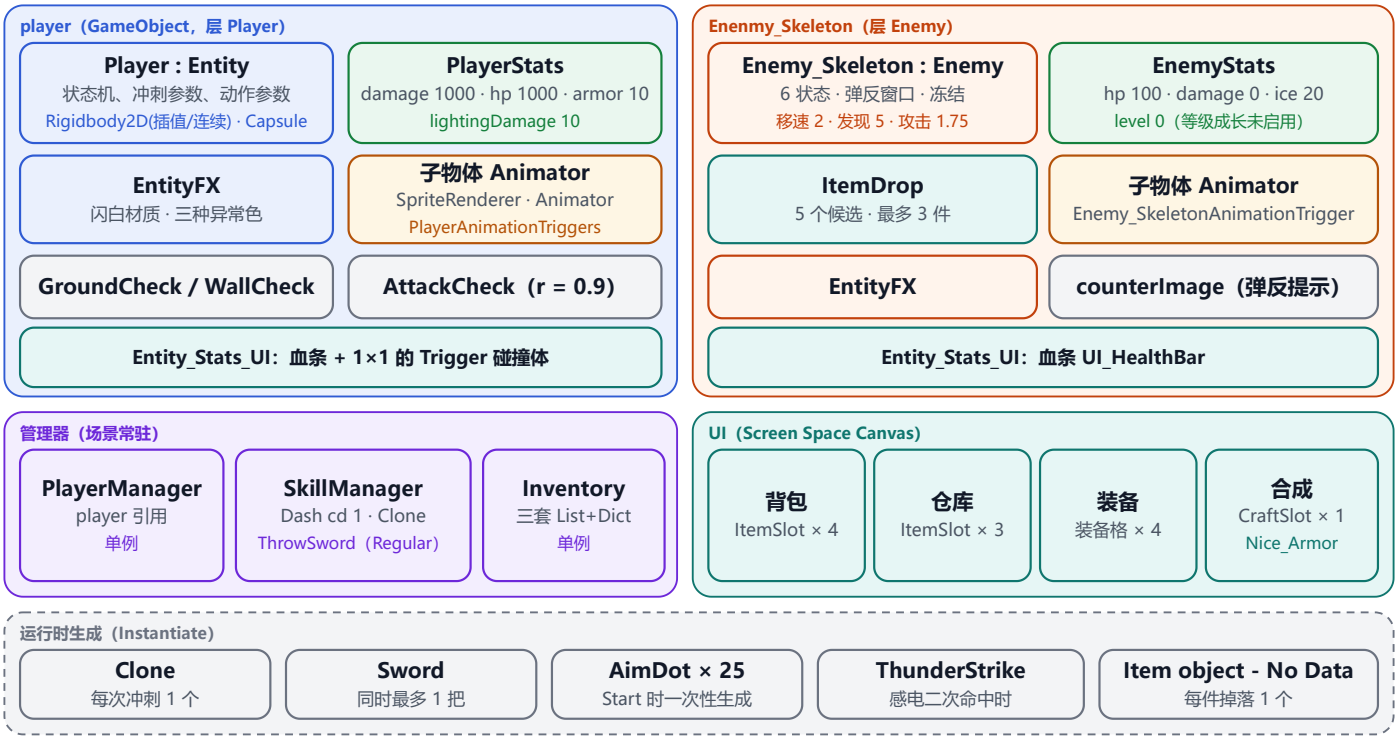
| 目录 / 文件 | 职责 | 面试关注点 |
| --- | --- | --- |
| Assets/Script | 57 个自写 C# 文件，3543 行 | |
| Entity.cs • EntityFX.cs | 角色基类：组件缓存、射线检测、翻转、击退；受击闪白与异常状态变色 | 继承 vs 组合；组件在 Start 里取 |
| player/ (18) | Player + 状态机 + 13 个状态类 + 动画事件转发 | 状态爆炸、转移守卫（第 3 章） |
| Enemy/ (12) | Enemy 基类、骷髅 6 个状态、弹反窗口、冻结 | 自转移、永不超时（第 9 章） |
| Skill/ (8) | Skill 基类（冷却）、冲刺 / 分身 / 飞剑、两个单例 | Can 函数副作用、标志汤（第 8 章） |
| Stats/ (4) | Stat（基础值 + 修正列表）、伤害公式、异常状态、等级成长 | off-by-one、复利（第 7 章） |
| Items and Iventory/ (7) | ItemData（SO）、背包 / 仓库 / 装备、掉落、拾取 | 模板 vs 实例、合成扣料（第 10 章） |
| UI/ (4) | 格子、装备格、合成格、血条 | 订阅生命周期（第 11 章） |
| Controller_ThunderStrike.cs | 感电连锁时召唤的追踪雷击 | 目标销毁、延迟结算 |
| ParalleBackGround.cs | 按相机 x 的视差 + 无限循环背景 | Find("Main Camera") |
| Assets/Animation | 5 个 AnimatorController、23 个片段（事件都在这里） | Entry/Exit + bool 结构（第 4 章） |
| Assets/Prefabs | Clone、Sword、AimDot、ThunderStrike、格子、掉落物 | 运行时 Instantiate 的对象 |
| Assets/Scenes/SampleScene | 唯一场景，所有数值都序列化在这里 | 数值不在代码里 |
| Logs/ • UserSettings/ | 本不该入库的本机文件（12 个） | .gitignore 写错（第 12 章） |

现状 图 1-1　目录与职责：括号里是文件数。数值几乎全部序列化在场景和 SO 资源里——**读代码只能看到“怎么算”，看不到“算什么”**，这是复盘 Unity 项目必须同时读场景的原因。

**现状问题** 局限 数值散在场景与 SO 里：读代码看不到“算什么”，策划改数要开场景

## 1.3 运行时对象图：场景里真正挂着什么

用脚本 `.cs.meta` 的 guid 反查 `SampleScene.unity` 和 Prefab，得到下图。**“类存在”不等于“被使用”**：`Skill.FindClosestEnemy`、`Entity.SlowEntityBy` 基类版本、`ItemObject` 的 M 键调试代码都存在，但前两者没有调用方。




**现状** 图 1-2　运行时对象图（由 guid 反查 SampleScene.unity / Prefab 得到）。注意玩家血条上挂了一个 Trigger 碰撞体，而项目开启了 Queries Hit Triggers——它会出现在玩家所有 OverlapCircleAll 的结果里

**现状问题** P2 血条上的 Trigger 碰撞体会混进玩家所有 OverlapCircleAll 的结果（Queries Hit Triggers 开启，#27）

## 1.4 类继承树

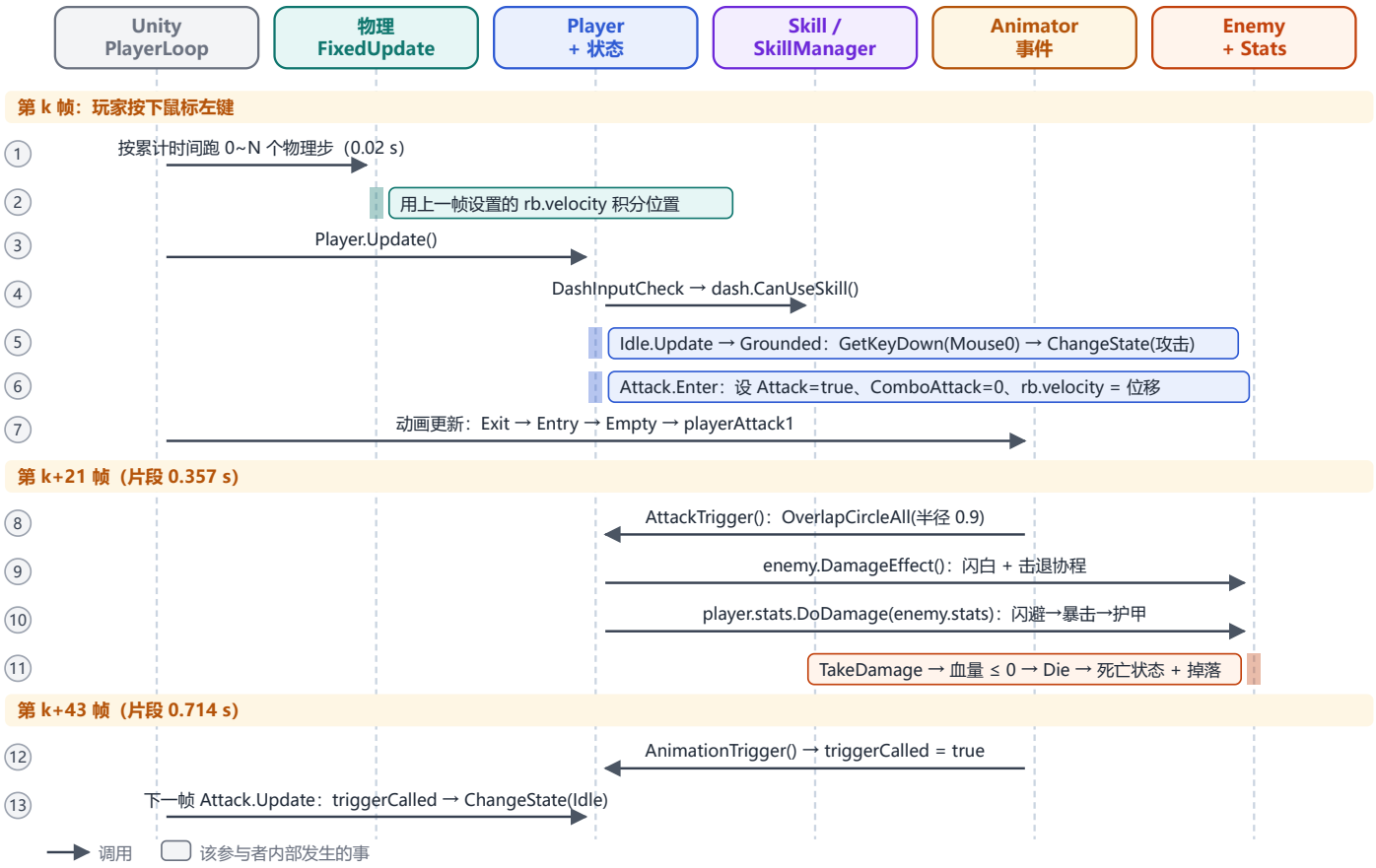
| 类 | 基类 / 形态 | 备注 |
| --- | --- | --- |
| MonoBehaviour | Unity 组件 | |
| Entity | 角色基类 | Awake 空、Start 取组件、虚函数 Die / DamageEffect |
| Player | Entity | 在 Awake 里 new 出 13 个状态对象 |
| Enemy | Entity | 弹反窗口、冻结、减速 |
| Enemy_Skeleton | Enemy | 6 个状态；重写 CanbeStunned / Die |
| CharacterStats | 属性与伤害 | Update 里推进异常状态计时 |
| PlayerStats | CharacterStats | Die → player.Die |
| EnemyStats | CharacterStats | 等级修正 + 掉落 |
| Skill | 冷却计时 | CanUseSkill 里直接 UseSkill |
| Skill_Dash • Skill_Clone • Skill_ThrowSword | Skill | ThrowSword 重写 Update 未调 base |
| UI_ItemSlot | IPointerDownHandler | 左键装备 / Ctrl 丢弃 |
| UI_EquipmentSlot • UI_CraftSlot | UI_ItemSlot | 卸下 / 合成 |
| 其余组件 | EntityFX、各种控制器 | AnimationTrigger、SkillController_*、ItemObject 等小组件 |
| PlayerState（普通类） | Enter / Update / Exit | 持有 Player、StateMachine、rb |
| PlayerGroundedState | PlayerState | 地面通用输入：攻击 / 反击 / 瞄准 / 跳 / 掉落 |
| Idle • Move | Grounded | Update 开头判 currentState != this |
| PlayerAirState | PlayerState | 空壳 |
| Jump • Fall | Air | Fall 有一帧双转移 |
| Dash • WallSlide • WallJump • PrimaryAttack | PlayerState | 直接继承基类 |
| CounterAttack • AimSword • CatchSword • Death | PlayerState | 直接继承基类；Death 隐藏了基类字段 player |
| EnemyState（普通类） | Enter / Update / Exit | Exit 时记录 lastAnimBoolName |
| Enemy_SkeletonGroundState | EnemyState | 发现玩家 → Battle；Enter 里 GameObject.Find |
| Idle • Move • Battle | Ground | Battle 继承它 → 每帧自转移 |
| Attack • Stun • Death | EnemyState | |
| ScriptableObject | 资源 | |
| ItemData | SO | 类型、名称、图标、掉率 |
| ItemData_Equipment | ItemData | 14 项属性 + 配方；AddModifiers 直接改玩家 |
| [Serializable] Stat • InventoryItem | 可序列化普通类 | InventoryItem 同时被当作“运行时堆叠”和“配方条目” |

现状 图 1-3　类继承树。三种“继承”值得区分：MonoBehaviour 组件继承（Entity 系）、普通 C# 状态类继承（复用 Enter/Update/Exit 模板）、ScriptableObject 数据继承（ItemData 系）。

**现状问题** 局限 Entity 基类集中了检测、翻转、击退、减速：非共有能力也被所有角色继承

## 1.5 一帧里发生了什么：从按键到怪物掉血

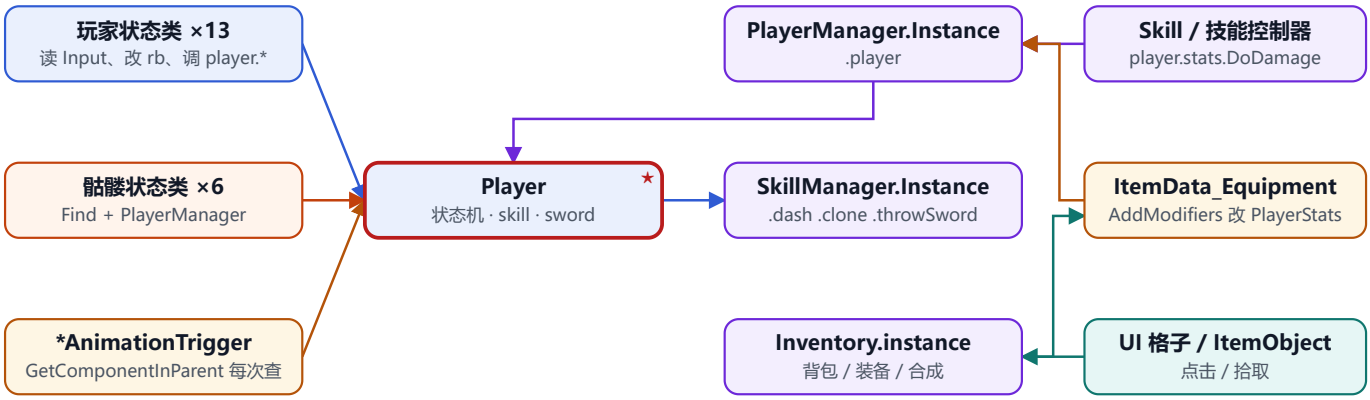
把“玩家按左键 → 第一段攻击命中骷髅”按 Unity 的帧顺序（规则 R1）展开。这张图是第 3~6 章的公共底图：状态机在 Update、命中在动画事件、位移在下一帧的物理步。




**现状** 图 1-4　一次近战命中的完整调用链（60 fps）。命中不在 Update 里，而在动画更新阶段——这决定了第 6 章弹反窗口的边界

**现状问题** 局限 命中在动画更新阶段结算：时机受播放速度 / 冻结影响，同帧互砍的先后取决于 Animator 更新顺序

## 1.6 依赖图：谁知道谁




现状 图 1-5 依赖图：几乎所有箭头最终指向 Player（★）。三个单例让任何代码都能“伸手”拿到玩家——方便，但也意味着没有任何一处能单独测试，换一个玩家（多人、分身、AI 托管）全都要改

**现状问题** P0 三个单例写反了两个（#4）；所有依赖指向唯一的 Player，无法单测、无法换玩家

## 1.7 架构评价：做对了什么、代价是什么、理想架构

### ✓ 状态是对象，不是 switch

每个状态一个类，Enter / Exit 成对设置 Animator bool，新增状态不碰其他状态的代码。 源码 PlayerState.cs:25-44

### ✓ 关键帧交给美术时间轴

命中、结束、掷剑由动画事件触发，改动画节奏不用改代码。

### ✓ 数值与资源分离的雏形

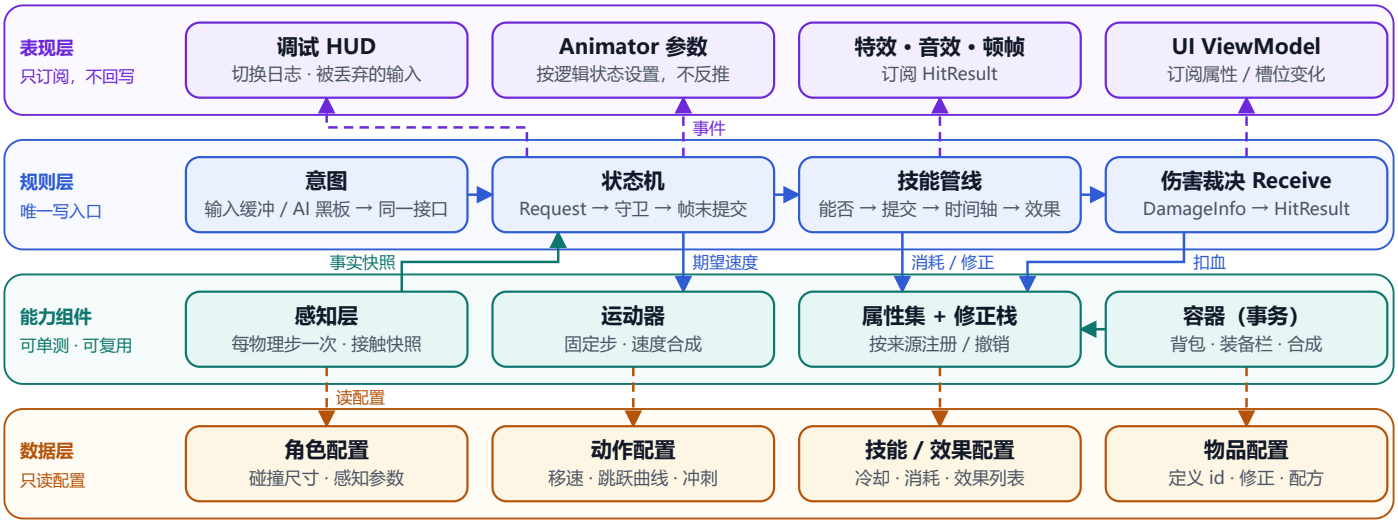
物品用 ScriptableObject，Stat 用“基础值 + 修正列表”，装备卸下能精确撤销。

### ✓ 自己做过的重构

异常状态计时器抽成 `HandleStatus(ref…)`；元素伤害改成排序选最大；掉落改成无放回抽样。

| 代价 | 源码证据 | 后果 | 章 |
| --- | --- | --- | --- |
| 状态类直接读 `Input` | PlayerState.cs:35-36、GroundedState.cs:25-46 | 无法重绑定按键、无法做输入缓冲 / 回放 / AI 托管、无法单测 | 3 |
| 状态切换没有统一入口规则 | Player.cs:117（全局冲刺）、:140（接剑强切） | 死亡可被打断，“吸收态”不成立 | 3 |
| 单例当服务定位器 | PlayerManager / SkillManager / Inventory | 隐式依赖、初始化顺序敏感、单例本身写反 | 2 |
| 多个系统写同一个字段 | moveSpeed、animator.speed、sr.color、rb.velocity | 减速 / 冻结 / 死亡 / 闪烁互相覆盖 | 7 · 11 |
| 字符串反射 | `StartCoroutine("…")` 5 处、`Invoke` 10 处、`InvokeRepeating` 4 处 | 改名不报错，调用不存在的方法只在运行时打日志 | 4 · 12 |
| 数据写在组件字段 | 技能参数、AI 参数都在 MonoBehaviour 上 | 无法复用到第二种怪物 / 第二套技能，策划改数要开场景 | 8 · 9 |

**理想** 把上表六条代价倒过来，就是理想架构的六条规则：**依赖只向下**（规则层 → 能力组件 → 只读配置，没有谁伸手去拿“唯一的玩家”）；**写入口唯一**（只有状态机能切状态、只有属性集能改属性、只有容器操作能改物品）；**表现层只订阅**（Animator、特效、UI 不回写逻辑）；**数据只读**（组件只存配置 id）；**输入与 AI 走同一个意图接口**（缓冲、回放、AI 托管都接在这里）；**组装根注入**（取代 3 个单例）。与图 1-5 对照看：箭头从“全部指向 Player”变成“分层向下 + 事件向上”。




**理想方案** 图 1-6　理想架构（设计方案，Demo 未实现）：依赖只向下（规则层 → 能力组件 → 只读配置），表现层只通过事件（紫色虚线）拿到结果；玩家输入和 AI 决策走同一个“意图”接口，输入缓冲、回放、AI 托管都接在这里；装备 = 容器之间移动物品 + 按来源注册属性修正（右侧短箭头）。对照图 1-5：原来所有箭头指向 Player，现在每一层只认识下一层

**图 1-6 是设计，不是 Demo**

Demo 里没有一个组件是按这张图拆的。面试时这样说：“现在的结构是所有东西都指向玩家对象、靠三个单例连接（图 1-5）；如果重做，我会按四层拆。顺序是先修错、再改结构（第 12 章图 12-2）：先把输入抽成意图、把伤害收口到受击方裁决，再把速度和颜色这类多方抢写的字段改成修正器推导，最后做物品实例和对象池。”

## 1.8 问题总览：36 条分布在哪里

| 模块 | P0 | P1 | P2 | P3 | 代表问题（附录 A 编号） |
|---|---|---|---|---|---|
| 生命周期 / 单例 / 构建 | 2 | 0 | 1 | 1 | #1 运行时代码引用 UnityEditor；#4 单例写反；#31 执行顺序配在父类；#35 .gitignore |
| 玩家状态机 | 1 | 2 | 0 | 0 | #3 死亡可打断；#14 墙到头还在滑；#15 一帧两次转移 |
| 动画事件 | 0 | 1 | 1 | 0 | #12 调用不存在的 RedColorBlink；#25 瞄准被打断仍掷剑（推断） |
| 时间与帧率 | 0 | 0 | 3 | 0 | #20 冲刺距离；#21 点燃跳数 / 旋转剑；#22 瞄准虚线偏差 |
| 战斗判定 | 0 | 1 | 2 | 0 | #5 弹反成功仍掉血；#26 击退方向；#27 无 LayerMask / 去重 |
| 属性与数值 | 0 | 3 | 4 | 0 | #7 死后重复 Die；#10 冰冻判错对象；#11 冻结协程重叠；#16 暴击 +1% |
| 技能 | 1 | 0 | 2 | 0 | #2 分身攻击空引用；#23 朝向判断写进循环；#24 冷却永不递减 |
| 敌人 AI | 0 | 2 | 1 | 0 | #8 每帧自转移 + Find；#9 永不超时；#32 尸体不回收 |
| 物品 | 0 | 2 | 1 | 0 | #6 合成少扣料；#13 点空格空引用；#17 掉率 +1% / 偏左 |
| UI / 表现 | 0 | 0 | 2 | 0 | #28 订阅不对称；#29 材质实例 / CancelInvoke |
| 代码质量 | 0 | 0 | 0 | 3 | #33 字符串反射；#34 Find / 热路径查组件；#36 命名与“Can”副作用 |
| **合计** | **4** | **11** | **17** | **4** | 36 条，完整清单见附录 A |

**面试时挑哪几个主动说**

选“**能演示、有数字、有通用教训**”的三个：① 死亡可被冲刺打断（状态机守卫，人人都会被问）；② 骷髅每帧自转移（继承复用带来的隐性行为，附带性能数字）；③ 合成少扣料（事务性，能自然引到服务器校验与防刷）。避免主动说 P3 的命名问题——它只会让面试官觉得你在凑数。

## 1.9 面试追问

追问树：从“介绍一下架构”出发

```
介绍一下项目架构（1.1）
├─ 为什么 Player/Enemy 用继承？什么时候该用组合？（Q1-1）
│   └─ 如果要加“会飞的弓箭手”和“可召唤的宠物”呢？（场景 1-1）
├─ 状态为什么是普通 C# 类而不是 MonoBehaviour？（Q1-2）
├─ 模块之间怎么通信？为什么用单例？（Q1-3 → 第 2 章）
│   └─ 不用单例怎么拿到玩家？依赖注入 / 服务定位器 / 事件？（Q1-4）
└─ 一帧里先执行什么？命中发生在哪个阶段？（Q1-5 → 第 4、6 章）
```

## Q1-1　为什么用继承 Entity?

**L1 • 30 秒**　玩家和怪物共享组件缓存、地面 / 墙检测、翻转、击退和受击表现，抽到 Entity 里避免重复。

**L2 • 正常回答**　Entity 提供的是“所有可战斗角色都有的能力”，并用虚函数留了扩展点：`DamageEffect` 玩家重写成“反击状态下先尝试弹反”，`Die` 各自切到死亡状态，`SlowEntityBy` 玩家多改跳跃和冲刺速度。

**L3 • 深入**　继承的问题在“能力组合”出现时暴露：想要一个不会受击闪白的木桩、一个会墙滑的怪物，就得在继承树上开岔或塞开关。组合的做法是把“地面检测”“击退”“受击闪白”做成独立组件（Unity 本身就是组件模型），角色按需挂载；状态机通过接口拿能力（`IMover`、`IDamageable`），而不是拿具体的 `Player`。

**L4 • 面试官攻击**　“那你为什么不一开始就用组合？” → 两个角色的规模下继承最省事，我也确实复用了一大块代码；但我在 Entity 里已经看到了坏味道：`SlowEntityBy` 的基类实现只调了一个 Invoke、什么都没减速，子类各自重写——这说明“减速”并不是所有实体的共同行为，它更适合成为一个修正器组件（第 7 章）。

## Q1-2　状态为什么不是 MonoBehaviour?

**L2**　状态只是行为的切片，不需要 Transform、生命周期和 Inspector；做成普通类可以在 Awake 里一次性 new 出来、零 GetComponent、切换时没有 enable / disable 的开销，也方便做成可测试的纯逻辑。代价是状态里不能直接用协程和 Invoke，所以代码里会看到 `player.StartCoroutine("BusyForSeconds", .1f)` 这样借宿主组件跑协程的写法。

**L3**　更进一步可以让状态完全不认识 `Player`，只依赖一个“上下文”接口（输入快照、移动能力、动画参数），这样同一套状态能给玩家、AI 托管、回放共用。

### Q1-3 → Q1-4　模块怎么通信？为什么用单例？不用单例怎么办？

**L2**　三种方式并存：直接引用（状态持有 Player）、单例（技能、装备、怪物通过 PlayerManager 拿玩家）、委托（`onFlipped`、`onHealthChanged` 通知血条）。单例的好处是写起来快，坏处是隐式依赖：看 `ItemData_Equipment` 的签名完全不知道它会去改玩家属性。

**L3**　替代方案按“侵入性”从低到高：① 在场景组装处显式注入（SkillManager 在 Awake 里把 player 传给每个 Skill）；② 服务定位器加接口（`Services.Get<IPlayerProvider>()`，可在测试里替换）；③ 事件总线（装备变化发 `EquipmentChanged`，由 PlayerStats 自己订阅修改属性）；④ DI 框架（VContainer / Zenject）。对这个规模我会选 ①+③：依赖显式、写操作收口到属性所有者。

**L4 • 攻击**　“事件总线不是也是全局的吗？” → 是，但它把“谁改玩家属性”收口到了 PlayerStats 自己；单例的问题不在全局，而在**任何人都能写**。

### Q1-5　一帧里先执行什么？命中发生在哪个阶段？

**L2**　FixedUpdate（物理）→ Update（脚本，状态机在这里）→ 协程恢复 → 动画更新（动画事件在这里触发，命中判定就在这时做）→ LateUpdate → 渲染。所以“按键在 Update 里被读到、速度在本帧设置、位移在下一帧的物理步才发生、命中在本帧或之后某帧的动画阶段”。

**L3**　这个顺序会直接影响结论：第 6 章算弹反窗口时，玩家的反击状态是在 Update 里退出的，而怪物的 AttackTrigger 在动画阶段触发——同一帧里玩家先退出、怪物后命中，窗口的右边界就差了一帧。

**常见错误回答**

**错误**：“我用了 MVC 架构，Player 是 Model，状态机是 Controller，Animator 是 View。”

**问题**：Player 里既有数据又有行为还有协程，状态类直接读输入也直接写刚体——套名词会被一行代码问倒。

**更好的回答**：“它是 Unity 常见的组件 + 状态机结构；数据和表现没有严格分层，例如状态在 Enter 里直接设 Animator 参数。如果要分层，我会先把输入抽成快照、把伤害抽成事件。”

## 1.10 发散场景题

**场景 1-1　策划要加“会飞的弓箭手”和“玩家的召唤宠物”，你要改哪些文件?**

为什么会问：考察你对继承结构的扩展成本有没有概念。

弓箭手：新建 `Enemy_Archer` + 一套状态类（照抄骷髅的 6 个，Battle 改成保持距离、Attack 改成发射投射物），新写投射物控制器；“会飞”要绕开 `IsGroundedDetected` 和 Move 状态里的边缘检测——**状态类与具体子类强绑定**（`Enemy_SkeletonMoveState` 持有 `Enemy_Skeleton`），几乎无法复用。宠物：它既不是 Player 也不是 Enemy，却要能攻击敌人、被敌人攻击、跟随玩家；现有的“敌人只打 `Player` 组件”“玩家攻击只认 `Enemy` 组件”需要换成阵营 / 接口判断。结论：先引入 `IDamageable` + 阵营（Team），再把感知、移动、攻击拆成可组合的组件或行为树节点（第 9 章）。

**场景 1-2　新人接手这个项目，你会先让他读哪三个文件?**

为什么会问：考察你能不能抓住系统的“主干”。

① `Player.cs`（状态在哪里创建、Update 里先做什么）；② `PlayerGroundedState.cs`（大部分输入分支在这里，读懂它就读懂了操作逻辑）；③ `CharacterStats.cs`（伤害、异常状态、死亡全在这里）。再配一张图 1-4，告诉他“命中在动画事件里”，否则他会在 Update 里找半天攻击判定。

**场景 1-3　如果要做成 ECS（DOTS），哪些部分最容易迁移，哪些最难?**

为什么会问：考察数据与行为分离的理解。

最容易：Stat / 伤害公式（纯数据 + 纯函数）、异常状态计时（可以是一个 System 批量递减）、掉落概率（纯函数）。最难：动画事件驱动的命中（ECS 里通常改成“技能时间轴组件 + System 在固定逻辑帧判定”）、状态类里读输入和直接写刚体的写法、依赖单例的所有代码。迁移顺序也就是重构顺序：先把数据和计算从 MonoBehaviour 里拔出来。

**本章速记**

- 三张图：**对象图**（场景里真挂了什么）、**继承树**（组件 / 状态类 / SO 三种继承）、**一帧顺序**（Fixed → Update → 协程 → 动画事件）。
- 依赖全部汇聚到 Player；三个单例让“谁都能写玩家”。
- 36 个问题：P0 4 个、P1 11 个；面试主动讲 **死亡可打断 / 每帧自转移 / 合成少扣料。**

▶ **自测 1：为什么说“读 Unity 项目必须同时读场景”？**

▶ **自测 2：一次近战命中的伤害结算发生在 Update 里吗?**

▶ **自测 3：图 1-5 里，哪条依赖最该先拆?**

CHAPTER 02

# Unity 生命周期、单例与对象存亡

Awake 和 Start 各该做什么、单例怎么写才对、Destroy 之后对象“什么时候真的没了”、为什么这个项目在编辑器里能跑而打包会失败——这些都是 Unity 客户端面试的必考地基。

---

**本章目标**

- 按“自我初始化 / 跨对象连接 / 首帧运行”三段说清 Awake、OnEnable、Start 的分工，并指出项目里哪里放错了
- 看懂两个单例为什么写反、会在哪种场景切换下出事；默写正确的模板
- 说清 fake null、Destroy 延迟、Script Execution Order 不继承这三个“坑”
- 知道运行时代码里引用 `UnityEditor` 为什么会让打包失败

## 2.1 30 秒面试版 • 现状 vs 理想

**30 秒回答**

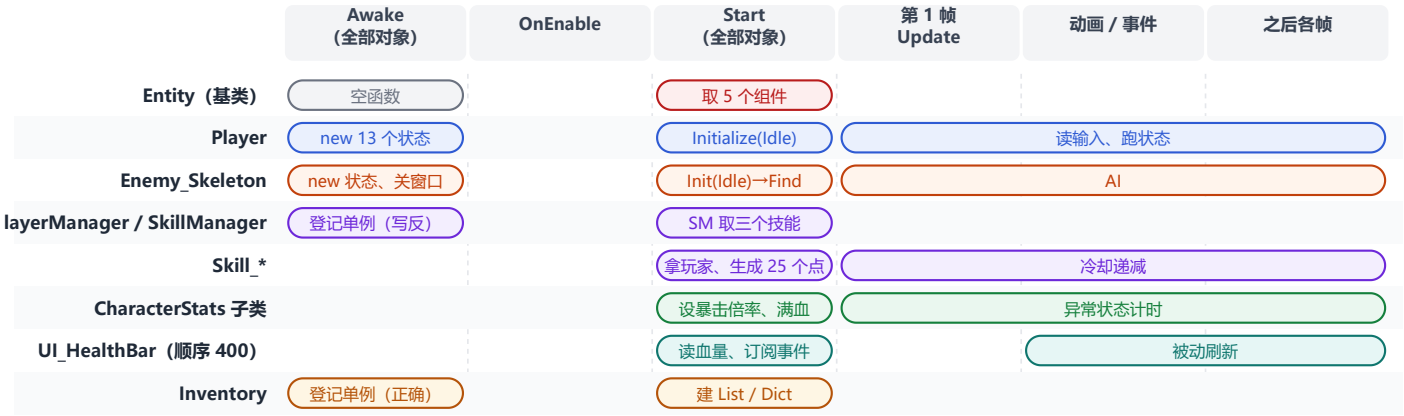
Awake 做“只依赖自己”的初始化（取自身组件、建内部数据），Start 做“依赖别人”的连接（拿单例、订阅事件），因为所有 Awake 都在任何 Start 之前；OnEnable / OnDisable 成对做订阅。项目里 Entity 把取组件放在 Start，导致对象在 Awake 和 Start 之间被访问会空引用；两个单例写反了——发现已有实例时销毁的是旧的、新的也不登记；脚本执行顺序配在父类 CharacterStats 上，对挂在场景里的子类无效；Inventory 和骷髅战斗状态 `using` 了 UnityEditor 命名空间，编辑器里能跑，打包会编译失败。

现状 → 问题 → 理想　**生命周期与对象存亡**

| 维度 | 当前做法（Demo 现状） | 存在的问题 | 理想重构方案 | 收益 / 代价 |
| --- | --- | --- | --- | --- |
| **取组件** | `Entity` 在 Start 里取刚体、动画机、特效、属性 | 隐患 生成后到 Start 之前被访问即空引用（例如生成当帧就被命中、被别人的 Awake 读取） | Awake 只做“我自己”（取自身组件），Start 做“我和别人” 设计方案 · 未实现 | 运行时生成的对象在 Awake 与 Start 之间被访问不会空引用 |
| **单例** | PlayerManager / SkillManager：发现旧实例时销毁旧的、新的也不登记；public 静态字段 | P0 多放一个管理器就连同上面的技能组件一起销毁，新的又不登记（推断：冲刺只能用一次，#4）；public 字段谁都能改 | 后来者自毁 + 只读属性 + OnDestroy 只清自己；或干脆改为注入 设计方案 · 未实现 | 场景里多放一个管理器也不会毁掉旧的（图 2-2） |
| **初始化顺序** | 执行顺序 300 配在父类 CharacterStats（对子类无效），400 配在血条（有效） | P2 配在父类上的 300 对 PlayerStats / EnemyStats 不生效，初始化先后靠运气（#31） | 显式初始化：Bootstrap 依次调用各系统 Init；必须用执行顺序时用 `[DefaultExecutionOrder]` 设计方案 · 未实现 | 顺序是看得见的代码，而不是隐式全局配置 |
| **编辑器依赖** | 两个运行时脚本 `using UnityEditor`（IDE 自动补全带入，并未使用） | P0 编辑器里一切正常，打包时编译失败——提测才发现（#1） | asmdef 分离运行时 / 编辑器代码 + CI 每日真打包 设计方案 · 未实现 | 问题在编辑器里就暴露 |
| **销毁与判空** | 依赖 fake null（`!player.sword`）；持有的引用从不置空 | 隐患 持有的是已销毁对象；改用 `?.` / `is null` 会绕过 fake null 判成“非空” | 持有方在回调 / OnDestroy 中置空，或用“id + 代数”句柄 设计方案 · 未实现 | 不依赖引擎重载，`?.` 也安全 |
| **出场重置** | 出场状态散在 Awake / Start（满血、关弹反窗口、初始状态） | 局限 改成对象池后，复用的对象带着上一次的血量、弹反窗口、状态 | 集中到 OnSpawn / OnDespawn，与对象池配合 设计方案 · 未实现 | 池化复用后状态正确（场景 2-3） |

现状问题：P0 ×2　P2 ×1　隐患 ×2　局限 ×1　（P0~P3 编号见附录 A，分级说明见 0.2）

## 2.2 Awake / Start 在项目里的分工




**现状** 图 2-1　各组件在生命周期里做了什么（红 = 放错阶段）。Unity 保证：同一批加载的对象，所有 Awake（及 OnEnable）先于任何一个 Start；所有 Start 先于第一次 Update

**现状问题** P2 组件在 Start 才取；执行顺序配在父类上，对子类无效（#31）

| 位置 | 源码 | 评价 |
| --- | --- | --- |
| `Entity.Start` | Entity.cs:45-52 取 Rigidbody2D、Animator、EntityFX、CharacterStats、Collider | 问题 这些只依赖自身，应放 Awake。放在 Start 意味着：**运行时 Instantiate 的怪物，在 Awake 之后、Start 之前被打中，`fx` / `stats` 都是 null。**当前场景只有预置的一只骷髅，所以没暴露。 |
| `Player.Awake` → `Start` | Player.cs:59-89：Awake 里 new 状态，Start 里先 `base.Start()` 再 `Initialize(idleState)` | 源码 顺序是对的：Idle.Enter 会读 `player.rb`，必须在 base.Start 取完组件之后。但这个约束只写在“调用顺序”里，换个人把两行对调就坏了。 |
| `Player.Start` | `skill = SkillManager.Instance`，而 SkillManager 在**它自己的 Start** 里才 GetComponent 三个技能 | 能工作是因为 Player.Start 只缓存了“管理器引用”，真正访问 `skill.clone` 发生在冲刺时（第 1 帧之后）。如果有人在 Player.Start 里写 `skill.dash.xxx`，就会随两个 Start 的先后随机空引用。 |
| `EnemyStats.Start` | 先 `ApplyLevelModifiers()` 再 `base.Start()`（EnemyStats.cs:17-23） | 源码 刻意的顺序：先加等级修正，再按修正后的上限回满血。写成 `base.Start()` 在前就会满血少一截。 |
| `UI_HealthBar.Start` | 读 `GetCurrentHealth()` 并订阅（UI_HealthBar.cs:13-25） | 依赖 CharacterStats.Start 先把 currentHealth 设满——作者用 Script Execution Order（400）保证了这一点，见 2.5。 |

**一句话规则**

**Awake：我自己。Start：我和别人。OnEnable / OnDisable：我和事件。**对象池复用时 Awake / Start 都不会再调，所以“每次出场都要重置的状态”只能放 OnEnable 或池的 OnTakeFromPool 回调里（第 12 章）。

## 2.3 单例：两个写反了，一个写对了

**当前源码 • 822270c** PlayerManager.cs:10-16（SkillManager.cs:21-31 完全相同）

```
public static PlayerManager _instance;      // public 字段：谁都能改写
private void Awake()
{
    if (_instance ≠ null)
        Destroy(_instance.gameObject);      // ← 销毁的是“已经登记的旧实例”
    else
        _instance = this;                   // ← 新实例只有在没有旧实例时才登记
}
```

**现状问题** **P0** 已有实例时销毁旧的、新的又不登记：多放一个管理器就连同技能组件一起毁掉（推断：冲刺只能用一次，#4）；public 字段谁都能改写

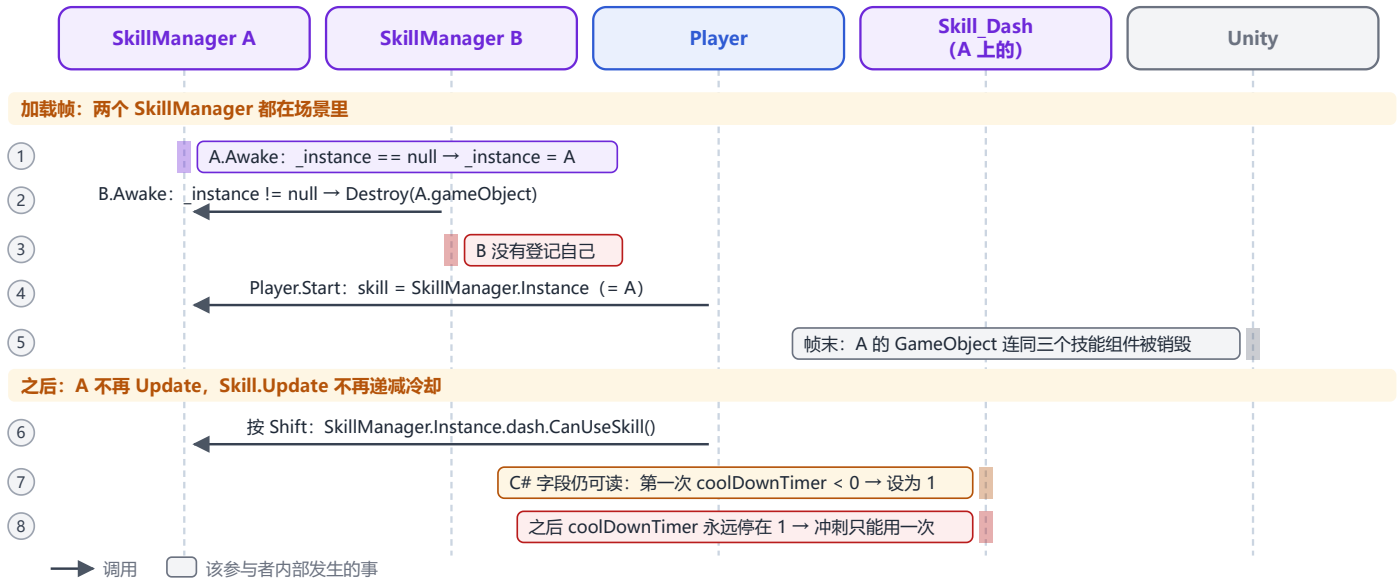
**当前源码 • 822270c** Inventory.cs:27-33（这一个是对的）

```
private void Awake()
{
    if (instance == null) instance = this;
    else Destroy(gameObject);               // 销毁“后来的自己”
}
```

**现状问题** ✓ 无问题 这一处写法正确（后来者自毁、已有实例保留），可作为对照模板

写反的版本在“只有一个实例”时表现正常，所以平时测不出来。出问题的三种情形：

| 情形 | 源码行为 | 正确模板的行为 |
| --- | --- | --- |
| ① 场景里误放了两个 SkillManager（复制粘贴、预制体嵌套） | A.Awake 登记 A；B.Awake 发现 A，**销毁 A 的整个 GameObject**（连同上面的 Skill_Dash / Clone / ThrowSword），B 不登记 → 帧末之后 `Instance` 指向已销毁的 A | B 销毁自己，A 继续工作 |
| ② 给单例加了 DontDestroyOnLoad，重新加载场景 | 新场景里的副本把常驻的旧实例毁掉，自己又不登记 → 两边都没了 | 新场景的副本自毁，常驻实例保留 |
| ③ 叠加加载（Additive）另一个也带 PlayerManager 的场景 | 同 ①：主场景的管理器被毁 | 后加载的自毁 |
| ④ 单场景切换（不常驻） | 旧场景卸载时 A 已销毁，`_instance ≠ null` 因 Unity 重载的 == 返回 false → 新实例正常登记 | 同左 |




现状 图 2-2　情形 ①：写反的单例让“冲刺只能用一次”【推断】——依据两条 Unity 规则：被销毁的组件不再收到 Update；对已销毁 MonoBehaviour 读写 C# 字段不会抛异常，只有访问引擎侧属性（transform 等）才抛 MissingReferenceException

**现状问题** P0 单例写反：多放一个管理器就销毁旧实例，新的又不登记（推断，#4）

图 2-2 的结论标为 推断 ，验证方法在附录 B：复制一个 SkillManager 进场景，按两次 Shift 观察第二次是否打印 “Skill [Dash] is on CoolDown” 且之后再也冲不出去。这个推断恰好是面试的好素材——它把“单例写反”这种看似无害的笔误，和 **fake null**、**销毁对象不再 Update** 两个引擎规则串在了一起。

**理想方案 • 骨架** 可复用的场景单例（C#，未在 Unity 中编译）

```
public abstract class SceneSingleton<T> : MonoBehaviour where T : SceneSingleton<T>
{
    public static T Instance { get; private set; }          // 只读属性，外部不能改写
    protected virtual void Awake()
    {
        if (Instance ≠ null && Instance ≠ this) { Destroy(gameObject); return; }  // 后来者
        Instance = (T)this;
    }
    protected virtual void OnDestroy()
    {
        if (Instance == this) Instance = null;               // 只清理“自己”
    }
}
// 子类重写 Awake 时必须先 base.Awake()，并在 base 返回后检查 Instance == this 再继续初始化
```

## 2.4 fake null 与 Destroy 延迟：项目里三处依赖它们

| 代码 | 依赖的规则 | 说明 |
| --- | --- | --- |
| `if (!player.sword)`（GroundedState.cs:57） | UnityEngine.Object 重载了 bool 转换 / ==：已销毁的对象“等于 null” | 源码 飞剑被 `Invoke("DestroyMe", 7)` 销毁后，`player.sword` 这个 C# 引用并没有清空，但判空返回 true，所以又能掷剑——**能工作，但靠的是引擎魔法。**如果写成 `player.sword is null` 或 `player.sword?.GetComponent…`，C# 层面的判空会认为它还在。 |
| `CatchTheSword()`：先 `ChangeState(catchSwordState)` 再 `Destroy(sword)`（Player.cs:138-142） | Destroy 在帧末才生效 | CatchSword.Enter 读 `player.sword.transform` 判断朝向。即使两行对调也能工作，因为 Destroy 之后本帧对象仍然存在——但别依赖这一点。 |
| `PickupItem()`：`AddItem` 后 `Destroy(gameObject)`（ItemObject.cs:39-43） | 同上 | 风险 如果同一物理步里触发器收到两次进入回调（例如玩家根物体上有两个碰撞体），两次都会 AddItem——Destroy 拦不住同一帧的第二次。当前玩家根物体只有一个胶囊体（血条上那个 Trigger 在子物体上，`GetComponent<Player>()` 取不到），所以没触发；做法是拾取时先置 `picked = true` 并关掉碰撞体。 |

**扩展：fake null 到底是什么**

UnityEngine.Object 是“C# 壳 + C++ 本体”。Destroy 释放的是 C++ 本体，C# 壳要等没人引用后被 GC 回收。为了让壳在本体消失后“看起来像 null”，Unity 重载了 `==` 和 bool 转换，检查本体指针。因此：① `?.`、`??`、`is null` 不走重载，会把已销毁对象当成活的；② 编辑器里 `GetComponent` 找不到组件时返回的也是一个“假 null”壳（为了给出更友好的报错），会产生少量托管分配，打包后不会。

## 2.5 Script Execution Order：配了 4 个，只有 1 个起作用

| 脚本（meta 中的 executionOrder） | 场景里实际挂的组件 | 是否生效 |
| --- | --- | --- |
| CharacterStats　300 | PlayerStats、EnemyStats（子类，各自的 MonoScript 顺序为默认 0） | **否：执行顺序按“具体脚本”记录，不会被子类继承（社区实测；代码里用 `[DefaultExecutionOrder]` 特性则会继承）** |
| UI_HealthBar　400 | UI_HealthBar ×2 | **是：它的 Start 晚于默认顺序的 PlayerStats / EnemyStats.Start，所以第一次读到的就是满血** |
| ItemData　500 | ScriptableObject | **否：执行顺序只对 MonoBehaviour 的事件函数有意义** |
| ItemDrop　600 | ItemDrop（没有 Awake / Start / Update） | **无可排序的函数** |

看得出作者遇到过“血条初始显示为 0”的问题，并用执行顺序修好了——这是个值得在面试里讲的经历，但要讲到下一层：**执行顺序是全局配置，是“隐式契约”**。更稳的做法是显式初始化：血条在 OnEnable 里订阅，并在 Stats 初始化完成时由 Stats 发一次 `onHealthChanged`；或者由一个 Bootstrap 组件按顺序调用各系统的 `Init()`。

## 2.6 打包会失败：运行时代码引用了 UnityEditor

**当前源码 • 822270c** `Inventory.cs:1-4` 与 `Enemy_SkeletonBattleState.cs:1-4`

```
using static UnityEditor.Progress;                  // Inventory.cs：IDE 自动补全带进来的
using UnityEditor.Experimental.GraphView;           // Enemy_SkeletonBattleState.cs：同上
```

**现状问题** **P0** 运行时脚本引用 UnityEditor 命名空间：编辑器里正常，打包编译失败（#1）

**问题** **P0** 编辑器里所有程序集都能看到 `UnityEditor.dll`，所以 Play Mode 一切正常；打包时运行时程序集（Assembly-CSharp）不引用编辑器程序集，这两行会报“找不到命名空间 UnityEditor”，构建直接失败。两处都没有真正用到其中的类型，删掉即可。

**工程上怎么提前发现**

① 用程序集定义（.asmdef）把运行时代码和编辑器代码分开，运行时 asmdef 不勾 Editor 平台，误用会在编辑器里就报错；② 必须混写时用 `#if UNITY_EDITOR … #endif` 包住；③ CI 每天跑一次真打包（哪怕只打 Windows）——“编辑器能跑 ≠ 能打包”是最常见的提测事故之一。另外几处 `using Unity.VisualScripting;`（Entity、Player、Skill、SkillManager 等）没用到任何类型，虽不致命，但会让代码隐式依赖 Visual Scripting 包。

## 2.7 面试追问

追问树

```
Awake 和 Start 的区别？（Q2-1）
├─ 为什么取组件要放 Awake？放 Start 会出什么问题？（Q2-1 L3）
├─ 单例怎么写？你的单例有没有问题？（Q2-2）
│   ├─ 场景切换 / DontDestroyOnLoad / 叠加加载时怎样？（2.3 表）
│   └─ 单例的缺点？怎么替代？（Q1-4、Q2-5）
├─ Destroy 之后马上访问会怎样？什么时候真正销毁？（Q2-3）
│   └─ 为什么 obj == null 是 true 但 obj?.x 还能执行？（Q2-4 fake null）
└─ 多个脚本的初始化顺序怎么控制？（Q2-5）
```

### Q2-1　Awake 和 Start 的区别?

**L1**　Awake 在对象实例化（且所在 GameObject 激活）时立刻调用，Start 在该脚本第一次 Update 之前调用；同一批加载的对象，所有 Awake 都先于任何 Start。

**L2**　所以 Awake 做自我初始化，Start 做跨对象连接。脚本被禁用时 Awake 仍会执行而 Start 不会，直到它被启用。

**L3 • 项目**　我的 Entity 把 GetComponent 放在了 Start，现在没出问题是因为怪物都是场景预置的；如果改成刷怪，刷出来的那一帧被飞剑打到，`DamageEffect` 里的 `fx` 就是空的。应该挪到 Awake。

**L4 • 攻击**　“运行时 Instantiate 的对象，Start 什么时候调？” → Instantiate 返回前 Awake / OnEnable 已经执行；Start 推迟到它的第一次 Update 之前——在 Update 里创建的对象，社区实测是本帧 Update 之后补调 Start、下一帧才第一次 Update。所以“生成后立刻配置参数”应该

在 Instantiate 之后、Start 之前完成，Start 里不要覆盖这些参数（项目里 `SetupClone`、`SetupSword` 正是在 Instantiate 之后立刻调用的）。

### Q2-2　单例怎么写?

**L2**　Awake 里：已有实例且不是自己 → 销毁自己并 return；否则登记自己；OnDestroy 里只在 Instance 是自己时清空；Instance 做成只读属性。需要跨场景就在登记后 DontDestroyOnLoad。

**L3 • 项目**　我项目里 PlayerManager 和 SkillManager 写反了：发现旧实例时销毁的是旧实例，新的还不登记。只有一个实例时看不出来；场景里多放一个就会把旧的连同上面的技能组件一起毁掉，按我的推断冲刺会变成只能用一次——因为被毁的组件不再 Update，冷却不再递减，而 C# 字段还读得到。

**L4**　“那你为什么不用 static class？” → 管理器需要 Inspector 配置（技能预制体、数值）和 Unity 生命周期（Update 递减冷却），所以是 MonoBehaviour；纯逻辑的工具（伤害公式）才适合 static。

### Q2-3 → Q2-4　Destroy 什么时候生效？fake null 是什么?

**L2**　Destroy 把对象标记为待销毁，当前帧的更新循环结束后才真正销毁（`DestroyImmediate` 立即销毁，只建议编辑器代码用）。所以 Destroy 后本帧内对象仍然可用，回调还会继续收到。

**L3**　销毁后 C# 壳还在，Unity 重载了 == 让它等于 null；`?.` / `??` 不走重载，会把它当成活对象。我项目里 `if (!player.sword)` 能在飞剑自动销毁后正确返回“没有剑”，靠的就是这个重载。

### Q2-5　多个脚本的初始化顺序怎么控制?

**L2**　三种手段：Script Execution Order（全局配置）、`[DefaultExecutionOrder]`（写在代码里，可被子类继承）、显式初始化（一个入口按顺序调用 Init，或依赖方在被依赖方就绪事件里再初始化）。

**L3 • 项目**　我用执行顺序 400 让血条的 Start 晚于属性组件，解决了初始显示 0 的问题；但我把 300 配在了父类 CharacterStats 上，对场景里的 PlayerStats / EnemyStats 并不生效。现在我更倾向显式初始化：执行顺序是隐式契约，人一多就没人记得为什么是 400。

**常见错误回答 ①**

**错误**：“Start 在 Awake 之后调用，所以在 Start 里访问别的对象一定安全。”

**问题**：只保证“所有 Awake 先于任何 Start”，不保证 A.Start 先于 B.Start；运行时生成的对象也不在同一批里。

**更好的回答**：“Start 里访问别人在 Awake 里准备好的东西是安全的；访问别人在 Start 里才准备的东西不安全，需要执行顺序或显式初始化。”

**常见错误回答 ②**

**错误**：“Destroy 之后对象就是 null 了。”

**问题**：本帧内它还活着；之后 == null 为真，但 C# 引用并未清空，`?.` 仍会执行。

**更好的回答**：“Destroy 在帧末生效；销毁后用 Unity 的 == 判空，自己持有的引用最好在 OnDestroy 或回调里手动置空。”

## 2.8 发散场景题

**场景 2-1　主菜单 → 战斗 → 主菜单，玩家的背包和属性怎么保留?**

为什么会问：项目只有一个场景，Inventory 和 PlayerStats 都跟场景同生共死。

把“数据”和“表现”拆开：背包、装备、属性基础值放进一个跨场景的纯数据对象（常驻的 GameSession / 存档服务，或 ScriptableObject 运行时容器），场景里的 Inventory / PlayerStats 只是视图和行为，在 OnEnable 时从数据层加载、OnDisable 时写回。不要给整个玩家 GameObject 加 DontDestroyOnLoad——它会把相机跟随、UI 引用、场景内对象引用全部带到新场景里变成悬空引用。

**场景 2-2　改成刷怪笼，刷出来的骷髅在出生那一帧被旋转剑命中，会发生什么?**

为什么会问：考察你能否把 Awake / Start 的区别落到具体的空引用上。

如果是在 Update 里 Instantiate，按社区实测 Start 会在本帧 Update 之后才补调；而飞剑的 SpinLogic 在 Update 里调用 `ThrowSwordDamage(enemy)` → `player.stats.DoDamage(enemy.stats)`——骷髅的 `stats` 是在 Entity.Start 里赋值的，此时可能还是 null，同帧的 `enemy.DamageEffect()` 里 `fx` 也可能为 null。修法：取组件挪到 Awake；或者刷怪后先置为“无敌 / 不可选中”一帧。

### 场景 2-3　加了对象池以后，复用的骷髅“满血复活”失败、弹反提示还亮着，为什么?

为什么会问：对象池是第 12 章的重构方向，面试官会提前埋雷。

池化对象第二次出场不会再调 Awake / Start：`currentHealth = GetMaxHealth()` 在 Start 里、`CloseCounterAttackWindow()` 在 Awake 里，`stateMachine.Initialize(idleState)` 在 Start 里，都不会重跑；死亡状态把 `cd.enabled = false`、`animator.speed = 0` 也不会恢复。所有“出场重置”要集中到一个 `OnSpawn()`（由池在取出时调用），并且和 `OnDespawn()` 成对。

### 场景 2-4　为什么同一份代码在同事电脑上偶现血条显示 0?

为什么会问：执行顺序未定义的经典表现。

两个默认顺序的脚本之间先后不确定（官方文档明确说不要依赖），可能随机器、构建、Unity 版本变化。我的项目恰好用执行顺序 400 消除了血条与属性的竞争；如果别人把它从列表里删掉，或者新增一个默认顺序、又依赖 currentHealth 的脚本（比如 Boss 血条），问题就会回来。根治是让“谁先初始化”显式化。

**本章速记**

- **Awake 我自己、Start 我和别人、OnEnable/OnDisable 我和事件**；池化对象只会重复 OnEnable/OnDisable。
- 单例：**后来者自毁 + 只读属性 + OnDestroy 只清自己**；项目里 PlayerManager / SkillManager 写反（#4）。
- fake null：Destroy 后 == null 为真，`?.` 不认；Destroy 在帧末生效。
- 执行顺序不被子类继承（CharacterStats 300 无效）；运行时代码 `using UnityEditor` → 打包失败（#1）。

▶ **自测 1：Player.Start 里如果先 Initialize(idleState) 再 base.Start()，会怎样?**

▶ **自测 2：情形 ① 里为什么不是立刻空引用，而是“冲刺只能用一次”？**

▶ **自测 3：怎样让基类上的执行顺序对所有子类生效?**

PART II

# 动作手感：状态机、动画、时间与判定

动作游戏的“手感”由四件事共同决定：状态怎么切（第 3 章）、动画事件什么时候触发（第 4 章）、计时与物理是否帧率无关（第 5 章）、命中与弹反怎么判（第 6 章）。这一部分的每个结论都有逐帧回放的实测。

CHAPTER 03

# 玩家状态机：13 个状态、三个切换入口与缺失的守卫

“状态机怎么设计”几乎是动作游戏客户端的必问题。这个项目的状态机结构清晰，但切换入口分散在三处、没有统一的守卫——于是出现了一帧两次转移、墙到头还在滑、死亡后冲刺“复活”。

---

**本章目标**

- 画出 13 个状态的转移图，说清 Grounded / Air 两个父类的作用
- 理解 `if (currentState ≠ this) return;` 这句惯用法在防什么，用 E1 回放看缺了它会怎样
- 找到全部三个切换入口（状态自身 / 全局输入 / 外部强制），说明“死亡不是吸收态”的根因
- 给出转移守卫、优先级、输入缓冲的设计，并能在面试中画出分层状态机

## 3.1 30 秒面试版 • 现状 vs 理想

**30 秒回答**

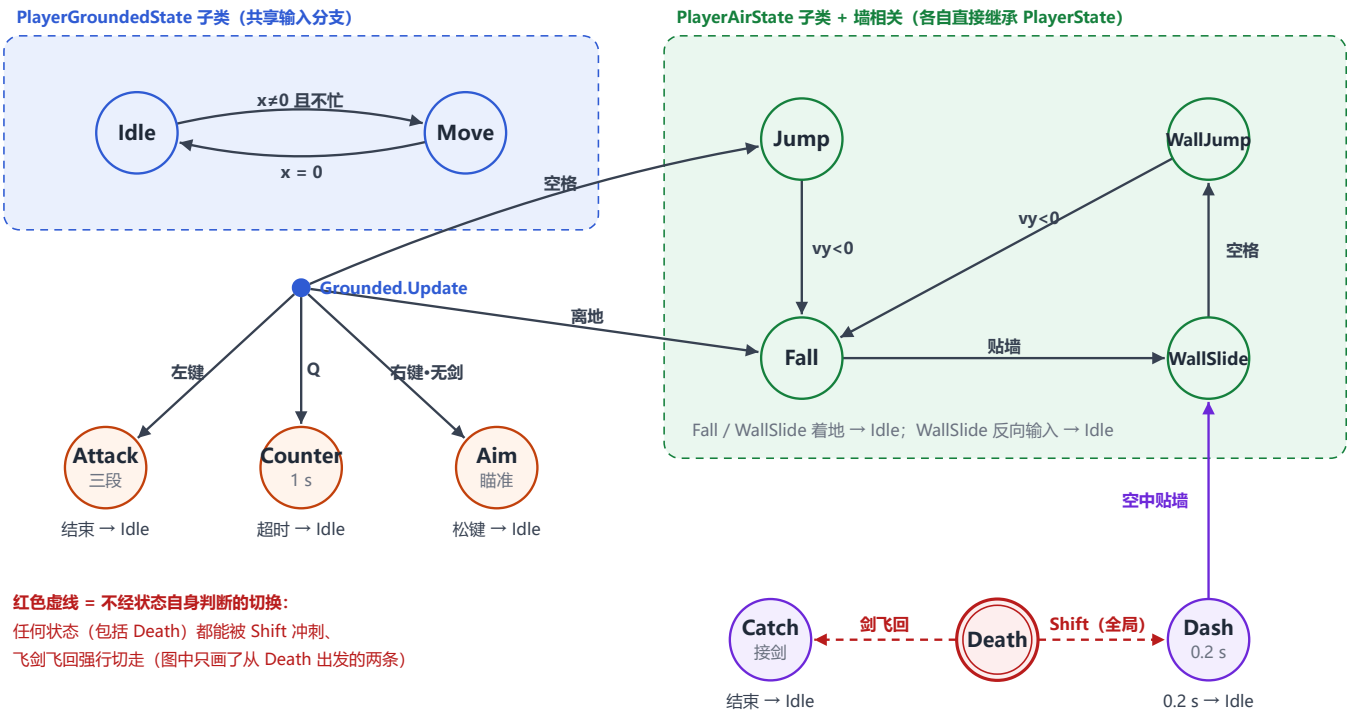
每个状态是一个继承 `PlayerState` 的普通类，Enter 时把同名 Animator bool 置真、Exit 时置假，Update 里读输入并决定是否切换；地面通用的输入（攻击、反击、瞄准、跳、掉落）放在父类 `PlayerGroundedState`。问题在于切换入口有三处：状态自己的 Update、`Player.Update` 里全局的冲刺检测、以及外部直接调用的 `Die` / `CatchTheSword`，它们都不检查“当前状态允不允许被打断”。逐帧回放发现：落地贴墙会一帧切两次、墙到头仍在墙滑、死亡后按冲刺或飞剑飞回都能“复活”。改法是给状态加打断优先级和标签，所有切换走同一个带守卫的入口，并把输入抽成带缓冲的快照。

现状 → 问题 → 理想　**玩家状态机**

| 维度 | 当前做法（Demo 现状） | 存在的问题 | 理想重构方案 | 收益 / 代价 |
| --- | --- | --- | --- | --- |
| **切换入口** | 三处：状态自身、Player.Update 全局冲刺、外部强切（Die / CatchTheSword），都不检查当前状态 | P0 死亡后按冲刺 / 接剑即“复活”（E1-S3/S4，#3） | 唯一的 Request(下一状态, 等级) 入口，按可打断等级守卫；Death 只接受 Revive<br>设计方案 · E1 已复现现状问题 | 死亡成为吸收态，“复活”不再可能 |
| **每帧切换次数** | 不限制：ChangeState 之后，旧状态的 Update 继续往下执行 | P1 切换后继续执行旧状态代码：一帧两次转移，新状态 Enter 的初始化被覆盖（E1-S1，#15） | 请求-提交：本帧只登记请求，帧末统一切换一次<br>设计方案 · 未实现 | Enter 的初始化不会被旧状态覆盖；代价是新状态晚半帧开始 Tick |
| **出口完整性** | WallSlide 的出口只有：着地、蹬墙、按离墙方向 | P1 墙在半空结束后对着空气慢落（E1-S2，#14） | 每个状态的出口覆盖“进入条件不再成立”，状态机每帧先检查 StillValid<br>设计方案 · 未实现 | 新增状态时不会漏出口 |
| **父状态** | Grounded / Air 只是代码复用，子类要手动判 `currentState ≠ this` | 局限 父状态不是状态，子类漏判 `currentState ≠ this` 就出错（#15 的成因）；“边跑边瞄准”只能再造组合状态 | 真正的分层状态机：父状态也是状态；移动层与动作层并行<br>设计方案 · 未实现 | 边跑边瞄准、空中攻击不需要组合新状态 |
| **输入** | 12 处 `Input.*` 直接在状态里读 | 局限 无法重绑定、无法缓冲 / 回放 / AI 托管、无法单测；落地前按的跳直接丢失（F3） | 输入采样成意图快照 + 带时间戳的预输入缓冲（第 20 章）<br>设计方案 · 未实现 | 可重绑定、可回放、可 AI 托管、可单测 |
| **可观测性** | 没有切换日志 | 局限 “动作偶尔抽一下”无法复现定位；E1 这类问题只有逐帧回放才看得到 | 最近 64 次切换的环形缓冲 + 屏幕 HUD<br>设计方案 · 未实现 | “动作偶尔抽一下”可以定位 |

现状问题：P0 ×1　P1 ×2　局限 ×3　（P0~P3 编号见附录 A，分级说明见 0.2）

## 3.2 状态与转移全图




**现状** 图 3-1　玩家状态转移全图（源码：player/*.cs）。蓝框内两个状态共享 Grounded 的输入分支（小圆点）；红色虚线是两个“全局入口”——它们从任何状态都能发生

**现状问题** **P0** 两个全局入口不检查当前状态：死亡后冲刺 / 接剑即“复活”（E1-S3/S4，#3）；WallSlide 缺“墙没了”出口（#14）

## 3.3 骨架代码：模板方法 + 父类兜底输入

当前源码 • 822270c PlayerState.cs:25-44 / PlayerStateMachine.cs:11-22

```
public virtual void Enter()  { player.animator.SetBool(animBoolName, true); rb = player.rb; t
public virtual void Update()
{
    stateTimer -= Time.deltaTime;                    // 每个状态共用一个倒计时
    xInput = Input.GetAxisRaw("Horizontal");         // 状态直接读输入
    yInput = Input.GetAxisRaw("Vertical");
    player.animator.SetFloat("yVelocity", rb.velocity.y);
}
public virtual void Exit()   { player.animator.SetBool(animBoolName, false); }

public void ChangeState(PlayerState _playerState)    // 没有任何检查：谁调都切
{
    currentState.Exit();
    currentState = _playerState;
    currentState.Enter();
}
```

**现状问题** 局限 状态直接读 Input、共用一个 stateTimer；ChangeState 无条件切换，不检查当前状态允不允许被打断（#3 的根源）

当前源码 • 822270c PlayerGroundedState.cs:21-53 + PlayerIdleState.cs:23-32

```
public override void Update()                        // Grounded：地面通用输入，按优先级 return
{
    base.Update();
    if (Input.GetKeyDown(KeyCode.Mouse1) && HasNoSword()) { stateMachine.ChangeState(player.a
    if (Input.GetKeyDown(KeyCode.Mouse0))                { stateMachine.ChangeState(player.pr
    if (Input.GetKeyDown(KeyCode.Q))                     { stateMachine.ChangeState(player.co
    if (Input.GetButtonDown("Jump") && player.IsGroundedDetected()) { stateMachine.ChangeStat
    if (!player.IsGroundedDetected())                    { stateMachine.ChangeState(player.fa
}
// Idle：父类可能已经切走了——这句守卫防的是“已经不是我了还继续执行我的逻辑”
public override void Update()
{
    base.Update();
    if (stateMachine.currentState != this) return;
    if (xInput != 0 && !player.isBusy) stateMachine.ChangeState(player.moveState);
}
```

**现状问题** P1 父类里的 return 只退出父类函数，子类必须再判 currentState != this，漏判即一帧两次转移（#15）；HasNoSword() 是带副作用的查询（#36）

这是典型的**模板方法**：父类定义 Enter / Update / Exit 的公共部分，子类 override 后先调 base。两个细节值得在面试里主动说：

- **父类里的 return 只能退出父类函数。**Grounded.Update 切到攻击后 return，回到 Idle.Update 里还会继续往下执行——所以 Idle / Move 都要再判一次 `currentState ≠ this`。这是“继承复用切换逻辑”的固有代价，第 9 章骷髅的 Battle 状态正是因为没意识到这一点而每帧自己切到自己。
- `HasNoSword()` 名字是查询，但它有副作用：剑在外面时会顺手 `ReturnSword()`。于是“右键”在有剑时变成“召回”，逻辑上没错，但读代码的人看条件判断时不会想到这里在改世界。

## 3.4 E1-S1：落地那一帧刚好贴墙，一帧切两次

**当前源码 • 822270c** PlayerFallState.cs:16-35（三个 if 之间没有 return，也没有 currentState 守卫）

```
public override void Exit()
{
    base.Exit();
    player.SetVelocity(xInput, rb.velocity.y);  // 原注释：修“落地保留水平速度”；没乘 moveSpeed
}
public override void Update()
{
    base.Update();
    if (player.IsGroundedDetected())
        stateMachine.ChangeState(player.idleState);
    if (xInput ≠ 0)                                    // ← 已经切到 Idle 了，仍然执行
        player.SetVelocity(xInput * player.moveSpeed, rb.velocity.y);
    if (player.IsWallDetected())                       // ← 再切一次
        stateMachine.ChangeState(player.wallSlideState);
}
```

**现状问题** P1 三个 if 之间没有 return：落地贴墙一帧切两次，Idle.Enter 清零的速度又被旧代码改回（E1-S1，#15）

脚本化输入：空中按住 → 下落，第 5 帧同时“着地 + 贴墙”（向墙根跳过去、正好落在墙脚时就是这样）。C++ 逐行移植后的回放 **实测 E1**：

| 帧 | 输入 / 传感器 | 进入时状态 | 本帧发生的转移 | 结束时状态 | vx | Animator 为真的 bool |
|---|---|---|---|---|---|---|
| 4 | → 空中 | Fall | — | Fall | 7 | Air |
| 5 | → 着地+贴墙 | Fall | Fall→Idle，<br>Idle→WallSlide | WallSlide | 7 | WallSlide |
| 6 | → 着地+贴墙 | WallSlide | WallSlide→Idle | Idle | 0 | Idle |
| 7 | → 着地+贴墙 | Idle | Idle→Move | Move | 0 | Move |
| 8 | → 着地+贴墙 | Move | — | Move | 7 | Move |

表 3-1　E1-S1 回放（60 fps）。第 5 帧在一次 Update 里连续两次 ChangeState，最终停在 WallSlide 且 vx = 7（Fall 里的 SetVelocity 在切到 Idle 之后仍执行）；下一帧 WallSlide 发现着地又切回 Idle

后果有三层：① Animator 上 WallSlide 被置真一帧，因为玩家 Animator 的转移都是“无退出时间、过渡 0”，墙滑动画有机会闪现一帧 推断 ；② Idle.Enter 刚把速度清零，Fall 的下一行又把 vx 设回 7——**Enter 里做的初始化被旧状态的后续代码覆盖**；③ 这类问题不会报错，只会表现为“偶尔动作抽一下”，最难查。修法有两种：每个分支切换后立刻 return；或者在状态机层面**把切换延迟到本帧状态 Update 结束后再统一执行**（下一节的“请求-提交”模式）。

## 3.5 E1-S2：墙到头了，还在墙滑

WallSlide.Update 只有三个出口：反向输入、跳跃、着地。**没有“墙没了 → Fall”**。当墙是平台侧面、下沿悬在半空时，贴墙滑过下沿之后：

| 帧 | 输入 / 传感器 | 进入时状态 | 本帧发生的转移 | 结束时状态 | vy |
|---|---|---|---|---|---|
| 9 | → 空中+贴墙 | WallSlide | — | WallSlide | -2 |
| 10 | → 空中 | WallSlide | — | WallSlide | -2 |
| 40 | → 空中 | WallSlide | — | WallSlide | -2 |
| 69 | → 空中 | WallSlide | — | WallSlide | -2 |
| 70 | → 着地 | WallSlide | WallSlide→Idle | Idle | 0 |
| 71 | → 着地 | Idle | Idle→Move | Move | 0 |

表 3-2　E1-S2 回放：第 10 帧起已经不贴墙，但一直停在 WallSlide，每帧被强制设成 vy = −2（重力被覆盖），直到第 70 帧着地——整整 70 帧里有 60 帧是“对着空气墙滑”

画面上就是角色在半空以 2 m/s 缓慢下落、播放墙滑动画。玩家的直接感受是“卡住了”。这个 bug 的教训是**每个状态都要问：进入条件消失时怎么办？**进入 WallSlide 的条件是“空中 + 贴墙”，那么“不贴墙”和“着地”都必须是出口。

## 3.6 E1-S3 / S4：死亡不是吸收态

正常的状态机里，Death 应该是**吸收态**（只能被“复活”这种显式事件离开）。这个项目有三个切换入口，后两个都不看当前状态：

| 入口 | 源码 | 能从哪些状态触发 |
|---|---|---|
| ① 状态自身 Update | 各状态类 | 只在本状态内，逻辑自洽 |
| ② 全局冲刺 | `Player.Update → DashInputCheck`（Player.cs:105-119），在 `currentState.Update()` 之前执行 | **全部 13 个**：包括 Death、CounterAttack、AimSword、CatchSword、WallSlide |
| ③ 外部强制 | `PlayerStats.Die → player.Die()`、`SkillController_ThrowSword.Update → player.CatchTheSword()`、`Entity.DamageEffect` | 全部；CatchTheSword 只要剑飞到 1 m 内就切 |

| 帧 | 输入 | 进入时状态 | 本帧发生的转移 | 结束时状态 | vx |
|---|---|---|---|---|---|
| 10 | 死亡 着地 | Idle | Idle→Death | Death | 0 |
| 11 | 着地 | Death | — | Death | 0 |
| 39 | 着地 | Death | — | Death | 0 |
| 40 | Shift 着地 | Death | Death→Dash | Dash | 25 |
| 52 | 着地 | Dash | Dash→Idle | Idle | 0 |
| 60 | → 着地 | Idle | Idle→Move | Move | 0 |

表 3-3　E1-S3：第 10 帧死亡，第 40 帧按 Shift——Death→Dash 直接发生（Skill_Dash 冷却早已好了），0.2 s 后进入 Idle，第 60 帧起能正常走路。Death.Exit 把 Die 置假，Animator 也离开了死亡动画

| 帧 | 输入 | 进入时状态 | 本帧发生的转移 | 结束时状态 | vx |
|---|---|---|---|---|---|
| 10 | 死亡 着地 | Idle | Idle→Death | Death | 0 |
| 11 | 着地 | Death | — | Death | 0 |
| 39 | 着地 | Death | — | Death | 0 |
| 40 | 剑飞回 着地 | Death | Death→CatchSword | CatchSword | -6 |
| 54 | 着地 | CatchSword | CatchSword→Idle | Idle | 0 |
| 61 | → 着地 | Idle | Idle→Move | Move | 0 |

表 3-4　E1-S4：死亡前掷出的飞剑在第 40 帧飞回 → CatchTheSword 把 Death 切成 CatchSword，接剑动画事件触发后回到 Idle（模型：事件时间 0.2143 s）

为什么测试时没发现？因为骷髅 damage = 0，玩家在当前场景里根本死不了；一旦给骷髅配上伤害，这就是一个**必现的 P0**。同样的问题也出在冲刺打断反击（弹反判定中途被取消）、打断接剑（剑已销毁但接剑状态没走完）等组合上——状态数 13，任意两两组合就有 156 种转移，靠人眼检查不可能覆盖。

## 3.7 两个小机关：isBusy 与连招计数

**isBusy**：攻击和接剑在 Exit 时 `player.StartCoroutine("BusyForSeconds", .1f)`，0.1 s 内 Idle 不允许切 Move——用来消掉“攻击刚结束角色滑一下”的手感问题。但它是**可重叠的协程**：两个 BusyForSeconds 叠在一起时，先开始的那个结束就会把 isBusy 置假，后一个的 0.1 s 被提前截断。这和第 7 章“冻结协程重叠”是同一种 bug，只是这里时间短、影响小。

**连招计数**：`comboCounter` 在 Exit 里 +1，Enter 里“超过 2 或距上次攻击超过 1.5 s”就归零。实测 E1-S5：第 1 段在命中帧（0.357 s）之前被冲刺取消，Exit 照样 +1，下一次左键直接打出第 2 段（进入时计数依次为 0、1）。

> **这是 bug 还是设计?**
>
> “取消后连招是否推进”没有标准答案：鬼泣系允许跳跃 / 冲刺取消后续段，有的游戏取消即重置。问题在于这里的行为**是 Exit 写法的副产品，不是被设计出来的**：第 3 段攻击位移是 3（最大），“冲刺取消第 1 段 → 直接第 2 段 → 第 3 段”可以让玩家用更短的时间打出位移最大的一段。面试时这样说最好：“我发现这个行为后先确认了它是不是想要的，再决定把推进条件改成‘命中帧之后才算完成一段’。”

## 3.8 输入直接写在状态里：手感与可测试性的天花板

玩家相关代码里有 **12 处** `Input.*` 调用，分散在 6 个文件（PlayerState 2、Grounded 4、WallSlide 1、PrimaryAttack 1、AimSword 2、Player 2），瞄准方向和松键时机在 Skill_ThrowSword 里又读了 3 次。带来的限制：

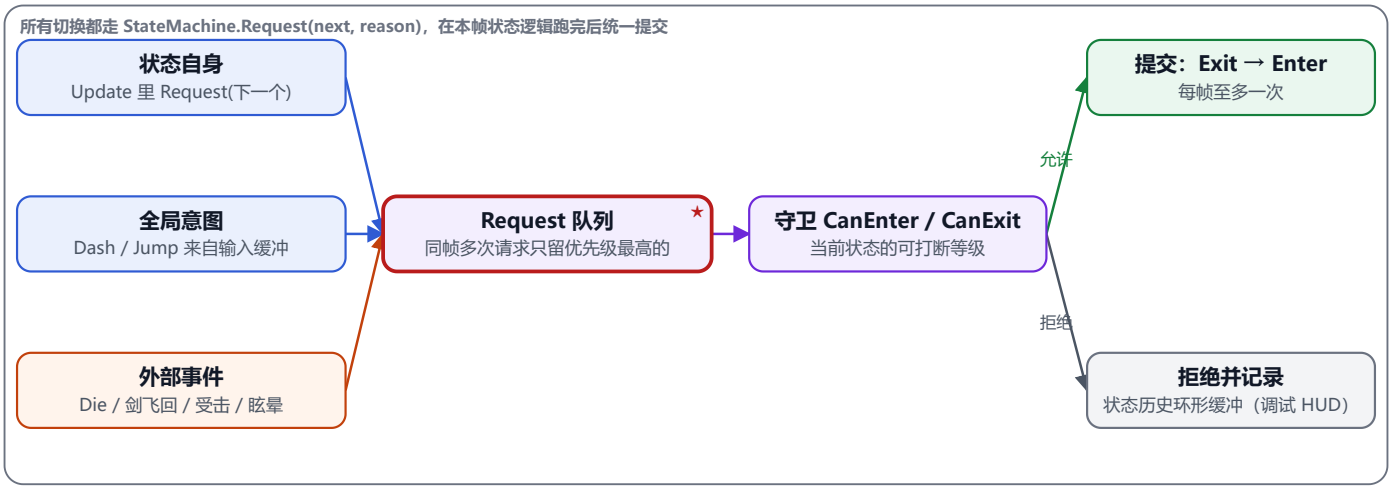
| 限制 | 表现 | 动作游戏里的标准做法 |
| --- | --- | --- |
| 没有**输入缓冲**（预输入） | 攻击动画还剩 50 ms 时按下的跳跃被丢弃，玩家觉得“按了没反应” | 按键进入带时间戳的缓冲队列，状态在“可取消窗口”里消费 100~200 ms 内的缓冲指令 |
| 没有 **Coyote Time** | 跑出平台边缘的下一帧就切 Fall，此时按跳无效 | 离地后 0.1 s 左右仍允许起跳 |
| `GetKeyDown` 只在一帧为真 | 两个状态在同一帧都想消费同一个按键时，先执行的抢走；状态切换那帧的按键会被新状态错过（新状态本帧不再 Update） | 输入在帧首采样成快照，由状态机统一分发 |
| 无法重绑定 / 手柄 / 回放 / AI 托管 | 按键写死 Mouse0 / Q / LeftShift | Input System 的 Action 映射；状态只认“意图”（Attack、Dash），不认键位 |
| 无法单元测试 | 状态逻辑依赖全局 Input | 注入输入接口；本书 E1 就是把 Input 替换成脚本化快照才得以逐帧回放 |

**理想方案 • 骨架** 输入快照 + 预输入缓冲（C#，未在 Unity 中编译）

```
public enum Intent { None, Attack, Jump, Dash, Counter, Aim }
public struct InputFrame { public float x, y; public Intent pressed; public bool aimHeld, aim

public class InputBuffer
{
    readonly Queue<(Intent intent, float time)> q = new Queue<(Intent, float)>();
    public float window = 0.15f;                        // 预输入有效期
    public void Push(Intent i, float now) { if (i ≠ Intent.None) q.Enqueue((i, now)); }
    public bool Consume(Intent want, float now)         // 状态在“可取消窗口”里调用
    {
        while (q.Count > 0 && now - q.Peek().time > window) q.Dequeue();   // 丢掉过期指令
        if (q.Count > 0 && q.Peek().intent == want) { q.Dequeue(); return true; }
        return false;
    }
}
// 每帧开头：frame = sampler.Sample(); buffer.Push(frame.pressed, Time.time); 然后才 stateMachi
```

## 3.9 改进：一个入口、带守卫、按优先级打断




理想方案 图 3-2　改进后的切换管线：三个入口不再直接调用 ChangeState，而是提交请求；状态声明自己的“可打断等级”，Death 等级最高（只接受 Revive），攻击前摇 / 后摇可以有不同等级

| 状态 | 可被谁打断（等级 ≥） | 标签 |
| --- | --- | --- |
| Idle / Move / Fall / Jump | 任何请求 | CanMove · CanDash · CanAttack |
| Attack 前摇（命中帧前） | 受击硬直、死亡（Dash 是否允许由设计决定） | Committed |
| Attack 后摇（命中帧后） | Dash、Jump、下一段攻击（消费预输入） | Cancelable |
| Counter / Aim / CatchSword | 受击、死亡、Dash | — |
| Death | **只接受 Revive** | Dead（所有“CanXxx”都为假） |

**理想方案 • 骨架** 带守卫的状态机（C#，未在 Unity 中编译）

```
public abstract class PlayerState
{
    public virtual int InterruptLevel => 0;          // 被打断所需的最低请求等级
    public virtual bool IsDead => false;
    // ...Enter / Tick / Exit
}
public class PlayerStateMachine
{
    public PlayerState Current { get; private set; }
    PlayerState pending; int pendingLevel = -1;
    public void Request(PlayerState next, int level)  // level：普通 0、受击 5、死亡 9、复活 10
    {
        if (next == Current || level < Current.InterruptLevel) return;    // 自转移、等级不够：拒
        if (level > pendingLevel) { pending = next; pendingLevel = level; }
    }
    public void Tick(in InputFrame f)
    {
        Current.Tick(f);                                  // 状态只 Request，不直接切
        if (pending ≠ null) { Current.Exit(); Current = pending; Current.Enter(); }
        pending = null; pendingLevel = -1;                // 每帧至多切一次
    }
}
// Death.InterruptLevel = 10；Shift 冲刺（0）、剑飞回（0）、受击（5）都进不来，只有 Revive（10）可
```

这样改有三个直接收益：① 同一帧至多切换一次，S1 的“一帧两跳”不可能发生；② Death 成为真正的吸收态，S3 / S4 被守卫拒绝；③ 拒绝的请求可以记日志，配合“最近 64 次切换”的环形缓冲画成屏幕 HUD，手感问题第一次有了可观测性。**代价**是切换晚了“半帧”：新状态的 Enter 在本帧末尾执行，新状态的第一次 Tick 在下一帧——对 60 fps 的动作游戏通常可接受，格斗类可以让提交后立即补一次 Tick。

**扩展：分层状态机（HFSM）与并行状态**

项目里 Grounded / Air 已经是“父状态”的雏形，但父类只用来复用代码，状态机本身是扁平的。真正的 HFSM 让父状态也是状态：Grounded 处理“离地 → Air”这类所有子状态共有的转移，子状态只关心自己；还可以拆成**并行的两层**——“移动层”（Idle / Move / Air / WallSlide）与“动作层”（None / Attack / Counter / Aim），这样“边跑边瞄准”“空中攻击”不需要再组合出新状态。Unity 的 Animator 本身就支持多层 + Avatar Mask，逻辑分层与动画分层一一对应最好维护。

## 3.10 面试追问

追问树

```
你的状态机怎么设计的？（Q3-1）
├─ 父类里切换了状态，子类还会继续执行吗？（Q3-2 → E1-S1）
├─ 怎么防止非法切换？死亡状态能被打断吗？（Q3-3 → E1-S3/S4）
│   └─ 同一帧有多个切换请求（受击 + 冲刺）听谁的？（Q3-4 优先级）
├─ 进入条件消失时怎么办？（Q3-5 → E1-S2 墙滑）
├─ 预输入 / 取消窗口怎么做？（Q3-6 → 3.8）
└─ 状态机和行为树怎么选？Animator 能不能直接当状态机？（Q3-7）
```

### Q3-1　你的状态机怎么设计的?

**L1**　每个状态一个类，Enter / Update / Exit；状态机只持有 currentState，ChangeState 先 Exit 旧的再 Enter 新的。

**L2**　状态在 Awake 里一次性创建，切换零分配；Enter / Exit 成对设置 Animator bool，Animator 用 Exit → Entry 的结构根据 bool 选片段；地面通用输入放在 Grounded 父类，按“瞄准 > 攻击 > 反击 > 跳 > 掉落”的顺序 return，相当于一个隐式的优先级。

**L3**　我后来发现最大的问题不在状态本身，而在**切换入口**：冲刺在 Player.Update 里全局检测，接剑和死亡由外部直接调用，三处都不检查当前状态。所以我会把切换统一成“请求 + 守卫 + 每帧至多提交一次”。

**L4 • 攻击**　“13 个状态而已，有必要搞这么复杂吗？” → 状态少的时候确实可以靠人脑检查，但 13 个状态两两组合就有 156 种转移，我用逐帧回放就找到了死亡后冲刺复活这种组合 bug；守卫的代码量很小，换来的是“新增状态时不用回头检查所有入口”。

### Q3-2　父类里切换了状态，子类还会继续执行吗?

**L2**　会。`base.Update()` 里的 return 只退出父类函数，子类的后续代码照常执行。所以 Idle / Move 在 base.Update 之后先判 `currentState ≠ this`。

**L3 • 项目**　Fall 没有继承 Grounded，但它自己的三个 if 之间没有 return：落地那一帧刚好贴墙，就会 Fall→Idle→WallSlide 连切两次，Idle.Enter 刚清零的速度还会被 Fall 的下一行设回 7。我用 C++ 移植逐帧回放确认过。

### Q3-3 → Q3-4　怎么防止非法切换？同一帧多个请求听谁的？

**L2**　给状态定义可打断等级，请求带等级，低于当前状态等级的请求直接拒绝；同一帧的多个请求只保留等级最高的。死亡等级最高，只接受复活。

**L3**　等级是“横切”的规则，不要写进每个状态的 if 里；标签（CanMove、CanDash、Dead）则给外部系统查询用——比如技能系统在释放前查 `CanCast`，AI 托管在死亡时停止输入。GAS 里的 Gameplay Tags + Blocked Tags 就是这个思路的通用化。

### Q3-5　进入条件消失时怎么办？

**L2**　每个状态的出口集合至少要覆盖“进入条件的否定”。WallSlide 的进入条件是“空中且贴墙”，我漏了“不贴墙 → Fall”，于是滑过平台侧面下沿时会在半空慢速下滑一秒。

**L3**　可以写成一条可检查的约定：状态声明 `bool StillValid()`，状态机每帧先检查，失效就回到父状态的默认子状态。这样任何人新增状态都不会忘。

### Q3-6　预输入和取消窗口怎么做？

**L2**　输入在帧首采样并进入带时间戳的缓冲；动作状态声明“可取消窗口”（例如命中帧之后到动画结束），窗口内消费缓冲里 0.1~0.2 s 以内的指令。没进窗口前的按键不丢，到窗口时自动生效。

**L3**　窗口最好由动画或技能时间轴标记（动画事件 / Notify State），而不是写死时间；我项目里的 `inertiaTime = 0.15f`、`BusyForSeconds(.1f)` 就是写死时间的反例，换了动画节奏就要回来改代码。

### Q3-7　状态机和行为树怎么选？能不能直接用 Animator 当状态机？

**L2**　玩家角色的状态少、切换受输入驱动、需要精确的帧级控制，FSM / HFSM 最合适；AI 的决策多、需要优先级和组合，行为树更合适（第 9 章）。Animator 可以用 StateMachineBehaviour 承载逻辑，但它的转移在动画更新阶段发生、受过渡时间和退出时间影响，逻辑时序不可控，还和美术资源绑死——所以通常“逻辑状态机驱动 Animator 参数”，而不是反过来。

**常见错误回答 ①**

**错误**：“状态之间切换由 Animator 的转移控制。”

**问题**：本项目恰恰相反——逻辑状态机在 Update 里切换，Animator 只是被动地读 bool；混淆两者会被追问“那动画事件触发时逻辑在哪个状态？”问倒。

**更好的回答**：“逻辑状态机是权威，Animator 跟随；关键时刻由动画事件回调到逻辑，所以两者在时间上需要对齐（第 4 章）。”

**常见错误回答 ②**

**错误**：“我的状态机没有问题，每个状态都测过。”

**问题**：单个状态都对，组合起来不对——S1（两状态在同一帧）和 S3（外部入口 × 死亡）都不是单状态测试能发现的。

**更好的回答**：“单状态行为我手测过；组合问题我用脚本化输入逐帧回放找过，发现了一帧双转移和死亡可打断，改成了统一入口加守卫。”

## 3.11 发散场景题

**场景 3-1　策划说“攻击后摇太长，想在后摇里按跳跃能直接跳”，你怎么做?**

为什么会问：取消窗口是动作手感的核心，考察你会不会写死时间。

① 在攻击动画上标一段“可取消窗口”（动画事件 Begin / End，或 Animation Curve），而不是写 `stateTimer < x`；② 窗口内 Attack 状态向输入缓冲查询 Jump / Dash / 下一段攻击，命中就 Request；③ 窗口外的按键留在缓冲里 0.15 s，进窗口自动生效；④ 给策划一个调试开关，在角色头顶显示当前是否处于取消窗口。注意不同指令可以有不同窗口（冲刺可以更早取消，跳跃晚一点）。

**场景 3-2　加入“受击硬直”和“霸体”，怎么和现有状态共存?**

为什么会问：现在受击只有闪白 + 击退协程，不改变状态。

新增 Hurt 状态（等级 5），受击时 Request(Hurt, 5)；霸体不是一个状态，而是一个**标签 / 修正器**（SuperArmor），存在时受击请求的等级降到 0（只扣血不打断）。攻击前摇可以在动画里临时

挂霸体。这样霸体、无敌、格挡都能用同一套“标签 + 等级”表达，不会出现 13 × N 的组合状态。

**场景 3-3　要加“爬梯子”，需要改哪些地方?**

为什么会问：考察扩展点是否集中。

现状要改：新建 LadderState；在 Grounded 和 Fall 里加进入条件（贴梯子 + 上下输入）；冲刺的全局检测要排除梯子状态；接剑强切要排除；WallSlide 的墙检测要排除梯子层。改进后：LadderState 声明自己的等级和标签（CanDash = false），进入条件注册到“移动层”的转移表里，其他状态一行不改。

**场景 3-4　玩家反馈“偶尔动作会抽一下”，你怎么查?**

为什么会问：考察调试方法论，而不是背知识点。

① 先加可观测性：状态机记录每次切换（帧号、旧 → 新、原因、请求来源），保存最近 64 条，屏幕 HUD 显示；② 复现时看是否出现“同一帧两次切换”或“极短的状态停留”（S1 就是停留 1 帧的 WallSlide）；③ 把输入录下来回放（这要求输入已经抽成快照），用二分法缩小到具体帧；④ 修完后把这段输入存成回归用例。

**场景 3-5　把玩家改成可由 AI 托管（掉线托管、自动战斗），状态机要怎么改?**

为什么会问：考察输入抽象。

状态机只消费 InputFrame，玩家输入采样器和 AI 决策器都输出 InputFrame（意图 + 轴向）；AI 不需要“模拟按键”，直接产出意图。这也是联网时把输入发给服务器 / 其他客户端的基础（第 14 章）。

**方法论延伸**

状态判定（着地 / 贴墙）应如何从状态里拆出来做成感知层，见第 17 章；滑墙状态的完整设计见第 18 章；预输入、土狼时间等输入宽容的量化与设计见第 20 章。

**本章速记**

- 结构：**状态类 + 模板方法 + Grounded/Air 父类**；Enter/Exit 成对设 Animator bool。
- 父类里的 return 只退出父类；子类要判 `currentState ≠ this`。Fall 没有守卫 → **落地贴墙一帧切两次**（E1-S1）。
- WallSlide 没有“墙没了”出口 → **半空慢滑 60 帧**（E1-S2）。
- 三个切换入口都不检查当前状态 → **死亡后冲刺 / 接剑复活**（E1-S3/S4）。
- 改进：**Request + 等级守卫 + 每帧至多一次 + 切换历史**；输入抽成快照 + 预输入缓冲。

▶ **自测 1：Grounded.Update 的 if 顺序隐含了什么规则?**

▶ **自测 2：为什么 S3 在当前场景里没人发现?**

▶ **自测 3：“请求-提交”模式的代价是什么?**

CHAPTER 04

# Animator 与动画事件：谁驱动谁、何时触发、何时失灵

项目里命中、攻击结束、掷剑、弹反开窗都由动画事件触发——逻辑的关键时刻交给了美术时间轴。这一章把 5 个 AnimatorController 和 23 个片段里的事件全部列出来，再讲动画事件在“速度为 0、被打断、同一时刻、字符串改名”四种情况下怎样失灵。

---

**本章目标**

- 说清项目 Animator 的“Exit → Entry + bool”结构，以及它和逻辑状态机的主从关系
- 掌握事件清单：每个事件在片段第几秒、回调到哪个组件、驱动哪段逻辑
- 能举出动画事件不可靠的四种情形，并在项目里各找到一个例子
- 比较“动画驱动逻辑”和“逻辑时间轴驱动动画”两种流派，知道帧同步游戏为什么选后者

## 4.1 30 秒面试版 • 现状 vs 理想

**30 秒回答**

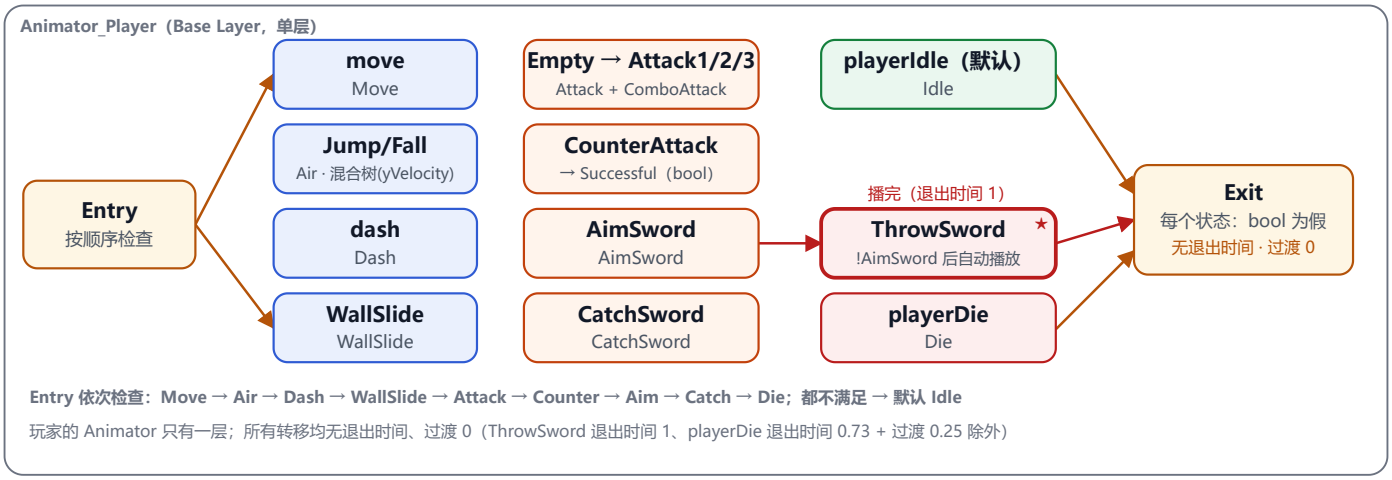
逻辑状态机是主，Animator 是从：状态 Enter / Exit 设同名 bool，Animator 每个状态都只有“bool 为假 → Exit”一条出边，回到 Entry 后按顺序检查各个 bool 进入新片段，过渡时间全是 0。关键帧用动画事件回调：攻击片段上的 AttackTrigger 做命中、AnimationTrigger 通知状态“动画结束”，骷髅攻击片段上还有弹反开窗 / 关窗。好处是手感和美术对齐；风险是逻辑依赖动画播放——速度为 0 就不触发、被打断就跳过、同一时刻的事件依赖排列顺序、方法名是字符串。项目里就有一个调用了不存在方法的 Invoke（眩晕闪烁从未生效），以及“瞄准被冲刺打断仍会掷剑”的推断问题。

现状 → 问题 → 理想 **Animator 与动画事件**

| 维度 | 当前做法（Demo 现状） | 存在的问题 | 理想重构方案 | 收益 / 代价 |
| --- | --- | --- | --- | --- |
| **主从关系** | 逻辑状态机设 bool，Animator 只读 bool；过渡时间全为 0 | ✓ 无问题 方向正确；唯一例外是掷剑靠 bool 变假触发（见“打断”行） | 保持；只把一次性事件（掷剑）改为显式命令 / Trigger<br>**保持 · 方向正确** | |
| **关键帧来源** | 命中、结束、掷剑、弹反开关窗都在动画事件里 | 局限 命中时机受播放速度、冻结（animator.speed = 0）与剔除影响；联网、回放无法复现 | 单机：动画事件可保留，但一个事件携带完整语义；联网 / 严格判定：逻辑时间轴，动画事件只做表现<br>**设计方案 · 未实现** | 不受帧率、播放速度、剔除影响 |
| **同时刻事件** | 弹反成功依赖 0.4167 s 处 AttackTrigger 排在 CloseCounterAttackWindow 之前 | 隐患 弹反成功依赖同一时刻两个事件的排序；美术把事件挪动一点就会悄悄失效 | 命中事件内部先判弹反再关窗，或改用区间窗口（Notify State）由逻辑查询<br>**设计方案 · 未实现** | 美术挪动事件不会悄悄破坏弹反 |
| **打断** | AimSword 变假即自动播掷剑并掷出（推断，图 4-2） | P2 瞄准被冲刺打断仍掷出一把方向陈旧的剑，瞄准点不收起（推断，#25） | 由退出原因决定是否掷剑；所有退出路径都收起瞄准点<br>**设计方案 · 未实现** | “选择冲刺”不会掷出一把方向错误的剑 |
| **方法引用** | 按字符串 InvokeRepeating 调用；退出时用不带参数的 `CancelInvoke()` | P1 眩晕闪红从未生效；`CancelInvoke()` 把点燃 / 冰冻的变色一起取消（#12） | 直接调用 / `nameof`；只取消自己的计时<br>**设计方案 · 未实现** | 改名编译期报错 |
| **保底** | 攻击只能靠动画结束事件退出 | 隐患 动画事件没触发（Animator 被禁用、片段被替换）时角色永远卡在攻击状态 | 逻辑时间保底超时（与动画时间同源，冻结时同步暂停）<br>**设计方案 · 未实现** | Animator 被禁用时角色不会卡死 |

现状问题：P1 ×1　P2 ×1　隐患 ×2　局限 ×1　✓ 无问题 ×1　（P0~P3 编号见附录 A，分级说明见 0.2）

## 4.2 Animator 结构：Exit → Entry + bool




**现状** 图 4-1　Animator_Player 的结构（由 tools/scan_anim.py 解析 .controller 得到）。逻辑负责设 bool，Animator 只做“读 bool 选片段”；唯一的例外是红框：AimSword 一旦变假就自动播放 ThrowSword，不管逻辑上是“松开右键”还是“被打断”

**现状问题** P2 AimSword 变假就自动掷剑：被打断也会掷出（推断，#25）

| 做法 | 本项目 | 优点 | 代价 |
| --- | --- | --- | --- |
| Exit → Entry + 每状态一个 bool | ✓ | Animator 图极简，逻辑完全掌控；新增状态只加一个 bool 和两条线 | 多个 bool 同时为真时，Entry 的检查顺序成了隐式优先级（E1-S1 那一帧就同时为真过）；每次切换都经过 Exit / Entry |
| Any State + Trigger | | “从任何状态进入”表达直接 | Trigger 会被消费 / 残留，容易出现“下一次才生效”；Any State 线多了难维护 |
| 代码直接 `animator.Play / CrossFade` | | 完全由代码控制，适合动作游戏 | Animator 图里看不到转移关系，美术调过渡要找程序 |

## 4.3 事件清单：逻辑的关键时刻都在这里

| 片段（长度 / 循环） | 事件 @ 时刻 | 接收组件 → 逻辑 |
| --- | --- | --- |
| playerAttack1（0.714 s） | AttackTrigger @0.357 · AnimationTrigger @0.714 | PlayerAnimationTriggers → OverlapCircleAll 命中；→ `triggerCalled` 结束攻击状态 |
| playerAttack2（0.333 s） | AttackTrigger @0.083 · AnimationTrigger @0.333 | |
| playerAttack3（0.700 s） | AttackTrigger @0.200 · AnimationTrigger @0.700 | |
| playerThrowSword（0.286 s） | ThrowSwordTrigger @0.143 | → `SkillManager.Instance.throwSword.SpawnSword()` |
| PlayerCatchSword（0.286 s） | AnimationTrigger @0.214 | → 结束接剑状态 |
| playeSuccessfulCounterAttack（0.3 s，**循环**） | AnimationTrigger @0.2 | → 结束反击状态（弹反成功时 stateTimer 被设成 10，只能靠这个事件退出） |
| Skeleton_Attack（1.333 s，**循环**） | OpenCounterAttackWindow @0.083 · **AttackTrigger @0.417** • **CloseCounterAttackWindow @0.417** · AnimationTrigger @1.25 | Enemy_SkeletonAnimationTrigger → 开窗 / 命中玩家 / 关窗 / 结束攻击 |
| 分身（复用 playerAttack1~3） | 同上 | SkillController_Clone 的同名方法（第 8 章：这里的 AttackTrigger 会空引用） |

动画事件按**方法名**派发给 Animator 所在 GameObject 上的所有组件（类似 SendMessage），所以接收方法可以是 private。这也是为什么玩家需要一个专门的 `PlayerAnimationTriggers` 挂在子物体 Animator 上，再 `GetComponentInParent<Player>()` 转发——Animator 和 Player 不在同一个 GameObject 上。

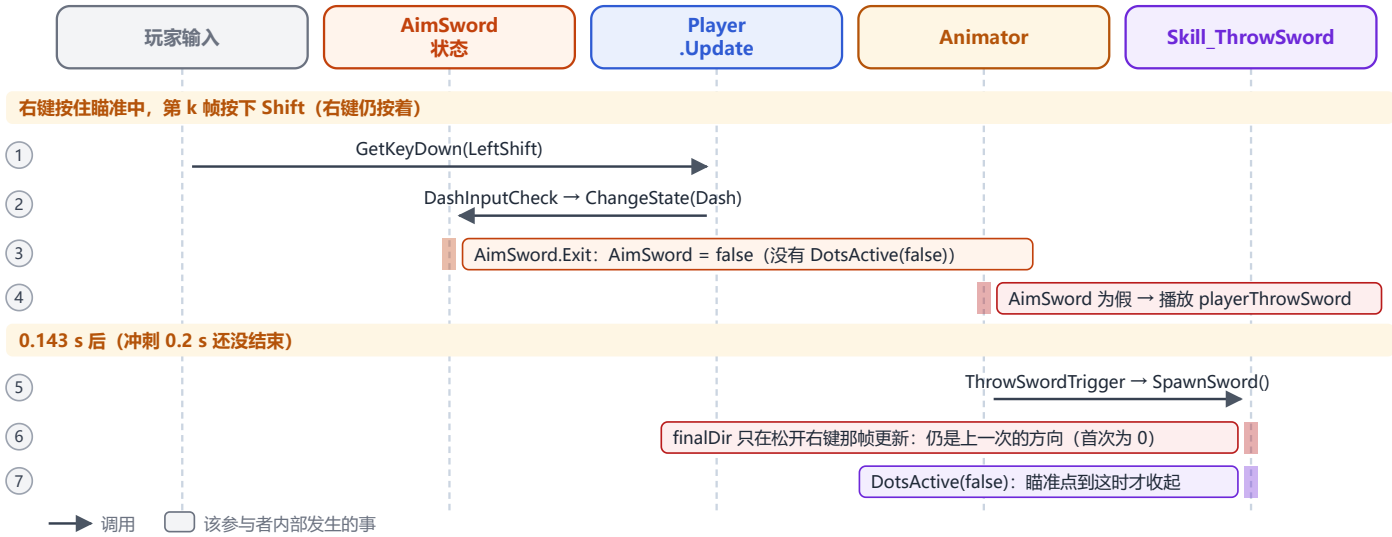
## 4.4 动画事件失灵的四种情形

### ① 播放速度为 0：事件不会来

骷髅被飞剑冻结时 `animator.speed = 0`（Enemy.cs:58-71）；死亡状态也 `animator.speed = 0`。如果冻结发生在攻击前摇，AttackTrigger 和 AnimationTrigger 都会推迟到解冻之后——攻击状态会一直停着，这正是“冻结”想要的效果。但反过来，**任何依赖“动画会播完”的逻辑都会被冻结卡住**：如果某天给攻击状态加一个“超时保底”，它必须用不受 animator.speed 影响的时间，否则冻结时会误触发。

### ② 被打断：事件被跳过，或者不该触发的事件被触发

攻击在 0.357 s 前被冲刺取消，命中帧就不会来——这是正确的。真正的问题在图 4-1 的红框：`AimSword → ThrowSword` 的条件是“AimSword 为假”，**它不知道 AimSword 为什么变假。**




现状 图 4-2 瞄准中被冲刺打断：Animator 仍会播放掷剑片段并触发掷剑事件【推断，由 .controller 与源码推出，附录 B 给验证步骤】。逻辑上玩家选择了冲刺，表现与结果却是“掷出了一把方向错误的剑”

**现状问题** P2 玩家选择了冲刺，却掷出一把方向陈旧的剑（推断，#25）

修法有两层：表现层把“掷剑”从“AimSword 变假”改成一个显式参数（逻辑在松开右键时设 `Throw` trigger）；逻辑层让 AimSword.Exit 根据退出原因决定是否掷剑，并在任何退出路径上收起瞄准点。原则是 **Animator 里不要藏逻辑决策。**

### ③ 同一时刻的多个事件：依赖排列顺序

骷髅攻击片段在 0.4167 s 同时有 AttackTrigger 和 CloseCounterAttackWindow。弹反判定发生在 AttackTrigger 里（玩家受击 → 检查 `canBeStunned`），**只有当 AttackTrigger 排在 Close 之前时弹反才可能成功**。当前 .anim 文件里的顺序恰好是 AttackTrigger 在前——这是一个没人写在任何地方的契约：美术在 Animation 窗口里挪一下事件，或者有人把 Close 提前 1 帧“修正判定”，弹反就再也不会成功，而且不报任何错。第 6 章会把这个窗口完整量化。

### ④ 方法名是字符串：改名即失效

**当前源码 • 822270c** `Enemy_SkeletonStunState.cs:13-25`

```
public override void Enter()
{
    base.Enter();
    stateTimer = enemy.stunDuration;
    enemy.fx.InvokeRepeating("RedColorBlink", 0, 0.1f);    // EntityFX 里根本没有这个方法
    rb.velocity = new Vector2(-enemy.facinDir * enemy.stunVector.x, enemy.stunVector.y);
}
public override void Exit()
{
    base.Exit();
    enemy.fx.Invoke("CancelColorChange", 0);                // CancelInvoke() 会取消 fx 上所有
}
```

**现状问题** `P1` 按名字调用不存在的 RedColorBlink，眩晕闪红从未生效；CancelInvoke() 把异常状态的变色一起取消（#12）

**问题** `P1` 教程原版的 EntityFX 有 `RedColorBlink`，这个项目重写 EntityFX 时删掉了它，但眩晕状态还在按名字调用：编译通过，运行时 Unity 打印“Trying to Invoke method: EntityFX.RedColorBlink couldn't be called.”，眩晕时没有红色闪烁。更隐蔽的是 Exit：`CancelColorChange` 里的 `CancelInvoke()` 不带参数，会把点燃 / 冰冻 / 感电的变色 InvokeRepeating 一起取消，并把颜色强制设回白色——骷髅在燃烧时被弹反眩晕，眩晕结束后燃烧的颜色效果就没了（伤害计时在 CharacterStats 里，不受影响）。

动画事件、`Invoke`、`InvokeRepeating`、`StartCoroutine(string)`、`SendMessage` 都是同一类“按名字反射调用”，项目里共有 5 + 10 + 4 处（不含动画事件）。替代：`nameof(Method)`（改名时编译器能跟着改）、直接 `StartCoroutine(Method())`、委托 / 计时器系统。

## 4.5 两种流派：动画驱动逻辑 vs 逻辑时间轴驱动动画

| | 动画驱动（本项目、UE AnimNotify） | 逻辑时间轴驱动（技能编辑器、HOK 式帧同步） |
|---|---|---|
| 关键帧在哪 | 动画片段的事件 / Notify | 技能配置的时间轴（第 N 逻辑帧做什么） |
| 谁是权威 | 动画播放进度 | 逻辑帧计数；动画只是表现 |
| 优点 | 美术调节奏立刻生效；手感与画面天然对齐 | 与帧率、动画速度、表现剔除无关；可确定性回放；服务器可以不跑动画 |
| 缺点 | 受播放速度 / 打断 / 剔除 / 帧率影响；服务器要跑动画才能判定 | 需要工具链（编辑器、预览）；动画改了要同步改时间轴 |
| 适合 | 单机动作、表现驱动的玩法 | 联网（尤其帧同步）、需要严格判定的竞技游戏 |

**为什么帧同步游戏不能让动画驱动逻辑**

帧同步要求每个客户端用相同输入算出完全相同的结果。动画事件的触发帧取决于渲染帧率、Animator 的更新方式、是否被剔除——各端都不一样。所以逻辑层用定点数的“第几逻辑帧”决定命中，表现层的动画只负责“看起来在那一刻挥刀”。即便是状态同步，服务器上通常也不跑完整动画（UE 的专用服务器会降低骨骼更新），命中判定同样依赖逻辑时间而不是动画回调。

## 4.6 面试追问

追问树

```
攻击命中是怎么判定的？（Q4-1）
├─ 动画事件什么时候触发？帧率低的时候会怎样？（Q4-2）
├─ 动画被打断 / 速度为 0 时逻辑怎么办？（Q4-3 → 4.4 ①②）
├─ 同一时刻两个事件的顺序可靠吗？（Q4-4 → 4.4 ③、第 6 章）
├─ 为什么 Animator 用 bool 不用 Trigger？（Q4-5）
└─ 如果要联网 / 帧同步，命中还能放在动画事件里吗？（Q4-6 → 4.5）
```

## Q4-1 → Q4-2　命中怎么判定？动画事件什么时候触发？

**L2**　攻击片段上在挥刀到位的那一帧放 AttackTrigger 事件，回调里以攻击点为圆心做 OverlapCircleAll，对检测到的敌人播受击表现并结算伤害。事件在 Animator 更新阶段触发——Update 之后、LateUpdate 之前——在“片段时间越过事件时刻”的那一帧触发。

**L3**　所以命中时刻会被量化到帧：30 fps 下最多晚 33 ms；一帧卡顿 200 ms 时，这一帧里越过的所有事件会按顺序一起触发。对单机动作游戏可以接受；对格斗或联网游戏，命中应该在固定逻辑帧上判定。

## Q4-3　动画被打断或速度为 0 时，逻辑怎么办？

**L2**　被打断时事件被跳过，所以“结束攻击状态”不能只靠动画结束事件——项目里攻击状态的出口只有 `triggerCalled`，如果攻击动画因为某种原因没播完（比如 Animator 被禁用），角色会永远卡在攻击状态。我会加一个按逻辑时间的保底超时。

**L3**　速度为 0（冻结 / 顿帧）时，保底超时也必须暂停，否则会提前结束；所以保底计时最好跟“动画时间”走同一个时间源，或者显式区分“逻辑暂停”和“逻辑继续”。

## Q4-4　同一时刻的两个事件，顺序可靠吗？

**L2**　我的弹反依赖骷髅攻击片段里 AttackTrigger 排在 CloseCounterAttackWindow 前面——它们在同一时刻。这类隐式契约很脆弱，美术挪一下事件就失效且不报错。

**L3**　更好的做法是让一个事件携带完整语义：AttackTrigger 自己先判弹反、再结算伤害、最后关窗；或者把窗口做成“区间”（Notify State），由逻辑查询“此刻是否在窗口内”，而不是靠两个点事件夹出区间。

### Q4-5　为什么用 bool 不用 Trigger?

**L2**　bool 表达“正处于某状态”，和逻辑状态一一对应，Enter 置真 Exit 置假，不会残留；Trigger 表达“发生了一次”，如果当帧没被转移消费会一直挂着，下次条件满足时意外触发。代价是需要成对维护，而且多个 bool 同时为真时依赖 Entry 的检查顺序。

**L3 • 项目**　唯一该用 Trigger 的地方恰恰用了 bool 的“副作用”：掷剑应该是一次性事件，现在却挂在“AimSword 变假”上，导致被打断也会掷剑。

### Q4-6　联网以后命中还能放在动画事件里吗?

**L2**　帧同步不行，事件触发帧各端不同；状态同步时服务器权威判定，也不应依赖客户端动画。做法是把命中时刻写进技能数据（第几逻辑帧、判定形状、持续几帧），服务器和客户端都按逻辑帧判定，动画只做表现；动画事件保留给纯表现（音效、特效、震屏）。

**常见错误回答**

**错误**：“动画事件是精确的，在指定时间点准时触发。”

**问题**：它只在“越过该时刻的那一帧”触发，受帧率、播放速度、打断、剔除（Culling Mode）影响。

**更好的回答**：“动画事件按帧量化，适合表现和单机判定；需要精确或可复现的判定时，我用逻辑时间轴。”顺带可以说：项目里所有 Animator 的 Culling Mode 都是 Always Animate，所以屏幕外的骷髅仍会正常出手——如果为了性能改成 Cull Completely，屏幕外怪物的攻击事件就不会触发了。

## 4.7 发散场景题

### 场景 4-1　QA 反馈“攻击有时打不到贴脸的敌人”，你怎么排查?

为什么会问：考察“逻辑 / 表现 / 物理”三条时间线的排查能力。

① 画出攻击点：Entity 里已经有 `OnDrawGizmos` 画 attackCheck 圆；② 确认事件是否触发（回调里打帧号）、是否被冲刺 / 受击打断；③ 确认 OverlapCircleAll 的结果：没有 LayerMask，敌人的碰撞体是否在圆内（attackCheck 在本地 x = 1.2、半径 0.9，只覆盖身前 0.3~2.1 m——**敌人被**

**挤进身体 0.3 m 以内或绕到身后就打不到**）；④ 确认敌人碰撞体没被关（死亡状态会关）；⑤ 若敌人在移动，物理位置与渲染插值位置差了一帧，看起来贴脸其实还没到。

### 场景 4-2　美术把攻击动画从 0.7 s 改成 0.5 s，逻辑要改哪些地方?

为什么会问：考察“数据写死在代码里”的识别。

事件随片段走，命中与结束自动跟着变；但写死在代码里的时间不会变：`inertiaTime = 0.15f`（攻击位移持续时间）、`comboWindow = 1.5f`、`BusyForSeconds(.1f)`、反击持续`counterAttackDuration`。这些应该和片段一起配置——要么也做成事件（“位移结束”事件），要么放进技能数据。

### 场景 4-3　做受击顿帧（Hit Stop），用 Time.timeScale = 0 可以吗?

为什么会问：顿帧是打击感的标配，实现方式会影响全局时间。

全局 timeScale 会让所有东西停下（包括 UI 动画、协程里的 WaitForSeconds），而且项目的 Animator 是 Normal 更新模式，会一起停——这对“整屏顿帧”可以接受。更常见的是**局部顿帧**：只把攻击者和受击者的 animator.speed 降到 0.05 持续 60~100 ms。但项目里已经有冻结、减速、死亡三个系统在写 animator.speed（第 7 章），再加顿帧就是第四个写者——必须先引入“速度修正器栈”，否则顿帧结束时会把冻结也一起恢复。

### 场景 4-4　设计一个技能编辑器的时间轴，你会放哪些轨道?

为什么会问：从“动画事件”自然发散到工具链。

轨道：动画（片段 + 起止 + 速度曲线）、判定（形状 / 偏移 / 持续帧 / 命中次数上限 / 去重组）、位移（曲线或根运动开关）、状态窗口（可取消、霸体、无敌）、表现（特效 / 音效 / 震屏，可以只在客户端执行）、条件分支（命中后跳转）。运行时由逻辑帧驱动，编辑器里能逐帧预览。项目里散落在代码和片段中的`inertiaTime`、命中事件、弹反窗口，都应该成为某条轨道上的一个区间。

**本章速记**

- 结构：**逻辑设 bool → Animator 读 bool**；所有状态 “bool 假 → Exit”，Entry 顺序即隐式优先级。

- 失灵四情形：**速度 0 不来、被打断跳过、同时刻靠顺序、字符串改名失效。**
- 项目实例：AimSword 变假即掷剑（推断）、弹反依赖同时刻事件顺序、`Invoke("RedColorBlink")` 方法不存在（#12）、`CancelInvoke()` 误伤异常状态变色。
- 联网 / 帧同步：**命中放逻辑时间轴，动画事件只做表现。**

▶ **自测 1：为什么玩家需要单独的 PlayerAnimationTriggers 组件?**

▶ **自测 2：弹反成功后为什么必须靠 AnimationTrigger 才能退出反击状态?**

▶ **自测 3：Jump/Fall 为什么是一个状态?**

CHAPTER 05

# 时间与帧率：Update、FixedUpdate 与“帧率无关”

“为什么要乘 deltaTime” “Update 和 FixedUpdate 有什么区别”是 Unity 面试的开胃菜。这个项目给了三个会被追问到底的实例：冲刺距离随帧率跳动、点燃和旋转剑的伤害次数随帧率变化、瞄准虚线和剑的真实轨迹对不上。

---

**本章目标**

- 画出“渲染帧 + 固定物理步”两条时间线，说清 Update 里设的速度何时生效
- 区分三类时间代码：一次性设速度（与帧率无关）、Update 计时决定持续时长（会量化）、计时器重置丢余量（会漂移）
- 用 E10 的数字说明冲刺 / 点燃 / 旋转剑在 30~240 fps 下的差异，以及一帧卡顿的后果
- 推导半隐式欧拉的离散轨迹，算出瞄准虚线与剑的偏差，并给出“完全对得上”的写法

## 5.1 30 秒面试版 • 现状 vs 理想

**30 秒回答**

Update 每渲染帧一次、间隔不定；FixedUpdate 按固定 0.02 s 一步、一帧里可能 0 次或多次，物理在它之后积分。项目的移动是“在 Update 里设 rb.velocity、由物理步积分”，持续移动没问题；但**持续时长在 Update 里用 deltaTime 计时**的动作（冲刺 0.2 s）会被量化：120 / 144 fps 下冲刺距离在 4.5~5.0 m 间跳，冲刺中卡一帧 250 ms 能冲到 7.0 m。计时器用“到点就重置为冷却值”而不是“累加”，点燃在 30 fps 下 2 秒只跳 6 次（60 fps 是 7 次），冷却为 0 的旋转剑命中次数直接等于帧数。瞄准虚线用连续抛物线，物理是离散积分，同一 x 处剑比虚线低 ½·g·s·Δt·t，1 秒时约 0.44 m。

现状 → 问题 → 理想 **时间与帧率**

| 维度 | 当前做法（Demo 现状） | 存在的问题 | 理想重构方案 | 收益 / 代价 |
|---|---|---|---|---|
| **持续时长** | 冲刺在 Update 里用 deltaTime 计时 0.2 s，位移由物理积分 | P2 120 / 144 fps 时冲刺有时只有 4.5 m（设计 5 m，E10，#20） | 按物理步计数（0.2 s = 10 步）或沿预设曲线 MovePosition<br>设计方案 · 未实现 | 任何帧率都是 5.00 m |
| **卡顿** | 没有单帧 dt 保护，冲刺照常按 Update 计时 | P2 卡一帧 250 ms 冲出 7.0 m，平台边缘会冲下悬崖（E10） | 固定步计数天然免疫；或给逻辑 dt 设上限<br>设计方案 · 未实现 | 平台边缘不会因卡顿冲下悬崖 |
| **周期计时** | 到点重置（点燃 0.3 s、旋转剑 hitCoolDown） | P2 30 fps 下点燃少跳；hitCoolDown = 0 时旋转剑命中数 = 帧数（E10，#21） | 累加计时 + while 处理一帧多次；伤害类放固定逻辑帧统一 tick<br>设计方案 · 未实现 | 30 fps 下点燃不再少跳；冷却 0 不再等于帧数 |
| **轨迹预测** | 瞄准虚线用连续抛物线，比剑高 ½·g·s·Δt·t | P2 剑总比瞄准虚线低：t = 1.68 s 时差 -0.74 m（E9，#22） | 离散闭式解按物理步画点；有碰撞时用 PhysicsScene2D 预演<br>模型验证：E9 离散闭式解与逐步积分最大误差 5.33e-14（三种角度） | 虚线与实际轨迹一致 |
| **投射物物理** | 飞剑 Discrete 碰撞、无插值；旋转停住后直接改 transform | P2 高速时可能穿过薄墙（Discrete）；高刷新率屏幕上飞剑抖动（无插值，#22） | Continuous + Interpolate；移动用 `rb.MovePosition`<br>设计方案 · 未实现 | 高速不穿薄墙、高刷屏不抖 |
| **输入读取** | 在 Update 里读按键 | ✓ 无问题 读取位置正确；问题在于读到当帧就消费、不缓冲（20.1） | 保持“Update 读输入”，但存成意图交给固定步消费<br>保持 · 读取位置正确 | |

现状问题：P2 ×5　✓ 无问题 ×1　（P0~P3 编号见附录 A，分级说明见 0.2）

## 5.2 两条时间线

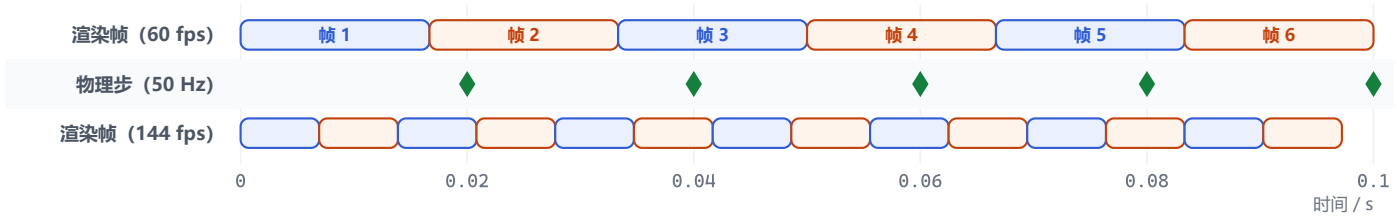



图 5-1　两条时间线：50 Hz 物理配 60 fps 渲染，平均每 6 帧里 5 帧各跑 1 个物理步、1 帧不跑；144 fps 时大多数帧不跑物理步。规则 R1：每帧先跑完累积的物理步，再跑 Update——**Update 里设的速度，最早在下一个物理步才产生位移**

| 类别 | 项目中的例子 | 与帧率的关系 |
| --- | --- | --- |
| ① 一次性设速度，交给物理 | 跳跃 `SetVelocity(vx, 20)`、墙跳、弹反击退、接剑反冲 | **无关：物理按固定步积分** |
| ② 每帧设同一个速度，持续移动 | Move、WallSlide、骷髅追击 | **基本无关（速度在帧间不变，物理步数 ∝ 时间）** |
| ③ **Update 计时决定持续时长** | 冲刺 0.2 s、攻击位移 0.15 s、弹反 1 s、骷髅待机 1 s | **量化：时长被向上取整到帧，物理步数随之跳动** |
| ④ **到点重置的周期计时器** | 点燃 0.3 s 一跳、旋转剑 hitCoolDown、Skill 冷却 | **漂移：每次丢掉超出的那一点，周期变长** |
| ⑤ 每帧按 dt 缩放的连续量 | 分身淡出 `alpha -= dt·speed`、剑回收 `MoveTowards(…, speed·dt)` | **无关（误差在最后一帧）** |
| ⑥ 协程 / Invoke 的等待 | `WaitForSeconds(0.2)` 闪白、`Invoke("DestroyMe", 7)` | **量化到帧（最多晚一帧）** |

## 5.3 E10：冲刺距离随帧率与卡顿变化

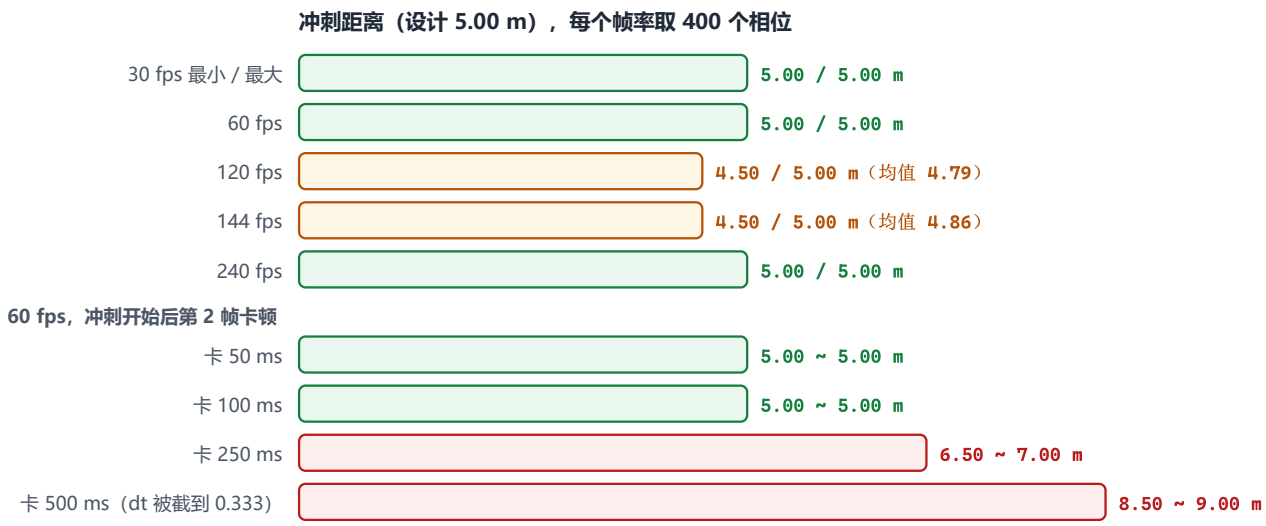
当前源码 • 822270c PlayerDashState.cs:16-45（只看与时间有关的部分）

```
public override void Enter()  { base.Enter(); stateTimer = dashDuration; /* 0.2 */  ... }
public override void Update()
{
    base.Update();                                      // stateTimer -= Time.deltaTime
    ...
    player.SetVelocity(player.dashDir * player.dashSpeed, 0);    // 25 m/s
    if (stateTimer < 0f) stateMachine.ChangeState(player.idleState);    // Exit 里把 vx 设回 0
}
```

现状问题 P2 时长按 Update 计时、位移按物理步积分：120 fps 时距离最短 4.50 m，卡顿 250 ms 冲出 7.0 m（E10，#20）

冲刺距离 = 25 m/s × 0.02 s × “看到 vx = 25 的物理步数”。物理步数取决于冲刺持续了几帧、每帧落了几个物理步。用 PlayerLoop 模型（R1、R4，物理步相位在 [0, 0.02) 内取 400 个值）扫一遍

实测 E10 模型：




现状 图 5-2　冲刺距离（前 5 行柱长取最小值，卡顿行取最大值）。120 / 144 fps 时有一部分相位只跑了 9 个物理步（4.5 m）；卡顿超过 0.2 s 时，冲刺在卡顿那一帧就结束，但这一帧里物理已经按 25 m/s 跑完了全部积压的步数

现状问题 P2 冲刺距离随帧率与卡顿变化（E10，#20）

两个现象各有原因：

- **120 / 144 fps 偶尔短 0.5 m**：冲刺在 Update 里共持续 24 / 29 帧，扣掉进入那一帧，速度只在 0.2 s 的“大约”时间里生效；0.2 / (1/120) 恰好是 24，float 累减 24 次后的结果落在 0 的哪一侧，决定

了冲刺多一帧还是少一帧——这是 R4（float 计时）的典型边界效应。60 fps 恰好落在另一侧，所以稳定 5.00 m。

- **卡顿冲得更远**：卡顿帧的 deltaTime 很大，状态在这一帧的 Update 里结束，但物理在这一帧的开头已经以 25 m/s 补跑了整段积压时间。卡 250 ms 时多冲 30%~40%，在平台边缘就是“冲刺掉下悬崖”。Maximum Allowed Timestep = 0.333 把 500 ms 卡顿也截成了 0.333 s，否则会更远。

**改法：让“时长”和“位移”用同一个时钟**

① 冲刺在 FixedUpdate 里按**物理步计数**：Enter 记 `remainSteps = round(0.2 / fixedDeltaTime) = 10`，FixedUpdate 里每步减 1，减到 0 结束——任何帧率、任何卡顿都是 10 步 5.00 m；② 或者干脆不用速度，用 `MovePosition` 沿预先算好的曲线走完 5 m，碰墙提前结束；③ 至少在逻辑上对单帧 dt 做上限（例如 1/20 s），避免卡顿放大位移。

## 5.4 E10：周期计时器“到点重置”会漂移

**当前源码 • 822270c** CharacterStats.cs:71-85 与 SkillController_ThrowSword.cs:144-157

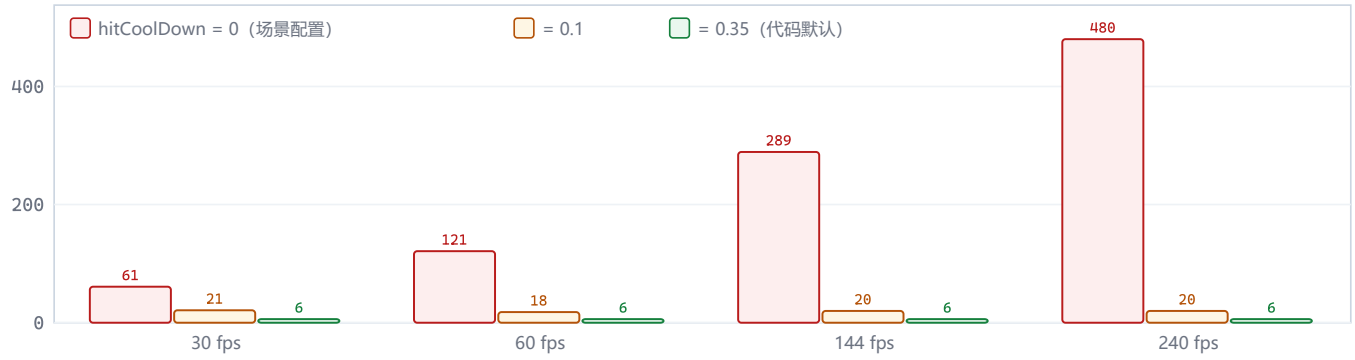
```
igniteDamageTimer -= dt;
if (igniteDamageTimer < 0)
{
    igniteDamageTimer = igniteDamageCooldown;    // 0.3：直接覆盖，丢掉了超出的那部分
    DecreaseHealthBy(igniteDamage);
}
// 旋转剑：同样的写法，hitCoolDown 在场景里配的是 0 → 每帧都命中
hitTimer -= Time.deltaTime;
if (hitTimer ≤ 0) { hitTimer = hitCoolDown; /* OverlapCircleAll + 伤害 + 冻结 */ }
```

**现状问题** P2 到点重置丢掉了超出的时间，实际周期被向上取整到帧；hitCoolDown = 0 时每帧命中（E10，#21）

到点重置时，实际周期是“冷却时间向上取整到帧”，近似为 $T_{实际} = \lceil T/\Delta t \rceil \cdot \Delta t$（float 累减让“恰好整除”的情况落在哪一侧不确定）。点燃持续 2 s、冷却 0.3 s：

| 帧率 | 2 s 内伤害次数 | 首跳时刻 / s | 平均间隔 / s |
|---|---|---|---|
| 30 | 6 | 0.033 | 0.3333 |
| 60 | 7 | 0.017 | 0.3000 |
| 144 | 7 | 0.007 | 0.3056 |
| 240 | 7 | 0.004 | 0.3042 |

表 5-1　点燃跳数（E10）。首跳总在第一帧：igniteDamageTimer 初值为 0，被点燃后的第一次 Update 就扣到负数。30 fps 下间隔被拉长到 0.333 s，少跳一次（伤害 −14%）




现状　图 5-3　旋转剑 2 秒内的命中次数（E10）。场景里 hitCoolDown 配成 0，命中次数 = 帧数：240 fps 的玩家伤害是 30 fps 的 8 倍；冷却 0.1 时 30 fps 反而比 60 fps 多打 3 次——又是 float 边界

现状问题 P2 hitCoolDown = 0 时命中数 = 帧数：高帧率玩家伤害成倍增加（E10，#21）

场景里的飞剑类型是 Regular，旋转模式目前不会触发；但 hitCoolDown = 0 是一个“换个枚举值就爆炸”的配置。改法与冲刺相同：**累加而不是覆盖**（`timer += cooldown`，一帧内用 while 处理多次），或在固定步里计数；伤害类周期效果最好放进固定逻辑帧（Buff 系统统一 tick，第 7 章）。

## 5.5 E9：瞄准虚线为什么总比剑高一点

当前源码 • 822270c Skill_ThrowSword.cs:145-151（虚线）与 SetupSword（实际发射）

```
// 虚线：连续时间抛物线 p(t) = p0 + v0·t + ½·g·s·t²
Vector2 position = (Vector2)player.transform.position + new Vector2(
        AimaDirection().x * launchForce.x * t,
        AimaDirection().y * launchForce.y * t) + 0.5f * (Physics2D.gravity * swordGravityScal
// 实际：rb.velocity = finalDir（= 方向 ⊙ launchForce），rb.gravityScale = swordGravityScale，
```

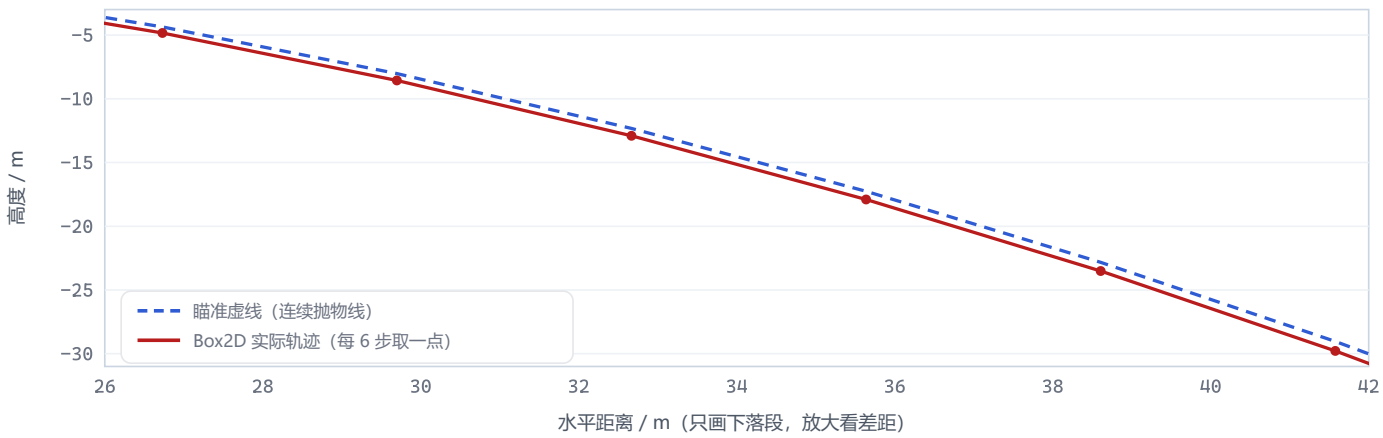
现状问题 P2 虚线用连续抛物线，实际是半隐式欧拉积分：剑总比虚线低 ½·g·s·Δt·t（E9，#22）

Box2D 用**半隐式欧拉**：先用重力更新速度，再用新速度更新位置。把它展开成闭式解：

**推导 • 半隐式欧拉的离散轨迹（竖直方向，线阻尼为 0）**

| | | |
|---|---|---|
| ① | $v_n = v_{n-1} + g s\,\Delta t = v_0 + n\,g s\,\Delta t$ | 每步先更新速度（g = −9.81，s = gravityScale） |
| ② | $y_n = y_{n-1} + v_n\,\Delta t = \sum_{k=1}^{n} v_k\,\Delta t$ | 再用“新”速度更新位置 |
| ③ | $= v_0\,n\Delta t + g s\,\Delta t^2\,\frac{n(n+1)}{2}$ | 等差数列求和 |
| ④ | $= v_0 t + \frac{1}{2} g s\,t^2 + \frac{1}{2} g s\,\Delta t\,t$ | 代入 t = nΔt |
| ⑤ | $y_n - y(t) = \frac{1}{2}\,g s\,\Delta t\,t$ | 与连续抛物线的差：随 t 线性增长，g < 0 → 剑更低 |

水平方向没有加速度，两者的 x 完全相同，所以“同一时刻的差”就是“同一 x 处的差”。代入场景值（gravityScale 4.5、Δt 0.02、launchForce (35, 28)）实测 E9：




现状 图 5-4　45° 掷剑：虚线与实际轨迹（E9）。最后一个瞄准点在 t = 24 × 0.07 = 1.68 s，此时剑比虚线低 0.742 m，与公式 ½·g·s·Δt·t 的 0.742 m 一致

**现状问题** P2 瞄准虚线与实际轨迹不一致，玩家照着虚线瞄会打低（E9，#22）

| 角度 / ° | t / s | x / m | 虚线 y | 实际 y | 偏差 / m | 公式 ½gsΔt•t |
|---|---|---|---|---|---|---|
| 20 | 0.50 | 16.45 | -0.73 | -0.95 | -0.221 | -0.221 |
| 20 | 1.00 | 32.89 | -12.50 | -12.94 | -0.441 | -0.442 |
| 20 | 1.50 | 49.33 | -35.30 | -35.96 | -0.662 | -0.662 |
| 20 | 1.68 | 55.25 | -46.21 | -46.95 | -0.742 | -0.742 |
| 45 | 0.50 | 12.37 | 4.38 | 4.16 | -0.221 | -0.221 |
| 45 | 1.00 | 24.75 | -2.27 | -2.71 | -0.441 | -0.442 |
| 45 | 1.50 | 37.12 | -19.96 | -20.63 | -0.662 | -0.662 |
| 45 | 1.68 | 41.58 | -29.04 | -29.78 | -0.742 | -0.742 |
| 70 | 0.50 | 5.99 | 7.64 | 7.42 | -0.221 | -0.221 |
| 70 | 1.00 | 11.97 | 4.24 | 3.80 | -0.441 | -0.442 |
| 70 | 1.50 | 17.96 | -10.20 | -10.86 | -0.662 | -0.662 |
| 70 | 1.68 | 20.11 | -18.09 | -18.84 | -0.742 | -0.742 |

表 5-2　三个角度的偏差完全相同——它只取决于时间，不取决于角度；离散闭式解与逐步积分的最大误差约 4e-14（只剩浮点舍入）

0.44 m（1 秒处）在 2D 横版里大约是角色身高的五分之一，玩家会觉得“剑总是比瞄准的低一点”。修法：虚线直接用离散闭式解 $p_n = p_0 + v_0 n\Delta t + gs\Delta t^2 \frac{n(n+1)}{2}$，取 $n$ 为物理步数，就和 Box2D 逐步积分逐位一致（只要没有阻尼和碰撞）；需要考虑碰撞时，用 `PhysicsScene2D` 在一个隐藏的物理场景里 `Simulate` 预演。

**另外三个与时间有关的细节**

- **插值**：玩家和骷髅的刚体开了 Interpolate，渲染位置在物理步之间平滑；飞剑没开，在 144 fps 屏幕上它每 0.02 s 才跳一次位置，快速飞行时会有“抖动感” `推断`。
- **直接改 transform 的刚体**：旋转剑停住后用 `transform.position = MoveTowards(...)` 移动一个带 Rigidbody2D 的物体（项目关闭了 Auto Sync Transforms），物理世界里的位置要到下一次模拟前同步——中间的 Overlap 查询会用旧位置。应改用 `rb.MovePosition`。
- **隧穿**：飞剑刚体是 Discrete 碰撞检测，速度约 35 m/s，每步移动 0.7 m，而剑的触发器半径只有 0.35 m——薄墙和瘦小的敌人可能被直接穿过。玩家和骷髅用的是 Continuous，飞剑也应该改。

## 5.6 面试追问

追问树

```
Update 和 FixedUpdate 的区别？（Q5-1）
├─ 为什么移动要乘 deltaTime？设 velocity 也要乘吗？（Q5-2）
├─ 冲刺 / 攻击位移在高帧率下距离会变吗？卡顿时呢？（Q5-3 → E10）
├─ 计时器怎么写才帧率无关？（Q5-4）
├─ 固定步长游戏循环怎么写？为什么要插值？（Q5-5）
└─ 轨迹预测怎么和物理对上？（Q5-6 → E9）
```

### Q5-1　Update 和 FixedUpdate 的区别?

**L1**　Update 每渲染帧一次，间隔是 deltaTime、不固定；FixedUpdate 按 fixedDeltaTime（默认 0.02 s）固定间隔，一帧里可能执行 0 次或多次，物理模拟紧跟在它后面。

**L2**　所以输入在 Update 读（GetKeyDown 只在一帧为真，FixedUpdate 可能漏掉），物理相关的力和持续推进放 FixedUpdate；在 Update 里直接设 velocity 是可以的，它在下一个物理步生

效。

**L3 • 项目**　我的冲刺在 Update 里计时、靠物理积分位移，测出来 120 / 144 fps 下距离会少 0.5 m，冲刺中卡顿 250 ms 能多冲 30%；改成按物理步计数后任何帧率都是 10 步 5 m。

### Q5-2　为什么移动要乘 deltaTime？设 velocity 也要乘吗？

**L2**　直接改 position 时，每帧位移 = 速度 × 这一帧经过的时间，不乘就变成“每帧固定位移”，帧率翻倍速度翻倍。设 velocity 不用乘——velocity 本来就是“每秒”的量，由物理按时间积分；反而乘了才是错的。AddForce 在 FixedUpdate 里调用是“每步一次”，放在 Update 里会随帧率变化（ForceMode.Impulse 的一次性冲量例外）。

### Q5-3 → Q5-4　计时器怎么写才帧率无关？

**L2**　周期性的计时器用累加：`timer -= dt; while (timer ≤ 0) { timer += period; Fire(); }`，保留超出的余量，一帧里该触发几次就触发几次；只关心“过没过”的一次性计时器，到点触发即可，但要接受最多一帧的量化。

**L3 • 项目**　点燃用的是“到点重置”，30 fps 下周期从 0.3 变成 0.333，2 秒少跳一次；旋转剑冷却配成 0 时命中次数等于帧数，240 fps 玩家的伤害是 30 fps 的 8 倍。这类伤害应该放到固定逻辑帧里统一结算。

**L4 • 攻击**　“累加写法在卡顿时会一口气触发很多次，不是更糟吗？” → 对伤害来说“总量正确”比“平滑”更重要；如果担心一帧里触发太多，可以给 while 设上限，把剩余的丢弃或延后，这是有意识的取舍，而不是漂移。

### Q5-5　固定步长的游戏循环怎么写？

**L2**　累加器模式：`acc += frameDt; while (acc ≥ step) { Simulate(step); acc -= step; }` `Render(lerp(prev, curr, acc / step));`。逻辑永远以固定步推进，渲染用两个逻辑状态之间的插值，代价是显示比逻辑晚最多一步。还要给 frameDt 设上限防止“死亡螺旋”（Unity 的 Maximum Allowed Timestep 就是这个）。

**L3**　帧同步游戏把这个循环推到极致：逻辑帧率固定（如 15 Hz），只用定点数，渲染层插值和预表现；这样不同设备、不同帧率得到逐位相同的结果。

### Q5-6　轨迹预测怎么和物理对上?

**L2**　连续抛物线和物理引擎的离散积分不一样：Box2D 是半隐式欧拉，位置比连续解多一项 ½·g·s·Δt·t。没有阻尼和碰撞时，用离散闭式解按物理步数画点就能逐位一致；有碰撞、阻尼、反弹时，在隐藏的物理场景里用 `PhysicsScene2D.Simulate` 预演若干步。

**常见错误回答 ①**

**错误**：“在 Update 里改 rb.velocity 是错的，所有物理操作都必须放 FixedUpdate。”

**问题**：设速度在 Update 里是安全的；真正有问题的是“在 Update 里施加持续力”和“用 Update 计时决定物理持续时长”。

**更好的回答**：“输入在 Update 读、意图存下来；持续的力和需要严格时长的推进放在 FixedUpdate；一次性的速度变化放哪都行。”

**常见错误回答 ②**

**错误**：“乘了 deltaTime 就帧率无关了。”

**问题**：乘 deltaTime 只解决“速率”，不解决“离散事件的次数”和“持续时长的量化”——点燃少跳一次、冲刺少走半米，都发生在乘了 deltaTime 的代码里。

## 5.7 发散场景题

### 场景 5-1　120 Hz 手机和 60 Hz 手机的玩家，跑图速度、冲刺距离、DPS 一样吗?

为什么会问：移动端高刷普及，帧率差异是真实的公平性问题。

跑图速度一样（类别 ②）；冲刺距离在 120 Hz 下有一部分情况短 0.5 m（类别 ③）；点燃、旋转剑这类周期伤害取决于计时写法（类别 ④），冷却为 0 的技能 DPS 直接翻倍。排查方法：把所有

“Update 里计时 + 结果有数值意义”的代码列出来，逐个改成累加或固定步。PVP 游戏还要求逻辑帧完全与渲染帧解耦。

### 场景 5-2　加入“子弹时间”（Time.timeScale = 0.3），哪些系统会出问题?

为什么会问：考察 scaled / unscaled 时间的边界。

deltaTime、WaitForSeconds、Invoke、FixedUpdate 的步长（fixedDeltaTime 也随 timeScale 缩放，但物理步的间隔不变 → 每秒物理步数减少）都会变慢，这是想要的；需要不受影响的是 UI 动画、输入缓冲窗口（否则预输入时间变长 3 倍）、相机震动等，要改用 unscaledDeltaTime / WaitForSecondsRealtime。Animator 的 Update Mode 要决定是 Normal 还是 Unscaled Time。注意项目里的 `Time.time` 比较（连招窗口、攻击冷却）是缩放时间，子弹时间里连招窗口会“变长”。

### 场景 5-3　低帧率时玩家冲刺穿墙，你怎么解释和修复?

为什么会问：隧穿是物理 + 帧率的综合题。

玩家刚体已经是 Continuous，单纯的物理隧穿不太会发生；但卡顿时冲刺多走的那 1~2 m（E10）可能让角色越过平台边缘；而飞剑是 Discrete，每步 0.7 m，薄墙可能被穿过。修复：飞剑改 Continuous；冲刺改成固定步计数；冲刺前用 `Rigidbody2D.Cast` 预测最远可达位置。

### 场景 5-4　要做“录像回放”，回放时结果和实战不一致，可能是哪些原因?

为什么会问：回放是确定性的试金石，自然引向帧同步。

本项目里至少有四个不确定源：① 逻辑依赖帧率（冲刺、点燃、旋转剑）；② `UnityEngine.Random` 没有固定种子且被很多地方共用（暴击、闪避、掉落、分身攻击编号）；③ 动画事件的触发帧依赖渲染；④ 物理引擎在不同帧时序下结果不同。要做输入回放，必须先把逻辑放进固定步、随机数按系统分流并记录种子、命中改成逻辑时间轴；否则只能录“状态”（位置、血量）而不是录“输入”。

**方法论延伸**

跳跃曲线（可变跳高、下落加重、顶点悬停）与输入延迟链路的分析见第 20 章。

**本章速记**

- 两条时间线：**Update 设速度 → 下一个物理步才位移**；输入在 Update 读。
- 三类问题代码：**Update 计时决定时长（量化）**、**到点重置的周期计时（漂移）**、**连续公式预测离散物理（偏差）**。
- E10：120 / 144 fps 冲刺有时只有 4.5 m，卡 250 ms 冲 7.0 m；点燃 30 fps 少跳一次；冷却 0 的旋转剑命中数 = 帧数。
- E9：剑比虚线低 **½•g•s•Δt•t**，1.68 s 时 0.74 m；用离散闭式解可逐位对齐。

▶ **自测 1：跳跃高度会随帧率变化吗?**

▶ **自测 2：为什么 60 fps 冲刺稳定 5.00 m，而 120 fps 不稳定?**

▶ **自测 3：分身的淡出写法为什么与帧率无关?**

CHAPTER 06

# 战斗判定：攻击检测、受击反馈与弹反窗口

“攻击判定怎么做”“弹反怎么实现”“为什么会多段命中”——这一章把一次命中拆成检测、结算、反馈三步，指出项目里“反馈和结算分家”导致的两个怪现象：闪避了也闪白击退、弹反成功照样掉血；再把弹反窗口按帧算清楚。

---

**本章目标**

- 说清 Hitbox / Hurtbox / LayerMask / 命中去重这四个概念，并对照项目里缺了哪些
- 理解“表现先行、结算在后、二者不通气”带来的两个 bug
- 用 E2 的数字讲清弹反成功窗口：本项目 1.01 s、原教程写法 1.33 s，以及为什么都偏宽
- 给出 DamageInfo → HitResult 的伤害管线，把弹反、闪避、无敌帧、霸体放进同一个判定入口

## 6.1 30 秒面试版 • 现状 vs 理想

**30 秒回答**

攻击判定在动画的命中帧用 `OverlapCircleAll` 取攻击点周围的碰撞体，对其中的敌人先调 `DamageEffect()`（闪白 + 击退协程），再调 `DoDamage()`（闪避 → 暴击 → 护甲）。问题有三：查询没有 LayerMask、每次分配数组、同一目标多个碰撞体会重复命中；反馈和结算是两个互不知情的调用，所以闪避时照样闪白击退，弹反成功时伤害照样结算；击退方向用的是“受击者自己的朝向”，背后挨打会被打向攻击者。弹反的判定是“受击那一刻处于反击状态、且怪物正在出手窗口内”，实测按键成功窗口约 1.01 s。改进是统一成一个伤害管线：攻击方构造 DamageInfo，受击方返回 HitResult（Evaded / Parried / Blocked / Hit），反馈根据结果播放。

现状 → 问题 → 理想 **战斗判定**

| 维度 | 当前做法（Demo 现状） | 存在的问题 | 理想重构方案 | 收益 / 代价 |
|---|---|---|---|---|
| **检测** | 9 处 OverlapCircleAll：无 LayerMask、按碰撞体遍历、每次分配数组 | P2 会查到地面与触发器；一个敌人两个碰撞体就挨两刀；每次调用都分配数组（#27） | Hurtbox 层 + ContactFilter2D + 复用缓冲 + 按实体去重 设计方案 · 未实现 | 不误查地面 / 触发器；多碰撞体不多段命中；零分配 |
| **裁决** | DamageEffect（表现）与 DoDamage（结算）两次独立调用 | P1 闪避了却闪白击退；弹反成功仍照常扣血（#5） | 攻击方构造 DamageInfo → 受击方 Receive 唯一裁决 → HitResult；表现只读结果 设计方案 · 未实现 | 闪避不再闪白；弹反成功不再掉血 |
| **弹反窗口** | 受击瞬间处于 1 s 的反击架势即成功；落空没有惩罚 | P2 按键窗口 1.01 s 且可预按、落空无惩罚：弹反没有风险，不需要读招（E2） | 完美窗口从按键起算约 0.15 s；之后算格挡；落空进入收招硬直 设计方案 · 未实现 | 难度可控、需要读招（参数为设计值） |
| **击退方向** | 按受击者自己的朝向 | P2 背对攻击者挨打会被打进攻击者怀里；玩家 knockBack 为 (0, 0)，受击时速度被清零（推断，#26） | 按攻击来源方向（DamageInfo.origin） 设计方案 · 未实现 | 背后挨打不会被打进攻击者怀里 |
| **免疫与打断** | 无无敌帧、无霸体；多次受击叠加多个击退协程 | 隐患 冲刺穿怪照样挨打；连续受击叠加多个击退协程，最早结束的那个提前解除击退 | 标签（Invulnerable / SuperArmor）+ 打断等级 设计方案 · 未实现 | 冲刺无敌、霸体攻击有统一表达 |
| **同帧互砍** | 结算顺序取决于 Animator 更新顺序（不确定） | 隐患 谁先扣血取决于 Animator 更新顺序：同样的操作结果不可复现 | 本帧命中先收集，帧末按规则统一结算 设计方案 · 未实现 | 结果可预期、可复现 |

现状问题：P1 ×1　P2 ×3　隐患 ×2（P0~P3 编号见附录 A，分级说明见 0.2）

## 6.2 检测：一个圆、没有过滤、没有去重

当前源码 • 822270c PlayerAnimationTriggers.cs:7-26

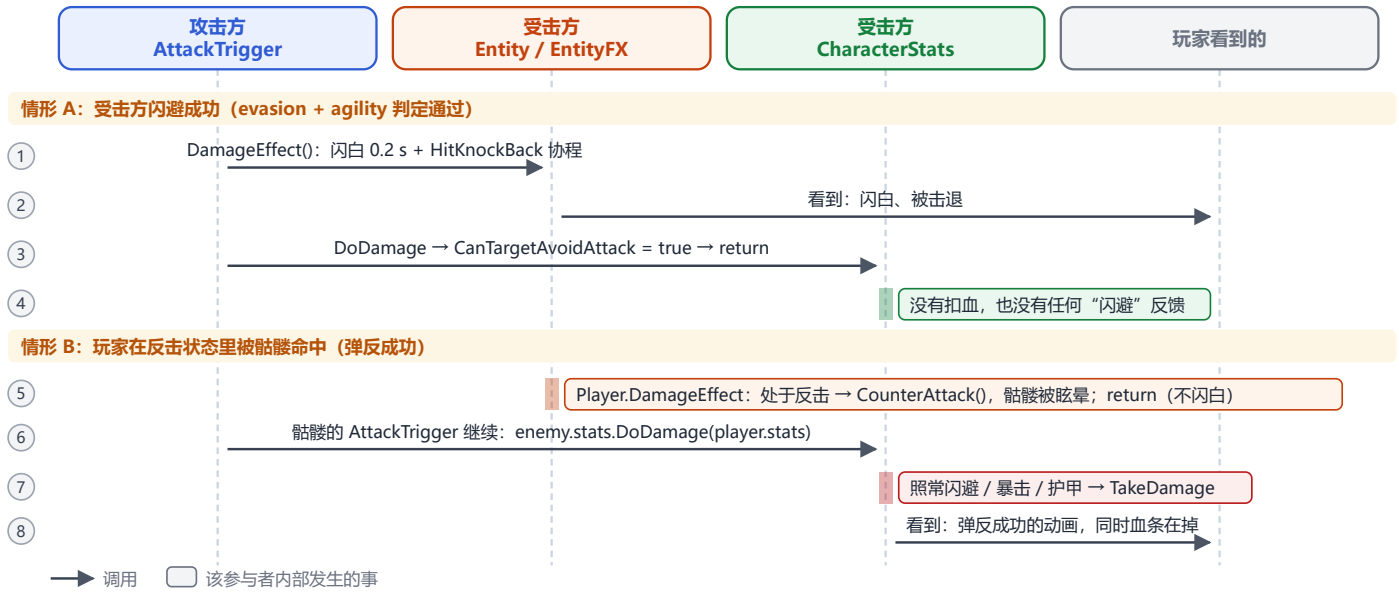
```
private Player player => GetComponentInParent<Player>();    // 属性：每次访问都向上查一次

private void AttackTrigger()
{
    Collider2D[] colliders = Physics2D.OverlapCircleAll(player.attackCheck.position, player.a
    foreach (Collider2D collider in colliders)
    {
        Enemy enemy = collider.GetComponent<Enemy>();
        if (enemy != null)
        {
            enemy.DamageEffect();                    // ① 先播受击表现
            player.stats.DoDamage(enemy.stats);      // ② 再结算（可能被闪避）
        }
    }
}
```

**现状问题** P1 表现（DamageEffect）与结算（DoDamage）分两次调用（#5）；OverlapCircleAll 无 LayerMask、按碰撞体不去重、每次分配（#27）；player 属性每次访问都向上查（#34）

| 概念 | 项目现状 | 后果 | 改法 |
| --- | --- | --- | --- |
| **Hitbox**（攻击判定区） | 一个圆：attackCheck（本地 x 1.2）、半径 0.9；三段攻击共用 | 三段攻击动作幅度不同却同一判定；空中 / 地面也同一判定 | 判定形状随技能数据配置（圆 / 盒 / 扇形 + 偏移 + 持续帧） |
| **Hurtbox**（受击区） | 直接用物理胶囊体 | 受击区与“推挤 / 站地面”的物理体绑死；想做“只打头”“蹲下躲刀”都做不了 | 单独的 Trigger 碰撞体，放在 Hurtbox 层 |
| **LayerMask** | 9 处 OverlapCircleAll 全都没传 | 结果里混着地面 Tilemap、玩家自己的血条 Trigger（Queries Hit Triggers 开着）、掉落物触发器，每个都要 GetComponent 一次 | 只查 Hurtbox 层；用 ContactFilter2D 统一配置 |
| **去重** | 按碰撞体遍历 | 一个敌人挂两个碰撞体就挨两刀（加头部判定、加盾牌碰撞体时立刻出现） | 本次攻击维护 HashSet<受击者>，按“实体”而不是“碰撞体”去重 |
| **分配** | 每次 new 一个数组 | 一刀一次 GC 分配；旋转剑冷却为 0 时每帧一次（第 12 章） | NonAlloc / 带 List 结果的重载，复用缓冲 |

## 6.3 反馈与结算分家：两个怪现象




**现状** 图 6-1　反馈与结算是两次独立调用：情形 A 闪避了却闪白击退（玩家攻击敌人时）；情形 B 弹反成功却照样结算伤害（骷髅攻击玩家时）。当前骷髅 damage = 0，情形 B 暂时看不出来

**现状问题** P1 闪避也闪白击退；弹反成功仍结算伤害（#5）

**问题** P1 两个现象是同一个设计缺陷：**“这次攻击的结果是什么”没有一个唯一的裁决点。**表现层（闪白、击退、弹反）和数值层（闪避、伤害）各自判断，互不知道对方的结论。修法见 6.6 的伤害管线。

## 击退的两个细节

**当前源码 • 822270c** `Entity.cs:66-73 与 SetVelocity:107-113`

```
protected virtual IEnumerator HitKnockBack()
{
    isKnocked = true;
    rb.velocity = new Vector2(knockBackDistance.x * -facinDir, knockBackDistance.y);    // 方向
    yield return new WaitForSeconds(knockBackDuration);
    isKnocked = false;
}
public void SetVelocity(float _xVelocity, float _yVeloctiy) { if (isKnocked) return;  ... }  //
```

**现状问题** P2 击退方向取受击者朝向而非攻击来源；多次受击叠加协程，最早结束的那个提前解除击退（#26）

- **问题** P2 击退方向取决于**受击者面朝哪边**，而不是攻击来自哪边。骷髅背对玩家时被砍，会被击退到玩家怀里。应传入攻击者位置：`dir = sign(victim.x - attacker.x)`。

- 推断 玩家的 knockBackDistance 是 (0, 0)、持续 0 s：每次受击都会把速度设成 (0, 0) 并锁住 SetVelocity 一帧——空中被打会瞬间失去上升 / 下落速度，表现为“顿一下再掉”。这可能被当成“受击硬直”接受了，但它是零值配置的副作用，而不是设计。
- 多次受击会叠加多个 HitKnockBack 协程，最早结束的那个把 isKnocked 置假——和 isBusy、冻结一样的“重叠协程最早者赢”问题（第 7 章 E8 量化）。

## 6.4 弹反：判定逻辑与 E2 成功窗口

当前源码 • 822270c Player.cs:123-131 · PlayerCounterAttackState.cs:55-72 · Enemy.cs:100-109

```
public override void DamageEffect()                                    // Player：受击时先看是不是在反击
{
    if (stateMachine.currentState == counterAttack) { counterAttack.CounterAttack(); return;
    base.DamageEffect();
}
public void CounterAttack()                                            // 以自己的攻击点找“正在出手”的敌人
{
    foreach (var c in Physics2D.OverlapCircleAll(player.attackCheck.position, player.attackCh
        if (c.GetComponent<Enemy>() is Enemy e && e.CanbeStunned())
        {
            player.animator.SetBool("SuccessfulCounterAttack", true);
            stateTimer = 10;                                           // 之后只能靠成功动画的事件退出
            player.isSuccessfulCounterAttacking = true;
        }
}
public virtual bool CanbeStunned()                                     // “查询”带副作用：关窗 + 子类里切
{
    if (canBeStunned) { CloseCounterAttackWindow(); return true; }
    return false;
}
```

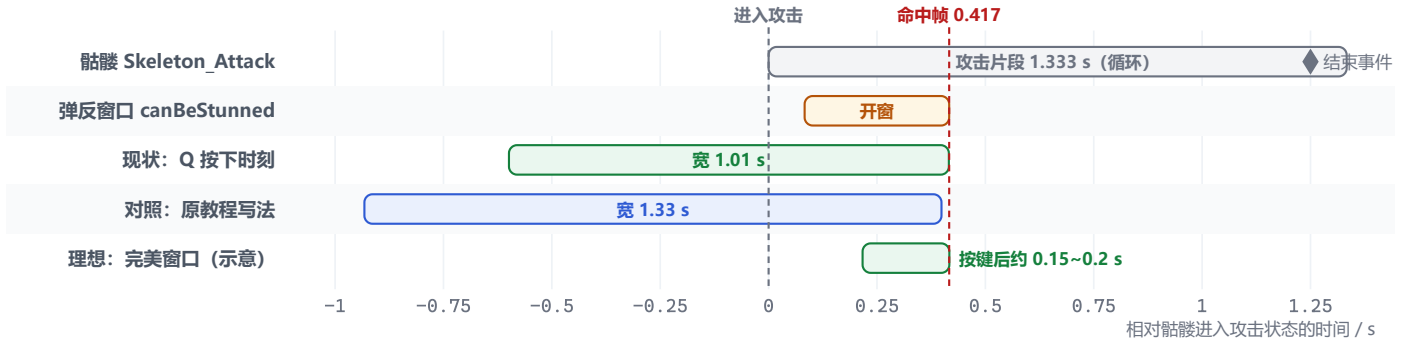
现状问题 P1 弹反只拦住了表现，伤害结算照常执行（#5）；架势 1 s 内任何时刻被打都算成功，按键窗口 1.01 s（E2）

作者把教程的写法（反击状态每帧检查“附近有没有正在开窗的敌人”，代码还以注释形式留在 Update 里）改成了“**被打中的那一刻**才检查”。两种写法的成功条件不同：

- **本项目**：骷髅的 AttackTrigger（0.4167 s）触发时，玩家仍处于反击状态。反击状态持续 1 s，所以 Q 必须在命中前 1 s 以内按下。

- **原教程写法**：反击状态的任意一帧 Update 里，骷髅的窗口开着（0.0833 s 开、0.4167 s 关）。反击状态与窗口有重叠即可。

用逐帧模型扫按键时刻 τ（相对骷髅进入攻击的那一帧，1 ms 步长，24 个子相位取公共部分） 实测 E2 模型 R1~R3 ：




现状 vs 理想 图 6-2　弹反成功的按键时刻（60 fps，E2）。本项目的成功区间是“命中前约 1 s 到命中帧”，原教程写法再往前多 1/3 s；两者都能在骷髅**还没抬手**时就按好 Q 等着——最后一行是动作游戏常见的“只奖励临近命中的按键”的示意

| 帧率 | 本项目：最早 / s | 最晚 / s | 宽度 / s | 原写法：最早 / s | 最晚 / s | 宽度 / s |
|---|---|---|---|---|---|---|
| 30 | -0.566 | 0.432 | 0.998 | -0.899 | 0.399 | 1.298 |
| 60 | -0.599 | 0.416 | 1.015 | -0.932 | 0.399 | 1.331 |
| 144 | -0.589 | 0.416 | 1.005 | -0.923 | 0.409 | 1.331 |

表 6-1　三种帧率下的成功区间。帧率只影响边界的量化：命中事件在“片段时间首次 ≥ 0.4167 s”的那一帧触发，30 fps 下这一帧在 0.433 s，所以 0.42 s 才按下也来得及；左边界由反击状态的 1 s 计时（float 累减）决定

三点结论：

1. **窗口宽度 ≈ 反击状态持续时间（1 s）**，而不是骷髅的“破绽”时长。把 `counterAttackDuration` 调短会同时缩短“摆架子等弹反”的时间，这才是真正控制难度的旋钮。
2. **可以无脑预按**：骷髅进入攻击前约 0.6 s 就按 Q，仍会成功。原写法更宽（只要求反击状态与 0.33 s 的窗口有重叠），提前 0.9 s 都行。
3. **没有失败代价**：Q 落空只是在原地站 1 s（反击状态每帧 SetVelocity(0,0)），没有硬直或冷却；配合第 2 点，弹反几乎是“看到骷髅靠近就按”。

**别忘了 6.3 的情形 B**

即使弹反成功，骷髅的 AttackTrigger 在调用 `player.DamageEffect()` 之后照样执行 `enemy.stats.DoDamage(player.stats)`。另外，弹反成功的判定还依赖第 4 章说的“同一时刻 AttackTrigger 排在 CloseCounterAttackWindow 之前”——这是一个只存在于 .anim 文件事件顺序里的契约。

## 6.5 无敌帧、霸体与多段命中

项目里没有任何“免疫”概念：冲刺没有无敌帧，受击后没有短暂无敌，攻击没有霸体。在只有一只骷髅、伤害为 0 的场景里看不出来；一旦多个敌人同时攻击，玩家会在同一帧被多次结算、被多个击退协程拉扯。动作游戏里通常这样组织：

| 机制 | 实现方式 | 放在哪里 |
| --- | --- | --- |
| 闪避无敌帧 | 冲刺的某一段给受击者挂 Invulnerable 标签 | 技能时间轴的区间轨道（第 4 章） |
| 受击后无敌 | 被击中后 0.3~0.5 s 内对同类攻击免疫（常见于平台跳跃） | 受击方的状态 / 修正器 |
| 霸体 / 削韧 | 受击只扣血不打断；韧性值归零才进入硬直 | 受击方的“打断等级”（第 3 章的 InterruptLevel） |
| 同一攻击不重复命中 | 每个攻击实例一个 HashSet<受击者> | 攻击方的命中记录 |
| 多段攻击 | 每段一个命中组，组内去重、组间不去重；或按间隔重置记录 | 技能数据（命中次数上限、间隔） |

## 6.6 改进：一个裁决点的伤害管线




**理想方案** 图 6-3　改进后的伤害管线：受击方的 Receive 是唯一裁决点，表现、数值、攻击方的后续（被弹反进眩晕）都只读 HitResult

**理想方案 · 骨架** DamageInfo / HitResult（C#，未在 Unity 中编译）

```
public struct DamageInfo
{
    public Entity source; public Vector2 origin;  // 击退方向 = sign(victim.x - origin.x)
    public int physical, fire, ice, lightning;
    public bool parryable; public int hitGroup;   // 同组只命中一次
}
public enum HitResult { Hit, Evaded, Parried, Blocked, Immune }

public interface IDamageable { HitResult Receive(in DamageInfo info); }

// 受击方（玩家）：所有"这次算不算"的判断都在这里，按优先级
public HitResult Receive(in DamageInfo info)
{
    if (tags.Has(Tag.Invulnerable))                return HitResult.Immune;
    if (info.parryable && parry.IsPerfect(Time.time)) { info.source.OnParried(); return HitRe
    if (stats.RollEvade())                         return HitResult.Evaded;
    stats.Apply(info);                             // 暴击、护甲、元素、异常状态
    fx.PlayHit(info.origin);                       // 反馈在结论之后
    return HitResult.Hit;
}
```

"完美弹反"的窗口 `parry.IsPerfect` 应该从**按下的时刻**算起（例如按下后 0.15 s 内），而不是"整个反击状态"；超过完美窗口但仍在架势中可以算格挡（减伤、扣耐力），落空则进入 0.3 s 的收招硬直——这三个参数就是弹反手感的全部旋钮。

## 6.7 面试追问

追问树

```
攻击判定怎么做的？（Q6-1）
├─ 为什么用 Overlap 不用碰撞回调 OnTriggerEnter？（Q6-2）
├─ 一刀会不会打中同一个敌人两次？（Q6-3 去重）
├─ 弹反怎么实现？窗口多宽？（Q6-4 → E2）
│   └─ 弹反成功为什么还掉血？（Q6-5 → 6.3 情形 B）
├─ 闪避 / 无敌帧 / 霸体怎么做？（Q6-6 → 6.5）
└─ 打击感怎么做？（Q6-7）
```

## Q6-1 → Q6-2　攻击判定怎么做？为什么用 Overlap 不用 OnTriggerEnter？

**L2**　在动画命中帧做一次 OverlapCircle 查询，是“瞬时判定”：在哪一帧判、判多大范围完全由逻辑控制，结果立刻可用。OnTriggerEnter 是“持续判定”：需要一个跟随武器的触发器，回调在物理步之后发生，进入后不会再次触发（同一敌人站在里面不会被第二段攻击命中），而且时机受物理步与帧的对齐影响。近战挥砍多用瞬时查询，投射物多用触发器——项目里飞剑正是用 OnTriggerEnter2D。

**L3**　无论哪种都要做三件事：只查 Hurtbox 层、按实体去重、结果交给唯一的裁决点。我项目里三件都没做全。

## Q6-3　一刀会不会打中同一个敌人两次？

**L2**　按碰撞体遍历就会：敌人有身体和头两个碰撞体，就会被结算两次。我的骷髅现在只有一个胶囊体所以没出现；正确做法是取 `collider.attachedRigidbody` 或向上找实体组件，放进本次攻击的 HashSet 去重。

**L3**　多段攻击（旋转剑）是另一回事：需要“同一目标间隔 x 秒才能再次命中”，记录表要带时间戳；我项目里旋转剑的间隔是全局的 hitTimer，对所有目标同时生效，而且冷却在场景里配成了 0。

## Q6-4 → Q6-5　弹反怎么实现？窗口多宽？为什么成功了还掉血？

**L1**　怪物出手前用动画事件开“可被弹反”窗口；玩家按 Q 进入 1 s 的反击架势；被打中时如果在架势里、且怪物窗口开着，就眩晕怪物。

**L2**　我用逐帧模型算过：成功的按键时刻是命中前约 1 s 到命中帧，宽 1.01 s；教程原来的“每帧检查重叠”写法更宽，1.33 s。两者都允许在怪物抬手前就按好等着，所以难度很低。

**L3**　成功还掉血，是因为怪物的命中事件先调玩家的 DamageEffect（这里完成了弹反），再调 DoDamage——两次调用互不知道结果。我会把它改成一个 Receive 返回 HitResult，Parried 时不结算伤害，并把“完美窗口”改成从按键时刻起算的 0.15 s。

**L4 • 攻击**　“1 s 的窗口是你故意设计的吗？” → 不是，它是反击状态持续时间的副产品；我确认之后才意识到真正的难度旋钮在 counterAttackDuration 上。这也是我后来坚持“先量化再调参”的原因。

### Q6-7　打击感怎么做?

**L2**　按时间顺序叠加：命中瞬间顿帧 50~100 ms（攻击方和受击方动画减速）、受击闪白（项目有）、击退 / 浮空（项目有但方向有误）、镜头震动（Cinemachine Impulse）、命中特效与音效、伤害数字、手柄震动。关键是“同一帧开始”且“顿帧期间其他反馈不停”。

**L3**　顿帧会和冻结、减速抢写 animator.speed（第 7 章），所以要先有速度修正器栈；镜头震动和顿帧要用不受 timeScale 影响的时间。

**常见错误回答**

**错误**：“弹反就是在怪物攻击的时候按键。”

**问题**：没有回答“按键和命中的时间关系”，也没有区分“架势持续时间”和“完美窗口”。

**更好的回答**：“弹反由三个参数决定：完美窗口（从按键起算多久内被打中算完美）、架势持续时间（之后算格挡或落空）、落空惩罚（收招硬直）。我的项目里只有第二个，所以窗口宽达 1 s；改进是把完美窗口单独做出来。”

## 6.8 发散场景题

### 场景 6-1　同一帧里玩家和骷髅互相砍中，谁先结算?

为什么会问：同帧事件顺序是实时战斗的经典难题。

现状：两边的命中都在各自 Animator 的事件里，执行顺序取决于 Animator 的更新顺序，**不确定**；先结算的一方可能把对方打死（骷髅死亡后关碰撞体），后结算的就落空。设计上要明确规则：① 同帧互相命中都生效（“相杀”）；② 按优先级（弹反 > 攻击）；③ 按攻击发起时间。实现上把命中“收集”到本帧的队列，帧末统一按规则结算，而不是在回调里立刻扣血。

### 场景 6-2　一刀砍中 20 个小怪，帧率掉了，怎么办?

为什么会问：考察战斗峰值的性能意识。

热点：OverlapCircleAll 分配、每个碰撞体 GetComponent、每个受击者启动两个协程（闪白、击退）+ 改材质、伤害数字 UI 实例化。优化：NonAlloc + 层过滤、受击者组件缓存在 Hurtbox 上、受击表现用 MaterialPropertyBlock 和统一的计时器而不是协程、飘字对象池并合批、超过 N 个受击者时只对最近的 K 个播放完整表现。

### 场景 6-3　改成联机（状态同步），近战命中在哪里判定?

为什么会问：从单机判定发散到网络。

服务器权威判定；客户端为了手感先播放挥刀和命中特效（预表现），但伤害数字和受击硬直等服务器结果。高延迟下“我看到砍中了却没伤害”的矛盾用延迟补偿缓解：服务器把受击者回溯到攻击者看到的时刻再做 Overlap。弹反这种“受击方按键”的机制更难：受击方的按键要发到服务器，服务器以“它看到的攻击时间”判定，窗口需要考虑 RTT——这也是很多联机动作游戏把弹反做成“预判架势”而不是“精确时机”的原因。

### 场景 6-4　策划要“背刺双倍伤害”，你会怎么加?

为什么会问：检验伤害管线是否可扩展。

在 DamageInfo 里已经有 origin，受击方 Receive 里根据 origin 与自身朝向判断是否背后，乘以修正系数——一处修改。现状下要同时改 DoDamage（数值）和 DamageEffect（表现），而且击退方向本身就是按朝向算的，背刺时还会被打向攻击者。

**本章速记**

- 检测四件事：**Hitbox 形状、Hurtbox 分层、LayerMask、按实体去重**；项目 9 处 Overlap 都没过滤。

- 反馈与结算分家 → **闪避也闪白击退、弹反成功仍掉血**（#5）；击退方向用了受击者朝向（#26）。
- E2：本项目弹反成功窗口 1.01 s（命中前 ~1 s 到命中帧），原写法 1.33 s；都能“预按”。
- 改进：**DamageInfo → Receive（唯一裁决）→ HitResult**；完美窗口从按键起算，落空有惩罚。

▶ **自测 1：为什么 30 fps 下最晚按键时刻（0.432 s）比事件时间 0.4167 s 还晚?**

▶ **自测 2：CanbeStunned 有哪些副作用?**

▶ **自测 3：只把 DamageEffect 和 DoDamage 的调用顺序对调能修好闪避闪白吗?**

PART III

# 数值、技能与 AI

数值系统决定“打多疼”，技能系统决定“怎么打”，AI 决定“谁来挨打”。这三章的共同主题是：**多个系统同时写同一份数据**，以及**名字像查询的函数在偷偷改世界。**

CHAPTER 07

# 属性、伤害公式与异常状态

伤害公式、Buff、属性修正器是 RPG 方向必问的三件套。项目的 Stat / CharacterStats 结构清楚，但实测出四个数值问题：暴击率凭空多 1%、冰冻判定看错了对象、等级成长是复利、多个冻结效果互相抵消；元素伤害整条链路写好了却没有接线。

---

**本章目标**

- 说清 14 项属性怎么派生出闪避率、暴击率、暴击倍率、最终伤害
- 用 E3 / E4 的数字讲清 off-by-one、写错判定对象、复利成长三个数值 bug
- 讲清“多个系统写同一字段”的危害，并用 E8 量化重叠冻结的失效比例
- 设计 Modifier（Flat / PercentAdd / PercentMult）与 Buff（叠层、刷新、来源、到期）系统

## 7.1 30 秒面试版 • 现状 vs 理想

**30 秒回答**

`Stat` 是“基础值 + 整数修正列表”，装备穿上加修正、卸下按值移除；`CharacterStats` 的物理伤害按“闪避 → 暴击 → 减护甲 → TakeDamage”结算，元素伤害取最高元素施加点燃 / 冰冻 / 感电。实测有四个问题：暴击用 `≤` 比较 0~99 的整数，暴击率 0 也有 1% 暴击；冰冻破甲判断的是攻击者而不是目标；敌人每级 +25% 是在“当前值”上加，10 级是 7.44 倍而不是 3.25 倍；冻结、减速、死亡三个系统都直接写 moveSpeed 和 animator.speed，多段冻结时实际只冻住 26%~85% 的时间。另外受伤没有死亡保护，死后再受伤会重复掉落；元素伤害函数 `DoMagicalDamage` 没有任何调用方。改进是 Modifier 栈 + 脏标记缓存，异常状态做成统一的 Buff。

**现状** → 问题 → **理想** **属性、伤害与异常状态**

| 维度 | 当前做法（Demo 现状） | 存在的问题 | 理想重构方案 | 收益 / 代价 |
|---|---|---|---|---|
| **概率判定** | 暴击用 `≤`、闪避用 `<` 与 0~99 的整数比较，两种写法 | P2 暴击率配 0 仍有 1% 暴击，配 c 实际 (c+1)%（E3，#16） | 统一 Chance.Roll（`Random.value × 100 < 概率`）<br>**模型验证：E3 闪避的 < 写法蒙特卡洛等于设计值** | 配置 0 = 绝不发生 |
| **冰冻破甲** | 判断的是攻击者的 isChilled | P1 被冰冻的目标不破甲，反而是被冰冻的攻击者打人时破甲（E3，#10） | 判断目标的 isChilled<br>**模型验证：E3 修正后结果** | 符合设计意图 |
| **等级成长** | 循环里每级按“当前值”的 25% 叠加 | P2 10 级是 1 级的 7.44 倍（线性应为 3.25 倍），与“每级 +25%”的命名不符（E4，#18） | 等级曲线表驱动，复利与否显式决定<br>**设计方案 · 未实现** | 策划直接看到每一级的绝对值 |
| **死亡** | TakeDamage 无守卫，尸体可重复 Die / 掉落 | P1 弹跳剑 / 雷击让尸体重复 Die、重复掉落——物品可以刷（#7） | isDead 守卫 + 死亡事件让所有目标列表注销<br>**设计方案 · 未实现** | 掉落不可被刷 |
| **多写者** | 冻结、减速、死亡直接给 moveSpeed / animator.speed 赋绝对值，结束时再赋回 | P1 重叠冻结只冻住 26%~85% 的时间；一个效果的恢复会解除另一个效果（E8，#11） | 速度修正器栈：效果只登记修正，最终值推导；冻结取最晚截止时间<br>**模型验证：E8 理想版 100% 冻结** | 效果互不覆盖 |
| **修正器** | 只有整数加法、按值删除、没有来源 | 局限 做不了百分比加成，无法按来源撤销或预览“换上之后是多少” | Flat / PercentAdd / PercentMult，带来源，脏标记缓存<br>**设计方案 · 未实现** | 能做百分比加成、按来源撤销、属性预览 |
| **异常状态** | 三套 bool + timer；DoMagicalDamage 没有调用方 | P2 魔法伤害没有入口，点燃 / 冰冻 / 感电在游戏里触发不了；新增异常要复制一套 bool + timer（#19） | 统一 Buff：叠层 / 刷新 / 互斥标签 / 固定逻辑帧 tick<br>**设计方案 · 未实现** | 新增异常不用复制代码 |

现状问题：P1 ×3　P2 ×3　局限 ×1　（P0~P3 编号见附录 A，分级说明见 0.2）

## 7.2 属性派生关系

**当前源码 • 822270c** Stat.cs:5-37（精简）

```
[System.Serializable]
public class Stat
{
    [SerializeField] private int baseValue;
    public List<int> modifiers = new List<int>();
    public int GetValue() { int v = baseValue; foreach (int m in modifiers) v += m; return v;
    public void SetDefaultValue(int _value) => baseValue = _value;
    public void AddModifier(int m)    => modifiers.Add(m);
    public void RemoveModifier(int m) => modifiers.Remove(m);    // 按“值”删第一个相等的
}
```

**现状问题** 局限 修正只有整数加法、按值删除、不记来源：做不了百分比加成，也无法按来源撤销或预览；每次 GetValue 都重新求和

| 派生量 | 公式（源码） | 场景值（玩家 / 骷髅） |
| --- | --- | --- |
| 最大生命 | maxHealth + vitality × 5（GetMaxHealth） | 1000 / 100 |
| 闪避率 | (evasion + agility)%，判定 `Random.Range(0,100) < 值` | 0 / 0 |
| 暴击率 | (critRate + agility)%，判定 `Random.Range(0,100) ≤ 值` | 0 / 0（实际 1%） |
| 暴击倍率 | (critPower + strength) × 1%，critPower 在 Start 里被 `SetDefaultValue(150)` **覆盖** | 150% |
| 物理伤害 | damage + strength，减去目标 armor（冰冻时 ×0.85），下限 0 | 1000 − 0 / 0 − 10 → 0 |
| 魔法伤害 | fire + ice + lightning + intelligence，减去目标 magicResistance + intelligence × 3 | 10 / 20（但从未被调用） |

两个容易被追问的细节：① critPower 的基础值在 Inspector 里配多少都没用，Start 会改成 150——**“代码默认值”应该放在字段初始化器或 Reset() 里**，而不是在运行时覆盖序列化数据；② `RemoveModifier` 按值删除，装备 A 和 B 都给 +5 力量时卸下 A 也只会删掉“某一个 5”，结果数值上正确，但没有“来源”就做不到“移除某个 Buff 的全部效果”或“UI 显示加成来源”。

# 7.3 物理伤害：E3 量化三个细节




DoDamage(target) 的五步（CharacterStats.cs:97-121）。闪避发生时直接 return——但调用方在此之前已经播放了受击表现（第 6 章）

现状 图 7-1　物理伤害结算顺序：红框是 E3 / 源码确认的三个问题

**现状问题** P1 冰冻破甲判错对象（#10）；TakeDamage 无死亡保护，尸体可重复掉落（#7）；暴击 <= 多 1%（#16）

## 暴击率 off-by-one

`Random.Range(int, int)` 返回 [0, 100) 的整数，即 0~99 共 100 个值。闪避用 `< e`，命中 e 个值，概率 e%；暴击用 `≤ c`，命中 c + 1 个值，概率 (c + 1)%。**实测 E3**（各 200 万次）：

| 暴击率 + 敏捷 c | 设计概率 / % | 实际 (c+1)/100 / % | 蒙特卡洛 / % |
|---|---|---|---|
| 0 | 0 | 1.0 | 0.99 |
| 5 | 5 | 6.0 | 6.02 |
| 20 | 20 | 21.0 | 21.03 |
| 50 | 50 | 51.0 | 51.05 |
| 99 | 99 | 100.0 | 100.00 |

表 7-1　暴击概率：暴击率为 0 时仍有 1% 暴击；c = 99 时已是必暴。闪避的蒙特卡洛结果与设计一致（10 → 10.01%，30 → 30.04%）

1% 看起来很小，但“暴击率 0 也会暴击”会让策划配置的“必不暴击”的单位（训练木桩、Boss 的护盾阶段）出现意外；更重要的是两处用了不同的比较符，说明**概率判定没有统一的函数**。应该写一个 `Chance.Roll(float percent)`，内部用 `Random.value * 100f < percent`，全项目只用它。

## 冰冻破甲判断错了对象

当前源码 • 822270c CharacterStats.cs:143–152

```
private int CheckTargetArmor(CharacterStats _targetStats, int totalDamage)
{
    if (isChilled)    // ← this（攻击者）是否被冰冻；应为 _targetStats.isChilled
        totalDamage -= Mathf.RoundToInt(_targetStats.armor.GetValue() * 0.85f);
    else
        totalDamage -= _targetStats.armor.GetValue();
    return Mathf.Max(totalDamage, 0);
}
```

**现状问题** P1 判断的是攻击者（this）是否被冰冻而不是目标：设计意图的“冰冻破甲”完全反了（E3，#10）

| 情形（伤害 30、目标护甲 10） | 现状（源码） | 理想（修正后） |
|---|---|---|
| 双方都正常 | 20 | 20 |
| 目标被冰冻（设计意图：破甲） | 20 | 22 |
| 攻击者被冰冻 | 22 | 20 |

表 7-2　设计意图是“目标被冰冻时护甲打 85 折”；源码判断的是攻击者自己，于是变成“被冰冻的攻击者打人更疼”。注意 RoundToInt(8.5) = 8：Mathf.RoundToInt 是银行家舍入（.5 取偶）

### 暴击倍率

| strength | 普通伤害（damage 20 + strength） | 暴击伤害 | 实际倍率 |
|---|---|---|---|
| 0 | 20 | 30 | 1.50 |
| 10 | 30 | 48 | 1.60 |
| 50 | 70 | 140 | 2.00 |

表 7-3　力量同时加伤害和暴击倍率（各 1 点 / 1%），倍率随力量线性上升；50 力量时暴击翻倍。这是有意的“属性耦合”，但要在数值表里写清楚

## 7.4 死亡结算：没有保护，掉落会重复

当前源码 • 822270c CharacterStats.cs:115-121 · EnemyStats.cs:50-56

```
public virtual void TakeDamage(int _damage)
{
    DecreaseHealthBy(_damage);
    if (currentHealth ≤ 0) Die();        // 已经死了还会再 Die
}
protected override void Die()            // EnemyStats
{
    base.Die();
    enemy.Die();                         // → ChangeState(deathState)：Exit + Enter 再来一遍
    myDropSystem.GenerateDrop();         // → 再掉一次
}
```

现状问题 P1 没有死亡守卫：弹跳剑 / 雷击能让尸体重复 Die、重复掉落（#7）

问题 P1 近战打不到死掉的骷髅（死亡状态关了碰撞体），所以平时不会触发。但两条路径能绕过碰撞体：① **弹跳剑**在第一次接触时把半径 10 m 内的敌人 Transform 存进列表，之后按列表依次飞过去造成伤害——目标中途死亡也会被继续“弹”到，每次都重新 Die + 掉落；② 雷击 `Controller_ThunderStrike` 持有目标引用，飞行途中目标死亡，到达时照样 TakeDamage。修法：`if (isDead) return;` 放在 TakeDamage 最前面，Die 只执行一次；死亡时从所有“目标列表”里注销（发 OnDeath 事件）。

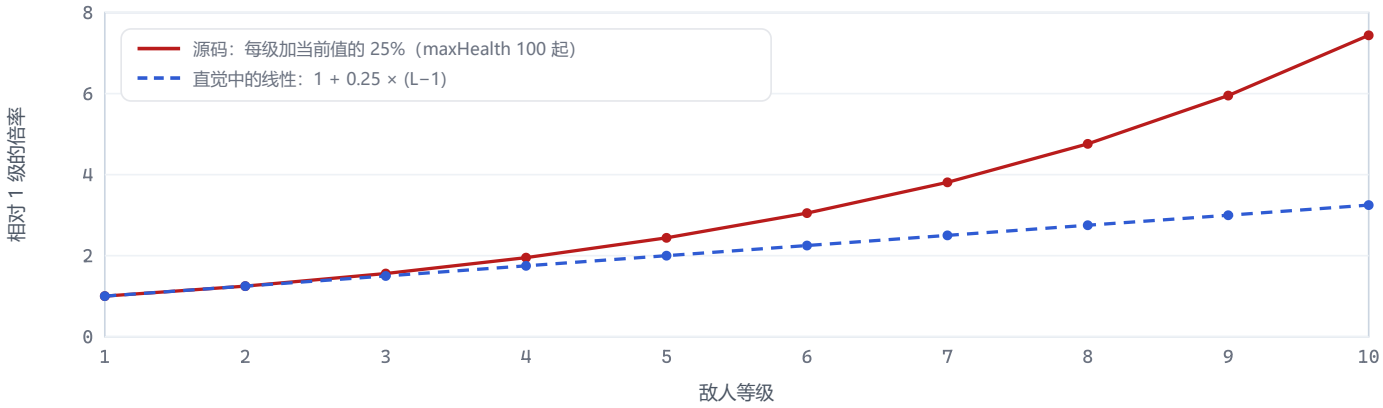
同一段代码里还有一个“上限变了血量不变”的问题：装备 Nice_Armor（+10 health）后 GetMaxHealth 变成 1010，currentHealth 还是 1000，血条也只在下一次受伤时才刷新上限 问题 P2。常见规则是“上限增加时当前值按比例 / 按差值增加，上限减少时截断”，并立刻通知 UI。

## 7.5 E4：每级 +25% 是复利

当前源码 • 822270c EnemyStats.cs:35–43

```
private void Modify(Stat _stat)
{
    for (int i = 1; i < level; i++)
    {
        float modifier = _stat.GetValue() * percantageModifier;    // GetValue 包含了前几轮加上
        _stat.AddModifier(Mathf.RoundToInt(modifier));
    }
}
```

现状问题 P2 GetValue 包含前几轮加上的修正 → 隐式复利，10 级 7.44 倍（E4，#18）；当前场景 level = 0，尚未生效




现状 图 7-2　敌人等级成长（E4）：10 级时生命是 1 级的 7.44 倍，线性只有 3.25 倍；伤害 10 起步时 10 级为 75（线性 32.5）

现状问题 P2 隐式复利，与命名“每级 +25%”的意图不符（E4，#18）

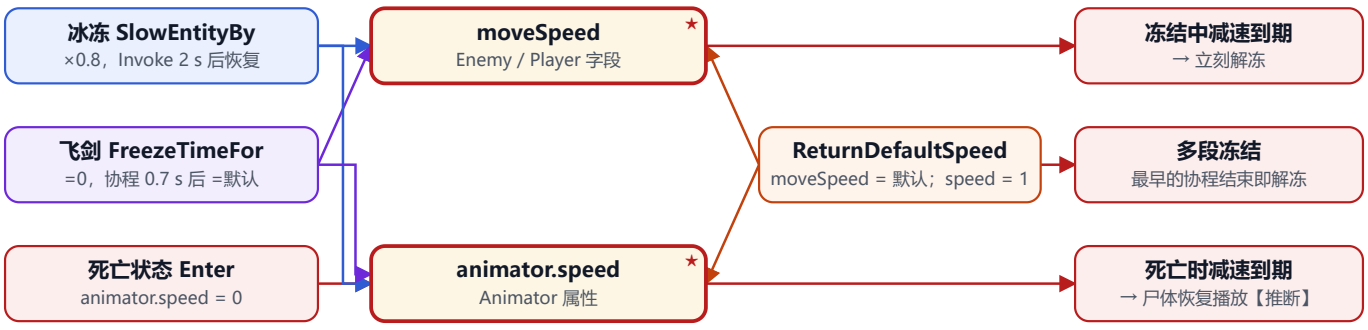
复利本身不是错——很多游戏的怪物成长就是指数的——问题是**它没有被当作设计决定写下来**：变量名 `percantageModifier`、注释“将基础属性实现等级修正”都暗示“每级加基础值的 25%”。数值成长应该由曲线表驱动（等级 → 系数），代码只查表，策划能直接看到第 10 级是多少。当前场景的骷髅 level = 0、percantageModifier = 0，这段代码还没有生效。

## 7.6 元素伤害与异常状态：写好了，没接线

`DoMagicalDamage` 的唯一调用是 DoDamage 里被注释掉的一行（“测试用”），所以点燃、冰冻、感电、连锁雷击在当前版本里都触发不了。代码本身值得读，因为它有几个设计点和几个 bug：

| 环节 | 源码行为 | 评价 |
| --- | --- | --- |
| 选元素 | 三种元素伤害放进 List 按值降序排序；唯一最大者施加对应异常；并列时 50% 概率从非零项里随机选一个 | 可扩展的写法；但每次调用都 new 一个 List（并列时 FindAll 再分配一个），并列分支里判断感电用的是 `damages[0]` 而不是随机选中的那一项（CharacterStats.cs:226）——`List.Sort` 不稳定，并列时 damages[0] 是谁不确定 |
| 互斥规则 | 点燃 / 冰冻：身上没有任何异常时才能上；感电：没有点燃和冰冻时可上 | “同一时间只有一种异常”是清晰的规则；感电可以叠加到感电上，用于触发连锁 |
| 点燃伤害 | 每 0.3 s 扣 fire × 10%（取整） | 首跳在第一帧；周期受帧率影响（第 5 章 E10） |
| 冰冻 | 减速 20%，持续 2 s | 通过 SlowEntityBy 直接改 moveSpeed（见 7.7） |
| 感电连锁 | 已感电再被感电：25 m 内找最近的**其他**敌人召唤雷击；找不到就劈自己 | `Controller_ThunderStrike` 到达后调用 `TakeDamage(1)`——**传进来的 shockDamage 完全没用**；目标在飞行途中被销毁时 `if (!targetStats) return;` 让雷击永远停在半空不销毁 |

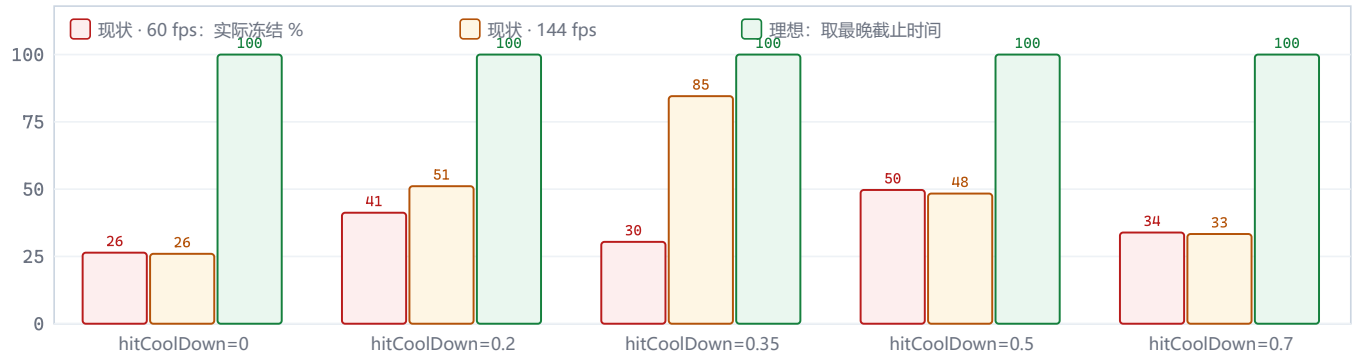
## 7.7 多个系统写同一个字段：E8 量化




现状 图 7-3　moveSpeed 与 animator.speed 各有 3~4 个写者，每个写者都用“赋绝对值”的方式恢复，谁最后写谁赢

现状问题 P1 多个写者赋绝对值：重叠冻结只冻住部分时间，一个效果的恢复解除另一个（E8，#11）

最容易被观察到的是飞剑冻结：每次命中都 `StartCoroutine("FreezeTimeFor", 0.7f)`，协程开头冻结、0.7 s 后解冻。多段命中时，**第一个协程到点就把敌人解冻了**，后面命中的“冻结 0.7 s”被截断；协程又在 Update 之后才恢复（R1），如果某一帧既有新命中又有旧协程到期，解冻会覆盖冻结。用旋转剑的命中节奏（持续 2 s）跑一遍 实测 E8 模型 R1 ：




现状 vs 理想 图 7-4 “应当全程冻结”的时间里实际冻住了多少（E8）。hitCoolDown = 0（场景配置）时只有约 26%；同一配置在 60 / 144 fps 下差异很大（0.35 s 时 30% vs 85%），因为截断发生在哪一帧取决于帧量化

理想方案 • 骨架 用“截止时间取最大值”替代重叠协程；速度用修正器栈（C#，未在 Unity 中编译）

```
float frozenUntil;                                  // 冻结：不开协程，只维护截止时间
public void Freeze(float seconds) => frozenUntil = Mathf.Max(frozenUntil, Time.time + seconds
bool IsFrozen => Time.time < frozenUntil;

readonly List<(object source, float mul, float until)> speedMods = new();
public void AddSpeedMod(object src, float mul, float sec) => speedMods.Add((src, mul, Time.ti
float SpeedMultiplier()                             // 每帧求一次：过期的删掉，其余相乘
{
    speedMods.RemoveAll(m => Time.time >= m.until);
    float k = IsFrozen || isDead ? 0f : 1f;         // 冻结 / 死亡是“硬覆盖”，优先级最高
    foreach (var m in speedMods) k *= m.mul;
    return k;
}
// moveSpeed、animator.speed 都由 base × SpeedMultiplier() 推导，任何系统都不再直接赋值
```

## 7.8 改进：Modifier 与 Buff

| 设计点 | 做法 | 解决了项目里的什么 |
| --- | --- | --- |
| 修正类型 | Flat（+5）、PercentAdd（+10%，同类相加）、PercentMult（×1.1，相乘），按固定顺序：(base + ΣFlat) × (1 + ΣPercentAdd) × ΠPercentMult | 现在只有整数加法，做不了“+20% 伤害” |
| 来源 | 每个修正带 source（装备实例、Buff 实例），可按来源批量移除 | 按值删除无法区分来源；UI 无法显示“加成来自哪里” |
| 缓存 | 修正列表变化时置脏，GetValue 只在脏时重算 | 每次 DoDamage 调 7~9 次 GetValue（暴击时 9 次），每次遍历列表 |
| Buff 生命周期 | OnApply / OnTick（固定逻辑帧）/ OnRemove；持续时间、叠层上限、刷新策略（刷新时长 / 叠加时长 / 独立实例） | 点燃 / 冰冻 / 感电三套 bool + timer 各写一遍；周期伤害随帧率漂移 |
| 互斥与免疫 | Buff 带标签，规则表声明“A 存在时 B 不能上 / 覆盖 / 共存” | 互斥规则硬编码在 ApplyAilments 的三个 bool 表达式里 |
| 死亡与清理 | 死亡时移除所有 Buff（触发 OnRemove 恢复速度等） | 死亡时只把三个 bool 置假，变色 Invoke 和减速恢复仍在跑 |

## 7.9 面试追问

追问树

```
伤害公式是怎么算的？（Q7-1）
├─ 暴击率怎么判定？会不会有偏差？（Q7-2 → E3）
├─ 护甲用减法还是百分比？（Q7-3）
├─ 装备加属性怎么实现？卸下怎么撤销？（Q7-4）
│   └─ 百分比加成怎么叠？顺序怎么定？（7.8）
├─ Buff 系统怎么设计？冻结和减速同时存在怎么办？（Q7-5 → E8）
└─ 怪物成长曲线怎么定？（Q7-6 → E4）
```

### Q7-1 → Q7-2　伤害怎么算？暴击判定有没有偏差？

**L2**　闪避判定 → 基础伤害（攻击 + 力量）→ 暴击判定，暴击时乘 (150 + 力量)% → 减目标护甲（下限 0）→ 扣血，血量 ≤ 0 则死亡。

**L3 • 项目**　我测出暴击判定用了 `≤`，而 Random.Range(0,100) 返回 0~99，所以实际暴击率是 (c+1)%，暴击率 0 也有 1%；闪避用的是 `<`，是对的。两处写法不一致说明概率判定应该收敛到一个函数里。

**L4 • 攻击**　“1% 无所谓吧？” → 对普通战斗无所谓，但它破坏了“配置 0 = 绝不发生”的约定；而且同样的 off-by-one 我在掉落里也找到了：dropChance 填 0 的物品仍有 1% 掉率（第 10 章）。

### Q7-3　护甲用减法还是百分比?

**L2**　减法（伤害 − 护甲）直观，但低伤害攻击会被完全免疫、高伤害攻击护甲几乎无效，数值容易失控——项目里骷髅的伤害 0 减玩家护甲 10 就是 0。百分比减伤常用 `减伤 = armor / (armor + K)`（K 与等级相关），护甲收益递减、永远不会 100% 免伤，便于平衡。

**L3**　也可以混合：先百分比再减一个小的固定值，让“刮痧”攻击仍有最低伤害（例如至少 1 点或 5%）。

### Q7-4　装备加属性怎么实现?

**L2**　ItemData_Equipment 的 AddModifiers 把 14 项数值逐个加到玩家对应 Stat 的修正列表，RemoveModifiers 按值删除；EquipItem 先卸下同部位旧装备再穿新的。

**L3**　两个问题：装备数据（ScriptableObject）直接通过 PlayerManager 单例去改玩家组件，依赖是隐式的；修正没有来源，只能按值删。我会让修正带上来源（装备实例），由 PlayerStats 自己订阅“装备变化”事件来应用；同时上限变化时同步调整当前血量并通知 UI。

### Q7-5　冻结和减速同时存在怎么办?

**L2**　我原来的写法是每个效果直接给 moveSpeed、animator.speed 赋值，恢复时也赋“默认值”，于是冻结期间减速到期会直接解冻；多段冻结时第一个协程结束就解冻，我建模算过，旋转剑场景下应当全程冻结的时间里只冻住了 26%~85%。

**L3** 改成修正器栈：每个效果是一个带来源和到期时间的修正，最终速度 = 基础 × 所有修正之积；冻结、死亡这种“硬覆盖”放在最高优先级；同类效果按规则合并（冻结取最晚截止时间）。这样任何效果的加入和移除都不需要知道别的效果。

### Q7-6 怪物成长曲线怎么定?

**L2** 由曲线表驱动：等级 → 生命 / 伤害系数，策划在表里看到每一级的绝对值；代码只查表。常见是“分段线性 + 每 N 级一个台阶”，或者指数但有上限。

**L3 • 项目** 我的 Modify 循环里每级取 GetValue() × 25%，GetValue 包含了前几轮的修正，于是成了复利：10 级 7.44 倍而不是线性的 3.25 倍。复利不一定错，但必须是显式决定。

**常见错误回答**

**错误**：“我的 Buff 用协程实现，每个 Buff 开一个协程计时，到点恢复。”

**问题**：重叠的同类 Buff 会互相截断；恢复时“赋默认值”会抹掉其他 Buff；对象死亡或禁用时协程停止，恢复代码永远不执行；协程计时还受帧率量化。

**更好的回答**：“Buff 是数据：开始时间、持续时间、叠层、来源；由一个系统在固定逻辑帧统一推进和到期；属性由所有生效 Buff 推导，而不是被 Buff 改写。”

## 7.10 发散场景题

**场景 7-1 设计一个 Buff 系统，要求支持“中毒可叠 5 层、每层独立计时”“加速只刷新时长”“净化移除所有负面效果”。**

为什么会问：RPG / MOBA 客户端必考的系统设计题。

数据：BuffConfig（id、标签、持续、周期、叠层上限、叠加策略：Refresh / Extend / Independent、互斥组、属性修正列表、周期效果）。运行时：BuffInstance（config、来源、层数、开始帧、下次 tick 帧）。管理器按固定逻辑帧推进：到期移除、到点 tick、层数变化时重算属性修正。中毒用 Independent（每层一个实例，或一个实例记录每层的到期帧），加速用 Refresh，净化按“负面”标签批量移除。表现（图标、特效）订阅 Buff 增删事件，不参与逻辑。

### 场景 7-2　玩家抱怨“暴击率 30% 却连续 10 次不暴击”，怎么办?

为什么会问：真随机的体验问题，引出伪随机分布。

独立 30% 连续 10 次不暴击的概率是 0.7^10 ≈ 2.8%，玩得多了一定会遇到。可以用伪随机分布（PRD）：第 N 次未暴击时概率为 C × N，暴击后重置为 C，C 由目标期望 30% 反解（约 0.084），既保持期望又压缩连败。也可以用“抽卡式”保底：每 K 次至少一次。注意 PRD 需要每个单位独立维护计数，且必须由权威端计算。

### 场景 7-3　改成联网后，伤害在哪里算？客户端显示的伤害数字从哪来?

为什么会问：数值的权威性与防作弊。

状态同步：服务器算伤害（随机数只在服务器），客户端收到“命中事件（攻击者、受击者、结果、数值）”后播放表现，伤害数字以服务器下发为准；为了手感可以先播命中特效，数字晚一个 RTT 出现。帧同步：每个客户端算同一份确定性逻辑，随机数用同一个种子的确定性 RNG，数值用定点数——项目里的 `Mathf.RoundToInt(float)` 和 `UnityEngine.Random` 都不能用。

### 场景 7-4　策划想在 Excel 里配所有数值，你怎么接?

为什么会问：工具链与数据驱动。

Excel → 导出工具（校验类型、范围、外键）→ 生成 ScriptableObject 或二进制表 + 强类型访问代码；运行时只读。项目里散落在场景序列化字段的数值（玩家移速、骷髅攻击距离、技能参数）都应该进表，组件只保存“表 id”。导出时就能检查 dropChance 是否在 0~100、等级曲线是否单调，把 off-by-one 这类问题挡在运行之前。

**本章速记**

- E3：暴击 `≤` → **(c+1)%**；闪避 `<` 正确；冰冻破甲判了**攻击者**；RoundToInt 是银行家舍入。
- TakeDamage 无死亡保护 → 弹跳剑 / 雷击可让尸体**重复 Die、重复掉落**（#7）。
- E4：每级 +25% 当前值 = **复利**，10 级 7.44× vs 线性 3.25×。
- E8：多个写者赋绝对值 → 重叠冻结只冻住 **26%~85%**；改成截止时间取最大 + 速度修正器栈。

- DoMagicalDamage **没有调用方**；雷击伤害写死 1。

▶ **自测 1：为什么 critPower 在 Inspector 里配了也没用?**

▶ **自测 2：RemoveModifier 按值删除会算错吗?**

▶ **自测 3：E8 修正版为什么能 100% 冻结?**

CHAPTER 08

# 技能系统：冷却、冲刺分身与四模式飞剑

“技能系统怎么设计”是 Gameplay 岗的高频系统题。项目里的技能是“每个技能一个 MonoBehaviour + 一个控制器”，能跑，但能看到三类典型问题：名字叫 Can 的函数在消耗冷却、分身的攻击回调拿不到玩家、飞剑用七个 bool 拼出四种模式。

---

**本章目标**

- 说清 Skill 基类的冷却模型，以及 `CanUseSkill` 为什么违反“命令-查询分离”
- 找到分身攻击的空引用（P0）和朝向判断写进循环的 bug，并用 E11 量化后者
- 画出飞剑的生命周期，指出“bool 标志汤”的组合爆炸，给出策略模式的重构
- 能在白板上画一个数据驱动的技能系统，并对照 UE GAS 的五个核心概念

## 8.1 30 秒面试版 • 现状 vs 理想

**30 秒回答**

SkillManager 上挂着三个继承 `Skill` 的组件（冲刺、分身、飞剑），Skill 只管冷却：`CanUseSkill()` 冷却好了就直接 UseSkill 并重置冷却。冲刺每次生成一个淡出的分身；飞剑由 `SkillController_ThrowSword` 控制，支持普通、弹跳、穿刺、旋转四种模式和召回。问题有三：`CanUseSkill` 名为查询实为命令；分身配置成可攻击，但它的攻击回调用 `GetComponentInParent<Player>()` 找玩家，分身不是玩家的子物体，范围内有敌人就空引用；飞剑用 7 个 bool 组合模式，任何新模式都要改控制器的每个分支。重构方向是“技能 = 配置 + 运行时实例 + 时间轴任务”，飞剑模式改成可组合的行为对象。

**现状** → 问题 → **理想** **技能系统**

| 维度 | 当前做法（Demo 现状） | 存在的问题 | 理想重构方案 | 收益 / 代价 |
|---|---|---|---|---|
| **冷却接口** | `CanUseSkill`：检查即释放、即扣冷却 | P2 被状态守卫拒绝也扣冷却；名字叫 Can 却有副作用，读代码的人以为只是查询（#36） | CanActivate（只读）+ Commit（扣消耗与冷却） 设计方案 · 未实现 | 被状态守卫拒绝时不会白扣冷却 |
| **技能数据** | 冲刺速度 / 时长在 Player，冷却在组件，飞剑 20 个序列化字段 | 局限 参数散在三处，改一个技能要改场景里三个组件；无法热更、无法复用 | SkillConfig 资源（定义）+ SkillInstance（等级、冷却剩余、充能） 设计方案 · 未实现 | 新增技能是配置工作；可热更 |
| **伤害来源** | 分身用 GetComponentInParent 向上找玩家 | P0 分身攻击从未生效，范围内有敌人就空引用（#2） | 创建时注入攻击者属性快照（来源） 设计方案 · 未实现 | 分身攻击真正生效 |
| **模式扩展** | 枚举 + 两个 switch + 7 个 bool | P2 加一种剑要改两个 switch 再加 bool；SetupGravity 覆盖序列化字段；Update 不调 base，冷却永不减（#24） | 行为组合（伤害 / 插住 / 弹跳 / 穿刺 / 旋转 / 追踪）+ 配置资源 设计方案 · 未实现 | 新增一种剑只加一个行为类 |
| **释放入口** | 冲刺在 Player.Update 里全局检测，绕过任何状态检查 | P0 死亡、眩晕时照样能冲刺——正是“复活”的入口（#3） | 统一激活管线 + 阻断标签（Dead / Stunned） 设计方案 · 未实现 | 死亡、眩晕时不能放技能 |
| **对象生成** | 分身、飞剑每次 Instantiate / Destroy | 局限 频繁生成 / 销毁带来分配与卡顿尖刺；销毁后的旧引用变成 fake null | 对象池 + 代数句柄（第 12 章） 设计方案 · 未实现 | 零分配、旧引用安全 |

现状问题：P0 ×2　P2 ×2　局限 ×2　（P0~P3 编号见附录 A，分级说明见 0.2）

## 8.2 Skill 基类：冷却与“Can”的副作用

当前源码 • 822270c Skill.cs:16-43

```
protected virtual void Start()  { player = PlayerManager.Instance.player; }
protected virtual void Update() { coolDownTimer -= Time.deltaTime; }       // 一直往负数减，从

public virtual bool CanUseSkill()
{
    if (coolDownTimer < 0)
    {
        UseSkill();                          // ← 名字是"能不能用"，实际"已经用了"
        coolDownTimer = coolDown;
        return true;
    }
    Debug.Log("Skill [" + skillName + "] is on CoolDown");    // 每次失败都拼字符串 + 打日志
    return false;
}
```

**现状问题** P2 CanUseSkill 检查即释放、即扣冷却（命令与查询不分离，#36）；冷却一直往负数减；子类不调 base 时冷却永不减（#24）

| 问题 | 源码事实 | 后果 |
| --- | --- | --- |
| 命令-查询不分离 | `CanUseSkill` 调用即消耗冷却 | 调用方无法“先问再决定”。加了状态守卫后（第 3 章），如果冲刺请求被拒绝，冷却已经白白消耗。正确的形状是 `bool TryActivate()`（名字说明会执行）或 `CanActivate()` + `Commit()` 两步 |
| 子类重写 Update 没调 base | `Skill_ThrowSword.Update` 完全重写（Skill_ThrowSword.cs:75-89） | 飞剑的 coolDownTimer 永不递减；目前掷剑不经过冷却检查所以没暴露，一旦接入就是“永远在冷却” |
| 每帧改写序列化字段 | `SetupGravity()` 每帧把 bounce / pierce / spin 的重力写进 `swordGravityScale`；Regular 分支什么都不做 | 运行中从 Pierce（重力 1）切回 Regular，重力停在 1 而不是原来的 4.5——原始值已被覆盖 |
| 冷却失败的日志 | 字符串拼接 + Debug.Log | 每次按键都分配并写日志（打包后 Debug.Log 依然执行，见第 12 章） |
| 未使用的代码 | `FindClosestEnemy` 没有调用方；分身和雷击各写了一份“找最近敌人” | 同一逻辑三份实现，修 bug 要修三处（其中一份就有 E11 的 bug） |

## 8.3 冲刺分身：一个 P0 和一个循环里的 bug

当前源码 • 822270c SkillController_Clone.cs:33-96（精简）

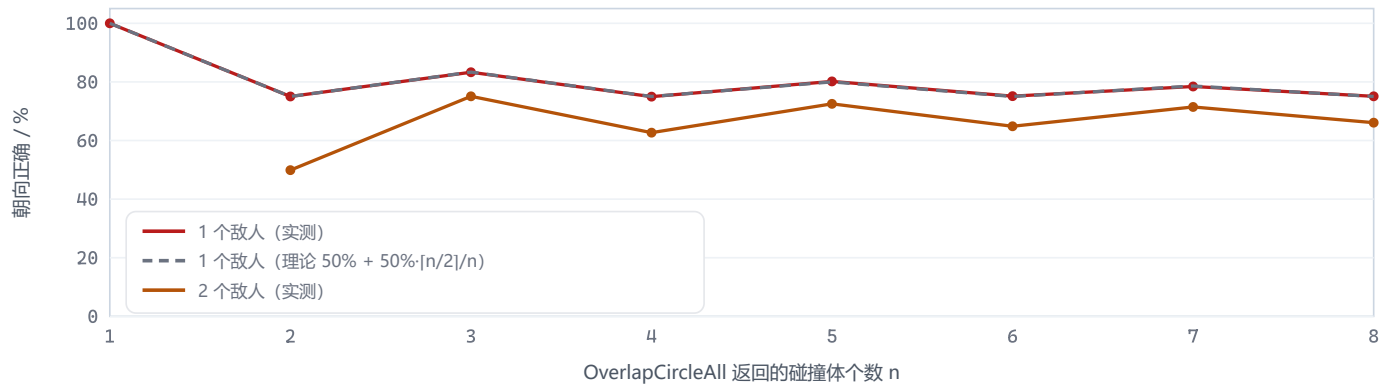
```
public void SetupClone(Transform _newTranform, float _cloneDuration, float _colorLosingSpeed,
{
    transform.position = _newTranform.position;      // 只拷了位置，没拷朝向
    cloneTimer = _cloneDuration;
    if (canAttack)                                    // 场景里 Skill_Clone.canAttack = 1
    {
        FaceClosesttTarget();
        animator.SetInteger("AttackNumber", Random.Range(1, 4));    // 随机播 1~3 段攻击
    }
}
private Player player ⇒ GetComponentInParent<Player>();          // 分身是 Instantiate(cloneP
private void AttackTrigger()                                      // 攻击片段 0.083~0.357 s 处
{
    foreach (var c in Physics2D.OverlapCircleAll(attackCheck.position, attackCheckRadius))
        if (c.GetComponent<Enemy>() is Enemy enemy)
        {
            player.stats.DoDamage(enemy.stats);        // ← player 为 null：NullReferenceExcep
            enemy.DamageEffect();
        }
}
private void FaceClosesttTarget()
{
    float closestDistance = Mathf.Infinity;
    foreach (Collider2D collider in Physics2D.OverlapCircleAll(transform.position, 25))
    {
        Enemy enemy = collider.GetComponent<Enemy>();
        if (enemy ≠ null && Vector2.Distance(transform.position, enemy.transform.position) <
        { closestDistance = ...; closetEnemy = enemy.transform; }
        if (closetEnemy ≠ null && transform.position.x > closetEnemy.position.x)
            transform.Rotate(0, 180, 0);              // ← 写在 foreach 里：之后每遍历一个碰撞体
    }
}
```

**现状问题** P0 用 GetComponentInParent 找玩家，独立生成的分身上必为 null：分身攻击从未生效、打到敌人就空引用（#2）；朝向判断写进循环（E11，#23）

问题 P0 分身由 `Instantiate(clonePrefab)` 生成，没有父物体，Clone.prefab 上也没有 Player 组件（已查 prefab：只有 SpriteRenderer、Animator、SkillController_Clone 和一个 AttackCheck 子物体）。`GetComponentInParent<Player>()` 必然返回 null。因为空引用发生在 `if (enemy ≠ null)` 里面，**只有分身攻击范围内有敌人时才会抛异常**——也就是“分身攻击从来没打出过伤害，而且每次打到敌

人都报错”。教程原版在 SetupClone 里把 player 作为参数传进来，这个项目改成了属性查找。修法：Skill_Clone 在 CreatClone 时把 Player（或者攻击者的 CharacterStats）传进去。

问题 P2 FaceClosesttTarget 把旋转写进了循环：一旦找到最近的敌人，之后每遍历一个碰撞体（地面、血条触发器、掉落物……OverlapCircleAll 半径 25 且没有过滤），只要“分身在敌人右边”就再转 180°，最终朝向取决于剩余碰撞体个数的奇偶。实测 E11 （每个 n 跑 20 万次，敌人位置和碰撞体顺序随机）：

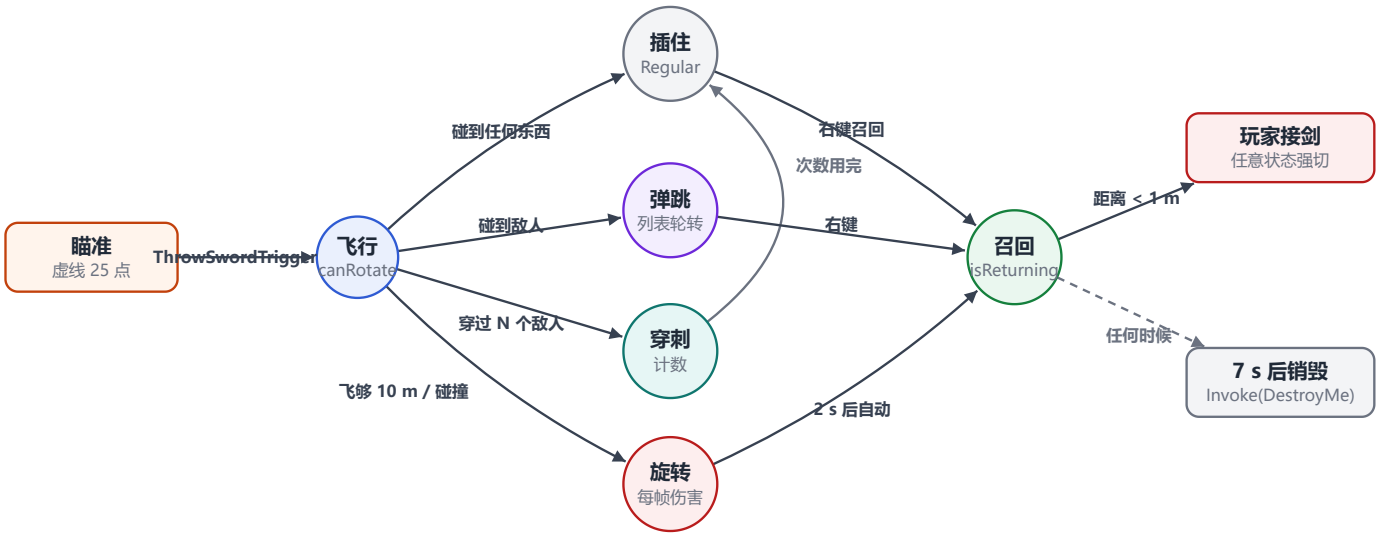



现状 图 8-1　分身朝向正确率（E11）。敌人在右边时不会转，总是对的；敌人在左边时转 n − i 次（i 是敌人在数组中的位置），奇数次才对。场景里 25 m 内光地面、背景、触发器就有多个碰撞体，正确率在 50%~83% 之间跳动

现状问题 P2 朝向判断写在循环里，正确率随碰撞体数量跳动（E11，#23）

还有一个表现问题 推断 ：SetupClone 只拷贝了位置没拷贝朝向，分身默认朝右；玩家向左冲刺且附近没有敌人时（不进入 FaceClosesttTarget 的旋转），残影朝向与玩家相反。最后，每次冲刺都 Instantiate 一个分身、淡出后 Destroy——冲刺冷却 1 s，频率不高，但它是对象池最典型的适用场景（第 12 章）。

## 8.4 飞剑：七个 bool 拼出四种模式




现状 图 8-2　飞剑生命周期（SkillController_ThrowSword.cs）。四种模式不是四个状态，而是 isRegular / isBouncing / isPiercing / isSpinning 四个 bool，加上 isReturning / isSpinStoped / canRotate，共 7 个标志在 Update 和 OnTriggerEnter2D 里交叉判断

**现状问题** P2 7 个 bool 标志交叉判断，加一种剑要改多处；子类 Update 不调 base，冷却永不减（#24）

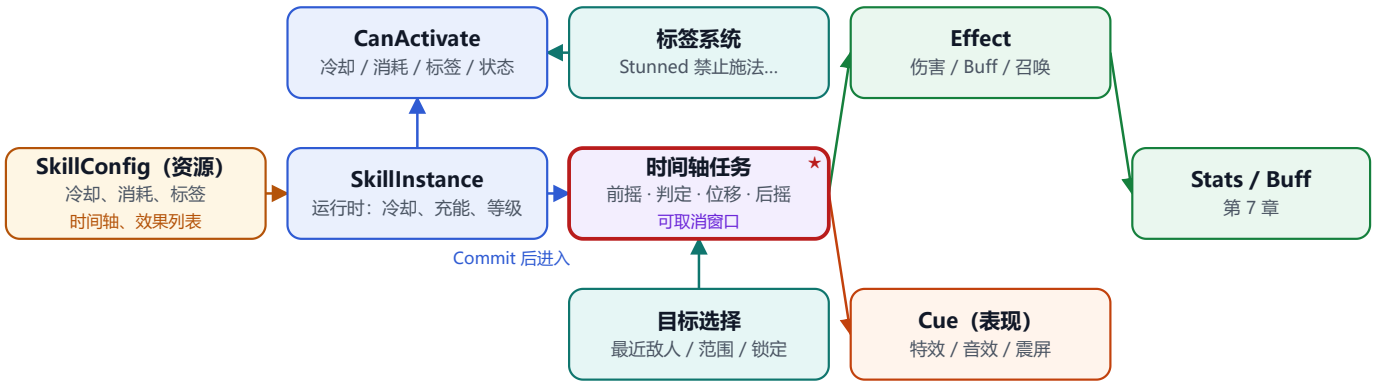
| 模式 | 关键逻辑 | 问题 |
| --- | --- | --- |
| Regular | OnTriggerEnter2D → 伤害 + 冻结 0.7 s → StuckInto：关碰撞、变运动学、挂到被击中物体下 | 挂到敌人下后，敌人翻转时剑跟着转；敌人若被销毁，剑作为子物体一起被销毁 |
| Bounce | 首次碰到敌人时收集 10 m 内所有敌人，按列表轮流飞过去，共 4 次 | 只有 1 个敌人时（`Count > 1` 不成立）直接插住；列表里的敌人死亡后仍会被“弹到”（重复 Die，第 7 章）；弹完后剑悬在半空、碰撞体重新打开 |
| Pierce | 每碰到一个敌人伤害一次、计数减一；计数用完后插住 | 计数在 StuckInto 里减，伤害在 OnTriggerEnter2D 里——两个函数共同决定一个规则 |
| Spin | 飞出 10 m 或碰撞后停下，2 s 内每 hitCoolDown 对 1 m 内敌人伤害 + 冻结，然后自动召回 | hitCoolDown 场景值 0 → 命中数 = 帧数（E10）；每次命中开一个冻结协程（E8）；停住后直接改 transform.position 而不是 rb.MovePosition |
| 召回 | MoveTowards 玩家，距离 < 1 m 时 `player.CatchTheSword()` | 无视墙体穿墙飞回；CatchTheSword 从任何状态（包括死亡）强切（第 3 章） |

Setup 的调用顺序也是隐式契约：`SpawnSword` 必须先调 `SetupBounce / SetupPierce / SetupSpin` 再调 `SetupSword`，因为 SetupSword 里要读 `isPiercing` 决定是否播放旋转动画。7 个 bool 理论上有 128 种组合，合法的只有十来种，非法组合没有任何东西阻止（例如 isRegular 默认就是 true，只有 Setup 系列函数会把它置假）。

**理想方案 · 骨架** 飞剑模式改为可组合的行为（C#，未在 Unity 中编译）

```
public interface ISwordBehavior
{
    void OnLaunch(Sword s) { }
    bool OnHit(Sword s, Collider2D other);    // 返回 true 表示“这一下消耗掉了”，继续交给下一个行为
    void Tick(Sword s, float dt) { }
}
// 普通剑 = [Damage, StickInto]；穿刺剑 = [Damage, Pierce(n), StickInto]
// 弹跳剑 = [Damage, Bounce(n, radius)]；旋转剑 = [SpinAfter(distance, duration, hitInterval)]
// 新增“追踪剑”= [Homing(turnRate), Damage, StickInto]——只加一个类，不碰其他模式
[CreateAssetMenu] public class SwordConfig : ScriptableObject
{
    public float gravity, launchSpeed, freezeTime;
    [SerializeReference] public List<ISwordBehavior> behaviors;    // 由策划在 Inspector 里拼
}
```

## 8.5 数据驱动的技能系统：白板版




**理想方案** 图 8-3　数据驱动技能系统的五个部件。对照 UE GAS：SkillConfig/Instance ≈ GameplayAbility（CDO / 实例），Effect ≈ GameplayEffect，标签 ≈ GameplayTag，时间轴任务 ≈ AbilityTask，Cue ≈ GameplayCue

| 项目里的写法 | 放进这个系统后 |
| --- | --- |
| Skill_Dash 的冷却 1 s 在组件字段；冲刺时长 0.2 s 在 Player 字段；冲刺速度 25 在 Player 字段 | 都在 DashConfig 里；时长与位移由时间轴的“位移轨道”定义（并且在固定步里执行，第 5 章） |
| Skill_Clone 由 DashState.Enter 直接调用 | 冲刺技能的时间轴第 0 帧挂一个“生成残影”效果，残影是否攻击是效果的参数 |
| 飞剑参数 20 个序列化字段 + swordType 枚举 + switch | 四个 SwordConfig 资源，切换模式 = 换资源 |
| “能否施放”散落在 GroundedState 的 if 和 HasNoSword 里 | CanActivate 统一检查：冷却、标签（Dead、Stunned）、外部条件（剑不在手上） |

## 8.6 面试追问

追问树

```
技能系统怎么设计的？（Q8-1）
├─ 冷却放在哪？为什么 CanUseSkill 会扣冷却？（Q8-2）
├─ 分身怎么拿到玩家的属性？（Q8-3 → P0）
├─ 飞剑四种模式怎么扩展？加一种追踪剑要改几处？（Q8-4）
├─ 技能能不能被打断？打断了冷却怎么算？（Q8-5）
└─ 和 GAS 比你的系统缺什么？（Q8-6）
```

### Q8-1 → Q8-2　技能系统怎么设计？冷却为什么在 CanUseSkill 里扣？

**L2**　Skill 基类只有冷却：Update 里递减，CanUseSkill 冷却好了就执行 UseSkill 并重置冷却，返回 true。冲刺、分身、飞剑各是一个子类组件，挂在 SkillManager 上。

**L3**　这个接口的问题是查询带副作用：调用方无法“先问能不能、再决定要不要”。比如冲刺被状态守卫拒绝时，冷却已经扣了。我会改成 TryActivate（名字表明会执行），或者 CanActivate + Commit 两步，Commit 才扣冷却和资源——GAS 的 CommitAbility 就是这个设计。

### Q8-3　分身怎么拿到玩家的属性来算伤害？

**L2** 现在是 `GetComponentInParent<Player>()`，但分身是独立生成的根物体，找不到玩家，范围内有敌人时空引用。这是我复盘时发现的最严重的问题之一：分身的攻击动画在播，却从来没有造成过伤害。

**L3** 正确的做法是创建时注入攻击者的属性（或者一个“伤害来源”对象），而且最好是**快照**：分身存在期间玩家换了装备，分身的伤害按创建时还是按当前？这是一个需要策划决定的规则，代码要能表达两种。

### Q8-4 加一种“追踪剑”要改几处?

**L2** 现在要改：SwordType 枚举、Skill_ThrowSword 的 SetupGravity 和 SpawnSword 两个 switch、控制器加 isHoming 标志、Update 加分支、OnTriggerEnter2D 和 StuckInto 里所有“isXxx”判断都要检查是否与新模式冲突——至少 6 处，而且每处都可能破坏已有模式。

**L3** 改成行为列表后，追踪剑 = [Homing, Damage, StickInto]，只新增一个 Homing 类和一个配置资源。这就是开闭原则：对扩展开放、对修改关闭。

### Q8-5 技能能不能被打断？打断了冷却怎么算?

**L2** 现状：冲刺可以打断一切（全局检测），瞄准被打断时仍会掷剑（第 4 章）。规则应该写进技能配置：前摇阶段被打断是否返还冷却 / 资源、后摇阶段是否可被移动取消。常见做法是“Commit 发生在前摇结束、判定之前”，所以前摇被打断不扣冷却。

### Q8-6 和 UE GAS 比，你的系统缺什么?

**L2** 缺四样：① 标签系统（技能之间的阻断、取消、需求关系）；② 效果（GameplayEffect）把“改属性”统一成数据，带持续时间、叠加、修正器；③ AbilityTask 这样的异步任务，把“等动画事件”“等输入”“等目标选择”写成可组合的步骤；④ 预测与网络复制。我的项目里这四样分别散落在状态机 if、ItemData_Equipment.AddModifiers、动画事件回调和……没有（单机）。

**常见错误回答**

**错误**：“我的飞剑支持四种模式，用了策略模式。”

**问题**：源码是一个枚举 + 两个 switch + 七个 bool，不是策略模式；面试官只要问“新增一种模式要改哪里”就会暴露。

**更好的回答**：“现在是枚举 + 标志位实现的，扩展性很差；我设计过改成行为组合的方案，新增模式只加一个类。”——把“做了什么”和“应该怎么做”分开说。

## 8.7 发散场景题

### 场景 8-1　加入“冷却缩减 30%”的装备和“可存 2 次的冲刺”，你怎么改?

为什么会问：冷却模型是技能系统的基础。

冷却时长不再是常量，而是 `baseCooldown × (1 − CDR)`，CDR 来自属性系统（带上限，例如 40%）；冷却缩减变化时，正在冷却的技能是按比例缩短剩余时间还是只影响下一次，要定规则。充能：SkillInstance 存 `charges` 与 `rechargeTimer`，使用时 charges − 1，计时器在 charges < max 时运行、到点 +1（累加式计时，第 5 章）。UI 显示层数和环形进度。

### 场景 8-2　技能 Z 的效果是“3 秒内下一次普攻附带火焰伤害”，放在哪里实现?

为什么会问：考察技能与 Buff 的边界。

技能 Z 只做一件事：给自己上一个 3 秒的 Buff“火焰附魔”（带“下一次普攻”计数 1）；普攻的伤害管线在构造 DamageInfo 时询问身上的 Buff，附魔 Buff 往 DamageInfo.fire 里加数值并消耗计数。这样技能、普攻、Buff 三者互相不知道具体实现，只通过 DamageInfo 和标签交流。

### 场景 8-3　联机时，飞剑由谁生成?命中由谁判?

为什么会问：投射物同步是联机技能的难点。

状态同步常见做法：客户端本地立刻生成一个“预测投射物”（只做表现），同时把“掷剑（方向、时间戳）”发给服务器；服务器生成权威投射物、判定命中、复制给其他客户端；本地预测物在服务器结果到达后与权威物对齐或替换。轨迹可预测的投射物（抛物线）甚至不需要逐帧复制位置，只复制发射参数，各端自己算（这时 E9 的离散 / 连续一致性就很重要）。

**场景 8-4　策划想在不发版的情况下调技能数值，怎么支持?**

为什么会问：热更新与配置管线。

数值必须先从组件字段里拿出来（SkillConfig 资源或表格），再走热更通道：资源热更（Addressables / AssetBundle 替换配置资源）或表格热更（下载新的数据表）。逻辑代码不变，只换数据；版本号与校验保证客户端和服务器用同一份配置。项目现在的数值都在场景里，热更只能整场景替换。

**方法论延伸**

技能与背包、装备如何统一成“配置 / 实例 / 规则 / 表现”四层，并通过属性修正、效果、标签三条通道协作，见第 21 章。

**本章速记**

- `CanUseSkill` = 查询 + 命令；应为 **TryActivate** 或 **CanActivate + Commit。** Skill_ThrowSword.Update 没调 base → 冷却永不递减。
- 分身攻击：`GetComponentInParent<Player>()` 在独立生成的分身上为 null → **有敌人就空引用、从未造成伤害**（#2，P0）。
- E11：旋转写进 foreach → 朝向正确率 75%~83%（n = 2~3 时）。
- 飞剑：**7 个 bool 的标志汤** → 改成行为组合 + 配置资源；数据驱动技能 ≈ GAS 的 Ability / Effect / Tag / Task / Cue。

▶ **自测 1：为什么分身的空引用在测试时不容易发现?**

▶ **自测 2：弹跳剑只有一个敌人时会怎样?**

▶ **自测 3：SpawnSword 里 Setup 函数的调用顺序为什么重要?**

CHAPTER 09

# 敌人 AI：骷髅的 6 个状态与“自己切到自己”

骷髅 AI 只有 6 个状态，却集中暴露了“用继承复用转移逻辑”的副作用：战斗状态继承了地面状态的“发现玩家 → 进入战斗”，于是看见玩家的每一帧都在自己切换到自己，顺带执行一次 GameObject.Find；战斗计时器每帧被重置，永远不会超时。

---

**本章目标**

- 画出骷髅状态图，说清感知（射线 + 2 m 圆）、追击、攻击冷却、眩晕、死亡的规则
- 用 E7 讲清自转移的成因与代价：10 s 内自转移 252 次、Find 258 次
- 理解“战斗永不超时 + 无边缘检测 + 射线不被墙挡”组合出的“平台下抽搐”
- 能设计可扩展到百只怪物的 AI：行为树 / 感知系统 / AI LOD / 攻击令牌

## 9.1 30 秒面试版 • 现状 vs 理想

**30 秒回答**

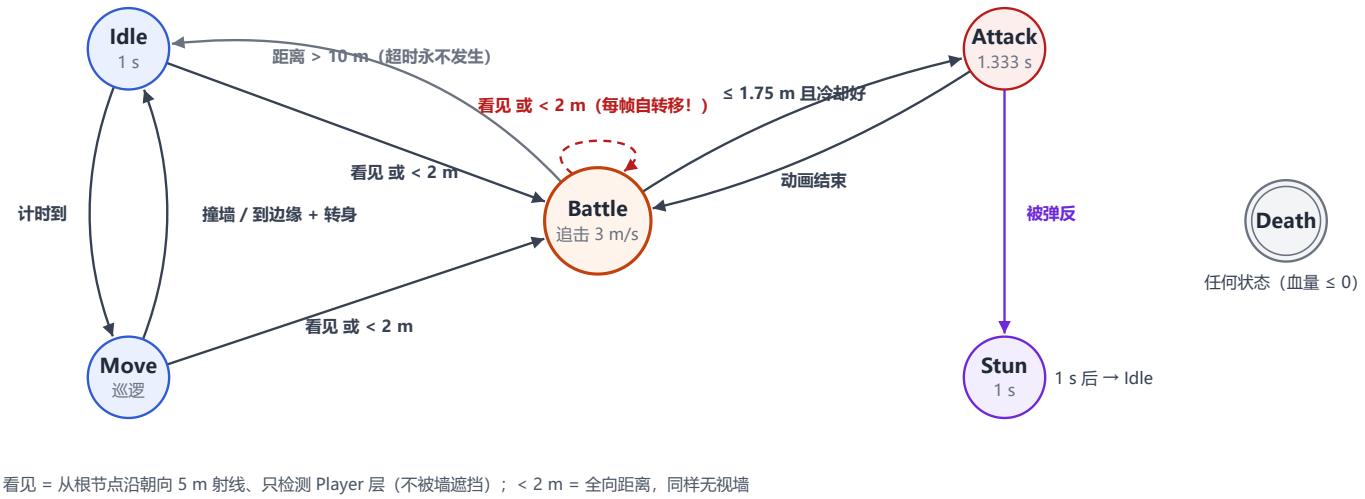
骷髅在 Idle 和 Move 之间巡逻（撞墙或到边缘就转身），从前方 5 m 射线或 2 m 范围发现玩家后进入 Battle：追击、进入 1.75 m 攻击距离且冷却好了就攻击，攻击动画里开关弹反窗口，被弹反则眩晕，血量归零进入死亡。问题出在 Battle 继承了 GroundState：父类的“发现玩家就切到 Battle”在 Battle 自己身上也生效，于是每帧 Exit + Enter 一次（Enter 里还有 GameObject.Find），实测 10 s 自转移 252 次；Battle 的超时计时器每帧被重置，玩家跳到高处后骷髅会在他脚下永远来回抽搐，10 s 转身 547 次。修法：Battle 不继承 GroundState、计时器只在看见玩家时续命；更进一步用行为树和感知组件。

现状 → 问题 → 理想　**敌人 AI**

| 维度 | 当前做法（Demo 现状） | 存在的问题 | 理想重构方案 | 收益 / 代价 |
|---|---|---|---|---|
| **状态复用** | Battle 继承 GroundState 的“看见玩家 → 切到 Battle”；Enter 里 GameObject.Find 找玩家 | P1 看见玩家时每秒约 26 次无意义的 Exit / Enter + 全场景 Find（E7，#8） | 状态机拒绝自转移 / Battle 不继承该转移；玩家引用注入<br>**模型验证：E7 理想版自转移 0** | Find 从 26 次/秒降到 0.6 次/秒 |
| **脱战** | battle 计时每帧被重置 | P1 玩家跳上平台后骷髅永不放弃，10 s 在平台下转身 547 次（E7-C，#9） | 只在看见玩家时续命，Enter 初始化计时<br>**模型验证：E7 理想版第 5.7 s 脱战** | 失去目标后会放弃 |
| **感知** | 前方 5 m 射线只检测 Player 层（穿墙）+ 2 m 全向圆 | P1 隔墙也能发现玩家；背后 2 m 内一律被发现（#9） | 视野 + 遮挡射线 + 最后看见位置，写入黑板，降频分帧<br>**设计方案 · 未实现** | 不再隔墙发现；能做搜索行为 |
| **追击** | 没有边缘检测，会追下平台 | P1 会追下平台 / 悬崖（#9） | 追击前判断可达性；移动层带边缘检测<br>**设计方案 · 未实现** | 不追下悬崖 |
| **决策结构** | 6 状态 FSM，状态类与具体怪物强绑定 | 局限 状态类绑死骷髅：第二种怪物要复制整套状态；做不了“谁先上”的群体协调与 AI LOD | 感知 → 黑板 → 行为树 → 动作（复用技能系统）+ 攻击令牌 + AI LOD<br>**设计方案 · 未实现** | 弓箭手、Boss 复用节点；百怪可控 |
| **死亡** | 尸体穿地坠落，不销毁、不回收 | P2 尸体穿地坠落后 Update 照跑，刷怪场景会不断累积（#32） | 死亡动画后回收入池<br>**设计方案 · 未实现** | 刷怪场景不累积“幽灵” |

现状问题：P1 ×4　P2 ×1　局限 ×1　（P0~P3 编号见附录 A，分级说明见 0.2）

## 9.2 状态图与感知规则




看见 = 从根节点沿朝向 5 m 射线、只检测 Player 层（不被墙遮挡）；< 2 m = 全向距离，同样无视墙

**现状** 图 9-1　骷髅状态图（Enemy_Skeleton*.cs）。红色虚线自环是本章的主角：Battle 继承了 GroundState 的转移，看见玩家的每一帧都会 Exit + Enter 自己

**现状问题** `P1` Battle 每帧自转移（E7，#8）；战斗计时每帧重置、永不脱战（#9）

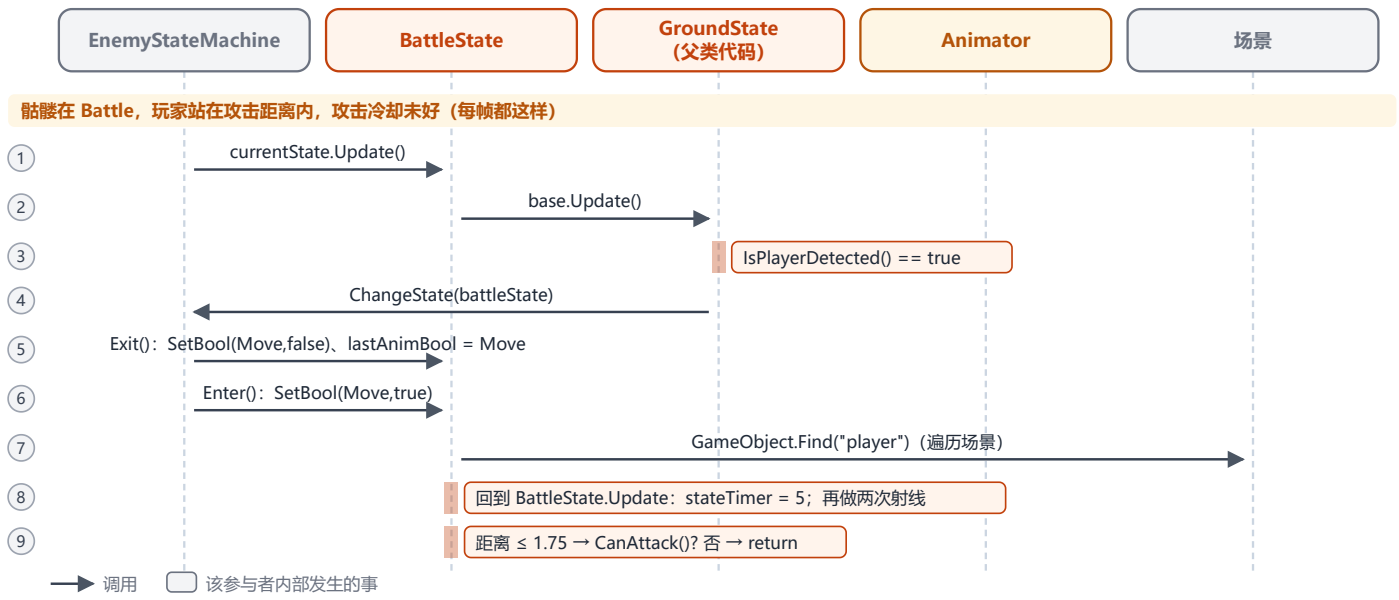
| 规则 | 源码 | 备注 |
|---|---|---|
| 视觉 | `Raycast(wallCheck.position, right * facinDir, 5, whatIsPlayer)`；wallCheck 就是骷髅根节点 | LayerMask 只含 Player 层，**射线穿墙**；只看前方 |
| 近身感知 | `Vector2.Distance( ... ) < 2` | 全向、穿墙 |
| 追击 | 速度 moveSpeed × 1.5 = 3 m/s，朝玩家 x 方向 | **没有边缘 / 墙检测**：会追下平台（Move 状态有边缘检测） |
| 攻击 | 射线命中距离 ≤ 1.75 且 `Time.time ≥ lastAttackTime + 1` | lastAttackTime 在 Attack.Exit 写入；冷却从动画结束算起 |
| 退出战斗 | 没看见时：`stateTimer < 0` 或 距离 > 10 | stateTimer 在同一函数开头被设为 battleTime → 第一个条件永远为假 |
| 死亡 | Death.Enter：播 Idle + 上一个动画 bool、animator.speed = 0、关碰撞体，0.1 s 内向上 10 m/s | 之后以重力倍率 12 穿过地面一直掉，**从不销毁**，Update 照跑（#32） |

## 9.3 E7：自转移与 Find

当前源码 • 822270c Enemy_SkeletonGroundState.cs:14-34 · Enemy_SkeletonBattleState.cs:15-57（精简）

```
public class Enemy_SkeletonGroundState : EnemyState
{
    public override void Enter() { base.Enter(); playerTransform = GameObject.Find("player").
    public override void Update()
    {
        base.Update();
        if (enemy.IsPlayerDetected() || Vector2.Distance(enemy.transform.position, playerTran
        { stateMachine.ChangeState(enemy.battleState); return; }      // ← 当前就是 battleStat
    }
}
public class Enemy_SkeletonBattleState : Enemy_SkeletonGroundState    // ← 继承了上面的转移
{
    public override void Enter() { base.Enter(); playerTransform = PlayerManager.Instance.pla
    public override void Update()
    {
        base.Update();                                  // 可能刚刚 Exit + Enter 了自己；returr
        stateTimer = enemy.battleTime;                  // ← 每帧重置，下面的 stateTimer < 0 永
        if (enemy.IsPlayerDetected())
        {
            if (enemy.IsPlayerDetected().distance ≤ enemy.attackRange) { ...; if (CanAttack(
        }
        else if (stateTimer < 0 || Vector2.Distance(playerTransform.position, enemy.transform
        { stateMachine.ChangeState(enemy.idleState); return; }
        // 追击：没有边缘检测
        enemy.SetVelocity(enemy.moveSpeed * moveDir * 1.5f, rb.velocity.y);
    }
}
```

**现状问题** P1 Battle 继承了“看见玩家 → 切到 Battle”：每帧自己切到自己，Enter 里还要 GameObject.Find（E7，#8）；战斗计时每帧被重置，永不脱战（#9）




**现状** 图 9-2　看见玩家时 Battle 的一帧：一次无意义的 Exit / Enter、一次 GameObject.Find、三次 Raycast。Animator 的 Move 在同一帧被置假又置真，不会触发转移，所以画面上看不出来

**现状问题** P1 每帧一次无意义的 Exit / Enter + 全场景 Find（E7，#8 #34）

用 C++ 逐行移植 4 个状态类，在 1D 平台上跑三个场景（60 fps，遭遇发生在开局 5 s 后） 实测 E7

**模型**：

| 场景 | 版本 | 时长 / s | Battle.Enter 次数 | 其中自转移 | GameObject.Find | Find / 秒 | 出手次数 |
|---|---|---|---|---|---|---|---|
| A 玩家站在前方 2.5 m | 现状（源码） | 10 | 257 | 252 | 258 | 25.8 | 5 |
| A 玩家站在前方 2.5 m | 理想（修正版） | 10 | 5 | 0 | 6 | 0.6 | 5 |
| B 玩家站在身后 1.5 m | 现状（源码） | 10 | 250 | 245 | 251 | 25.1 | 5 |
| C 3 s 后玩家跳上 4 m 高平台 | 现状（源码） | 13 | 72 | 69 | 73 | 5.6 | 2 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| C 3 s 后玩家跳上 4 m 高平台 | 理想（修正版） | 13 | 3 | 0 | 6 | 0.5 | 2 |

表 9-1　E7 汇总。理想（修正版）= Battle 不继承自转移 + Enter 初始化计时器 + 只在看见玩家时续命。出手次数两版相同——自转移不改变行为，只浪费 CPU，所以测不出来；“一帧两次转移”发生在冷却到点那一帧（自转移 + 进入攻击）

每秒 26 次 GameObject.Find 对一只骷髅不算什么，但它的成本与场景对象数成正比、与怪物数成正比：100 只骷髅同时进入战斗、场景里有几千个对象时，这就是每帧几百次全场景遍历。更关键的是它说明了一个结构性问题：**“用继承复用转移规则”会让规则作用到你没想到的状态上**，而且因为行为不变，任何功能测试都发现不了。

## 9.4 平台下的抽搐：三个小问题叠出的大 bug

场景 C：前 3 s 玩家在骷髅前方地面上，骷髅进入战斗并出手；随后玩家跳上骷髅头顶 4 m 高的平台（水平偏移 0.5 m）。此时：射线是水平的，看不见玩家；直线距离约 4 m，不满足 < 2 m 的近身感知，也不满足 > 10 m 的脱战条件；超时条件永远为假。于是：

| 版本 | 跳上平台后仍在追击 / s | 结局 | 这段时间转身次数 | 相对玩家的 x 范围 / m |
|---|---|---|---|---|
| 现状（源码） | 10.0 | 从不放弃 | 547 | -0.50 ~ 0.00 |
| 理想（修正版） | 5.7 | 第 5.7 s 放弃 | 289 | -9.68 ~ 0.00 |

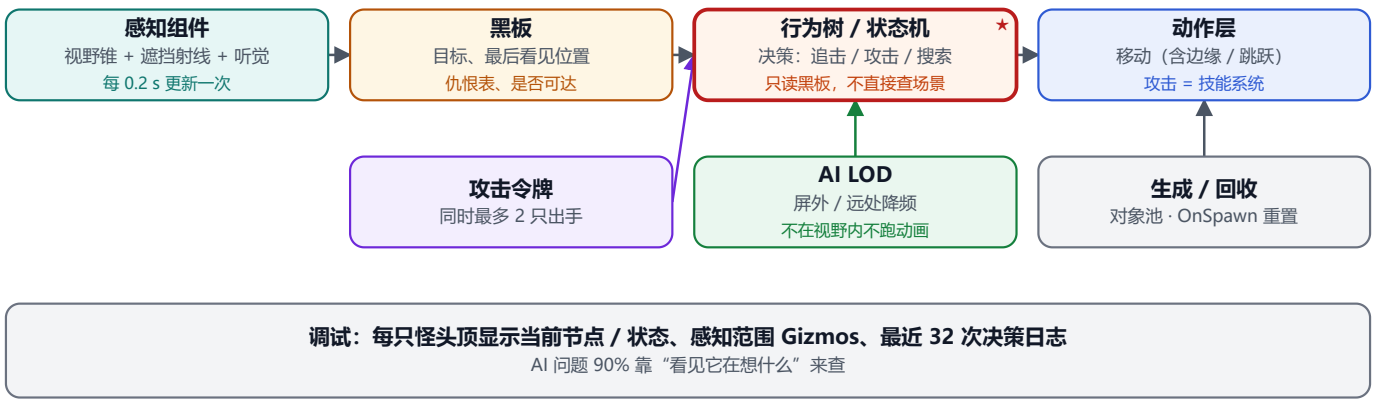
表 9-2　E7 场景 C：源码版在玩家脚下 0.5 m 范围内每帧转身（追击速度 3 m/s，一帧 5 cm，越过玩家 x 就掉头），10 s 转身 547 次；修正版 5 s 后脱战回到巡逻

真实游戏里还会叠加一条：Battle 没有边缘检测，如果平台下方是悬崖，骷髅会直接追下去。三个问题单看都很小——计时器写错一行、追击少一个检测、射线只看水平——组合起来就是玩家截图发论坛的那种 bug。

## 9.5 其余细节

- **两种拿玩家的方式**：GroundState 用 `GameObject.Find("player")`（依赖对象名，改名即空引用），BattleState 用 `PlayerManager.Instance`；BattleState 又声明了一个同名私有字段 `playerTransform` 隐藏了父类字段。同一个类层次里三份 `enemy` 字段（基类 enemyBase、Ground 的 enemy、Idle / Death / Stun 各自又声明一次）。
- **Idle / Move 同样有“切走后继续执行”**：Idle.Update 在 base.Update 切到 Battle 之后仍会检查自己的计时器并可能切到 Move；Move 切走后仍 SetVelocity 并检查边缘（第 3 章同一个模式）。
- **眩晕在对方的回调里发生**：弹反成功时，骷髅的状态是在**它自己的 AttackTrigger 动画事件**执行过程中被切到 Stun 的；回调返回后，同一时刻的 CloseCounterAttackWindow 仍会执行。目前无害，但“在回调里改变调用者的状态”是重入问题的温床。
- **死亡不回收**：尸体关掉碰撞体后穿地坠落，GameObject、EnemyStats.Update、血条都还在；刷怪时就是线性增长的“幽灵”。

## 9.6 改进：从 FSM 到可扩展的 AI




理想方案 图 9-3　可扩展的 AI 分层：感知写黑板、决策读黑板、动作复用技能系统；令牌和 LOD 控制“同时有多少只在认真思考 / 出手”

## 9.7 面试追问

追问树

```
怪物 AI 怎么写？（Q9-1）
├─ 状态之间怎么复用代码？有没有副作用？（Q9-2 → E7）
├─ 怪物怎么发现玩家？隔着墙能看见吗？（Q9-3）
├─ 追不到玩家时怎么办？（Q9-4 → 场景 C）
├─ 为什么用状态机不用行为树？（Q9-5）
└─ 100 只怪同屏怎么保证性能？（场景 9-1）
```

## Q9-1 → Q9-2　怪物 AI 怎么写？状态复用有没有副作用？

**L2**　和玩家一样是状态类：Idle / Move 巡逻，GroundState 父类负责“发现玩家进入战斗”，Battle 追击并在距离与冷却满足时攻击。

**L3 • 项目**　我把 Battle 也继承了 GroundState，结果父类的“发现玩家 → 切到 Battle”对 Battle 自己也生效，每帧 Exit + Enter 一次，Enter 里还有 GameObject.Find。行为完全正常，所以功能测试测不出来；我是用 C++ 移植逐帧计数才发现的：10 s 自转移 252 次。修法是让状态机拒绝自转移（或 Battle 不继承），并把 Find 换成注入。

**L4 • 攻击**　“状态机拒绝自转移会不会影响需要重入的状态？” → 需要“重新开始”的状态（比如连续两次受击硬直）应该显式调用 Restart，而不是靠 ChangeState(this)；把这两种语义分开，比依赖隐式行为安全。

## Q9-3　怪物怎么发现玩家？隔着墙能看见吗？

**L2**　前方 5 m 的射线（只检测 Player 层）加 2 m 的全向距离。因为射线的 LayerMask 只有 Player，墙不会挡住射线；距离判断也无视墙，所以能隔墙发现。

**L3**　正确的视线检测是两步：先用 LayerMask = Player 找候选（或者距离 + 视野角），再从眼睛到目标打一条 LayerMask = 地形 的射线，命中地形就说明被遮挡。感知结果写进黑板并记录“最后看见的位置”，丢失目标后去那里搜索，而不是立刻放弃或永远追。

## Q9-4　追不到玩家时怎么办？

**L2**　设计上 Battle 有两个出口：超时（battleTime 5 s 没看见）和距离（> 10 m）。源码每帧重置计时器，超时永不发生；玩家跳到头顶平台时，骷髅在他脚下来回转身，10 s 547 次。

**L3** 修复后 5 s 脱战。更完整的做法：追击前判断可达性（2D 平台游戏常用“平台图”：哪些平台之间可以走 / 跳过去），不可达时切到“守望”或“远程攻击”；同时给追击加边缘检测，避免追下悬崖。

### Q9-5 为什么用状态机不用行为树?

**L2** 6 个状态、规则简单时 FSM 最直接，调试也容易。当出现“优先级 + 条件组合”（低血量逃跑、有队友时包抄、玩家远程时找掩体）时，FSM 的转移数量会平方增长，行为树用 Selector / Sequence 的优先级结构表达更自然，也更容易让策划配置。

**L3** 行为树不是银弹：每帧从根节点重新评估有开销，需要“事件驱动 + 装饰器中断”优化；决策特别依赖打分时用 Utility AI，需要规划多步行动时用 GOAP。很多项目是“行为树做决策、FSM 做动作”（行为树叶子节点驱动一个小状态机）。

**常见错误回答**

**错误**：“怪物 AI 很简单，就是状态机，没有什么问题。”

**问题**：面试官会追问“状态之间怎么复用”“丢失目标怎么办”“隔墙能不能看见”，每个问题在这个项目里都有一个具体的 bug。

**更好的回答**：“状态机本身很简单，我复盘时发现了三类问题：继承复用导致的每帧自转移、计时器写错导致的永不脱战、感知不考虑遮挡。修法分别是……”

## 9.8 发散场景题

### 场景 9-1 同屏 100 只骷髅，帧率掉到 30，你怎么优化?

为什么会问：AI 性能是动作 / 割草游戏的必考题。

先用 Profiler 定位（Update 本身、Physics2D 查询、Animator、GC）。AI 侧：① 去掉每帧 Find 和重复射线（E7 每帧 3 次射线 + 1 次 Find）；② 感知降频（每 0.1~0.3 s，分帧错开）；③ AI LOD：屏幕外 / 远处的怪只跑简化逻辑或不跑；④ 统一由 AIManager 批量 Tick，替代 100 个 MonoBehaviour.Update（减少引擎回调开销）。动画侧：屏外 Animator 设为 Cull Update Transforms 或 Cull Completely（注意它会停止动画事件，第 4 章）；GPU 动画 / 合批。物理

侧：查询加 LayerMask、NonAlloc；骷髅之间关闭碰撞。再往上是 DOTS / Job System 批量计算移动与感知。

### 场景 9-2　十只骷髅围住玩家时挤成一团、同时出手，怎么办？

为什么会问：群体 AI 的表现与公平性。

攻击令牌：全局（或以玩家为中心）只发 2 个令牌，拿到令牌的怪才能进入攻击，其他怪在攻击距离外“踱步”；站位：以玩家为中心分配环形槽位，避免重叠；节奏：出手后令牌回收并加个人冷却。这既是性能优化（大多数怪在做便宜的事），也是体验优化（玩家能读清攻击）。

### 场景 9-3　加一个远程弓箭手，用行为树怎么搭？

为什么会问：考察行为树的实际搭建能力。

Selector：① [Sequence] 低血量 → 逃离；② [Sequence] 目标可见 且 距离 < 3 m → 后撤步；③ [Sequence] 目标可见 且 距离在射程内 且 拿到攻击令牌 → 转向 → 拉弓（技能）；④ [Sequence] 有最后看见位置 → 移动过去搜索；⑤ 巡逻。感知组件负责“可见”，技能系统负责“拉弓”的时间轴与投射物，行为树只做选择。

### 场景 9-4　怪物需要“仇恨”：谁打它就追谁，但坦克可以嘲讽，怎么做？

为什么会问：多目标选择是联机 / 召唤物出现后的必然需求。

仇恨表：受击者在 Receive 里给攻击者加仇恨（伤害 × 系数），治疗者获得“治疗仇恨”，嘲讽直接把仇恨设为当前最高 + 一个余量并锁定 N 秒；仇恨随时间衰减、目标离开太远时移除。决策时取仇恨最高且可达的目标。这要求先完成第 6 章的 DamageInfo（知道“谁打的”），以及第 1 章说的“敌人不再只认 Player 组件”。

**本章速记**

- Battle 继承 GroundState → **看见玩家就每帧自转移 + Find**（E7：10 s 252 次自转移、258 次 Find；行为不变，测不出来）。
- stateTimer 每帧重置 → **永不超时**；叠加“射线只看水平、追击无边缘检测” → 平台下抽搐 10 s 转身 547 次。

- 感知：射线 LayerMask 只有 Player → **穿墙可见**；正确做法是候选 + 遮挡射线 + 最后看见位置。
- 扩展：**感知** → **黑板** → **行为树** → **动作（技能）**，加攻击令牌、AI LOD、对象池。

▶ **自测 1：为什么自转移在画面上看不出来?**

▶ **自测 2：修正版为什么必须在 Enter 里初始化计时器?**

▶ **自测 3：骷髅死亡后为什么还在消耗性能?**

PART IV

# 物品、UI 与工程化

最后一次提交 “IventorySystem” 加入了背包、装备、合成、掉落和格子 UI。这是 RPG 客户端面试里最容易被追问到“数据结构 + 事务 + 防刷 + 存档”的部分；第 11、12 章再看表现层和整个工程的性能与规范。

CHAPTER 10

# 背包、装备、合成与掉落

背包用“List 保序 + Dictionary 查找”的双结构，以 ScriptableObject 作键堆叠；装备通过修正器加属性；合成先校验后扣料；掉落先按概率筛选再无放回抽样。实测发现：合成每种材料只扣一份，10 皮 10 铁能做 9 件而不是 5 件；掉落判定多 1%，掉率填 0 的物品也会掉。

---

**本章目标**

- 说清背包双结构的取舍，以及“ScriptableObject 当键”意味着没有物品实例
- 走一遍装备流程，找出两处点击空格就空引用的原因（含 Unity 序列化器的一个特性）
- 用 E6 / E5 量化合成少扣料和掉落概率偏差
- 设计一个可存档、可联网校验的物品系统：定义 / 实例 / 容器 / 事务

## 10.1 30 秒面试版 • 现状 vs 理想

**30 秒回答**

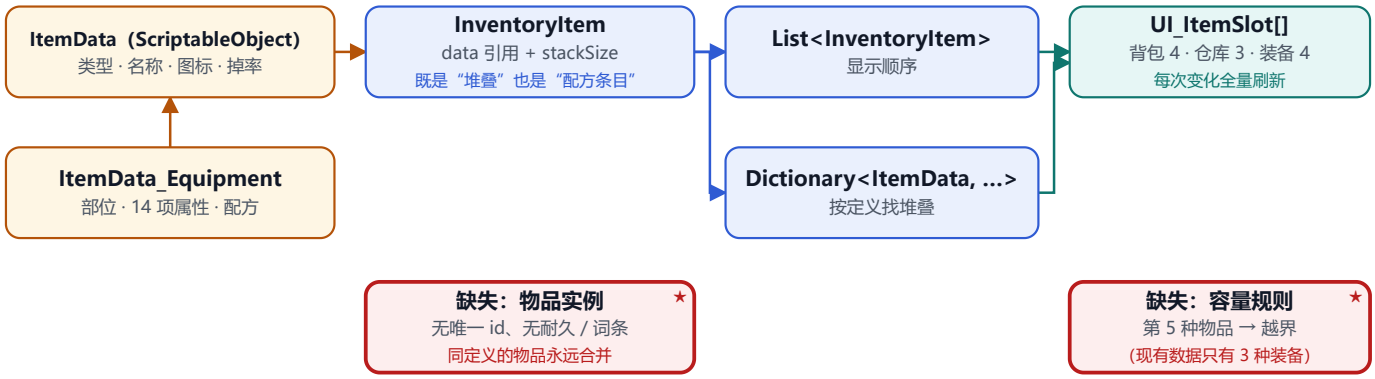
物品定义是 ScriptableObject（类型、名称、图标、掉率；装备再加 14 项属性和配方）；背包、仓库、装备栏各用一个 List 保持显示顺序、一个 Dictionary 以 ItemData 为键找堆叠，任何变化后全量刷新格子 UI。装备时先卸同部位旧装备放回背包，再把新装备的属性加到玩家 Stat 上。合成先检查每种材料数量够不够，再扣料加成品；掉落先让每个候选按掉率独立判定，再从入选的里无放回抽最多 3 个。问题：以定义作键意味着没有物品实例（耐久、随机词条都做不了）；合成每种材料只调一次 RemoveItem，需要 2 张皮只扣 1 张；掉落用 `≤` 比较 0~99，每件都多 1%，且“最多 3 件”的截断让真实掉率低于配置；点空的装备格和首次刷新前点空背包格都会空引用。

现状 → 问题 → 理想　**背包、装备、合成与掉落**

| 维度 | 当前做法（Demo 现状） | 存在的问题 | 理想重构方案 | 收益 / 代价 |
| --- | --- | --- | --- | --- |
| **物品身份** | 以 ItemData（ScriptableObject）作字典键，同种物品永远完全相同 | 局限 做不了“+5 的剑”、耐久、词条；存档只能存配置引用 | 稳定定义 id + 物品实例（uid、数量、耐久、词条）<br>设计方案 • 未实现 | “+5 的剑”可以存在；存档只存实例 |
| **容器** | 背包 / 仓库 / 装备栏三套 List + Dictionary 复制代码；没有容量规则 | 局限 三套几乎相同的代码，改一处要改三处；背包没有容量上限 | 统一 Container（容量、格子、过滤规则）<br>设计方案 • 未实现 | 邮件、商店、交易复用同一套 |
| **合成** | 先校验数量，再对每种材料调用一次 RemoveItem | P1 需要 2 张皮只扣 1 张：10 皮 10 铁做出 9 件，应为 5 件——可以刷装备（E6，#6） | 事务：只读校验全部条件 → 一次性提交 → 结果码<br>模型验证：E6 理想版 5 件 | 不再“少扣料” |
| **掉落** | 整数 Random.Range + ≤ 逐件判定；最多掉 3 件；初速度用 int 版 Random.Range | P2 配置值 ≠ 实际概率：35.4% 实际 32.3%，填 0 也有 1%；掉落物整体偏左（E5，#17） | 独立判定（Random.value）或权重表；float 版随机；由权威端抽取<br>设计方案 • E5 只量化了现状偏差 | 配置值 = 实际概率 |
| **UI 刷新** | 任何变化全量清空重画（一次装备刷新 3 次） | 局限 一次装备操作把所有格子重建 3 次；格子一多就卡 | SlotChanged(i) 事件增量刷新；格子多时虚拟列表<br>设计方案 • 未实现 | 大背包不卡 |
| **格子视图** | 格子上序列化了一个 InventoryItem 字段 | P1 开局点空背包格 / 空装备格即空引用（#13） | 视图只持有（容器, 下标），数据从容器读<br>设计方案 • 未实现 | UI 不拥有数据 |
| **存档 / 联网** | 没有存档；所有操作在本地 | 局限 关游戏全部丢失；联网后客户端可以任意改背包 | 稳定 id + 版本迁移；联网时操作在服务器执行<br>设计方案 • 未实现 | 可存档、可防刷 |

现状问题：P1 ×2　P2 ×1　局限 ×4　（P0~P3 编号见附录 A，分级说明见 0.2）

## 10.2 数据模型：双结构 + 定义作键

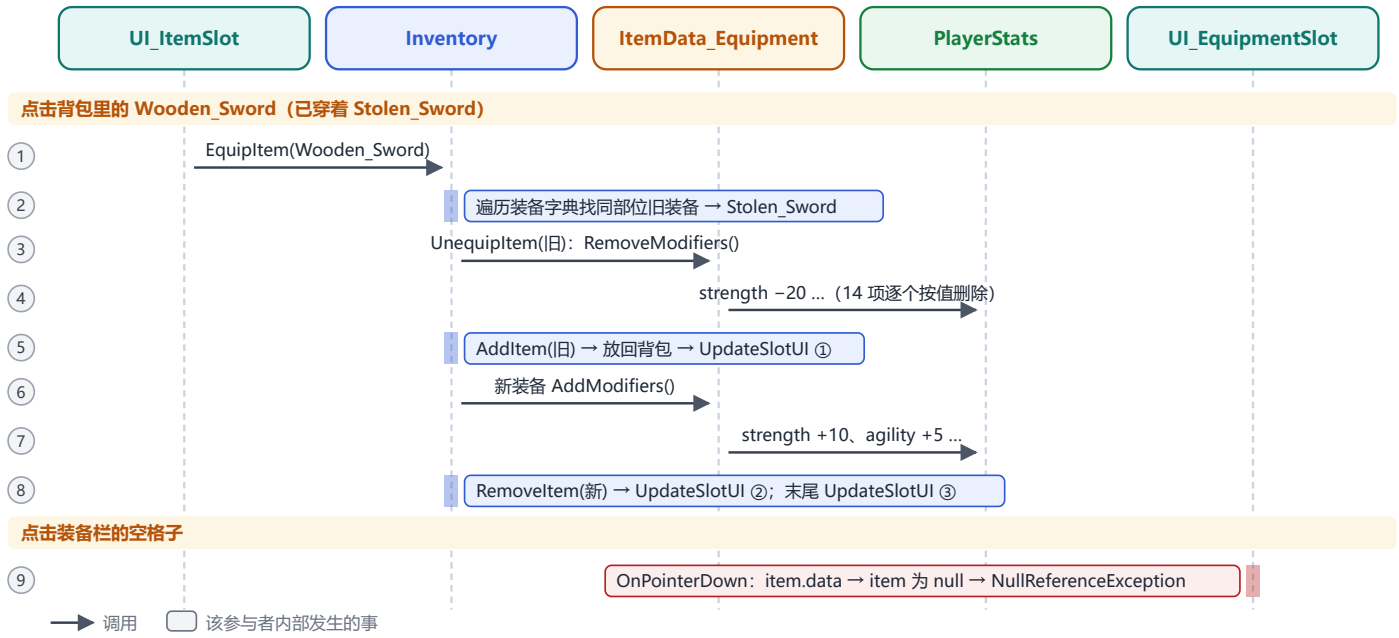



**现状** 图 10-1　背包数据模型（Inventory.cs）。三套“List + Dictionary”（背包 / 仓库 / 装备）结构相同、代码复制了三份；红框是两个结构性缺失

**现状问题** 局限 以配置作键、没有物品实例；三套容器代码复制

| 设计点 | 项目做法 | 好处 | 代价 |
| --- | --- | --- | --- |
| 双结构 | List 保序 + Dictionary 查找 | 堆叠查找 O(1)，显示顺序稳定 | 两个结构要同步增删，任何一处漏写就不一致；删除时 `List.Remove` 是 O(n) |
| 定义作键 | `Dictionary<ItemData, InventoryItem>` | 同类物品自动堆叠，引用比较极快 | 没有“这一把剑”的概念：耐久、强化等级、随机属性、绑定状态、获取时间都无处存放 |
| 全量刷新 | 每次增删都清空所有格子再按 List 重画 | 简单，不会不同步 | 一次装备操作刷新 3 次（AddItem、RemoveItem、EquipItem 末尾各一次）；格子多时浪费；格子数少于物品种类时直接越界 |
| InventoryItem 复用为配方 | Nice_Armor 的配方是 `List<InventoryItem>`（材料 + 数量） | 少写一个类 | 运行时“堆叠”和设计期“配方条目”语义不同，构造函数里的 AddStack 对配方毫无意义 |

## 10.3 装备流程与两个空引用




现状 图 10-2　装备流程（Inventory.cs:152-187）。一次操作刷新 UI 三次；装备栏格子的点击处理没有判空（UI_EquipmentSlot.cs:14-19）

**现状问题** P1 点击空装备格空引用（#13）；一次操作全量刷新 3 次

问题 P1 两处点击空格的空引用原因不同：

- **装备格**：`UI_EquipmentSlot.OnPointerDown` 直接 `item.data as ItemData_Equipment`，卸下后 `CleanUpSlot()` 把 item 置为 null，再点一次就空引用。基类 UI_ItemSlot 有 `if (item == null) return;`，子类重写时丢了。
- **背包格，首次刷新之前**：`public InventoryItem item` 是一个 `[Serializable]` 普通类字段。**Unity 序列化器不支持自定义类字段为 null**，会为它构造一个默认实例——ItemSlot.prefab 里就序列化着 `item: {data: {fileID: 0}, stackSize: 0}`。于是开局时每个格子的 item 都“不为 null 但 data 为 null”，基类的判空拦不住，点击空格走到 `item.data.itemType` 空引用。第一次拾取物品触发 UpdateSlotUI 之后，所有格子被 CleanUpSlot 置为真正的 null，问题就消失了——典型的“只在开局出现一次”的 bug。

**怎么避免**

格子不要把数据对象序列化进 Inspector：字段加 `[System.NonSerialized]` 或做成 private；判空写成 `item?.data == null` 一并检查；更根本的是格子只持有“容器 + 下标”，数据从容器读——UI 不拥有数据。

## 10.4 E6：合成只扣一份材料

当前源码 • 822270c　Inventory.cs:189–223（精简）

```
public bool CanCraft(ItemData_Equipment _itemToCraft, List<InventoryItem> _requiredMaterials)
{
    List<InventoryItem> materialsToRemove = new List<InventoryItem>();
    foreach (var need in _requiredMaterials)
    {
        if (stashDictionary.TryGetValue(need.data, out InventoryItem have) && have.stackSize
            materialsToRemove.Add(have);                          // 校验的是“够不够 need.stackS
        else { Debug.Log("not enough materials"); return false; }
    }
    foreach (var m in materialsToRemove) RemoveItem(m.data);     // ← 每种材料只移除 1 个
    AddItem(_itemToCraft);
    return true;                                                  // 名字是 Can，实际已经合成了
}
```

现状问题　P1　每种材料只 RemoveItem 一次：需要 2 张皮只扣 1 张（E6，#6）；名字叫 Can 却完成了合成

| 初始 皮 | 初始 铁 | 现状：合成件数 | 剩皮 | 剩铁 | 理想：合成件数 | 剩皮 | 剩铁 |
|---|---|---|---|---|---|---|---|
| 2 | 1 | 1 | 1 | 0 | 1 | 0 | 0 |
| 3 | 1 | 1 | 2 | 0 | 1 | 1 | 0 |
| 4 | 2 | 2 | 2 | 0 | 2 | 0 | 0 |
| 6 | 6 | 5 | 1 | 1 | 3 | 0 | 3 |
| 10 | 10 | 9 | 1 | 1 | 5 | 0 | 5 |

表 10-1　Nice_Armor 配方 = Animal_Skin × 2 + Iron × 1（E6）。源码每件只消耗 1 张皮，皮的实际成本减半：当皮成为瓶颈时，同样的材料能多做接近一倍的成品

问题　P1　这是一个**经济漏洞**：在有交易或数值成长的游戏里，合成成本减半会直接破坏产出平衡。修法一行：`for (int k = 0; k < need.stackSize; k++) RemoveItem(need.data);`，更好的是给容器一个 `TryRemove(def, count)`。另外这个函数还有三个设计问题值得一起说：名字叫 Can 却完成了合成（应拆成 CanCraft + Craft）；校验和扣除分两步，单线程下没问题，但联网后必须在服务器上作为一个事务执行；失败时只打日志，UI 拿不到“缺什么”。

## 10.5 E5：掉落概率与“最多 3 件”

当前源码 • 822270c ItemDrop.cs:13-50（精简）

```
for (int i = 0; i < possibleDrop.Length; i++)
    if (Random.Range(0, 100) ≤ possibleDrop[i].dropChance)    // 0~99 的整数 ≤ 84.8 → 85 个
        dropList.Add(possibleDrop[i]);
int itemsToDrop = Mathf.Min(PossibleItemDrop, dropList.Count); // 最多 3 件
for (int i = 0; i < itemsToDrop; i++)
{
    int randomIndex = Random.Range(0, dropList.Count);         // 无放回抽样
    DropItem(dropList[randomIndex]); dropList.RemoveAt(randomIndex);
}
// DropItem：new Vector2(Random.Range(-5, 5), Random.Range(15, 20)) —— int 版，x ∈ {-5..4}
```

现状问题 P2 整数随机 + <=：84.8% 等于 85%、填 0 仍有 1%；“最多 3 件”让 35.4% 变成 32.3%（E5，#17）

骷髅的 5 个候选掉率合计 261.7%，平均有 2.6 个通过筛选，经常超过 3 个而被截断。精确解（枚举 $2^5$ 种筛选结果）与 100 万次蒙特卡洛 实测 E5 ：

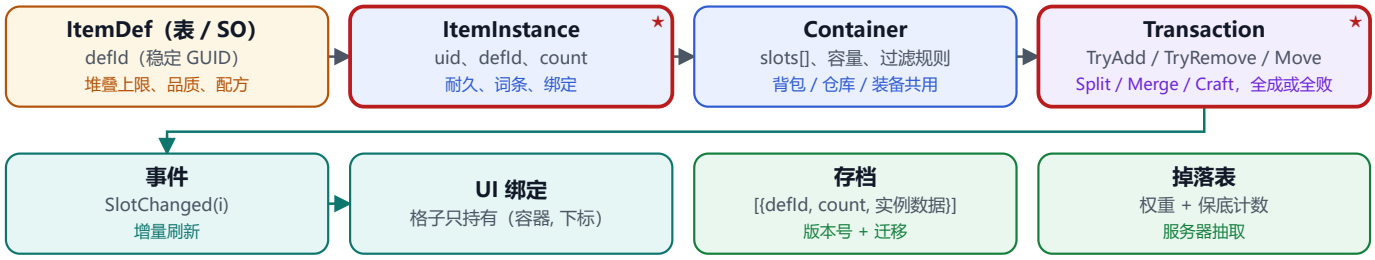
| 物品 | 配置 dropChance / % | 通过筛选 / % | 最终掉落 / % | 蒙特卡洛 / % |
|---|---|---|---|---|
| Animal_Skin | 84.8 | 85.0 | 80.5 | 80.46 |
| Iron | 88.2 | 89.0 | 84.5 | 84.51 |
| Nice_Armor | 35.4 | 36.0 | 32.3 | 32.32 |
| Stolen_Sword | 32.0 | 33.0 | 29.4 | 29.33 |
| Wooden_Sword | 21.3 | 22.0 | 19.2 | 19.20 |

表 10-2　每件物品的真实掉率：筛选阶段因 <= 多 1%（并把 84.8 这样的小数截掉），截断阶段又让每件少 3~5 个百分点。平均每只骷髅掉 2.46 件，什么都不掉的概率 0.55%

- **配置值不是实际掉率**：策划在 Inspector 里看到 35.4%，实际 32.3%。只要有“最多 N 件”，各物品就不再独立；要么文档写清“掉率 = 入选概率”，要么改成权重表一次抽 N 件。
- **小数位无效**：`Random.Range(0,100)` 是整数，84.8 和 84.0 效果相同（都是 85 个值）；dropChance 填 0 仍有 1% 掉率。

- **掉落向左偏**：x 初速度来自 int 版 `Random.Range(-5, 5)`，取值 −5~4，向左 50%、向右 40%、0 占 10%，均值 -0.5 m/s——掉落物整体偏左。应写 `Random.Range(-5f, 5f)`。
- 其他：`item` 字段序列化了却没用；每次掉落 `Debug.Log(_itemData)`；ItemObject 每帧检查 M 键（调试代码遗留，每个掉落物一份）。

## 10.6 改进：定义 / 实例 / 容器 / 事务




**理想方案** 图 10-3　可存档、可联网的物品系统：所有修改都走事务（失败不留半截状态），UI 只订阅“哪个格子变了”

**理想方案 • 骨架** 容器事务（C#，未在 Unity 中编译）

```
public bool TryCraft(Recipe r, Container stash, Container bag)
{
    foreach (var need in r.inputs)                          // 1. 只读校验（含成品是否放得下）
        if (stash.Count(need.defId) < need.count) return false;
    if (!bag.CanAdd(r.output, 1)) return false;
    foreach (var need in r.inputs)                          // 2. 提交：已校验过，不会半途失败
        stash.Remove(need.defId, need.count);
    bag.Add(ItemInstance.Create(r.output, 1));
    Events.Raise(new Crafted(r.id));                        // 3. 通知（UI、任务、成就）
    return true;
}
```

## 10.7 面试追问

追问树

```
背包用什么数据结构？（Q10-1）
├─ 为什么 List + Dictionary？删除的复杂度？（Q10-1 L2）
├─ 一把“+5 的剑”和一把“+0 的剑”怎么区分？（Q10-2 → 定义 vs 实例）
├─ 合成怎么保证不出错？联网后在哪做？（Q10-3 → E6）
├─ 掉落概率怎么做？策划配 35% 实际是多少？（Q10-4 → E5）
└─ 背包怎么存档？改了物品表旧存档怎么办？（场景 10-3）
```

## Q10-1　背包用什么数据结构?

**L2**　List 保持显示顺序，Dictionary 以物品定义为键找到对应堆叠，加物品时 O(1) 判断是否已有堆叠；删除时字典 O(1)、List.Remove O(n)。三个容器（背包、仓库、装备栏）各一套。

**L3**　如果背包是“格子制”（物品放在固定格子里、允许空洞），更自然的结构是定长数组 slots[capacity] + “defId → 格子下标集合”的索引；堆叠满了要找下一个同类格子或空格子。我项目里没有容量规则，UI 只有 4 格，物品种类超过 4 就会越界——现在只有 3 种装备，所以没触发。

## Q10-2　“+5 的剑”和“+0 的剑”怎么区分?

**L2**　现在区分不了：字典以 ScriptableObject 定义为键，同一定义的物品会被合并成一个堆叠。要支持实例数据，必须引入 ItemInstance（唯一 id + 定义 id + 实例属性），可堆叠的材料按定义合并，不可堆叠的装备每件一个实例。

**L3**　实例数据还决定了装备修正的来源：修正器挂“装备实例”，强化等级变化时重算该实例的修正；存档存实例列表。

## Q10-3　合成怎么保证不出错?

**L2**　先只读校验全部条件（材料数量、成品放不放得下），再一次性提交；提交阶段不应该有可能失败的步骤。我原来的代码校验是对的，但提交时每种材料只 RemoveItem 一次，需要 2 张皮只扣 1 张，测出来 10 皮 10 铁能做 9 件而不是 5 件。

**L3** 联网后合成必须在服务器执行：客户端只发“合成配方 X”，服务器按自己的背包数据做事务并回包；客户端可以预表现（先播特效）但背包以服务器为准。重复请求要用请求 id 去重，防止网络重发造成双扣或双得。

### Q10-4 掉落概率怎么做?

**L2** 我的做法是每个候选独立判定，再从入选的里无放回抽最多 3 个。测出来两个偏差：判定用 `≤` 比较 0~99 的整数，每件多 1%，掉率填 0 也会掉；“最多 3 件”的截断又让每件实际掉率比配置低 3~5 个点，比如 Nice_Armor 配 35.4% 实际 32.3%。

**L3** 按需求选模型：独立掉落（每件按概率，不设上限）、权重表（一次抽 N 件，概率 = 权重 / 总权重）、分组（先抽品质再抽物品）、保底（连续 K 次未出就必出，计数存在玩家身上）。权威端（服务器）抽取，客户端只表现。

**常见错误回答**

**错误**：“背包用 Dictionary 存，查找 O(1)，性能最好。”

**问题**：只回答了查找。面试官真正想听的是堆叠规则、实例数据、容量与格子、事务、存档和联网校验。

**更好的回答**：从“定义 / 实例 / 容器 / 事务”四层回答，再用自己项目的 bug 举例说明为什么需要每一层。

## 10.8 发散场景题

### 场景 10-1 设计一个 MMO 背包：200 格、可堆叠、可拆分、可整理、可交易。

为什么会问：最常见的客户端系统设计题之一。

数据：slots[200] 存 ItemInstance 引用（或 uid），索引 defId → 格子集合；操作：Add（先填同类未满堆叠再找空格）、Split（从堆叠拆出 n 个到空格）、Merge、Move / Swap、Sort（按规则排序后重排，注意稳定排序与锁定格子）；交易：物品在交易期间被“锁定”，服务器以交易单为事务原子交换。客户端：所有操作发请求，服务器回包后应用（可做乐观预表现 + 回滚）；UI 用虚拟列表只渲染可见格子（场景 10-4）。

### 场景 10-2　有人用加速器 / 改包刷合成材料，你在客户端能做什么?

为什么会问：防刷的边界意识。

客户端几乎做不了“防”，只能“不信任”：所有产出（掉落、合成、拾取）由服务器决定；客户端提交的只是意图。能做的是减小攻击面（不在客户端计算掉落、不在本地存可修改的权威数据）、上报异常（短时间大量请求）、协议加签和重放保护。项目里掉落在本地随机、合成在本地扣料，联网化时这两处都必须搬到服务器。

### 场景 10-3　物品表改版（删了一个物品、改了堆叠上限），旧存档怎么办?

为什么会问：存档兼容是线上事故高发区。

存档里存稳定的 defId（GUID 或数字 id，不能存数组下标或资源路径）；存档带版本号，加载时按版本逐级迁移（删除的物品转换成补偿货币、堆叠超上限的拆成多格）；ScriptableObject 引用在存档里要转换成 id，不能直接序列化对象引用。项目里背包以 ScriptableObject 引用为键，直接序列化的话换个资源 GUID 就全丢。

### 场景 10-4　背包 1000 格，打开界面卡 200 ms，怎么优化?

为什么会问：UI 性能的经典题。

① 虚拟列表 / 循环滚动：只实例化可见的 30~40 个格子，滚动时复用并重新绑定数据；② 增量刷新：容器发 SlotChanged(i)，只更新变化的格子，而不是像项目这样全量 CleanUp + Update；③ 图标异步加载、图集合批；④ 把背包 Canvas 与常驻 HUD 分开，避免格子变化触发整张 Canvas 重建；⑤ 打开时分帧填充。

**方法论延伸**

背包、装备、技能三个系统的整体设计思路与“为什么这样设计”，见第 21 章；关卡与瓦片地图见第 19 章。

**本章速记**

- 背包：**List 保序 + Dictionary 查找**，以 SO 定义作键 → **没有物品实例**，没有容量规则。

- 空引用：装备格子类丢了判空；**Unity 序列化器给自定义类字段造默认实例** → 开局点空格空引用（#13）。
- E6：合成每种材料只扣 1 份 → 10 皮 10 铁做 9 件（应为 5），经济漏洞（#6）。
- E5：≤ 多 1%、0 也掉；“最多 3 件”截断后 35.4% → 32.3%；x 速度 int 随机偏左（#17）。
- 改进：**定义 / 实例 / 容器 / 事务** + 事件增量刷新 + 稳定 id 存档 + 服务器抽取。

▶ **自测 1：为什么点空背包格的空引用“只出现在开局”？**

▶ **自测 2：10 皮 10 铁时，为什么源码版最后剩 1 皮 1 铁?**

▶ **自测 3：怎样让“配置掉率 = 实际掉率”？**

CHAPTER 11

# UI 与表现层：血条、受击闪白与视差背景

表现层的面试题看起来“简单”，追问却很深：事件订阅放在哪个生命周期、`renderer.material` 为什么会悄悄复制材质、多个系统同时改颜色谁说了算、UI 刷新怎么不卡。项目里已经写好了一个没用上的 Shader，正好拿来讲“从改材质到改参数”的演进。

---

**本章目标**

- 说清事件订阅的对称原则，以及 `public Action` 字段和 `event` 的区别
- 理解 `sr.material` 的复制语义、颜色写者冲突，以及 MaterialPropertyBlock / Shader 参数方案
- 能推导视差背景的公式，并说出 Canvas 重建、虚拟列表、飘字池这些 UI 性能要点

## 11.1 30 秒面试版 • 现状 vs 理想

**30 秒回答**

血条挂在角色下面的世界空间 Canvas 上，Start 里订阅 `onHealthChanged` 和 `onFlipped`（角色翻转时把血条反向旋转回来），OnDisable 里退订——订阅与退订不对称，禁用再启用后血条就不再刷新；两个回调都是 public 委托字段而不是 event，外部可以用 = 把别人的订阅冲掉。受击闪白是把 SpriteRenderer 的材质临时换成纯色材质 0.2 s，异常状态用 InvokeRepeating 每 0.3 s 切换颜色；问题是 `sr.material` 会为每个角色复制一份材质实例，闪白、异常色、眩晕取消三方都直接写颜色。项目里已经有一个支持闪白强度和闪烁参数的 SpriteStatusFX Shader，但没有材质用它——用 MaterialPropertyBlock 改它的参数就是正确的方向。

**现状** → 问题 → **理想**　**UI 与表现层**

| 维度 | 当前做法（Demo 现状） | 存在的问题 | 理想重构方案 | 收益 / 代价 |
|---|---|---|---|---|
| **事件订阅** | Start 订阅、OnDisable 退订（不对称） | P2 禁用再启用后血条不再刷新；Start 之前被禁用时退订空引用（#28） | OnEnable / OnDisable 成对，OnEnable 里立即刷新一次 设计方案 · 未实现 | 禁用再启用、对象池复用后仍能刷新 |
| **回调类型** | `public System.Action` 字段 | P2 外部可以 `= null` 冲掉所有订阅，或直接调用伪造通知（#28） | `event` 设计方案 · 未实现 | 外部不能冲掉订阅或伪造通知 |
| **受击闪白** | 临时换纯色材质；`sr.material` 为每个角色克隆材质实例 | P2 每个角色一份材质副本：打断合批，销毁时不释放（#29） | 所有角色共享 SpriteStatusFX 材质，用 MaterialPropertyBlock 写参数 设计方案 · 未实现 | 保持合批、没有泄漏 |
| **颜色写者** | 闪白协程、异常色 InvokeRepeating、眩晕取消三方直接写 color | P2 异常在闪白期间结束时，颜色被“恢复”成过期的异常色并永久残留（推断，#29） | 每帧由“当前生效的效果集合”推导参数（反馈导演） 设计方案 · 未实现 | 不会残留异常色 |
| **视差背景** | Update 里读相机位置，用 GameObject.Find 找相机 | 隐患 相机在 LateUpdate 才移动，背景读到上一帧位置，快速移动时抖动（推断）；按名字找相机 | LateUpdate 且在相机更新之后读取；Camera.main 或注入 设计方案 · 未实现 | 相机移动时背景不滞后一帧 |
| **血条规模** | 每个角色一个世界空间 Canvas（只有两个角色，够用） | 局限 两个角色没问题；同屏几十个时每个角色一张 Canvas 会成为瓶颈 | 数量多时屏幕空间统一绘制 + 对象池 + LOD 设计方案 · 未实现 | 千怪同屏可行 |

现状问题：P2 ×4　隐患 ×1　局限 ×1　（P0~P3 编号见附录 A，分级说明见 0.2）

## 11.2 血条：订阅放在哪里

当前源码 • 822270c UI_HealthBar.cs:13-41

```
private void Start()
{
    entity   = GetComponentInParent<Entity>();
    myStats = GetComponentInParent<CharacterStats>();
    UpdateHealthUI();
    entity.onFlipped        += FlipUI;           // 订阅在 Start
    myStats.onHealthChanged += UpdateHealthUI;
}
private void FlipUI() => myTransform.Rotate(0, 180, 0);   // 父物体转 180°，子 UI 再转回来，文字
private void OnDisable()
{
    entity.onFlipped        -= FlipUI;           // 退订在 OnDisable
    myStats.onHealthChanged -= UpdateHealthUI;
}
```

现状问题 P2 Start 订阅、OnDisable 退订不对称：禁用再启用后不再刷新，Start 前被禁用时退订空引用；public Action 字段可被外部冲掉（#28）




现状 vs 理想 图 11-1 订阅生命周期：源码“Start 订阅、OnDisable 退订”不对称——对象被禁用再启用（UI 面板开关、对象池复用）后不会重新订阅。规则：OnEnable / OnDisable 成对，或 Start / OnDestroy 成对

- 问题 P2 如果对象在 Start 之前就被禁用（例如生成时立即 SetActive(false)），OnDisable 先于 Start 执行，`entity` 还是 null，退订时空引用。
- **public Action vs event**：`public System.Action onHealthChanged;` 允许任何人写 `stats.onHealthChanged = Foo;`，一行代码就冲掉了所有已有订阅，也允许外部直接 `onHealthChanged()` 伪造通知。加上 `event` 关键字后，外部只能 += / -=。
- **只刷新血量不刷新上限**：装备改变最大生命时不触发事件，血条上限要等下一次受伤（第 7 章）。
- **翻转的写法**：Entity.Flip 用 `transform.Rotate(0, 180, 0)` 翻转整棵层级（攻击点、检测点、子 UI 一起翻），所以 UI 需要“反向转回来”。另一种写法是只翻 SpriteRenderer.flipX 或 scale.x = −1，各有对子物体的影响，选定一种后要在整个项目里一致。

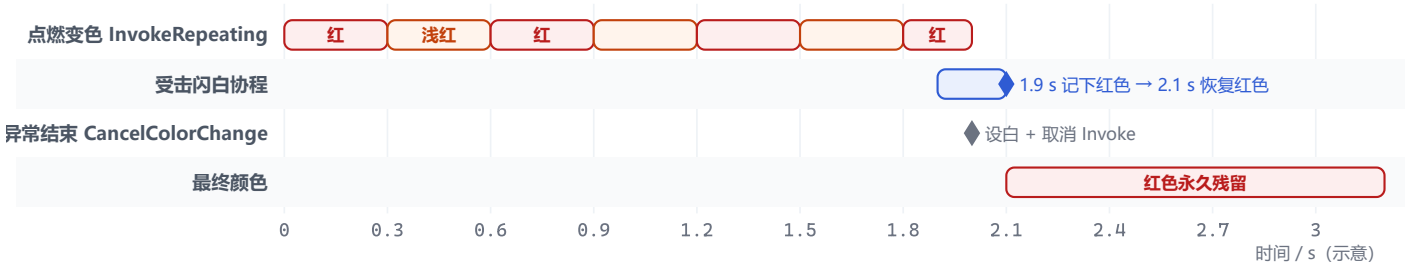
# 11.3 受击闪白与异常色：谁在写颜色

当前源码 • 822270c EntityFX.cs:21-59（精简）

```
private void Start() { sr = GetComponentInChildren<SpriteRenderer>(); originMat = sr.material
private IEnumerator FlashFX()
{
    isFlashing = true;
    sr.material = hitMat;                  // FlashFXMaterial：内置纯色 Shader，整张图变白
    Color currentColor = sr.color;         // 记下"此刻"的颜色（可能是异常色）
    sr.color = Color.white;
    yield return new WaitForSeconds(0.2f);
    sr.color = currentColor;               // 0.2 s 后恢复成记下的颜色
    sr.material = originMat;
    isFlashing = false;
}
private void CancelColorChange() { CancelInvoke(); sr.color = Color.white; }    // 取消该组件上
```

**现状问题** P2 读 sr.material 为每个角色克隆材质（打断合批、不释放）；闪白、异常色、眩晕取消三方抢写颜色（推断残留异常色，#29）

| 问题 | 说明 |
| --- | --- |
| `sr.material` 复制材质 扩展 | Renderer.material 的 getter 在材质被共享时会克隆一份并让该渲染器从此使用克隆体。每个角色一份 Sprites-Default 的副本：破坏动态合批（材质不同不能合批），而且克隆体不会随对象销毁自动释放，需要在 OnDestroy 里 `Destroy(originMat)`。只想读应使用 `sr.sharedMaterial`。 |
| 颜色的三个写者 | 闪白协程（记下颜色、0.2 s 后恢复）、异常状态 InvokeRepeating（每 0.3 s 在两种颜色间切换，闪白期间靠 isFlashing 跳过）、CancelColorChange（取消全部 Invoke 并设白）。推断 如果异常状态恰好在闪白的 0.2 s 内结束，CancelColorChange 先把颜色设白，随后闪白协程把颜色恢复成"闪白开始时记下的异常色"——异常结束了，颜色却永久残留。 |
| CancelInvoke() 误伤 | 眩晕结束时调用 CancelColorChange，会把点燃 / 冰冻的变色一并取消（第 4 章）。 |
| 未使用的 Shader | `Assets/Script/SpriteStatusFX.shader` 已经支持 `_FlashAmount`（闪白强度）和 `_BlinkColor / _BlinkIntensity / _BlinkSpeed`（状态闪烁），但没有任何材质引用它。 |




**现状** 图 11-2　颜色写者冲突（示意，推断）：闪白开始时记下的是异常色，异常在闪白期间结束，闪白结束时把异常色“恢复”回去。根因是每个写者都写绝对值，没有“颜色 = f(当前所有效果)”的推导

**现状问题** P2 异常结束后颜色永久残留（推断，#29）

**理想方案 • 骨架** 用 MaterialPropertyBlock 驱动 SpriteStatusFX 的参数（C#，未在 Unity 中编译）

```
static readonly int FlashId = Shader.PropertyToID("_FlashAmount");
static readonly int BlinkId = Shader.PropertyToID("_BlinkIntensity");
static readonly int BlinkColorId = Shader.PropertyToID("_BlinkColor");
MaterialPropertyBlock mpb;                    // 所有角色共用同一个 SpriteStatusFX 材质，参数各自
float flashUntil;

void LateUpdate()                             // 每帧由“当前所有效果”推导一次，不再有写者冲突
{
    sr.GetPropertyBlock(mpb);
    mpb.SetFloat(FlashId, Time.time < flashUntil ? 1f : 0f);
    var top = buffs.TopVisualAilment();       // 优先级最高的异常（来自 Buff 系统）
    mpb.SetFloat(BlinkId, top ≠ null ? 1f : 0f);
    if (top ≠ null) mpb.SetColor(BlinkColorId, top.color);
    sr.SetPropertyBlock(mpb);
}
public void Flash(float sec) ⇒ flashUntil = Mathf.Max(flashUntil, Time.time + sec);
```

注意 MaterialPropertyBlock 与 SRP Batcher 不兼容（使用 MPB 的渲染器会退出 SRP Batcher 路径）；2D 内置管线下它能保持“同一材质”，对动态合批友好。若使用 URP 2D，也可以把闪白做成顶点色或改用 `sr.color` 的一个通道编码，避免打断合批——具体取舍要用 Frame Debugger 看批次。

## 11.4 视差背景：一行公式

当前源码 • 822270c ParalleBackGround.cs:14–34

```
void Start()  { cam = GameObject.Find("Main Camera"); length = GetComponent<SpriteRenderer>()
void Update()
{
    float distanceToMove = cam.transform.position.x * parallaxEffect;          // 背景跟随相机
    float distanceMoved  = cam.transform.position.x * (1 - parallaxEffect);    // 相机相对背景
    transform.position = new Vector3(xPosition + distanceToMove, transform.position.y);
    if (distanceMoved > xPosition + length)      xPosition += length;          // 无限循环：整
    else if (distanceMoved < xPosition - length) xPosition -= length;
}
```

**现状问题** 隐患 在 Update 里读 Cinemachine 相机位置（它在 LateUpdate 才移动），背景滞后一帧（推断）；按名字 Find 相机

parallaxEffect = 1 时背景完全跟随相机（看起来无限远），= 0 时静止（和前景同深度）。三个追问点：① 用 Update 读相机位置，而 Cinemachine 默认在 LateUpdate 更新相机——背景用的是上一帧的相机位置，相机移动时背景会滞后一帧抖动，应改成 LateUpdate 并保证在相机之后执行（或订阅 CinemachineBrain 的更新事件） 推断 ；② `GameObject.Find("Main Camera")` 依赖对象名，`Camera.main` 或注入更稳；③ 世界坐标离原点很远时 float 精度下降，大地图需要“原点重置”（floating origin）。

## 11.5 UI 性能要点（扩展）

| 主题 | 要点 |
| --- | --- |
| Canvas 重建 | 同一 Canvas 下任一元素变化会触发整张 Canvas 的网格重建（Rebuild）；动静分离：常驻 HUD、频繁变化的血条 / 飘字、背包面板各用独立 Canvas 或子 Canvas |
| 布局 | LayoutGroup / ContentSizeFitter 在任一子元素变化时会重算整组，背包格子这类固定布局可以在编辑期算好 |
| 射线检测 | 不需要交互的 Image / Text 关掉 Raycast Target，减少 GraphicRaycaster 的遍历 |
| 世界空间血条 | 每个角色一个 Canvas 开销大；数量多时改为屏幕空间统一绘制（一个 Canvas、对象池、每帧把世界坐标转屏幕坐标） |
| 飘字 | 对象池 + TextMeshPro + 共享材质；数字图集化；同一帧同一目标的多次伤害合并显示 |
| 数据绑定 | UI 订阅数据变化（MVP / MVVM），而不是每帧轮询；项目的血条是事件驱动的，方向正确 |

## 11.6 面试追问

追问树

```
血条怎么刷新？（Q11-1）
├─ 订阅在哪里做？为什么要退订？（Q11-1 L2）
├─ event 和 Action 字段有什么区别？（Q11-2）
├─ 受击闪白怎么做？为什么不直接改 material？（Q11-3）
│   └─ 闪白和中毒变色同时发生怎么办？（Q11-4 → 图 11-2）
└─ 1000 个怪的血条怎么做？（场景 11-2）
```

### Q11-1　血条怎么刷新？订阅放在哪？

**L2**　属性组件在血量变化时触发 onHealthChanged，血条订阅后刷新 Slider。订阅和退订必须成对：OnEnable / OnDisable 或 Start / OnDestroy。不退订的话，血条被销毁后属性组件还持有它的委托，下次触发会访问已销毁对象（MissingReferenceException），也阻止 GC 回收。

**L3 • 项目**　我写成了 Start 订阅、OnDisable 退订，禁用再启用后就不刷新了；如果对象在 Start 前被禁用，退订时还会空引用。改成 OnEnable / OnDisable，并在 OnEnable 里立即刷新一次。

### Q11-2　event 和 public Action 字段有什么区别?

**L2**　event 对外只暴露 += 和 −=，只有声明它的类能调用和清空；public 字段可以被外部直接赋值（冲掉所有订阅）或直接调用（伪造事件）。我项目里两个回调都是 public 字段，应该加 event。

### Q11-3 → Q11-4　受击闪白怎么做？和中毒变色冲突怎么办?

**L2**　我现在是临时换成一个纯色材质 0.2 s 再换回来。问题是 `sr.material` 读取时会克隆材质，每个角色一份，破坏合批也要手动销毁；而且闪白、中毒变色、眩晕取消三方都在直接写颜色，可能出现“中毒结束了颜色还残留”。

**L3**　改成一个支持闪白和闪烁参数的 Shader（项目里已经写好了 SpriteStatusFX），所有角色共享一个材质，用 MaterialPropertyBlock 写各自的参数；每帧由“当前生效的效果”推导参数（闪白截止时间、最高优先级异常），而不是由效果去改颜色——和第 7 章的速度修正器是同一个思想。

**常见错误回答**

**错误**：“改颜色用 GetComponent<SpriteRenderer>().material.color = Color.red 就行。”

**问题**：每个角色克隆一份材质，合批失效、内存泄漏；而且多个效果直接改同一个 color 会互相覆盖。

**更好的回答**：“只读用 sharedMaterial；按对象改参数用 MaterialPropertyBlock 或顶点色；颜色由效果集合推导，不由效果直接写。”

## 11.7 发散场景题

**场景 11-1　设计一个伤害飘字系统，要求暴击放大、同目标短时间多次伤害合并、不卡帧。**

为什么会问：客户端最常见的表现系统之一。

数据：伤害事件（目标、数值、类型、是否暴击）进入队列；合并：同一目标 0.1 s 内的同类伤害累加成一条；表现：对象池里的 TMP 文本，位置 = 目标头顶世界坐标转屏幕坐标 + 随机偏移，动画

用曲线驱动（缩放 / 上浮 / 淡出，暴击用另一条曲线和颜色）；性能：单独 Canvas、共享材质、同屏上限（超出时丢弃最小的）。数据来自伤害管线的 HitResult（第 6 章），闪避显示 Miss。

### 场景 11-2　1000 个怪都要显示血条，怎么做?

为什么会问：世界空间 UI 的规模问题。

不要每个怪一个世界空间 Canvas。方案一：一个屏幕空间 Canvas + 对象池，只给“受过伤 / 在镜头内 / 距离近”的怪分配血条，每帧更新位置；方案二：血条做成 SpriteRenderer（一张 9-slice 图 + Shader 控制填充比例），和怪物一起合批，甚至 GPU Instancing；方案三：完全由一个 Mesh 批量绘制。还要做 LOD：远处只在受伤后显示 2 s。

### 场景 11-3　策划要求“UI 不能直接调用游戏逻辑”，你怎么组织背包界面?

为什么会问：UI 架构分层。

MVP：View（格子组件，只负责显示和把点击转成意图）、Presenter（把“点击格子 3”翻译成“请求装备容器 A 的第 3 格”，并订阅容器变化刷新 View）、Model（容器 + 事务，第 10 章）。项目里 UI_ItemSlot.OnPointerDown 直接调 Inventory.instance.EquipItem、还读 Input.GetKey(LeftControl)，View 与逻辑耦合；改完后 View 可以换皮、逻辑可以单测、联网时 Presenter 改成发请求即可。

**本章速记**

- 订阅 **OnEnable / OnDisable 成对**；回调用 **event** 而不是 public 字段（#28）。
- `sr.material` **克隆材质**（合批失效 + 需手动销毁）；只读用 sharedMaterial（#29）。
- 颜色有三个写者 → 可能残留异常色；改为 **MPB 写 Shader 参数 + 每帧由效果集合推导**（项目里的 SpriteStatusFX 已备好）。
- 视差：x = 起点 + 相机 x × 系数；在 LateUpdate、相机之后读取；Find 换 Camera.main。

▶ **自测 1：为什么 UI 需要在角色翻转时“反向旋转”?**

▶ **自测 2：FlashFXMaterial 用的是什么 Shader?**

▶ **自测 3：MaterialPropertyBlock 有什么限制?**

CHAPTER 12

# 性能、GC 与工程化：热点地图与两周重构路线

“你的项目有哪些性能问题？怎么定位？” “如果给你两周重构，你先做什么？”——这一章把前 11 章散落的性能与工程问题汇总成一张热点地图，再给出按风险排序的重构路线，最后讲对象池、程序集、版本管理这些“项目之外”的工程素养。

---

**本章目标**

- 能报出项目里每一类热点 API 的调用次数、位置和替代方案
- 说清 Unity 里哪些写法会产生 GC 分配，以及用 Profiler 验证的步骤
- 设计一个不会踩“复用后状态没重置 / 旧引用指向新对象”的对象池
- 给出两周重构计划：先修 P0、再修数值与状态、最后做结构性重构

## 12.1 30 秒面试版 • 现状 vs 理想

**30 秒回答**

规模小，帧率没问题，但写法里有会随规模放大的热点：9 处 OverlapCircleAll 每次分配数组且不过滤层；骷髅战斗时因自转移每秒 26 次 GameObject.Find；分身、飞剑、雷击、掉落物都是 Instantiate / Destroy；19 处按字符串反射调用（协程、Invoke）；8 处 Debug.Log 留在游戏逻辑里（冷却失败每次按键都打）；瞄准时每帧 50 次屏幕转世界坐标。工程上运行时代码引用了 UnityEditor（打包失败），.gitignore 的写法让 Logs 和 UserSettings 进了仓库。重构顺序：先修打包与 P0 逻辑，再修数值和状态守卫，最后做输入层、伤害管线、Buff、物品实例和对象池这些结构性改动，并配上纯逻辑单元测试。

现状 → 问题 → 理想　**性能与工程**

| 维度 | 当前做法（Demo 现状） | 存在的问题 | 理想重构方案 | 收益 / 代价 |
|---|---|---|---|---|
| **物理查询** | 9 处 OverlapCircleAll：每次分配、无过滤 | P2 每次调用都分配新数组（GC）；结果混入地面与触发器（#27） | 带 ContactFilter2D + List 的重载（或 NonAlloc），复用缓冲 设计方案 · 未实现 | 零分配、结果干净 |
| **对象查找** | GameObject.Find 2 处（其一每帧触发）；热路径与属性里 GetComponent | P3 战斗中每秒约 26 次全场景 Find（E7）；属性里每次访问都向上查组件（#34） | 注入 / Awake 缓存 / Hurtbox 直接持有实体 设计方案 · 未实现 | 去掉每帧全场景遍历 |
| **对象生成** | 分身、飞剑、雷击、掉落物 Instantiate / Destroy | 局限 生成 / 销毁带来分配与卡顿尖刺；销毁后的旧引用变成 fake null | 对象池 + OnSpawn / OnDespawn + 代数句柄 设计方案 · 未实现 | 没有尖刺、旧引用安全 |
| **日志** | 8 处 Debug.Log 在发布版照样执行（含字符串拼接） | P3 发布版照样拼字符串、写日志（#34） | `[Conditional]` 包装的分级日志 设计方案 · 未实现 | 发布版零开销 |
| **构建与仓库** | 运行时 using UnityEditor；.gitignore 字符集写法让 12 个本机文件入库；美术无 LFS | P0 打包编译失败（#1）；本机文件入库导致协作冲突（#35） | asmdef + CI 真打包；修正 .gitignore 并 git rm --cached；美术走 LFS 设计方案 · 未实现 | 不在提测时才发现打包失败 |
| **测试** | 只能进 Play Mode 手测 | 局限 E1~E11 这类问题只能靠手测碰运气；重构没有回归保护 | 纯逻辑抽出 + EditMode 单元测试（用本书 E1~E11 当用例） 设计方案 · 未实现 | 重构有回归兜底 |

现状问题：P0 ×1　P2 ×1　P3 ×2　局限 ×2　（P0~P3 编号见附录 A，分级说明见 0.2）

## 12.2 热点地图

| 热点（不含注释） | 次数 | 主要位置 | 代价 | 替代 |
| --- | --- | --- | --- | --- |
| `Physics2D.OverlapCircleAll` | 9 | 玩家 / 骷髅 / 分身攻击、弹反、旋转剑、弹跳收集、找最近敌人 ×3 | 每次分配数组；无 LayerMask，结果里混着地面、触发器；半径 25 的“找最近敌人”遍历大片场景 | 带 `ContactFilter2D` + 复用 List 的重载（或 NonAlloc 版本）；找最近敌人用敌人注册表而不是物理查询 |
| `GetComponent*` | 59 | 每个检测结果 GetComponent<Enemy>（OnTriggerEnter2D 里同一行调用 2~4 次）；`player ⇒ GetComponentInParent` 属性 3 处 | 单次很便宜，但在循环和属性里被重复调用；编辑器里查找失败会分配“假 null” | Awake 缓存；Hurtbox 上直接存实体引用；`TryGetComponent` |
| `GameObject.Find` | 2 | 骷髅 GroundState.Enter（每次进入 Idle / Move / Battle）；视差背景 Start | 遍历场景；依赖对象名；E7 中每秒约 26 次 | 注入 / 注册表 / `Camera.main` |
| `Instantiate / Destroy` | 5 / 11 | 分身（每次冲刺）、飞剑、雷击、掉落物、25 个瞄准点 | 实例化开销 + GC + 内存碎片 | 对象池（12.4） |
| 字符串反射调用 | 19 | `StartCoroutine("…")` 5、`Invoke` 10、`InvokeRepeating` 4 | 反射查找；改名不报错（已有一处调用了不存在的 RedColorBlink） | `StartCoroutine(F())`、`nameof`、计时器系统 |
| `Debug.Log` | 8 | 冷却中按键、每次掉落、每次点燃伤害、穿刺计数、合成结果、骷髅被眩晕 | 字符串拼接分配 + 写日志 + 采集堆栈；**打包后照样执行** | 用 `[Conditional("UNITY_EDITOR")]` 包装的日志函数，或分级日志系统 |
| 每次命中的协程 | 2~3 / 次 | FlashFX、HitKnockBack、FreezeTimeFor；每次 `new WaitForSeconds` | 协程对象 + WaitForSeconds 分配；重叠导致逻辑错误（E8） | 截止时间 + 统一 Tick；缓存 WaitForSeconds |

| | | | | |
|---|---|---|---|---|
| 瞄准虚线 | 50 / 帧 | DotsPosition 里每个点调用两次 AimaDirection（各一次 ScreenToWorldPoint） | 重复计算 | 每帧算一次方向，循环里复用 |
| 背包刷新 | 3 / 次装备 | AddItem / RemoveItem / EquipItem 各调一次 UpdateSlotUI | 全量清空重画 | 事件增量刷新（第 10 章） |

**关于“数字”的边界**

上表的“次数”是对源码的静态统计（排除注释行），E7 的 Find 频率来自 C++ 移植模型。**这些写法分别分配多少字节、耗时多少微秒，本书没有在 Unity 里测**——面试时请说“我在 Profiler 的 GC Alloc 列和 Timeline 里确认过”，并且只报你自己测过的数。

## 12.3 Unity 里哪些写法会分配（扩展）

| 来源 | 例子 | 项目里有没有 |
|---|---|---|
| 返回数组的引擎 API | `OverlapCircleAll`、`GetComponentsInChildren`、`Mesh.vertices` | 有（9 处 Overlap；Inventory.Start 里 GetComponentsInChildren 只调一次，可以接受） |
| 字符串 | 拼接、`ToString()`、格式化 | 有：Debug.Log 拼接、`itemText.text = stackSize.ToString()`、`gameObject.name = "Item object - " + …` |
| new 引用类型 | `new List`、`new WaitForSeconds` | 有：DoMagicalDamage 每次 new List、CanCraft 每次 new List、每次命中 new WaitForSeconds |
| 闭包捕获 | lambda 捕获局部变量 / this | DoMagicalDamage 的两个 lambda 不捕获，编译器会缓存委托，不分配 |
| 装箱 | 值类型转 object、枚举作 Dictionary 键（旧 Mono） | Dictionary 以 ScriptableObject 为键，不涉及装箱 |
| 协程 | 每次 StartCoroutine 生成一个迭代器对象 | 有：每次受击 2 个 |

验证步骤：Profiler → CPU 模块打开 GC Alloc 列，按帧找分配峰值 → 开 Deep Profile 或加 `Profiler.BeginSample` 定位到函数 → 修复后用同一段录制对比。项目的 Player Settings 里 `gcIncremental = 1`（增量 GC 已开启），单次停顿被摊到多帧，但分配总量不变；移动端更要关注总量和频率。

## 12.4 对象池：三个必须处理的坑




理想方案 图 12-1　对象池的生命周期与三个坑。Unity 2021 起自带 UnityEngine.Pool.ObjectPool<T>，提供创建 / 取出 / 归还 / 销毁四个回调，但“重置什么”和“旧引用怎么办”仍然要自己设计

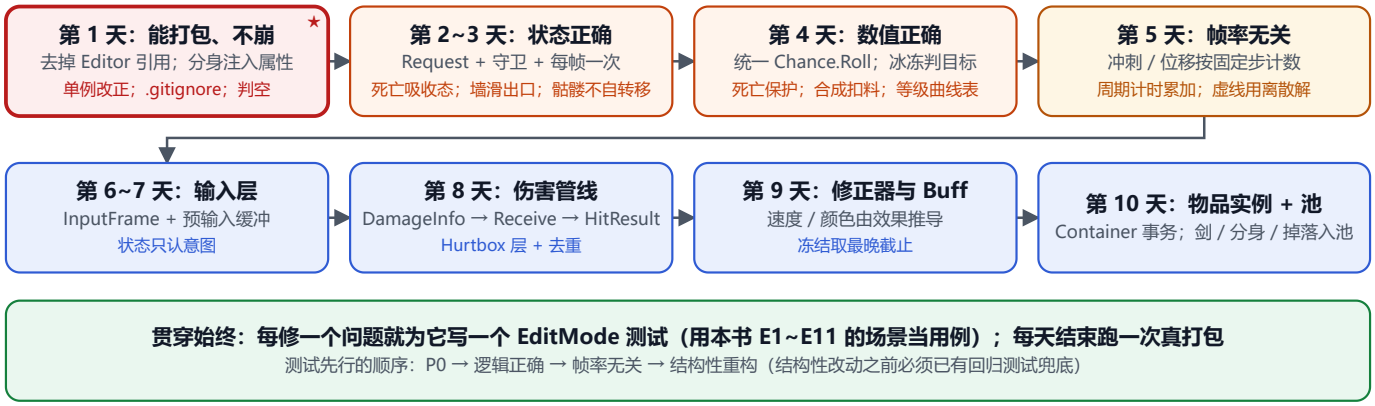
对照项目：飞剑有 `Invoke("DestroyMe", 7)`——池化后必须在归还时 `CancelInvoke`，否则一把被复用的新剑会在 7 秒后被旧的 Invoke 归还；`player.sword` 是裸引用，剑被归还又被别人取出后，玩家按右键会召回“别人的剑”。这正是“句柄 + 代数（generation）”要解决的问题：持有方保存 (id, gen)，访问前比较当前代数，不一致即视为已失效。

## 12.5 工程化：四个“项目之外”的问题

| 问题 | 事实 | 改法 |
| --- | --- | --- |
| 打包失败 P0 | Inventory.cs、Enemy_SkeletonBattleState.cs `using UnityEditor…` | 删除；用 asmdef 分离运行时 / 编辑器代码；CI 每日真打包 |
| .gitignore 写错 P3 | `/[Logs]/`、`/[Build]/`、`/[Builds]/`、`/[MemoryCaptures]/` 是“字符集”写法——`[Logs]` 匹配单个字符 L / o / g / s，而不是目录 Logs；结果 Logs/ 下 7 个日志、UserSettings/ 下 5 个本机文件进了仓库 | `/[Ll]ogs/`、`/[Bb]uild/`、`/[Bb]uilds/`、`/[Uu]ser[Ss]ettings/`、`/[Mm]emory[Cc]aptures/`；已入库的用 `git rm --cached` |
| 大文件 | 482 张 png、6 个 psd 直接进 Git（无 LFS） | 美术资源走 Git LFS；或像公司项目一样用 SVN / Perforce 管理二进制 |
| 没有测试 | 伤害、掉落、合成、状态机都是纯逻辑，却只能进 Play Mode 手测 | EditMode 单元测试 + 可注入的随机数（`IRandom`）与时间；本书 E1~E11 就是“把逻辑抽出来跑”的演示 |

代码规范层面：拼写（`facinDir`、`Eenmy_SkeletonIdleState`、`CreatClone`、`attaclMovement`、`Iventory`、`Wood_Sowrd`）会让搜索失败；同一个类层次里多次声明同名字段（`PlayerDeathState.player`、骷髅各状态的 `enemy`、Battle 的 `playerTransform`）会产生“改了一个、读的是另一个”的隐患；四个名为 Can / Has 的函数有副作用（`CanUseSkill`、`CanbeStunned`、`CanCraft`、`HasNoSword`）。这些都是 P3，但面试官看代码时第一眼就会注意到。

## 12.6 两周重构路线




**理想方案** 图 12-2　两周重构路线：前 5 天只修“错”，后 5 天才改“结构”。顺序背后的原则是风险递增、收益递减，且每一步都有回归测试兜底

**面试里怎么讲重构**

不要说“我会全部重写”。说“我会按风险排序：先保证能打包、不崩溃（1 天），再保证逻辑正确（3 天，每个修复配一个测试），再保证帧率无关（1 天），最后才做结构性重构（5 天），每一步都可以单独上线”。面试官要听的是**排序的理由**和**回归的手段**。

## 12.7 面试追问

追问树

```
项目有哪些性能问题？（Q12-1）
├─ 你怎么定位的？用了什么工具？（Q12-2）
├─ OverlapCircleAll 为什么不好？NonAlloc 有什么坑？（Q12-3）
├─ 对象池怎么设计？复用后出过什么问题？（Q12-4 → 12.4）
├─ Debug.Log 打包后还会执行吗？（Q12-5）
└─ 给你两周你先做什么？（Q12-6 → 12.6）
```

### Q12-1 → Q12-2　项目有哪些性能问题？怎么定位？

**L2**　按“频率 × 单次成本”排序：每帧发生的（骷髅自转移带来的 Find、瞄准时 50 次坐标转换）优先于每次命中发生的（Overlap 数组分配、两个协程），再优先于偶发的（冲刺时实例化分身）。定位用 Profiler：CPU 时间线找最贵的帧、GC Alloc 列找分配、Deep Profile 下钻；内存用 Memory Profiler 看材质实例（`sr.material` 克隆）和泄漏的对象。

**L3** 我会先确认问题是否真实：一只骷髅每秒 25 次 Find 在 PC 上测不出差异，但它的成本和场景对象数、怪物数成正比，所以我把它定性为“结构问题顺带的性能隐患”，而不是“性能瓶颈”。

### Q12-3 OverlapCircleAll 为什么不好？NonAlloc 有什么坑？

**L2** 每次返回新数组，频繁调用产生 GC；不传 LayerMask 时结果包含所有碰撞体（项目开了 Queries Hit Triggers，触发器也算）。NonAlloc 版本要预分配缓冲区，**缓冲区满了会静默截断**，结果数量要看返回值而不是数组长度，缓冲区里还残留着上次的引用。较新的版本推荐带 ContactFilter2D 和 List 结果的重载，List 会自动扩容。

### Q12-5 Debug.Log 打包后还会执行吗？

**L2** 会。Release 包里 Debug.Log 仍然执行：参数字符串照样拼接、照样采集堆栈（可在 Player Settings 里把 Stack Trace 设为 None 降低成本），只是没有编辑器 Console 显示。项目里冷却中每按一次冲刺就拼接一次字符串。做法是封装一个带 `[System.Diagnostics.Conditional("UNITY_EDITOR")]` 或自定义宏的日志函数，调用点连同参数表达式一起在发布构建中被编译器移除。

**常见错误回答**

**错误**：“GetComponent 很慢，所以我把所有 GetComponent 都缓存了。”

**问题**：GetComponent 单次很快，真正的问题是“在循环、属性、每帧里重复调用”；不加区分的“全部缓存”也会带来生命周期问题（缓存了被销毁的组件）。

**更好的回答**：“我用 Profiler 找到热点里的重复查找，把它们挪到 Awake 缓存，或者让 Hurtbox 直接持有实体引用；不在热点里的就保持原样。”

## 12.8 发散场景题

### 场景 12-1 移动端玩 10 分钟后发热降频，帧率从 60 掉到 40，你怎么排查？

为什么会问：移动端性能是客户端岗的硬技能。

先区分 CPU / GPU / 热：Profiler（连真机）看主线程、渲染线程、GPU 时间；看是否“逐渐变慢”（泄漏：对象数、内存、协程数持续增长——本项目的骷髅尸体不回收、材质实例不释放都属于这一类）还是“一直很重”。CPU 侧查 GC 频率与分配峰值、物理查询数量、动画数量；GPU 侧查 Overdraw（2D 透明叠层多）、分辨率、后处理。降频应对：限帧 30 / 动态分辨率 / 降低特效 LOD。

### 场景 12-2　QA 报告“玩一小时后内存涨了 300 MB”，怎么定位?

为什么会问：内存泄漏是线上高频问题。

用 Memory Profiler 在两个时间点各抓一次快照做 Diff：看哪类对象数量增长（GameObject、Material、Texture、托管对象）。项目里可疑点：每个角色 `sr.material` 克隆的材质（对象销毁后不释放）、不销毁的尸体、静态委托或单例持有已销毁对象的回调、Debug.Log 的日志缓冲。托管侧看谁持有引用链（静态字段 → 事件 → 对象）。

### 场景 12-3　新人提交了一行 using UnityEditor，第二天出包失败，如何从流程上避免?

为什么会问：工程化与团队协作。

① asmdef：运行时程序集不引用编辑器程序集，编辑器里就会编译报错；② 提交前钩子 / CI：每次提交跑一次脚本编译（可用 batchmode 只编译不打包），每日跑真打包；③ 代码评审清单加一条“Editor 代码放 Editor 目录”；④ IDE 设置关闭自动 using 补全的部分命名空间。

**本章速记**

- 热点：**Overlap 9 • GetComponent 59 • Find 2 • Instantiate/Destroy 5/11 • 字符串反射 19 • Debug.Log 8**；按“频率 × 单次成本”排序。
- 分配来源：返回数组的 API、字符串、new 引用类型、协程；用 Profiler 的 GC Alloc 验证，不报没测过的数。
- 对象池三坑：**状态没重置、旧引用指向新对象（用代数句柄）、残留 Invoke / 协程。**
- 工程：**UnityEditor using 打包失败（#1）**、.gitignore 字符集写法（#35）、无 LFS、无测试。
- 重构：**先错后构**——P0 → 逻辑 → 帧率 → 结构，每步配测试。

▶ **自测 1：`/[Logs]/` 匹配什么？**

▶ **自测 2：飞剑池化后，`Invoke("DestroyMe", 7)` 会带来什么问题？**

▶ **自测 3：为什么重构要先修“错”再改“结构”？**

PART V

# 面试实战：追问链、场景题与速问速答

前 12 章是“弹药”，这一部分是“打法”：第 13 章把知识点串成 12 条面试官真实会走的追问链，第 14 章是 30 道项目之外的发散场景题，第 15 章是 60 道速问速答与速记卡。

CHAPTER 13

# 12 条追问链：从一句项目介绍追到底

面试官的追问很少是随机的：他会抓住你话里的一个词往下挖三到五层。每条链给出“面试官的路径”和每一步能直接说出口的回答，括号里是展开时去哪一章找论据和数字。

---

**怎么用这一章**

- 先读每条链的路径，自己口头回答一遍，再对照下面的要点
- 注意每条链的第 3~4 步——那里通常是“你项目里的具体问题 + 数字”，是拉开差距的地方
- 最后一节的“危险说法对照表”是面试前一晚必看的

## 13.1 链 1：项目介绍 → 难点 → 验证 → 反思

面试官的路径

介绍一下这个项目 → 你自己做了哪些部分？→ 最难的是什么？→ 你怎么确认它是对的？→ 现在回头看会怎么改？

**① 介绍** 按第 0 章的一分钟版：结构（Entity + 状态机 + 子系统）→ 亮点（动画事件驱动命中、弹反、飞剑四模式、数值物品）→ 反思（用移植实测找到的问题）。

**② 自己做的** 主动说明骨架来自教程，自己改了弹反判定方式、异常状态计时、元素选择、掉落抽样，并做了背包 / 装备 / 合成。“哪些是照着做的、哪些是自己改的”说清楚，比含糊地说“都是我写的”可信。

**③ 难点** 选一个能讲出因果的：状态机的切换入口分散导致死亡可被打断（Ch3）；或弹反判定依赖动画事件同时刻的顺序（Ch4、Ch6）。

**④ 验证** “我把状态机和计时逻辑逐行移植成 C++，用脚本化输入逐帧回放，确认了落地贴墙一帧切两次、死亡后冲刺复活；概率类的用 200 万次蒙特卡洛对照解析解。”——这句话本身就是加分项。

⑤ **怎么改**　第 12 章的两周路线：先修错后改结构，每步配测试。

## 13.2 链 2：状态机 → 非法切换 → 同帧多请求 → 预输入

面试官的路径

状态机怎么设计的？→ 怎么防止非法切换？→ 同一帧有受击和冲刺两个请求听谁的？→ 预输入怎么做？→ 和行为树怎么

① 状态类 + Enter/Update/Exit，Grounded / Air 父类复用输入分支，Enter / Exit 成对设 Animator bool（Ch3.3）。

② 原来没有防：三个切换入口（状态自身、全局冲刺、外部强切）都不检查，死亡后能冲刺复活（E1-S3/S4）；改成 Request + 可打断等级，Death 只接受 Revive（Ch3.9）。

③ 请求带等级，同帧只保留最高的，每帧至多提交一次——顺带消灭了“落地贴墙一帧两跳”（E1-S1）。

④ 输入采样成快照进带时间戳的缓冲，动作在“可取消窗口”里消费 0.15 s 内的指令；窗口由动画 / 时间轴标记而非写死（Ch3.8）。

⑤ 玩家：FSM / HFSM（帧级精确、输入驱动）；AI：行为树（优先级与组合）（Ch3、Ch9）。

## 13.3 链 3：动画事件 → 帧率 → 打断 → 联网

面试官的路径

命中帧怎么做？→ 动画事件在哪个阶段、哪一帧触发？→ 被打断 / 速度为 0 怎么办？→ 两个事件在同一时刻呢？→

① 攻击片段上放 AttackTrigger 事件，回调里 OverlapCircleAll（Ch4.3）。

② Update 之后的动画阶段，在片段时间首次越过事件时刻的那一帧；30 fps 下最多晚 33 ms（Ch4、表 6-1）。

③ 事件会被跳过或推迟；逻辑出口不能只靠“动画结束”事件；瞄准被冲刺打断时 Animator 仍会播掷剑片段并触发掷剑（推断，图 4-2）。

④ 骷髅的命中与关窗同在 0.4167 s，弹反成功依赖 AttackTrigger 排在前面——隐式契约，应让一个事件携带完整语义或用区间窗口（Ch4.4 ③）。

⑤ 帧同步 / 服务器权威都不能依赖动画事件：命中进逻辑时间轴，动画事件只做表现（Ch4.5）。

## 13.4 链 4：Update / FixedUpdate → 冲刺距离 → 卡顿 → 确定性

面试官的路径

`Update 和 FixedUpdate 区别？→ 冲刺在高帧率下距离一样吗？→ 卡一帧会怎样？→ 计时器怎么写才对？→ 要做回`

① 渲染帧 vs 固定 0.02 s 物理步；Update 设的速度在下一个物理步生效（图 5-1）。

② 不一样：冲刺用 Update 计时，120 / 144 fps 时部分情况只有 9 个物理步，4.5 m 而不是 5 m（E10）。

③ 卡 250 ms 时冲刺在这一帧结束，但物理已以 25 m/s 补跑完积压时间，冲到 7.0 m（图 5-2）。改成按物理步计数。

④ 周期计时用累加不用重置；点燃 30 fps 少跳一次、冷却 0 的旋转剑命中数等于帧数（E10）。

⑤ 逻辑固定步 + 输入快照 + 可记录种子的 RNG + 不依赖动画与物理引擎；帧同步再加定点数（Ch5 场景 5-4）。

## 13.5 链 5：命中判定 → 去重 → 弹反 → 伤害管线

面试官的路径

`攻击判定怎么做？→ 会不会一刀打两次？→ 弹反怎么实现、窗口多宽？→ 为什么弹反成功还掉血？→ 你会怎么重新设计`

① 命中帧 OverlapCircleAll；缺 LayerMask、缺 Hurtbox 分层（Ch6.2）。

② 按碰撞体遍历会：多碰撞体的敌人挨多刀；按实体 HashSet 去重（Q6-3）。

③ 受击时处于反击状态且怪物窗口开着就眩晕；成功按键区间宽 1.01 s，原教程写法 1.33 s，都能预按（E2，图 6-2）。

④ 表现（DamageEffect）和结算（DoDamage）是两次独立调用，互不知道结果；同理闪避时也会闪白击退（图 6-1）。

⑤ DamageInfo → 受击方 Receive 唯一裁决 → HitResult；完美窗口从按键起算 0.15 s，落空有收招硬直（Ch6.6）。

## 13.6 链 6：伤害公式 → 概率 → 护甲 → 伪随机 → 权威

面试官的路径

伤害怎么算？→ 暴击率准吗？→ 护甲用减法有什么问题？→ 连续不暴击玩家骂怎么办？→ 联网后在哪算？

① 闪避 → 攻击 + 力量 → 暴击 ×(150 + 力量)% → 减护甲 → 扣血（图 7-1）。

② 暴击用 `≤` 比较 0~99，实际 (c+1)%，暴击率 0 也有 1%；闪避用 `<` 是对的；冰冻破甲判断的是攻击者（E3）。

③ 低伤害被完全免疫、高伤害几乎无视护甲；改百分比 armor/(armor+K)（Q7-3）。

④ 伪随机分布 PRD：未暴击时概率递增、暴击后重置，期望不变、连败被压缩（场景 7-2）。

⑤ 状态同步：服务器算，客户端表现；帧同步：确定性 RNG + 定点数（场景 7-3）。

## 13.7 链 7：Buff → 叠加 → 修正器栈 → 表现推导

面试官的路径

`Buff` 怎么做？→ 冻结和减速同时存在怎么办？→ 多次冻结重叠呢？→ 属性加成的顺序？→ 表现（颜色）怎么跟上？

① 现状：三个 bool + 计时器，协程 / Invoke 恢复（Ch7.6）。

② 三个系统都给 moveSpeed、animator.speed 赋绝对值，谁最后写谁赢（图 7-3）。

③ 每次命中开一个冻结协程，最早的到点就解冻，实际只冻住 26%~85% 的时间（E8）；改成截止时间取最大。

④ (base + ΣFlat) × (1 + ΣPercentAdd) × ΠPercentMult，修正带来源、脏标记缓存（Ch7.8）。

⑤ 颜色同样由“当前所有效果”推导，MPB 写 Shader 参数（Ch11.3）。

## 13.8 链 8：技能系统 → 冷却 → 分身 → 扩展 → GAS

面试官的路径

技能系统怎么设计？→ 冷却在哪扣？→ 分身的伤害从哪来？→ 加一种飞剑要改几处？→ 和 GAS 比缺什么？

① Skill 基类管冷却，三个子类组件挂 SkillManager（Ch8.2）。

② CanUseSkill 查询带副作用，应是 TryActivate 或 CanActivate + Commit（Q8-2）。

③ GetComponentInParent 在独立生成的分身上为 null，范围内有敌人即空引用——分身攻击从未生效（P0，Ch8.3）；创建时注入攻击者属性快照。

④ 现在至少 6 处（枚举、两个 switch、标志、Update、碰撞回调）；改行为组合后只加一个类（Ch8.4）。

⑤ 标签、Effect、AbilityTask、预测与复制（Q8-6）。

## 13.9 链 9：AI → 自转移 → 脱战 → 感知 → 规模

面试官的路径

怪物 AI 怎么写？→ 状态之间怎么复用？→ 追不到玩家怎么办？→ 能隔墙看见吗？→ 100 只怪呢？

① 6 状态 FSM，GroundState 负责发现玩家（图 9-1）。

② Battle 继承了 GroundState，看见玩家每帧自转移 + Find，10 s 252 次；行为不变所以测不出（E7）。

③ 计时器每帧重置永不超时，玩家跳上平台后骷髅在脚下 10 s 转身 547 次；修复后 5 s 脱战（表 9-2）。

④ 能：射线 LayerMask 只有 Player；改为候选 + 遮挡射线 + 最后看见位置（Q9-3）。

⑤ 感知降频、AI LOD、批量 Tick、攻击令牌、对象池（场景 9-1、9-2）。

## 13.10 链 10：背包 → 实例 → 合成 → 掉落 → 存档防刷

面试官的路径

背包用什么结构？→ +5 的剑怎么存？→ 合成怎么保证正确？→ 掉率怎么算？→ 存档和防刷？

① List 保序 + Dictionary 以 SO 定义作键（Ch10.2）。

② 存不了：没有物品实例；引入 ItemInstance（uid + defId + 实例数据）（Q10-2）。

③ 先校验后提交；源码每种材料只扣一份，10 皮 10 铁做 9 件而不是 5 件（E6）。

④ 判定 `≤` 多 1%，“最多 3 件”的截断让 35.4% 变成 32.3%（E5）。

⑤ 存稳定 defId + 版本迁移；产出全在服务器、请求 id 去重（场景 10-2、10-3）。

## 13.11 链 11：生命周期 → 单例 → fake null → 执行顺序 → 池化

面试官的路径

```
Awake 和 Start？→ 单例怎么写？→ Destroy 之后是 null 吗？→ 初始化顺序怎么控制？→ 对象池复用后为什么出
```

① Awake 我自己、Start 我和别人；项目把取组件放在 Start（Ch2.2）。

② 后来者自毁；项目里两个单例写反，可能导致“冲刺只能用一次”（推断，图 2-2）。

③ 帧末才销毁；之后 == null 为真但 `?.` 不认（Ch2.4）。

④ 执行顺序不被子类继承，CharacterStats 300 无效；更推荐显式初始化（Ch2.5）。

⑤ Awake / Start 不再调用，重置要放 OnSpawn；残留 Invoke 与旧引用用代数句柄防（Ch12.4）。

## 13.12 链 12：性能 → 工具 → GC → 工程化

面试官的路径

项目有哪些性能问题？→ 怎么定位？→ 哪些写法会 GC？→ 怎么防止“编辑器能跑、打包失败”？→ 给你两周先做什么？

① 按频率 × 单次成本排：自转移带来的 Find、瞄准的重复坐标转换、Overlap 分配、协程、实例化（Ch12.2）。

② Profiler（CPU 时间线 + GC Alloc + Deep Profile）、Memory Profiler 快照 Diff、Frame Debugger（Q12-1）。

③ 返回数组的 API、字符串、new 引用类型、协程；不报没测过的字节数（Ch12.3）。

④ asmdef 分离、CI 每日真打包；项目里两处 `using UnityEditor` 就是例子（Ch2.6）。

⑤ 先错后构：P0 → 逻辑 → 帧率 → 结构，每步配测试（图 12-2）。

## 13.13 危险说法对照表

| 危险说法 | 为什么危险（会被哪一行代码戳穿） | 更好的说法 |
| --- | --- | --- |
| “我的状态机很完善。” | Player.Update 的全局冲刺能打断死亡 | “结构清晰，但切换入口分散，我发现并修了死亡可被打断。” |
| “动画事件是精确的。” | 事件按帧量化、可被打断、受速度影响 | “动画事件按帧量化，适合表现；严格判定我用逻辑时间轴。” |
| “乘了 deltaTime 就帧率无关。” | 冲刺距离和点燃跳数随帧率变 | “乘 dt 只管速率；时长和离散次数要用固定步或累加计时。” |
| “弹反窗口是 0.33 秒。” | 那是骷髅开窗的时长，按键成功区间约 1 s | “开窗 0.33 s，但判定在命中时刻，按键区间由 1 s 的架势决定。” |
| “暴击率配多少就是多少。” | `≤` 让实际值多 1% | “我测过，判定写成了 <=，统一成一个 Roll 函数后修好了。” |
| “我的飞剑用了策略模式。” | 实际是枚举 + switch + 7 个 bool | “现在是标志位实现，我设计了行为组合的重构。” |
| “分身会攻击敌人。” | 攻击回调里 player 为 null | “分身的攻击动画在播，但伤害有空引用，这是我复盘时发现的 P0。” |
| “怪物 AI 没什么问题。” | 每帧自转移、永不脱战、穿墙可见 | “行为看起来正常，但我用逐帧计数发现了三个结构性问题。” |
| “背包用字典，O(1)。” | 没有实例、没有容量、合成少扣料 | “从定义 / 实例 / 容器 / 事务四层讲。” |
| “我优化了 GC，减少了 X KB 分配。” | 没在 Profiler 里测过的数字 | “我定位到这些分配点，并在 Profiler 里确认过修复前后的差异。”（只报测过的） |

**本章速记**

- 每条链第 3~4 步是“项目里的具体问题 + 数字”：**E1 复活、E2 1.01 s、E3 +1%、E6 9 vs 5、E7 252 次、E8 26%。**
- 第 5 步永远是“我会怎么改”，并且能说出代价。

- 危险说法的共同点：**把设计意图说成实现事实。**

CHAPTER 14

# 30 道发散场景题：从这个项目出发能被问到的“你会怎么做”

前面各章末尾的场景题紧贴源码；这一章是更开放的“设计一个……”“如果要……”。每题给出考点、答题框架和“本项目里能拿来举例的地方”——场景题最怕空谈，用自己项目的一个具体问题开头，答案立刻就有了分量。

---

**答场景题的四步**

- **澄清约束**：规模多大、单机还是联网、平台、要不要策划可配
- **先给结构**：数据 / 逻辑 / 表现三层各是什么，画一张框图
- **再讲关键难点**：时序、一致性、性能、扩展，每个点给一个具体做法
- **落到项目**：“我在 RPGDemo 里就踩过……”，然后说代价与取舍

## 14.1 手感与战斗（8 题）

### S1　让 2D 跳跃“好手感”，你会加哪些机制？

考点：平台跳跃手感的标准工具箱。

可变跳高（松开跳键时把上升速度乘 0.5）、顶点重力减小（apex hang）、下落重力加大（更干脆）、Coyote Time（离地后 0.1 s 仍可起跳）、Jump Buffer（落地前 0.1 s 按跳也算）、最大下落速度、空中水平加速度比地面小。**本项目**：跳跃是 Enter 时一次性 `vy = 20`，重力倍率 5，下落状态里水平速度直接设为 7——没有可变跳高与 Coyote，跑出平台边缘的下一帧就切 Fall。

### S2　列一份“打击感”清单，并说明它们的时序。

考点：表现系统的协同。

命中瞬间同时启动：顿帧（攻防双方动画减速 60~100 ms）→ 受击闪白（0.1 s）→ 击退 / 浮空 → 镜头震动（衰减曲线）→ 特效与音效（音效可略早于画面）→ 伤害数字 → 手柄震动。全部由 HitResult 驱动（Ch6.6），顿帧需要速度修正器栈支持（Ch7.7）。**本项目**：有闪白与击退，但击退方向按受击者朝向计算（#26）。

### S3　设计一个带分支的连招系统（轻 / 重攻击组合出不同招式）。

考点：数据驱动的连招树与取消窗口。

连招树：节点 = 招式（动画、判定、位移、可取消窗口），边 = 输入（轻 / 重 / 方向）+ 时间窗口；运行时当前节点在窗口内消费输入缓冲决定下一节点，超时回根。数据放 SO / 表，编辑器可视化。**本项目**：三段攻击由 comboCounter 线性递增，取消后计数照样 +1（E1-S5），位移和窗口写死在代码。

### S4　做一个三阶段 Boss：50% 血量换招式、30% 召唤小怪、狂暴计时。

考点：AI 的层次化与事件驱动。

外层阶段状态机（Phase1 / 2 / 3，转移条件为血量阈值或计时，转移时播过场并清理场上效果），内层每阶段一棵行为树；阶段切换期间无敌、打断当前技能（用 Ch3 的打断等级）；召唤用刷怪器 + 对象池；血量阈值监听 onHealthChanged（用 event，Ch11）。注意“一次伤害跨过两个阈值”要依次触发。

### S5　实现“子弹时间”：玩家正常速度，敌人和投射物 0.3 倍速。

考点：局部时间缩放。

全局 timeScale 做不到“玩家不变慢”。给实体一个本地时间系数：逻辑用 `dt × localScale`，Animator.speed 与刚体速度按系数缩放（或物理用自定义积分），粒子 simulationSpeed 同步；计时器、Buff 都读本地时间。这正需要 Ch7 的速度修正器栈——项目里冻结、减速、死亡已经在抢写 animator.speed，再加子弹时间只会更乱。

### S6　2D 横版的相机怎么做才舒服?

考点：相机系统的常见参数。

死区（小范围移动相机不动）、前瞻（朝移动方向偏移）、垂直方向只在落地后跟随（避免跳跃时镜头上下晃）、平滑阻尼、边界约束、震动（脉冲 + 衰减）、战斗时拉远 / Boss 时锁定。Cinemachine 提供大部分；注意相机在 LateUpdate 更新，依赖相机位置的视差背景要在它之后读取（Ch11.4）。

**S7　角色控制器：用 Rigidbody2D 还是自己写 Kinematic 控制器?**

考点：物理驱动 vs 运动学驱动的取舍。

动态刚体：碰撞、推挤、斜坡自然得到，但手感受物理参数影响、帧率相关问题多（E10），状态机要和物理“抢”速度（项目里 isKnocked、SetVelocity 互相阻挡）。Kinematic：自己做射线 / Cast 检测与移动，手感完全可控、可确定性，代价是斜坡、平台、推挤都要自己写。精品动作 / 平台游戏多选后者；本项目用前者，冲刺距离随帧率与卡顿变化就是代价之一。

**S8　设计浮空连段：把敌人打上天后可以空中追击。**

考点：受击状态、重力与判定的配合。

受击方新增 Launched / Airborne 状态（重力减小、每次被击中刷新滞空时间，次数上限防无限连）、落地进入 Knockdown；攻击方空中攻击有自己的判定与位移；连段计数衰减（每多一击击退 / 浮空效果递减）。全部写进技能数据与受击规则，伤害管线的 HitResult 告诉表现层该播哪种受击。

## 14.2 系统设计（8 题）

**S9　设计任务系统（击杀 N 只骷髅、收集 M 张皮、和 NPC 对话）。**

考点：事件驱动 + 条件组合 + 存档。

任务配置：前置条件、目标列表（类型 + 参数 + 数量）、奖励；运行时进度存在玩家数据里；目标通过订阅游戏事件推进（EnemyKilled、ItemAdded、DialogueFinished），而不是在各系统里调用任务系统。**本项目**：要先有“击杀事件”——现在骷髅死亡只是切状态 + 掉落，且可能重复 Die（#7），任务计数会被刷。

**S10　设计成就系统，要求不侵入战斗代码。**

考点：观察者模式与解耦。

与任务系统共用事件总线；成就是“监听器 + 条件 + 一次性奖励”；统计类成就（累计伤害）由统计服务汇总后推送。关键是事件定义稳定、带足够上下文（谁、对谁、用什么技能、是否暴击），这又回到 DamageInfo / HitResult（Ch6）。

### S11　设计存档系统：背包、装备、位置、任务进度、设置。

考点：序列化、版本兼容、可靠性。

可序列化的纯数据模型（与 MonoBehaviour 分离）→ 序列化（JSON 便于调试 / 二进制更小）→ 原子写（写临时文件再替换，防断电损坏）→ 版本号与迁移 → 引用用稳定 id（defId / GUID），不能存场景对象引用或数组下标。**本项目**：背包以 ScriptableObject 引用为键，直接序列化就会和资源 GUID 绑死（场景 10-3）。

### S12　做一个技能编辑器给策划用。

考点：工具链与数据结构。

时间轴编辑器（Timeline 扩展或自研 EditorWindow）：轨道 = 动画 / 判定 / 位移 / 状态窗口 / 表现；数据存 SO 或表；编辑器内逐帧预览（在编辑器里驱动 Animator 采样和判定 Gizmos）；运行时由逻辑帧驱动；校验（判定帧不能超出动画长度）。**本项目**：攻击位移 0.15 s、连招窗口 1.5 s、弹反持续 1 s 这些写死的数字都是编辑器的第一批“轨道”（Ch4 场景 4-4）。

### S13　设计对话 / 剧情系统，支持分支和条件。

考点：图结构数据与解释执行。

对话图（节点：台词 / 选项 / 条件分支 / 命令），用 Ink / Yarn 或自研节点编辑器；运行时解释执行，命令节点通过事件调用游戏系统（给物品、开任务）；本地化 key 而不是直接存文本；存档记录变量与已读节点。

### S14　设计刷怪器：波次、权重、上限、不刷在玩家眼前。

考点：配置、对象池、生成时机。

波次配置（时间 / 条件触发、怪物权重表、数量、上限）；生成点筛选（不在镜头内、与玩家距离区间、可达）；对象池 + OnSpawn 重置（Ch2 场景 2-3）；生成当帧不可被选中（Ch2 场景 2-

2）。**本项目**：怪物取组件在 Start、死亡不回收（#32），都需要先改。

**S15　给游戏做本地化（中 / 英 / 日），哪些地方要改?**

考点：数据与显示分离。

所有文本改为 key + 表格；字体回退（中日文字库体积）与 TMP 动态字体；数字 / 日期格式；文本长度变化导致 UI 布局溢出（自动缩放或预留）；图片中的文字单独出图。**本项目**：物品名直接写在 SO 的 itemName 里，`gameObject.name = "Item object - " + itemName` 也用它，改成 key 后要区分“调试名”和“显示名”。

**S16　做新手引导：高亮按钮、强制玩家完成一次弹反。**

考点：引导系统与游戏逻辑的解耦。

引导步骤配置（条件、高亮目标、遮罩、等待事件）；高亮通过 UI 路径 / id 找目标；“完成一次弹反”监听 Parried 事件（需要 HitResult）；引导期间可以临时修改规则（骷髅只出可弹反的攻击、窗口放宽），用修正器实现而不是改源码。

## 14.3 性能与工程（7 题）

**S17　游戏启动要 8 秒，怎么优化?**

考点：启动链路分析。

测：从进程启动到首帧、到可交互，分段打点。常见大头：首场景资源量、Resources 目录（会在启动时建立索引）、大量 Awake / Start 里同步加载、Shader 预热、单例初始化串行。优化：最小启动场景 + 异步加载主场景、按需加载、Shader 变体裁剪与分批预热、初始化分帧。

**S18　包体超标 200 MB，怎么减?**

考点：资源与代码体积。

Build Report 看构成：纹理（压缩格式 ASTC / ETC2、尺寸、图集、去掉未用的 mip）、音频（压缩、流式）、Shader 变体、未引用资源被 Resources 带入、TMP 示例（本项目 Assets 里就带着

TextMesh Pro 的 Examples & Extras）；代码裁剪（IL2CPP + Managed Stripping）；首包 + 分包下载（Addressables）。

### S19　进战斗瞬间卡顿 300 ms，怎么查?

考点：尖刺分析。

Profiler 抓那一帧：同步资源加载 / 实例化（第一次生成某种怪、第一次播特效）、Shader 首次编译、GC、大量 Awake。手段：预加载与预热（对象池 prewarm、ShaderVariantCollection.WarmUp）、异步实例化、分帧生成。**本项目**：瞄准点在 Start 一次性生成 25 个，是“预热”的正面例子；分身、飞剑、雷击则是首次使用时才实例化。

### S20　2D 场景 Draw Call 很高，怎么降?

考点：2D 合批。

Sprite Atlas 合图、同材质、同排序层连续绘制（排序打断合批）、减少材质实例（`sr.material` 克隆，Ch11）、Tilemap 使用 Chunk 模式、UI 动静分离、粒子共享材质。用 Frame Debugger 看每个批次被打断的原因。

### S21　资源加载从 Resources 换成 Addressables，要注意什么?

考点：异步加载与引用计数。

所有加载变为异步，调用方要处理“未加载完成”的状态；加载与释放成对（引用计数），忘记释放就是内存泄漏；分组策略（按场景 / 按功能 / 按更新频率）决定下载粒度；Play Mode 下用不同的模拟模式测试；直接引用（场景里拖的引用）仍会被打进依赖。

### S22　线上需要热修一个数值 bug（比如合成少扣料），怎么做?

考点：热更新方案。

数值类：配置热更（表 / SO 资源）即可。逻辑类（像 CanCraft 的扣料）：需要代码热更——Lua（xLua / ToLua）或 C# 热更（HybridCLR / ILRuntime）；更根本的是联网游戏里产出逻辑在服务器，修服务器即可、不必发客户端。热更前后的存档 / 数据要兼容。

### S23　多人协作时，场景和预制体经常冲突，怎么办?

考点：Unity 项目的协作规范。

Force Text 序列化 + 可读的 YAML；场景拆分（多场景叠加，按功能分人）；预制体嵌套与变体，减少对大场景的直接修改；SmartMerge（UnityYAMLMerge）作为合并工具；二进制资源用锁（SVN / Perforce 的锁定，或 Git LFS lock）。**本项目**：数值全在唯一的 SampleScene 里，任何调参都会改这个大文件。

## 14.4 联网与扩展（7 题）

### S24　把这个 Demo 改成双人联机（状态同步），你的方案?

考点：联网化的全景。

服务器权威：移动、伤害、掉落、合成都在服务器；客户端预测自己的移动与攻击动画（预表现），服务器纠正；其他玩家和怪物插值显示；命中判定服务器回溯；弹反按键带时间戳上传。改造前提：输入抽成 InputFrame（Ch3.8）、伤害管线（Ch6.6）、物品事务（Ch10.6）、随机数只在服务器。项目里“单例拿唯一玩家”的写法要全部改成按 id 查找。

### S25　改成帧同步呢？哪些代码必须重写?

考点：确定性的要求。

逻辑层必须确定：定点数代替 float（伤害公式、移动、计时器）、确定性 RNG 代替 UnityEngine.Random、固定逻辑帧代替 Update / deltaTime、自写碰撞检测代替 Physics2D、命中由逻辑时间轴而非动画事件、容器遍历顺序确定（Dictionary 遍历顺序不能依赖）。表现层（动画、特效、UI）保持 Unity。几乎所有 Player / Enemy / Stats 代码都要重写为“逻辑层”。

### S26　做录像回放，要求能拖动进度条。

考点：输入回放 vs 状态快照。

确定性逻辑下录输入（小、精确），拖动进度需要定期存“关键帧快照”以便跳转后重放；非确定性逻辑只能录状态（每 N 帧的实体状态 + 事件），回放时插值。本项目当前不满足确定性（Ch5 场景 5-4），只能走状态录制。

### S27　单机游戏也要防作弊（排行榜、成就），能做什么?

考点：客户端安全的边界。

关键数据加密 / 混淆存储（防内存修改器直接搜索）、数值校验（伤害上限、时间合理性）、存档签名、排行榜提交附带录像或关键统计由服务器复核、IL2CPP 提高反编译成本。承认边界：客户端没有真正的安全，只能提高成本、事后检测。

### S28　同时支持键鼠、手柄、触屏，输入怎么设计?

考点：输入抽象。

New Input System 的 Action Map：游戏逻辑只认 Action（Attack、Dash、Aim），不同设备绑定不同控件；瞄准在键鼠下是鼠标位置、手柄是右摇杆方向、触屏是拖拽，统一成“瞄准方向”向量；UI 提示随当前设备切换。**本项目**：瞄准直接读 `Input.mousePosition`，冲刺写死 LeftShift，需要先抽象。

### S29　如果把这个项目移植到 Unreal，各部分对应什么?

考点：跨引擎概念映射（游戏客户端岗常考）。

Entity → ACharacter（或 APawn + 组件）；状态机 → 自写 C++ 状态机 / StateTree / 动画蓝图状态机（只做表现）；动画事件 → AnimNotify / AnimNotifyState（区间窗口天然支持）；Stat / Buff / 技能 → GAS（Attribute、GameplayEffect、GameplayAbility、GameplayTag）；Inventory → UObject 数据 + 组件；ScriptableObject → DataAsset / DataTable；单例 → Subsystem；对象池 → 自写或插件；OverlapCircle → SweepMulti / OverlapMulti + Collision Channel。

### S30　用 ECS（DOTS）重写战斗，你会怎么拆?

考点：数据导向设计。

组件（纯数据）：Position、Velocity、Health、Faction、AttackState、HitboxTimeline、BuffBuffer、Cooldown；系统：InputSystem → StateSystem → MovementSystem → HitDetectionSystem（批量查询）→ DamageSystem（事件缓冲）→ BuffTickSystem → DeathSystem → PresentationSync（把结果同步给 GameObject 表现）。好处是批量与多线程；难点是动画事件驱动的命中要改成时间轴、表现层与 ECS 世界的同步（Ch1 场景 1-3）。

**本章速记**

- 场景题四步：**澄清约束 → 先给结构 → 关键难点 → 落到项目。**
- 几乎所有“扩展”题的前置条件都是同几件事：**输入抽象、伤害管线、修正器 / Buff、物品实例与事务、对象池、固定逻辑帧**——这也是第 12 章重构路线的后五天。

CHAPTER 15

# 60 题速问速答与速记卡

每题先给能直接说出口的一两句话，括号里是展开时去哪一章找论据和数字。面试前一天按组过一遍，答不上来的回到对应章节。

---

## 15.1 项目与架构（1–8）

▶ **1. 一句话介绍这个项目。**

▶ **2. 项目规模?**

▶ **3. 为什么 Player 和 Enemy 继承 Entity? 代价?**

▶ **4. 状态为什么是普通 C# 类?**

▶ **5. 模块间怎么通信? 问题在哪?**

▶ **6. 一帧里命中发生在哪个阶段?**

▶ **7. 项目最严重的 4 个问题?**

▶ **8. 你怎么验证这些问题的?**

## 15.2 生命周期与引擎（9–16）

▶ **9. Awake 和 Start 的区别?**

▶ 10. 项目里哪个初始化放错了阶段?

▶ 11. 正确的单例怎么写?

▶ 12. Destroy 后对象立刻是 null 吗?

▶ 13. Script Execution Order 能被子类继承吗?

▶ 14. 为什么编辑器能跑、打包失败?

▶ 15. Unity 序列化器对 [Serializable] 自定义类字段有什么特殊行为?

▶ 16. renderer.material 和 sharedMaterial 的区别?

## 15.3 状态机与动画（17–26）

▶ 17. 父类 Update 里切了状态，子类代码还会执行吗?

▶ 18. 墙滑有什么 bug?

▶ 19. 死亡能被打断吗?

▶ 20. 怎么防非法切换?

▶ 21. 预输入怎么做?

▶ 22. Animator 为什么用 bool 不用 Trigger?

▶ 23. 动画事件什么时候触发?

▶ 24. 瞄准时按冲刺会怎样?

▶ 25. 眩晕闪烁为什么不生效?

▶ 26. 帧同步游戏能用动画事件做命中吗?

## 15.4 时间与帧率（27–32）

▶ 27. Update 和 FixedUpdate?

▶ 28. 设 velocity 要乘 deltaTime 吗?

▶ 29. 冲刺距离随帧率变吗?

▶ 30. 周期计时器怎么写?

▶ 31. 旋转剑 hitCoolDown = 0 有什么问题?

▶ 32. 瞄准虚线为什么比剑高?

## 15.5 战斗与数值（33–42）

▶ 33. 攻击判定缺了什么?

▶ 34. 为什么闪避了还会闪白击退?

▶ 35. 弹反成功窗口多宽?

▶ 36. 弹反成功为什么还掉血?

▶ **37. 击退方向有什么错?**

▶ **38. 暴击率 0 会暴击吗?**

▶ **39. 冰冻破甲写错了什么?**

▶ **40. 敌人每级 +25% 有什么问题?**

▶ **41. 多段冻结为什么会提前解冻?**

▶ **42. 属性修正怎么叠加?**

## 15.6 技能与 AI（43–50）

▶ **43. CanUseSkill 有什么问题?**

▶ **44. 分身为什么打不出伤害?**

▶ **45. 分身朝向为什么会错?**

▶ **46. 飞剑四种模式怎么扩展?**

▶ **47. 骷髅为什么每帧自己切到自己?**

▶ **48. 骷髅为什么追不到也不放弃?**

▶ **49. 骷髅能隔墙看见玩家吗?**

▶ **50. 百怪同屏怎么优化 AI?**

## 15.7 物品、UI 与工程（51–60）

▶ **51. 背包为什么用 List + Dictionary?**

▶ **52. 合成有什么 bug?**

▶ **53. 配置掉率 35.4% 实际是多少?**

▶ **54. 掉落物为什么偏左?**

▶ **55. 事件订阅放哪?**

▶ **56. event 和 public Action 字段的区别?**

▶ **57. 项目有哪些 GC 来源?**

▶ **58. 对象池的三个坑?**

▶ **59. .gitignore 哪里写错了?**

▶ **60. 给你两周你先做什么?**

## 15.8 速记卡

**E1 状态机**

落地贴墙一帧两跳；墙到头慢落 60 帧；死亡后 Shift / 接剑复活；取消第 1 段连招仍 +1。

**E2 弹反**

开窗 0.083~0.417 s；按键成功区间 1.01 s（原写法 1.33 s）；成功仍掉血。

### E3 伤害

暴击 (c+1)%；闪避正确；冰冻判攻击者；RoundToInt 银行家舍入。

### E4 成长

每级 +25% 当前值 → 10 级 7.44×（线性 3.25×）。

### E5 掉落

每件 +1%；0 也掉；35.4% → 32.3%；平均 2.46 件；x 速度偏左。

### E6 合成

每种材料只扣 1 份：10 皮 10 铁 → 9 件（应 5）。

### E7 AI

10 s 自转移 252 次、Find 258 次；平台下 10 s 转身 547 次、从不放弃。

### E8 冻结

重叠协程最早者解冻 → 实际冻住 26%~85%；改截止时间取最大。

### E9 轨迹

半隐式欧拉比连续抛物线低 ½·g·s·Δt·t；1.68 s 时 0.74 m。

### E10 帧率

冲刺 120 / 144 fps 偶尔 4.5 m、卡 250 ms 冲 7.0 m；点燃 30 fps 少一跳；冷却 0 命中数 = 帧数。

### E11 分身朝向

旋转写进循环 → 正确率 50%~83% 随碰撞体个数跳动；分身攻击另有空引用（P0）。

### 四个 P0

using UnityEditor；分身 player 为 null；死亡可打断；单例写反。

PART VI

# 方法论篇：Demo 只是起点，如果重新实现应该怎么做

面试官看不到你的 Demo，只能通过你的语言理解它。这一部分把每个功能讲成“需求场景 → 我在 Demo 里怎么做的 → 这类功能最容易踩的坑（以我踩过的为例）→ 更好的实现方法 → 取舍”，**只讲思路与方法，不讲代码。**第 16 章先统一口径；第 17~21 章依次回答五个问题：状态判定怎么做、滑墙怎么设计、平台关卡怎么搭（Tilemap 原理与操作）、没有输入缓冲时手感怎么优化、背包 / 装备 / 技能系统为什么这样设计。

CHAPTER 16

# 面试口径：面试官看不到 Demo 时怎么讲

同一个 Demo，讲成“我做了冲刺、连招、背包”和讲成“我做了一个 2D 动作原型，复盘时总结出状态判定、输入宽容、数据分层三类常见坑，并设计了对应的正确做法”，在面试官心里是两个完全不同的候选人。这一章给出口播稿、术语对齐表和“把 bug 讲成经验”的转换表。

---

**本章目标**

- 知道面试官“看不到 Demo”带来的三个风险，以及对应的讲法
- 能用 60 秒把关卡画面、操作和系统结构“讲出画面”，必要时在白板上画出来
- 把项目内的命名换成行业通用术语，避免面试官听不懂
- 把 Demo 里的问题统一讲成“这类功能常见的坑 + 根因 + 正确做法”，而不是自曝 bug

## 16.1 30 秒：为什么要专门对齐口径

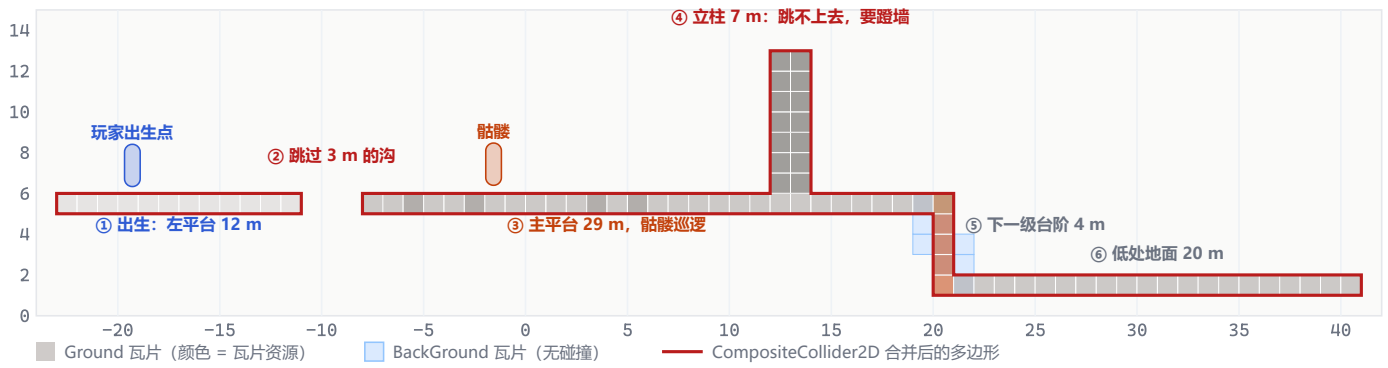
**核心观点**

面试官对项目的判断完全来自你的叙述，所以要同时让他看到三样东西：**画面**（玩起来是什么样）、**结构**（系统怎么组织）、**判断力**（哪里是难点、哪里有坑、正确做法是什么）。Demo 本身只是原型级实现，它的价值不在于“做得多完善”，而在于你能以它为例讲清楚“这类功能应该怎么做、为什么”。因此本部分所有章节都按同一个五句式组织：**场景 → 我的做法 → 常见的坑 → 更好的实现 → 取舍**。

| 风险 | 表现 | 对策 |
| --- | --- | --- |
| ① 听不懂 | 满口项目内命名：“我的 AnimationTrigger 里调 DoDamage” “isBusy 期间不能 Move” | 换成通用术语（16.4）：“在动画的命中帧事件里做伤害判定” “动作后摇期间锁住移动输入” |
| ② 看不出深度 | 功能清单式介绍：“有冲刺、三段连招、弹反、飞剑、背包、合成” | 每个功能只用一句话描述，然后挑 2~3 个讲“难点在哪、坑是什么、怎么做才对” |
| ③ 自毁或夸大 | “我的代码有 36 个 bug”或“我实现了一套完整的技能系统” | 讲成“复盘时总结出的几类典型坑”（16.5），并明确区分“Demo 里已实现的”和“我设计好的改进方案” |

## 16.2 讲出画面：关卡、操作、系统三张图

面试官最先需要的是“画面感”。下面是可以直接在白板上画的关卡草图（几何来自场景里的瓦片地图）：




**现状** 图 16-1　白板上可以照着画的关卡草图（由场景文件解析得到，1 格 = 1 m）。讲的时候按 ①→⑥ 的顺序“走一遍”，面试官就有了画面；第 19 章会用同一张图讲关卡度量

**现状问题** P2 关卡度量先画后定：3 m 沟太宽松，7 m 立柱依赖不稳定的墙交互（F4 / F5）

| 操作 | 玩家看到的 | 关键参数（说得出数字 = 你真的做过） |
| --- | --- | --- |
| 左右移动 / 空格跳跃 | 城堡里的像素小人，跳起有起跳 / 下落两段动画 | 移速 7 m/s；起跳初速 20、重力倍率 5 → 顶点约 3.9 m、滞空 0.8 s |
| 空中贴墙 | 自动进入滑墙，慢速下滑；按跳向反方向蹬出 | 滑墙 2 m/s；蹬墙速度（离墙 4，向上 18） |
| Shift 冲刺 | 瞬间冲出一段距离，原地留下一个会挥刀的残影 | 25 m/s × 0.2 s ≈ 5 m，冷却 1 s |
| 左键普攻 | 三段连招，每段带一点向前位移 | 位移 2 / 1 / 3；1.5 s 内连续按才接下一段 |
| Q 弹反 | 摆出架势，骷髅出刀时被打中则骷髅被弹开眩晕 | 架势 1 s；骷髅出刀动作的第 0.08~0.42 s 是“可弹反窗口”，头顶亮提示 |
| 右键瞄准 / 松开掷剑 | 按住出现 25 个点的抛物线，松开飞剑出手，插在墙或敌人身上；再按右键召回，飞回途中还能伤敌 | 出手速度 (35, 28)、重力倍率 4.5；四种剑型（普通 / 弹跳 / 穿刺 / 旋转），场景里用的是普通 |
| 背包界面 | 骷髅死亡掉落 1~3 件物品；背包、材料仓库、装备栏、合成格 | 5 种物品；骷髅最多掉 3 件；Nice_Armor = 兽皮 ×2 + 铁 ×1 |

第三张图是系统结构（第 1 章图 1-2、图 1-5），口头版只需要一句：**“角色由状态机驱动，关键帧由动画事件驱动，属性、技能、背包各是一个子系统，彼此通过玩家对象和几个单例连接。”**——最后半句恰好是后面要讲的坑之一。

## 16.3 五句式：每个功能都这样讲

| 句子 | 回答什么 | 例：滑墙（第 18 章） |
| --- | --- | --- |
| ① 场景 | 这个功能解决玩家的什么问题 | “横版动作里，滑墙给玩家在空中‘抓住’的机会，也是垂直移动的手段。” |
| ② 我的做法 现状 | Demo 里怎么实现的（一两句，不念代码） | “空中下落时用一条腰部射线检测前方墙面，碰到就进入滑墙状态，把下落速度压到 2 m/s；按跳就向反方向蹬出。” |
| ③ 坑 | 这类功能常见的问题，用 Demo 的现象举例 | “滑墙最容易漏的是退出条件和检测覆盖：我一开始只有一条腰部射线，墙在半空结束时角色会对着空气继续滑；接近墙顶时脚还贴着墙、射线已经越过墙顶，就出现‘贴着墙却滑不了’的死区。” |
| ④ 正解 理想 | 更好的实现方法（分层、流程、参数） | “墙检测至少用脚、腰两条射线或一个盒子投射，按法线判断是墙；状态的出口必须覆盖‘进入条件不再成立’；再加粘墙时间、墙宽限时间和蹬墙后的强制离墙时间。” |
| ⑤ 取舍 | 代价与适用范围 | “检测多了有成本，但每帧几条射线可以忽略；更重要的是先决定‘单墙能不能爬’，这是设计决定，不应该是参数的副作用。” |

**为什么第 ③ 句要用“这类功能常见的坑”开头**

面试官想考察的是“你知不知道这里有坑”，而不是“你的代码有没有 bug”。用“常见的坑是……，我在 Demo 里就遇到了……”的说法，**既诚实，又把话题从“你的失误”转到“你的经验”**。再配一个你量化过的数字（比如“墙顶附近有一段 1.3 m 的死区”），可信度最高。

| 你要讲的东西 | 时态与句式 | 例：滑墙 |
| --- | --- | --- |
| 现状 Demo 里做了的 | 过去时 + 证据：“我当时是……，后来测到……” | “我当时用一条腰部射线判墙，后来发现接近墙顶时会有一段贴着墙却滑不了的死区。” |
| 模型验证 只在移植模型里验证过的 | 完成时 + 限定范围：“我把规则放进移植模型对比过……” | “我在 C++ 移植模型里加了一条脚部射线，反应慢到 100 ms 也能翻上墙——这是模型结论，Unity 工程里还没改。” |
| 理想 只设计、没实现的 | 条件句 + 理由：“如果重做，我会……因为……” | “如果重做，我会把检测抽成每个物理步只跑一次的感知层，状态机只读结果。” |

三种时态对应第 0 章 0.2 的标记：橙色 = 过去时，绿徽标“模型验证” = 限定范围的完成时，绿色 = 条件句。**最危险的是把第三行说成第一行**——面试官只要追问一个细节（参数、调试过程、遇到的问题），就能分辨“做过”和“想过”。

## 16.4 术语对齐表：项目内命名 → 行业通用说法

| 项目里的名字 | 面试时这样说 | 一句话解释 |
| --- | --- | --- |
| Entity / Player / Enemy | 角色基类、玩家角色、敌人角色 | 公共的检测、翻转、受击放在基类 |
| PlayerState + Enter / Update / Exit | 有限状态机，状态类用模板方法 | 每个状态一个类，进入 / 每帧 / 退出三段 |
| PlayerGroundedState / AirState | 父状态（用于复用地面 / 空中的公共转移） | 注意：只是代码复用，还不是真正的分层状态机 |
| groundCheck / wallCheck | 地面 / 墙面探测点（射线传感器） | 从这两个点发射线判断是否着地、是否贴墙 |
| isBusy / BusyForSeconds | 动作后摇输入锁 | 攻击结束后 0.1 s 内不允许切换到移动 |
| AnimationTrigger / AttackTrigger | 动画事件（UE 叫 AnimNotify）：动画结束通知、命中帧 | 在动画的特定帧回调逻辑 |
| comboCounter | 连招段数 | 1.5 s 内连续攻击推进到下一段 |
| CounterAttack / canBeStunned | 弹反（招架）与可弹反窗口 | 敌人出刀前的一段时间可被弹反 |
| Skill.CanUseSkill | 技能冷却与释放入口 | 实际是“尝试释放”：冷却好了就直接释放 |
| SkillController_ThrowSword | 投射物（飞剑）控制器 | 普通 / 弹跳 / 穿刺 / 旋转 + 召回 |
| Stat + modifiers | 属性 + 属性修正器 | 基础值加上所有修正的总和 |
| ignite / chill / shock | 元素异常状态：点燃 / 冰冻 / 感电 | 持续伤害、减速、连锁雷击 |
| inventory / stash / equipment | 装备背包 / 材料仓库 / 装备栏 | 三个容器，结构相同 |
| ItemData（ScriptableObject） | 物品配置（物品模板） | 策划可在编辑器里配的数据资源 |

| | | |
|---|---|---|
| ItemDrop | 掉落表 | 每件按概率筛选，再随机抽最多 3 件 |
| Ground Tilemap + Composite Collider | 瓦片地图地形 + 合并碰撞体 | 79 格地形合并成 2 个多边形 |

## 16.5 把 bug 讲成经验：12 条“坑 → 正解”口径

下表左列是 Demo 里实际出现的现象（前 15 章和本部分实验测出来的），中间列是你在面试里应该说的“这类功能的坑”，右列是正确做法。**面试时只说中间和右边两列，左列作为例子带一句即可。**

| 现状：Demo 里的现象 | 抽象成的“坑” | 理想：正确做法（一句话） | 章 |
| --- | --- | --- | --- |
| 死亡后按冲刺能“复活” | 状态切换入口分散，没有打断优先级 | 所有切换走一个带守卫的请求入口，死亡是只接受“复活”的吸收态 | 3 |
| 落地瞬间贴墙，一帧切两次状态 | 切换后旧状态的代码还在执行 | 请求-提交：本帧只记录请求，帧末统一切换一次 | 3 |
| 斜坡超过约 25.5° 就判定离地 | 单点单射线做地面判定 | 每个物理步一次的感知层：形状投射 / 接触法线分类地面、墙、天花板 | 17 |
| 墙在半空结束，角色对着空气继续滑 | 状态缺少“进入条件不再成立”的出口 | 每个状态的出口覆盖进入条件的否定；滑墙检测用多条射线 + 覆盖率 | 3 · 18 |
| 能不能单墙爬上去取决于手速 | 可玩性规则成了参数的副作用 | 先定设计（允许 / 禁止单墙攀爬），再用强制离墙时间、推力把它做成稳定规则 | 18 |
| 跑出平台边缘的下一帧按跳没反应 | 没有输入宽容（土狼时间、预输入） | 离地 0.1 s 内仍可起跳；落地前 0.1~0.15 s 的跳跃键缓冲生效 | 20 |
| 冲刺距离随帧率和卡顿变化 | 用渲染帧计时去控制物理位移 | 需要严格时长的位移按物理步计数，周期计时用累加 | 5 |
| 弹反窗口实测宽达 1 s，还能“预按” | 窗口由状态持续时间而非按键时刻决定 | 完美窗口从按键时刻起算（约 0.15 s），落空有收招硬直 | 6 |
| 闪避了还闪白，弹反了还掉血 | 表现与结算分开调用，没有唯一裁决点 | 攻击方构造伤害信息，受击方统一裁决并返回结果，表现只读结果 | 6 |
| 多段冻结只冻住 26%~85% 的时间 | 多个系统直接改同一个字段 | 修正器栈：效果只登记修正，最终值由所有修正推导 | 7 |
| 合成少扣一份材料 | 物品操作不是事务 | 先只读校验全部条件，再一次性提交；联网后放到服务器 | 10 · 21 |
| “+5 的剑”无处存放 | 以物品定义作键，没有物品实例 | 定义 / 实例 / 容器 / 操作四层分离 | 10 · 21 |

**口径红线**

① **不报没测过的数字**：本书的实测来自“把逻辑移植成 C++ 逐帧复现”，面试时就这样说（“我把状态机逻辑抽出来做了逐帧复现”），不要说成“我在 Unity Profiler 里测的”。② **不把方案说成已实现**：第 17~21 章的“更好的实现”是设计方案，说“我设计了 / 我会这样改”，不要说“我实现了”。③ **不贬低教程**：说“骨架参考了一门教程，我在此基础上改了弹反判定、异常状态、掉落和背包”，比“教程写得很烂”更专业。

## 16.6 方法论版的项目介绍

### 1 分钟版

“这是一个 Unity 2D 横版动作 RPG 原型：一段城堡关卡，玩家能跑、跳、滑墙蹬墙、冲刺、三段连招、弹反，还能掷出可召回的飞剑；骷髅会巡逻、追击、出刀，死亡掉落装备和材料，可以穿戴和合成。

实现上，角色用有限状态机驱动，命中、动作结束这些关键帧放在动画事件里，属性、技能、背包各是一个子系统。

做完之后我做了一次系统性的复盘，把问题归成三类：**一是状态与判定**——切换入口分散、地面和墙只用单射线判断；**二是手感**——没有输入宽容、计时依赖帧率；**三是数据组织**——物品没有实例层、多个系统直接改同一份数据。每一类我都设计了更合理的做法，也用逐帧复现的方式量化过问题的影响。”

### 3 分钟版：在 1 分钟版之后，按面试官的兴趣展开其中两类

**如果对方关心手感**（动作 / 平台类岗位）：讲第 17、18、20 章——“着地判定我原来是脚底一条 0.15 m 的射线，真正伸出脚底的只有 4 cm，坡度超过约 25.5° 就判定离地；更好的做法是每个物理步做一次感知，用形状投射或接触法线区分地面、墙和天花板。滑墙我原来只有一条腰部射线，墙顶附近有一段死区，导致单墙攀爬能不能成功取决于手速；正确做法是多射线 + 明确的设计规则。手感上我量化过：在‘跑出平台边缘后 20 ms 左右按跳’的玩家里，没有土狼时间只有 24% 能跳起来，加 0.1 s 土狼时间是 96%。”

**如果对方关心系统**（RPG / 通用客户端岗位）：讲第 21 章——“背包、装备、技能我会统一成配置、实例、规则、表现四层，装备和 Buff 都通过属性修正器影响属性，技能是‘配置 + 实例 + 激

活管线 + 效果’ 。Demo 里没有实例层，所以‘+5 的剑’做不了；合成不是事务，少扣了一份材料，这些都是分层能直接解决的问题。”

## 16.7 面试防守：关于 Demo 本身的追问

追问树

```
介绍一下你的 Demo
├─ 我看不到 Demo，你怎么证明是你做的？（Q16-1）
├─ 你说的这些改进，你都实现了吗？（Q16-2）
├─ 为什么用教程的框架？你自己做了什么？（Q16-3）
├─ 最得意的部分是什么？（Q16-4）
└─ 最大的不足是什么？如果重做你会先改什么？（Q16-5）
```

### Q16-1　我看不到 Demo，你怎么证明是你做的?

**L2**　用只有作者才知道的细节：参数（移速 7、跳跃初速 20、重力倍率 5、冲刺 25 × 0.2 s、蹬墙速度 (4, 18)）、关卡几何（立柱 7 m 高、跳跃顶点只有约 3.9 m，所以设计成必须蹬墙）、实现选择的原因（为什么弹反改成“受击时判定”）、以及踩过的坑的具体现象（墙顶附近贴着墙却滑不了）。面试官判断真假靠的是“细节密度”和“因果链”。

### Q16-2　你说的改进都实现了吗?

**L2**　如实区分：“Demo 里是原型实现；改进方案是我复盘之后设计的，其中关键的判断我做过验证——比如把状态机和计时逻辑抽出来逐帧复现，确认了问题存在、量化了影响。下一步按优先级先修会导致错误结果的问题，再做结构性改造。”

**L3**　如果对方追问“为什么不直接改好”：说明取舍——Demo 的目标是验证玩法和练习系统设计，复盘的价值是把经验抽象出来；真正改的时候要先补回归测试，再动结构（第 12 章的路线）。

### Q16-3　为什么用教程的框架? 你做了什么?

**L2**　“框架参考了一门 Unity 2D RPG 教程，好处是能快速搭出可玩的东西；我自己改动的有：弹反从‘每帧检查’改成‘受击时判定’，异常状态计时抽成通用函数，元素伤害改成按最大值选择，掉落改成‘先筛选再无放回抽样’，背包、装备、合成是最后一次提交自己加的。更重要的是，做完之后我把它当成一份‘别人的代码’来做 Code Review，这个过程让我真正理解了每个设计的代价。”

### Q16-4 → Q16-5　最得意的部分？最大的不足？

**得意**　选“动画事件驱动命中 + 弹反”这条链：它把动画、状态机、判定、数值四个系统串起来，能讲出时序（第 6 章的窗口推导）。

**不足**　选“状态与判定”：“我的状态切换没有统一入口，地面和墙的判定也太简单，所以出现了死亡可被打断、墙顶死区这类问题。重做的话我会先建感知层和带守卫的状态机，这两样是动作游戏手感的地基。”——不足要选**有通用价值、你能讲出正解**的，不要选“美术不好”“没时间”。

**本章速记**

- 面试官看不到 Demo：要让他看到**画面、结构、判断力**；白板上画关卡草图 + 操作表。
- 五句式：**场景 → 我的做法 → 常见的坑 → 更好的实现 → 取舍。**
- 说通用术语：探测点 / 射线传感器、动作后摇输入锁、动画事件、可弹反窗口、属性修正器、物品配置。
- 红线：**不报没测过的数、不把方案说成已实现、不贬低教程。**

CHAPTER 17

# 状态判定：着地、贴墙、顶头与边缘怎么检测

问题 ①：“平台跳跃游戏里不同状态怎么判定？当前挂一个 Transform 做射线检测的做法合适吗？能否进一步优化？”——答案是：原型阶段合适，但它把“感知”和“决策”混在一起，并且只用一个点、一条线去代表一个有体积的角色。这一章先拆清楚需要判定什么，再列出单射线方案的六个坑，最后给出“每个物理步一次的感知层”这一更好的结构。

---

**本章目标**

- 列出平台动作游戏需要的“环境判定”：着地、坡度、贴墙、顶头、边缘、站在什么上面
- 用场景里的真实尺寸算清单射线方案的边界：可识别坡度约 25.5°、墙检测高度 1.32 m
- 比较射线、多射线、形状投射、接触法线、触发器五种检测手段的适用场景
- 说清“感知层 → 状态机 → 表现”的分层，以及宽限、滞回这两个让判定“不抖”的机制

## 17.1 口径：30 秒怎么讲 • 现状 vs 理想

**面试口播**

现状 **我的做法**：角色身上挂两个空子物体作为探测点，脚底那个向下发 0.15 m 的射线判断着地，腰部那个向朝向方向发 0.25 m 的射线判断贴墙，各个状态在每帧更新里自己调用这两个检测。

**评价**：原型阶段这样做是合适的——直观、成本低、编辑器里能用 Gizmos 画出来调。但它有两个结构性问题：一是**用一个点代表有体积的角色**（坡度超过约 25.5° 就判定离地、矮墙测不到、墙顶有死区）；二是**感知散落在状态里、在渲染帧里做**（同一帧多次查询、结果和物理不同步、没有宽限和滞回）。

理想 **更好的做法**：每个物理步只做一次“感知”，用形状投射或接触法线把接触面分成地面 / 墙 / 天花板，结果写进一个“接触信息”结构，加上宽限时间和滞回；状态机只读这个结构做决策。

现状 → 问题 → 理想　**状态判定**

| 维度 | 当前做法（Demo 现状） | 存在的问题 | 理想重构方案 | 收益 / 代价 |
|---|---|---|---|---|
| **检测形状** | 脚底 1 条 0.15 m 射线（只伸出脚底 0.042 m）、腰部 1 条 0.25 m 射线 | P2 坡度超过约 25.5° 就判成离地；站在平台边缘、身体中心悬空时射线打空（F1） | 从碰撞体推导：向下形状投射 / 多射线 + 读接触点法线<br>**模型验证：F1 几何计算** | 坡度上限从 25.5° 变为由“可站坡度”参数决定 |
| **检测时机** | 各状态在 Update 里各自查询，起点是插值后的 Transform | 隐患 同一帧查询三四次；起点是插值后的位置，落地判定可能晚一个物理步（推断） | 每个物理步只做一次，起点用刚体物理位置<br>**设计方案 · 未实现** | 查询次数下降；结果与物理同步 |
| **分类依据** | 只靠 LayerMask（Ground 层）；Wall 层定义了却没用 | P2 墙顶被当成地面、地面块的侧面被当成墙；单向平台、移动平台、敌人头顶无法区分 | 按法线分地面 / 墙 / 天花板 + 按组件识别单向平台、移动平台<br>**设计方案 · 未实现** | 墙顶、墙侧、单向平台不再混淆 |
| **结果形态** | 每帧重算的布尔值，没有记忆 | 局限 不知道“离地多久、墙在哪侧”，土狼时间、墙宽限都无从实现 | 接触信息结构：着地、坡度、墙在哪侧、脚下对象、离地 / 离墙时长<br>**设计方案 · 未实现** | 状态机只读一份数据，可单测 |
| **稳定性** | 没有宽限、没有滞回 | P2 边缘上来回切换着地 / 下落；离边一帧就不能跳（准时型玩家起跳成功率 24%，F2） | 土狼时间 / 墙宽限 + 进入与退出用不同阈值<br>**设计方案 · 未实现** | 边缘不抖、晚按也能跳 |
| **物理材质** | 玩家使用零摩擦材质，避免贴墙被挂住 | ✓ 无问题 零摩擦材质避免贴墙被挂住，做法正确 | 保持<br>**保持 · 现状已正确** | |

现状问题：P2 ×3　隐患 ×1　局限 ×1　✓ 无问题 ×1　（P0~P3 编号见附录 A，分级说明见 0.2）

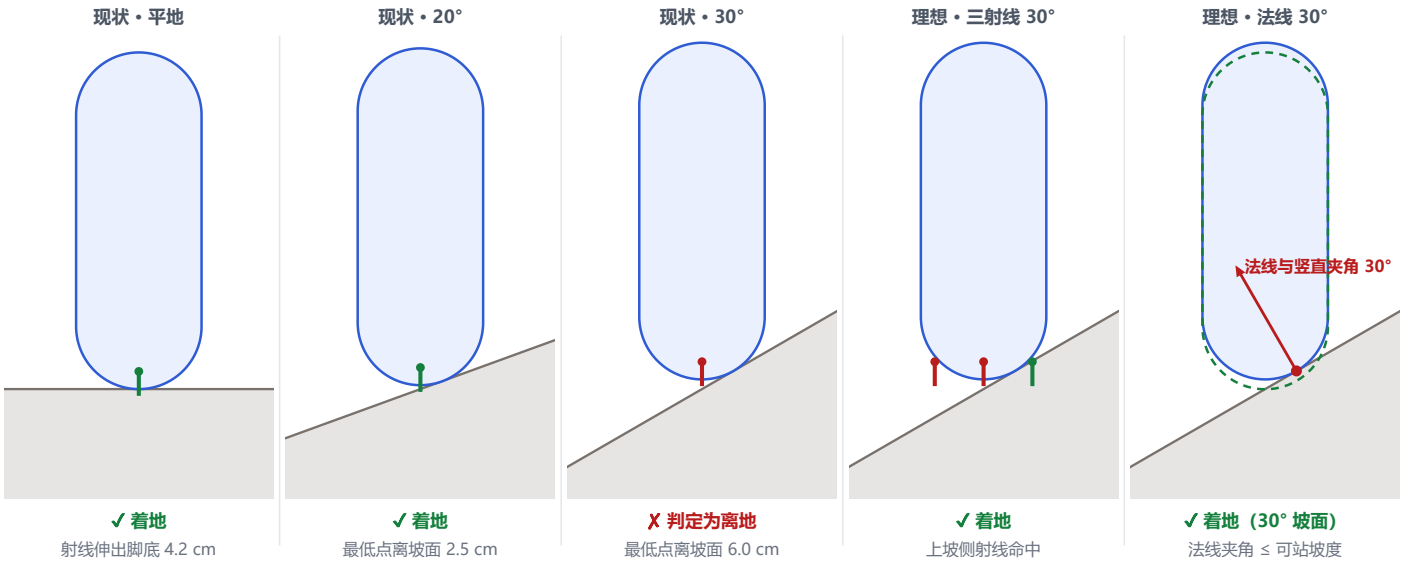
## 17.2 先拆需求：到底要判定什么

| 判定 | 用途 | 需要的信息 | Demo 现状 |
| --- | --- | --- | --- |
| 着地 | 能否起跳、切换地面 / 空中状态、落地动画与音效 | 是否接触、接触面法线、离地多久了 | 脚底单射线，只有 是 / 否 |
| 坡度 | 上下坡速度、能否站立、下坡贴地 | 法线与竖直方向的夹角 | 无 |
| 贴墙（左 / 右） | 滑墙、蹬墙跳、撞墙停止 | 哪一侧、墙的覆盖高度、法线 | 腰部单射线，只看前方 |
| 顶头 | 跳跃撞到天花板时截断上升速度、角落修正 | 上方接触、法线朝下 | 无（靠物理碰撞自然停住） |
| 边缘 | AI 巡逻转身、角色“边缘踮脚”动画、防止冲刺掉落 | 前方脚下是否有地面 | 骷髅把地面探测点放在身体前方 0.144 m，兼作边缘检测 |
| 站在什么上面 | 单向平台下跳、移动平台带动、冰面 / 泥地手感、脚步声 | 接触对象的类型 / 速度 / 材质 | 只有 Ground 一层，不区分 |
| 能否站起 / 容纳 | 蹲下后能否起身、狭窄通道 | 头顶空间的重叠查询 | 无 |

这张表本身就是面试里的好答案：**先把“状态判定”拆成具体问题，再谈用什么技术。**很多候选人一上来就说“用射线检测”，被追问“坡怎么办、单向平台怎么办”就卡住了。

## 17.3 用场景尺寸算一算：单射线的边界

玩家胶囊体宽 0.77 m、高 2.07 m，脚底探测点在胶囊最低点上方 0.108 m，射线长 0.15 m——**真正伸出脚底的只有 0.042 m（约 4 cm）**。站在斜坡上时，胶囊是用圆弧底部的切点接触坡面的，中心正下方的最低点会悬在坡面上方 $r(1/\cos\theta - 1)$。当这个间隙超过 4 cm，中心射线就够不着地面了：




现状 vs 理想 图 17-1　斜坡上的地面判定（几何按场景数值计算：胶囊半径 0.385 m、探测点在最低点上方 0.108 m、射线长 0.15 m）。单中心射线能识别的最大坡度约 25.5°；三射线（±0.3 m）约 78°；接触法线 / 形状投射没有这个上限，而且还能告诉你坡度是多少

| 坡度 / ° | 胶囊最低点离坡面 / m | 现状：单射线 0.15 m | 单射线加长到 0.30 m | 理想：三射线 ±0.3 m |
|---|---|---|---|---|
| 0 | 0.000 | ✓ | ✓ | ✓ |
| 10 | 0.006 | ✓ | ✓ | ✓ |
| 20 | 0.025 | ✓ | ✓ | ✓ |
| 25 | 0.040 | ✓ | ✓ | ✓ |
| 26 | 0.043 | ✗ | ✓ | ✓ |
| 30 | 0.060 | ✗ | ✓ | ✓ |
| 35 | 0.085 | ✗ | ✓ | ✓ |
| 45 | 0.160 | ✗ | ✓ | ✓ |

表 17-1　不同坡度下各方案能否判定着地。注意“加长射线”也能识别更陡的坡，但代价是起跳后的前几厘米仍判定为着地（见坑 ②）

当前关卡全是直角平台，所以这个坑没有暴露；但面试官问“如果加斜坡呢？”时，你能算出“约 25.5° 就会出问题、为什么、怎么改”，就比“我会加斜坡检测”有说服力得多。

## 17.4 单射线方案的六个坑

### 坑 ①　一个点代表一个有体积的角色

**现状** 坡度超过约 25.5° 判定离地；站在平台边缘、身体中心已经悬空但胶囊圆弧还搭在边上时，射线打空 → 播下落动画、按跳无效，或者在“着地 / 离地”之间每帧抖动。

**理想做法** 用覆盖脚底宽度的检测：两三条射线（左、中、右）取“任一命中”，或者用和脚底同宽的盒子 / 胶囊做向下的形状投射。

### 坑 ②　射线长度是一个两头为难的魔法数字

**现状** 0.15 m 里有 0.108 m 在碰撞体内部，只伸出脚底 4 cm。太短：下台阶、过接缝、下坡时会短暂判定离地；太长：起跳后的前几厘米仍判定着地，可能重复起跳或落地动画提前播放。

**理想做法** 长度只负责“即将着地”的预判，“是否着地”由实际接触决定；起跳后加一个很短的“着地判定屏蔽时间”（例如 0.05 s），避免起跳那一刻被判回地面。

### 坑 ③　墙检测只有一条腰部射线

**现状** 墙检测高度在脚底上方 1.32 m，伸出碰撞体表面 0.188 m。矮于 1.32 m 的墙测不到；接近墙顶时脚还贴着墙、射线已越过墙顶，形成“贴着墙却滑不了”的死区（第 18 章 F5 量化）；只看朝向一侧，背对墙时不算贴墙。

**理想做法** 脚部 + 腰部（必要时 + 头部）两到三条射线，或侧向盒子投射；左右两侧都检测，结果里带上“哪一侧”和“覆盖了多少高度”。

### 坑 ④　感知散落在各个状态里，在渲染帧里做

**现状** 地面状态、下落、冲刺、滑墙各自调用检测，同一帧最多查询三四次；检测在 Update 里做，而物理世界只在物理步更新，两者节奏不同。更隐蔽的一点：玩家刚体开了插值，Update 里的 Transform 位置是两个物理步之间的**插值位置**，比真实的物理位置落后最多一步——高速下落时（约 15 m/s）可能差出 0.3 m，射线从“落后的位置”打出去，落地判定会晚一个物理步 推断。

**理想做法** 在每个物理步（FixedUpdate）里做**一次**感知，射线起点用刚体的物理位置而不是 Transform；结果缓存起来，本步所有状态都读这一份。

**坑 ⑤　只靠 LayerMask，不看法线和对象类型**

**现状** 地面与墙在同一个 Ground 层（项目里定义了 Wall 层但没用），检测只知道“碰到了 Ground”，不知道是地面、墙还是墙顶；单向平台、移动平台、敌人头顶、箱子都无法区分。另外项目开启了 “Queries Start In Colliders”，脚底射线的起点在玩家自己的碰撞体内部——之所以没打到自己，只是因为 LayerMask 恰好不含 Player 层；哪天把掩码改成 Everything，第一个命中的就是自己。

**理想做法** 按接触法线分类：法线与竖直向上的夹角 ≤ 最大可站坡度（如 45°）是地面，接近水平是墙，朝下是天花板；对象类型用组件或标签识别（单向平台、移动平台、可攀爬墙）；查询时显式排除自身碰撞体。

**坑 ⑥　判定是每帧重算的布尔值，没有记忆**

**现状** 离开地面的那一帧就切到下落，按跳无效（没有土狼时间，第 20 章 F2）；判定结果没有滞回，在边界上会来回切换。

**理想做法** 感知结果带时间信息——“上次着地是多少秒前” “上次贴墙是多少秒前”；状态机按时间做宽限（离地 0.1 s 内仍可起跳、离墙 0.1 s 内仍可蹬墙）；进入和退出用不同阈值（滞回），例如“连续 2 个物理步命中才算着地”。

## 17.5 五种检测手段怎么选

| 手段 | 原理 | 优点 | 缺点 | 适合 |
| --- | --- | --- | --- | --- |
| 单 / 多射线 | 从探测点沿方向打线 | 最便宜、最直观、易调试 | 覆盖不全（点状）；长度难调；不知道体积 | 原型；辅助判断（边缘、前方是否有墙） |
| 形状投射 | 把与脚底同宽的盒子 / 胶囊沿方向“扫”一段距离 | 覆盖整个脚底；返回命中点、法线、距离 | 比射线贵一些；要处理“起点已重叠” | 地面 / 墙 / 天花板的主判定；“即将着地”预判 |
| 接触点法线 | 读取刚体当前所有接触点及其法线（物理引擎已经算好） | 零额外查询；就是“真实的接触” | 只有真正碰上才有；动态刚体才有 | 动态刚体角色的着地 / 贴墙判定 |
| 重叠查询 / 触发器 | 脚底放一个小盒子，查询与谁重叠 | 简单，适合“区域”判断 | 没有法线；触发器回调时机在物理步后 | 水、梯子、风区、伤害区 |
| 运动学控制器的移动结果 | 自己做移动时逐轴投射，碰撞结果就是感知 | 手感完全可控、可确定性 | 斜坡、台阶、推挤都要自己写 | 精品平台跳跃、格斗、需要回放 / 帧同步 |

**推荐组合（动态刚体角色，就是这个 Demo 的情况）**

① **主判定**：每个物理步读接触点，按法线分类出地面 / 左墙 / 右墙 / 天花板；② **补充判定**：向下做一次很短的形状投射，用于“即将着地”（提前切落地动画、下坡贴地吸附）和坑 ① 的平台边缘；③ **辅助射线**：前方脚下一条射线做边缘检测（巡逻 AI、防冲刺掉落）。玩家碰撞体用零摩擦物理材质（项目里已经这样做了：forPlayer 材质摩擦为 0），避免贴墙按方向键时被摩擦力挂住——这是 Demo 里做对的一个细节。

## 17.6 更好的结构：感知层 → 状态机 → 表现




**理想方案** 图 17-2　感知层的位置：物理之后、状态机之前，每个物理步只做一次。状态机从“自己去查世界”变成“读一份已经整理好的接触信息”，查询次数、时序一致性、可测试性三个问题一起解决

- **接触信息里存什么**：是否着地、地面法线与坡度、脚下对象（普通 / 单向 / 移动平台及其速度）、左右墙是否接触及覆盖高度、是否顶头、前方是否有地面、离地时长、离墙时长。
- **宽限与滞回放在哪**：感知层只报告“事实 + 时间”，宽限（土狼时间、墙宽限）由状态机按时间判断——这样调手感时只改状态机参数，不改检测。
- **可测试**：感知层的输出是纯数据，状态机可以喂一串伪造的接触信息做单元测试（本书第 3 章 E1 就是这样回放的）。

## 17.7 那么，“挂一个 Transform 做检测”到底合不合适?

| 场景 | 结论 | 理由 |
| --- | --- | --- |
| 原型、Game Jam、只有直角平台 | **合适** | Transform 作为探测点直观、可在编辑器里拖动调整、能画 Gizmos；成本最低 |
| 有斜坡、单向平台、移动平台 | **需要升级** | 单射线判不了坡、分不清对象类型；至少多射线 + 法线 |
| 角色尺寸可变（蹲下、变身）、多种角色复用 | **不合适** | 探测点位置要跟着碰撞体改；应由碰撞体尺寸推导检测形状 |
| 精品手感、回放、帧同步 | **不合适** | 需要每物理步一次、确定性的感知；常见做法是运动学控制器自己做碰撞 |

一句话口径：**“Transform 探测点适合原型；要做正式产品，检测形状应该从碰撞体推导、在物理步里集中做一次、按法线分类，并把结果和宽限时间一起交给状态机。”**

## 17.8 面试防守

追问树

```
你怎么判断角色着地？（Q17-1）
├─ 为什么不用 OnCollisionEnter / Exit 判断？（Q17-2）
├─ 斜坡怎么处理？上坡变慢、下坡飞出去怎么办？（Q17-3）
├─ 单向平台怎么做？怎么从上面跳下来？（Q17-4）
├─ 站在移动平台上为什么会滑走？（Q17-5）
└─ 检测放在 Update 还是 FixedUpdate？（Q17-6 → 坑 ④）
```

### Q17-1　你怎么判断角色着地?

**L1**　Demo 里是脚底探测点向下 0.15 m 的射线，只检测地面层。

**L2**　这个方案的边界我算过：射线只伸出脚底 4 cm，坡度超过约 25.5° 就会判定离地；站在边缘、中心悬空时也会误判。

**L3**　更好的做法是在物理步里统一做感知：动态刚体直接读接触点，按法线和竖直方向的夹角分出地面、墙、天花板，再补一个短距离的向下形状投射处理“即将着地”和边缘；结果带上“离地多久”，由状态机做土狼时间和滞回。

**L4 • 攻击**　“接触点法线在角落会同时出现地面和墙，怎么办？” → 分别统计：只要有一个法线落在地面范围就算着地，同时记录墙接触；角落里“既着地又贴墙”是合法状态，由状态机按优先级处理（着地优先，墙滑只在空中生效）。

### Q17-2　为什么不用碰撞进入 / 退出回调判断着地?

**L2**　进入 / 退出是“事件”，着地是“状态”。多个接触点、同一物理步里先进后出、穿过单向平台、碰撞体被禁用时不发退出事件，都会让“进入计数 − 退出计数”出错，常见结果是角色在空中被判定为着地。正确做法是每步根据当前接触重新计算（持续回调或主动读取接触点），事件只用来触发一次性表现（落地尘土）。

### Q17-3　斜坡怎么处理?

**L2**　三件事：① 判定——法线角 ≤ 可站坡度算地面，超过的当成墙或滑坡；② 移动——把输入速度投影到坡面切线方向，否则上坡会“顶”着坡、下坡会每帧腾空；③ 下坡贴地——向下短投射，如果离地面很近就把角色吸附到地面并保持着地，避免下坡时“一路小跳”。另外站在坡上不按键时要防止下滑（静止时提高摩擦或关闭重力分量）。

### Q17-4 → Q17-5　单向平台、移动平台?

**单向平台**　Unity 2D 有 PlatformEffector2D：只有从上方来的接触才生效。感知层要识别“脚下是单向平台”，下跳时临时忽略与这个平台的碰撞 0.2~0.3 s；着地判定还要求“竖直速度 ≤ 0”，否则从下往上穿过时会在中途被判着地。

**移动平台**　角色在平台上时，把平台本步的位移（或速度）叠加到角色上；用父子关系带动最简单，但会和刚体、缩放冲突，更稳的做法是感知层报告“脚下平台的速度”，角色移动时加上它。离开平台时是否继承平台速度（惯性）是设计决定。

**常见错误回答**

**错误**：“用 tag 判断碰到的是不是 Ground 就行。”

**问题**：标签只说明“碰到了什么”，不说明“从哪个方向碰到”——撞到地面块的侧面也会被当成着地，于是贴墙时可以无限起跳（“爬墙 bug”）。

**更好的回答**：“对象类型用标签 / 组件识别，接触方向用法线判断，两者都要。”

## 17.9 发散场景题

### 场景 17-1　角色可以蹲下（碰撞体变矮），蹲着的时候头顶有天花板就不能站起来。怎么做?

考点：检测形状与碰撞体尺寸的绑定。

蹲下时切换到矮碰撞体；想站起时，用“站立尺寸”的形状在当前位置做一次重叠查询（排除自身），有重叠就保持蹲姿。所有检测形状都从当前碰撞体尺寸推导，而不是挂固定位置的 Transform——这正是“挂探测点”方案在角色尺寸会变时不适用的原因。

### 场景 17-2　加入冰面（滑）和泥地（慢），怎么让判定和移动知道脚下是什么?

考点：表面属性的数据化。

两种来源：物理材质（摩擦）只影响物理，不够表达“加速度 / 最高速度 / 脚步声”；更好的是瓦片或地面对象携带“表面类型”数据（瓦片地图可以按格子查到瓦片资源，第 19 章），感知层把“脚下表面类型”写进接触信息，移动参数按表面类型查表。

### 场景 17-3　为什么角色偶尔会卡在两块地砖的接缝处?

考点：逐格碰撞体的内部接缝（ghost collision）。

逐格生成碰撞体时，相邻两格之间有一条内部竖边，角色快速滑过时可能被这条边的角“绊”一下。解决办法是把瓦片碰撞体合并成一个整体轮廓（项目里已经用了 CompositeCollider2D，79 格合并成 2 个多边形，第 19 章图 19-2），角色碰撞体用胶囊（圆角）而不是方盒也能大大缓解。

**本章速记**

- 先拆判定：**着地、坡度、左右墙、顶头、边缘、脚下对象、能否站起。**
- 单射线的边界（场景数值）：伸出脚底 **4 cm**、可识别坡度 **≈ 25.5°**、墙检测高度 **1.32 m。**
- 六个坑：**点代表体积、长度两难、墙单射线、散落在渲染帧（且读的是插值位置）、不看法线、没有记忆。**
- 正解：**物理步里一次感知 → 法线分类 → 接触信息（含时间）→ 状态机做宽限与滞回。**
- 口径：**“Transform 探测点适合原型，正式产品要从碰撞体推导检测形状并集中感知。”**

▶ **自测 1：为什么射线“加长”也能识别更陡的坡，却不是好办法?**

▶ **自测 2：开启刚体插值后，为什么检测起点不该用 Transform?**

▶ **自测 3：骷髅的地面探测点放在身体前方，有什么好处和问题?**

CHAPTER 18

# 滑墙与蹬墙跳：先定规则，再定检测与参数

问题 ②：“滑墙状态具体应该怎么设计？”——滑墙不是“贴墙时把重力调小”这么简单。它包含进入、维持、退出、蹬墙四段规则，每段都有玩家容错的需求；而“能不能单墙爬上去”这种关键玩法规则，应当是设计决定，而不是参数和检测精度的副作用。本章用关卡里那根 7 m 立柱做实验，说明 Demo 的做法会出现什么、正确的设计是什么。

---

**本章目标**

- 说清滑墙的三个设计目的，以及“单墙能不能爬”为什么必须先决定
- 画出完整的墙交互状态图：进入条件、维持行为、四个出口、蹬墙跳、墙顶处理
- 用 F5 实验讲清 Demo 的两个坑：墙顶死区、攀爬成败取决于反应速度
- 给出每个参数的作用与调参思路：滑速、粘墙时间、墙宽限、蹬墙推力与强制离墙时间

## 18.1 口径：30 秒怎么讲 • 现状 vs 理想

**面试口播**

现状 **我的做法**：空中下落时，腰部射线碰到墙就进入滑墙状态，下落速度被固定为 2 m/s（按下键则恢复重力加速）；按跳向反方向蹬出（离墙 4、向上 18）；按离墙方向立刻脱离；着地回到待机。

**常见的坑（以我遇到的为例）**：① 出口不完整——墙在半空结束时角色会对着空气继续滑；② 检测覆盖不够——只有一条腰部射线，墙顶附近有 1.3 m 的“贴着墙却滑不了”的死区；③ 规则不稳定——我用关卡里 7 m 的立柱测过，单墙能不能爬上去完全取决于玩家在滑墙第几帧按跳：3 帧以内能上，晚 100 ms 就永远卡在墙顶下面。

理想 **正确做法**：先决定“单墙能否攀爬”这条规则；再用多射线 / 形状投射检测墙面覆盖；状态出口覆盖进入条件的否定；加粘墙时间、墙宽限时间和蹬墙后的强制离墙时间；墙顶用翻越或角落修正处理。

现状 → 问题 → 理想 **滑墙与蹬墙跳**

| 维度 | 当前做法（Demo 现状） | 存在的问题 | 理想重构方案 | 收益 / 代价 |
|---|---|---|---|---|
| **玩法规则** | 没有明确决定“单墙能否攀爬” | P2 同样的操作，滑墙后 0~3 帧按跳能爬上 7 m 立柱，100 ms 就卡住：规则成了手速的副作用（F5） | 先定规则（允许 / 禁止），参数服从规则，并用自动化测试固定<br>模型验证：F5 | 玩家体验到的是稳定规则 |
| **墙检测** | 一条腰部射线（脚底上方 1.32 m） | P1 射线在脚底上方 1.32 m：矮墙测不到，接近墙顶时贴着墙却滑不了、蹬不上（F1 / F5） | 脚 + 腰（+ 头）射线或侧向盒投射，按覆盖率判定<br>模型验证：F5 加脚部射线后 100 ms 反应 3.5 s 翻上 | 墙顶不再“贴着墙却滑不了” |
| **进入条件** | 腰部射线命中即进入（不要求朝墙按住） | P2 空中擦到任何墙面都会被吸住滑墙，不想贴墙也不行 | 下落中 + 墙覆盖 + 朝墙按住（或刚按过）<br>设计方案 · 未实现 | 不想滑墙时不会被吸住 |
| **退出条件** | 没有“墙没了”出口；按离墙方向立刻脱离 | P1 墙结束后对着空气慢落（#14）；“先按反方向再按跳”的蹬墙常变成直接掉落 | 墙不再覆盖即回空中；离墙方向按住超过粘墙时间才脱离<br>设计方案 · 未实现 | 不会对着空气滑；“反方向 + 跳”能蹬出 |
| **蹬墙** | 只认滑墙状态那一帧的按键；蹬墙后立即可朝墙移动 | P2 早按、离墙后晚按都无效；蹬出后立即能转回墙面，单墙攀爬看手速（F5） | 跳跃缓冲 + 墙宽限 + 强制离墙时间<br>公开参考：Celeste 强制离墙 0.16 s | 时机更宽容；能否单墙爬由参数明确控制 |
| **滑速与墙顶** | 每帧直接设 vy = −2；墙顶没有处理 | P2 进入滑墙时速度突变为 −2，没有摩擦感；墙顶没有翻越 / 角落修正，差一点够到只能掉回去 | 限制速度上限并平滑减速；墙顶翻越或角落修正<br>设计方案 · 未实现 | 有“摩擦感”，墙顶不掉落 |

现状问题：P1 ×2　P2 ×4（P0~P3 编号见附录 A，分级说明见 0.2）

## 18.2 先问“滑墙是为了什么”

### ① 空中容错

跳歪了还能“抓住”墙，给玩家第二次机会。要求：进入要宽松（自动吸附 / 宽限），减速要明显。

### ② 垂直移动

两面墙之间来回蹬（墙井），或者单墙连续蹬上去。要求：蹬墙跳的高度、距离稳定可预期。

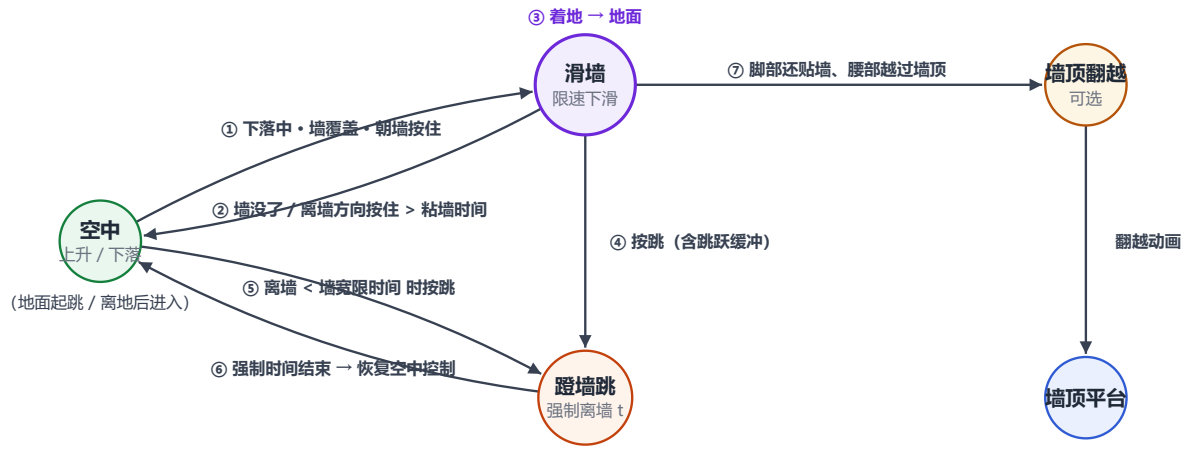
### ③ 节奏与关卡语法

下落减速让玩家有时间观察下方；“这面墙能滑 / 那面不能”是关卡设计的词汇。要求：可滑墙面要能被识别和配置。

**最关键的设计决定：单墙能不能爬?**

不同游戏的选择不同：有的游戏允许贴着同一面墙连续蹬墙攀爬（洛克人 X 系列的“踢墙”是经典例子）；有的游戏在蹬墙后的短时间内强制给一个离墙的水平速度，让玩家必须离开墙一段距离才能再回来——例如 Celeste 公开源码里的常量 WallJumpForceTime = 0.16 s 公开资料 。两种都对，**错的是“没有决定”**：让它取决于推力大小、检测精度和玩家手速，结果就是“有时能爬有时不能”。下面的 F5 实验正好说明了这一点。

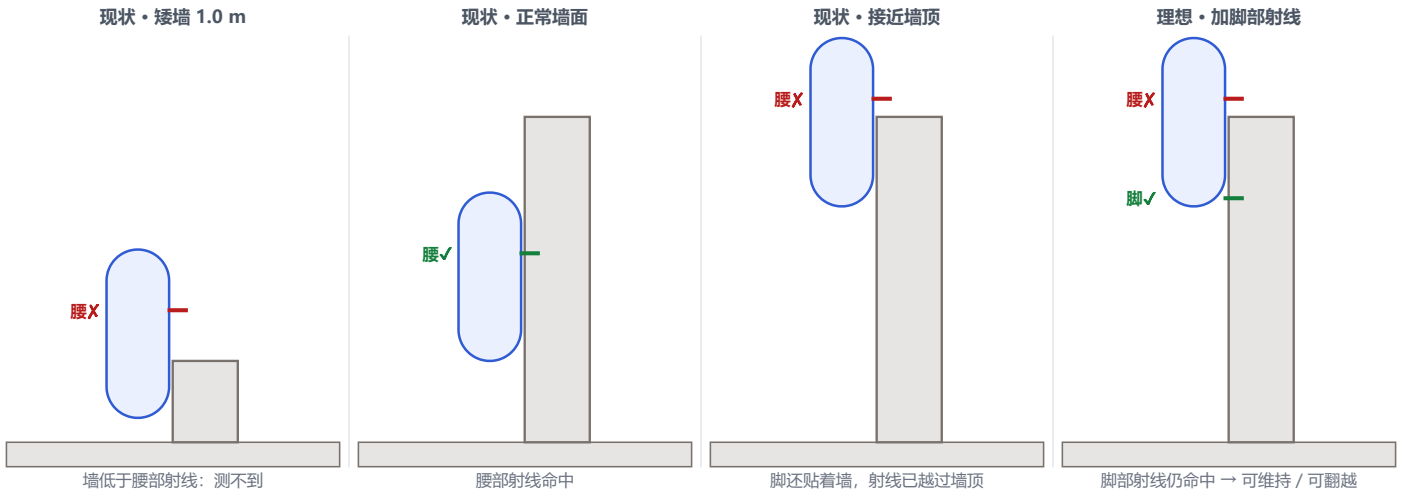
## 18.3 完整的墙交互状态图




**理想方案** 图 18-1　推荐的墙交互状态图。①~⑥ 是每个滑墙系统都必须回答的六条转移；⑦ 是“墙顶死区”的处理方式之一（另一种是角落修正）。Demo 里有 ①（但不要求朝墙按住）、③、④，缺 ②（墙没了）、⑤、⑥ 和 ⑦

| 转移 | 要回答的问题 | 推荐做法 | Demo 现状 |
| --- | --- | --- | --- |
| ① 进入 | 必须朝墙按方向键吗？上升中能进入吗？墙要多高才算？ | 下落中（竖直速度 ≤ 0）+ 墙覆盖脚到腰 + 朝墙按住（或短时间内按过）；可滑墙面可配置 | 腰部射线命中即进入，不要求按键 |
| ② 离开（空中） | 墙结束了怎么办？玩家想离开墙呢？ | 墙不再覆盖 → 立即回空中；离墙方向按住超过粘墙时间（0.1~0.2 s）才脱离 | 没有“墙没了”的出口（第 3 章 E1-S2）；按离墙方向立刻切走 |
| ③ 着地 | 滑到底 | 回地面状态，播落地 | ✓ |
| ④ 蹬墙 | 按跳的时机要多准？ | 接入跳跃缓冲（第 20 章）：进入滑墙前 0.1 s 内按的跳也算 | 只认滑墙状态里那一帧的按键 |
| ⑤ 墙宽限 | 刚离开墙一点点时按跳 | 离墙 0.08~0.12 s 内按跳仍算蹬墙（墙版土狼时间） | 无 |
| ⑥ 蹬墙后的控制 | 蹬出去之后玩家马上按回墙怎么办？ | 强制离墙时间内忽略朝墙的输入（决定能否单墙攀爬），之后恢复空中控制 | 无：蹬墙后下落状态立刻按 7 m/s 朝墙移动 |
| ⑦ 墙顶 | 脚还贴着墙、身体已过墙顶 | 翻越（Ledge Climb）、角落修正（把角色推上平台），或至少让脚部射线也能维持滑墙 | 腰部射线越过墙顶就不再算贴墙 → 死区 |

## 18.4 检测：墙要“覆盖”多高才算贴墙




现状 vs 理想 图 18-2　墙检测的覆盖问题（尺寸按场景数值：腰部射线在脚底上方 1.32 m，伸出碰撞体 0.19 m）。只有一条腰部射线时，“矮于 1.32 m 的墙”和“墙顶以下 1.32 m 的区间”都会出现“身体贴着墙、系统却认为没有墙”

检测方案按精度从低到高：**两条射线（脚 + 腰）** → **三条射线（脚 + 腰 + 头）**，用“命中条数”表示覆盖程度 → **侧向盒子投射**，返回接触面法线（法线接近水平才算墙，避免把斜坡当墙）。再加两条规则：只在空中检测墙（地面上贴墙是“撞墙停止”，不是滑墙）；左右两侧都检测，这样背对墙时按跳也能蹬墙（是否允许由设计决定）。

## 18.5 F5 实验：在 7 m 立柱上单墙攀爬

关卡主平台上有一根宽 2 m、高 7 m 的立柱，而普通跳跃的顶点只有约 3.9 m，所以它是一个“必须靠墙交互才能上去”的设计点。实验：玩家从立柱左侧起跳，一直按住朝墙方向（→），每次进入滑墙后经过“反应时间”按一次跳。逐行复刻 Demo 的下落 / 滑墙 / 蹬墙三个状态与检测几何，物理按 0.02 s 步长积分 实测 F5 模型 ：

| 方案 | 配置 | 20 s 内结果 | 蹬墙次数 | 第一次蹬墙净升高 / m | 最后一次进入滑墙时脚底高度 / m | “ |
|---|---|---|---|---|---|---|
| 现状 | 反应 0 帧（进入墙滑当帧按跳） | ✓ 3.90 s | 5 | 1.38 | 11.66 | 8 |
| 现状 | 反应 1 帧 | ✓ 2.48 s | 3 | 1.32 | 11.66 | 3 |
| 现状 | 反应 2 帧 | ✓ 4.78 s | 6 | 1.26 | 11.61 | 10 |
| 现状 | 反应 3 帧（50 ms） | ✓ 9.38 s | 12 | 1.26 | 11.59 | 33 |
| 现状 | 反应 6 帧（100 ms） | ✗ 20 s 内爬不上 | 25 | 1.08 | 11.54 | 54 |
| 现状 | 反应 12 帧（200 ms） | ✗ 20 s 内爬不上 | 23 | 0.78 | 11.50 | 24 |
| 参数调整 | 推力 5、反应 100 ms | ✗ 20 s 内爬不上 | 25 | 0.52 | 7.99 | 0 |

| | | | | | | |
|---|---|---|---|---|---|---|
| 参数调整 | 推力 6、反应 100 ms | ✗ 20 s 内爬不上 | 4 | -0.46 | 6.32 | 0 |
| 参数调整 | 推力 8、反应 100 ms | ✗ 20 s 内爬不上 | 2 | -2.95 | 6.89 | 0 |
| 理想（检测） | 加脚部墙射线、反应 100 ms | ✓ 3.48 s | 4 | 1.08 | 12.64 | 0 |
| 理想（检测） | 加脚部墙射线、反应 200 ms | ✓ 5.57 s | 6 | 0.78 | 12.87 | 0 |

表 18-1　F5 结果（立柱从 y = 6 到 13，翻上去需要脚底到达 13）。“最后一次进入滑墙时脚底高度”卡在 11.5 左右的配置，都是被墙顶死区挡住了：腰部射线在脚底上方 1.32 m，脚底高于 11.68 时就检测不到墙

三个结论，正好对应三类坑：

1. **规则不稳定**：Demo 的参数下，单墙攀爬是“可能”的，但取决于手速——滑墙第 0~3 帧内按跳能上去（耗时 2.5~9.4 s），晚 100 ms 就永远上不去。对玩家来说，这是一条“有时灵有时不灵”的隐藏规则，最伤手感。

2. **检测死区才是卡住的直接原因**：每次蹬墙其实都在上升（反应 100 ms 时第一次净升高 1.08 m），但到了墙顶以下 1.3 m 的区间，弧线回到墙边时腰部射线已经越过墙顶，角色贴着墙往下掉，直到重新低于 11.68 才又进入滑墙——无限循环。只加一条脚部射线，同样 100 ms / 200 ms 的反应就能翻上去（3.5 s / 5.6 s）。
3. **推力是“能否单墙攀爬”的开关**：离墙推力从 4 提到 6，第一次蹬墙的净升高就变成负数（-0.46 m）——蹬出去太远，回到墙边时已经低于起点。所以想“禁止单墙攀爬”，加大推力或加强制离墙时间都可以；想“允许”，就要保证在合理反应时间内每次都有正的净升高，并处理好墙顶。

**从实验得到的设计方法**

① 先写下规则：“允许单墙攀爬，每次蹬墙净升高约 1 m，反应窗口 ≥ 150 ms”或“禁止单墙攀爬，必须在两面墙之间来回”。② 用“蹬墙弧线”反推参数：竖直速度决定弧线高度，水平推力 × 强制离墙时间决定离墙最远距离，空中加速度决定回墙所需时间。③ 用自动化测试固定下来：像 F5 一样让脚本按不同反应时间去爬，CI 里断言“150 ms 反应能爬上去”或“任何反应都爬不上去”。

## 18.6 参数清单：每个旋钮管什么

| 参数 | 作用 | 常见取值 / 做法 | Demo |
| --- | --- | --- | --- |
| 滑墙速度上限 | 滑墙时的最大下落速度 | 普通下落上限的 20%~40%；**限制上限而不是直接设定速度** | 每帧直接设 vy = −2 |
| 滑墙减速 | 从快速下落进入滑墙时，速度怎样降到上限 | 短时间内平滑减速（有“摩擦”的感觉），配摩擦粒子与音效 | 瞬间变成 −2（“啪”地粘住） |
| 粘墙时间 | 按离墙方向要持续多久才脱离 | 0.1~0.2 s：让“先按反方向、再按跳”的蹬墙操作能成立 | 无：按反方向立即脱离，导致“反方向 + 跳”蹬不出去 |
| 墙宽限时间 | 离墙后多久内按跳仍算蹬墙 | 0.08~0.12 s | 无 |
| 蹬墙速度 | 蹬出去的方向和大小 | 竖直分量略小于普通起跳，水平分量 ≈ 跑速 + 额外推力 | (4, 18)；跑速 7 |
| 强制离墙时间 | 蹬墙后多久内忽略朝墙输入 | 0.1~0.2 s（Celeste 为 0.16 s 公开资料）；为 0 时允许立即回墙 | 0 |
| 检测距离 | 离墙多远算贴墙 | 很短（几厘米）：太长会“隔空滑墙”，太短会抖动 | 伸出碰撞体 0.19 m |
| 可滑墙面 | 哪些墙能滑 | 按层 / 标签 / 瓦片数据配置（冰墙不能滑、藤蔓墙可爬） | 所有 Ground 层表面 |

表现层同样重要：滑墙时角色朝向（面向墙还是背对墙）、手部与墙接触的动画、摩擦粒子、持续的摩擦音效，以及蹬墙瞬间的尘土——它们让玩家“感觉到”自己抓住了墙。

## 18.7 面试防守

追问树

```
滑墙怎么做的？（Q18-1）
├─ 什么时候进入、什么时候退出？（Q18-2 → 图 18-1 ①②）
├─ 为什么到墙顶会掉下来？（Q18-3 → 墙顶死区）
├─ 一面墙能不能一直蹬上去？（Q18-4 → F5）
├─ 蹬墙跳的方向和力度怎么定？（Q18-5）
└─ 滑墙和下落之间会不会来回抖？（Q18-6）
```

## Q18-1 → Q18-2　滑墙怎么做？进入和退出条件？

**L2**　滑墙是空中状态的一个子状态：下落中、前方墙面覆盖脚到腰、并且朝墙按住时进入；维持时把下落速度限制在一个较低的上限；出口有四个——墙不再覆盖（回空中）、离墙方向按住超过粘墙时间（回空中）、着地、按跳（蹬墙）。

**L3 • Demo**　我原来的实现漏了“墙没了”这个出口，墙在半空结束时角色会对着空气继续滑；这让我形成了一个习惯：每写一个状态，先列出它的进入条件，再检查出口是否覆盖了“进入条件不再成立”。

## Q18-3　为什么到墙顶会掉下来？

**L2**　因为墙检测只有一条腰部射线，在脚底上方 1.32 m。接近墙顶时，脚还贴着墙，射线已经越过墙顶，系统认为没有墙，角色就按普通下落掉下去。修法有三层：加脚部射线（最小改动）；到达墙顶时触发翻越；或者做角落修正——蹬墙弧线差几个像素就能上平台时，把角色直接推上去。

## Q18-4　一面墙能不能一直蹬上去？

**L2**　这应该是设计决定。我在 Demo 里测过，原参数下能不能爬上 7 m 的立柱取决于手速：滑墙后 3 帧内按跳能上，晚 100 ms 就卡住。所以我会先定规则：要么允许，保证合理反应时间内每次都有稳定的正升高；要么禁止，用强制离墙时间或更大的推力让玩家必须离开墙。

**L4 • 攻击**　“玩家觉得禁止单墙攀爬不爽怎么办？” → 那就在关卡层面给替代路线：两面墙之间的墙井、可抓的边缘、墙上的落脚点。规则一旦稳定，关卡设计才能围绕它展开。

## Q18-5 → Q18-6　蹬墙跳参数怎么定？会不会来回抖？

**参数**　由目标弧线反推：想让玩家在两面相距 d 米的墙之间来回，就让“水平速度 × 到达顶点的时间”大约等于 d；竖直速度决定每次上升多少。强制离墙时间决定玩家多早能改变方向。

**抖动**　会出现在“刚好贴着墙边缘”的情况：这一步检测到墙、下一步没检测到。用滞回解决：进入滑墙需要墙覆盖（例如两条射线都命中），退出需要完全不覆盖（两条都不命中）并持续 1~2 个物理步。

**常见错误回答**

**错误**：“滑墙就是贴墙时把重力调小。”

**问题**：只回答了“维持”这一段；进入条件、四个出口、粘墙、墙宽限、蹬墙后的控制、墙顶处理都没有覆盖，面试官追问任何一个都会卡住。

**更好的回答**：“滑墙是空中状态的子状态，我会先定规则（能不能单墙爬），再定检测（墙覆盖率），最后定参数（滑速上限、粘墙时间、墙宽限、蹬墙速度和强制离墙时间）。”

## 18.8 发散场景题

**场景 18-1　加入“抓墙 + 体力”（抓住墙不滑，可以上下爬，体力耗尽后滑落）。**

考点：在滑墙基础上扩展新的墙状态。

新增“抓墙”状态（按住抓取键进入），竖直速度由上下输入决定；体力是一个随时间和动作消耗的数值（静止抓墙慢耗、向上爬快耗、抓墙跳一次性扣），着地恢复；体力不足时只能滑墙；UI 和角色表现（出汗、颤抖）提示体力。检测层不变，只多一个状态和一个资源——这正是“感知层与状态机分离”的收益。

**场景 18-2　有些墙不能滑（冰墙），有些墙滑得更慢（粘液墙），怎么配?**

考点：墙面属性的数据化。

墙面属性放在数据上：瓦片资源或墙对象携带“可滑 / 滑速系数 / 可抓取”等字段；感知层报告“接触的墙面属性”，状态机据此决定能否进入以及用哪个滑速上限。这比在代码里按层名、标签写 if 好维护，关卡设计师可以直接刷不同的墙瓦片。

**场景 18-3　手柄玩家反馈“蹬墙跳老是失败”，你怎么排查?**

考点：用数据定位手感问题。

录制输入与接触信息：看失败时是“没进入滑墙”（检测 / 进入条件过严）、“按跳太早”（需要跳跃缓冲）、“按跳太晚已离墙”（需要墙宽限）、还是“先按了反方向导致提前脱离”（需要粘墙时间）。手柄摇杆有死区和回中延迟，“松开方向键”在手柄上不是瞬间的，所以粘墙时间对手柄尤其重要。

**本章速记**

- 先定规则：**单墙能不能爬**是设计决定（洛克人 X 允许、Celeste 用 0.16 s 强制离墙）。
- 六条转移：**进入、墙没了 / 主动离开、着地、蹬墙、墙宽限、蹬墙后控制**，外加墙顶处理。
- F5：Demo 参数下单墙攀爬**取决于手速**（3 帧内能上、100 ms 卡住）；卡住的原因是**墙顶以下 1.32 m 的检测死区**，加一条脚部射线即可翻上。
- 参数：**滑速上限（限上限不设定值）、粘墙时间、墙宽限、蹬墙速度、强制离墙时间、检测距离、可滑墙面。**

▶ **自测 1：为什么 Demo 里“先按离墙方向、再按跳”蹬不出去?**

▶ **自测 2：推力从 4 提高到 6，为什么第一次蹬墙净升高就变成负数?**

▶ **自测 3：只给墙检测加长射线距离能解决墙顶死区吗?**

CHAPTER 19

# 关卡搭建：Tilemap 的原理、操作流程与平台设计

问题 ③：“跳跃平台本身怎么搭建？”——Demo 用的是 Unity 的瓦片地图（Tilemap）。面试时要能讲清三层：**原理**（格子数据怎么存、怎么变成画面和碰撞）、**操作**（从导入贴图到画出可玩关卡的完整流程）、**设计**（平台的尺寸由角色的跳跃能力决定）。本章的场景数据全部由脚本从 SampleScene 解析得到。

---

**本章目标**

- 讲清 Grid、Tilemap、Tile 资源、TilemapRenderer、TilemapCollider2D、CompositeCollider2D 六个部件各自的职责
- 说出一个 Tilemap 在场景文件里是怎么存储的（稀疏格子 + 资源数组 + 引用计数）
- 掌握十步搭建流程，并能对照 Demo 指出哪些步骤做对了、哪些漏了
- 理解“跳跃度量决定关卡尺寸”，以及单向平台、可破坏地形、大地图这些扩展怎么做

## 19.1 口径：30 秒怎么讲 • 现状 vs 理想

**面试口播**

现状 **我的做法**：关卡用 Unity 的 Tilemap 搭：一个 1 m × 1 m 的矩形网格下面两层瓦片地图——地形层 79 格，带瓦片碰撞体并交给复合碰撞体合并成 2 个多边形（共 16 个顶点），挂静态刚体；背景层 10 格，没有碰撞，排在地形后面。贴图统一 32 像素 / 米、点过滤、不压缩，保证像素风清晰。

**原理一句话**：瓦片地图不是“每格一个物体”，而是一张稀疏表——格子坐标映射到瓦片资源，渲染器按 32 × 32 的块把格子合成网格批量绘制，碰撞体按瓦片形状生成，再合并成整体轮廓。

理想 **坑与改进**：没有打图集（157 张散图，块渲染模式官方要求图集）、墙和地面同在一层、同一张图生成了两份同名瓦片资源、没有用规则瓦片自动拼边角、关卡尺寸没有先按跳跃能力定度量。

现状 → 问题 → 理想 **关卡搭建与 Tilemap**

| 维度 | 当前做法（Demo 现状） | 存在的问题 | 理想重构方案 | 收益 / 代价 |
|---|---|---|---|---|
| **数据与碰撞** | Grid + Tilemap；地形 TilemapCollider2D + CompositeCollider2D + 静态刚体（79 格合并为 2 个多边形） | ✓ 无问题 已合并为轮廓碰撞，做法正确 | 保持；运行时改地形时把复合碰撞体改为手动生成<br>**保持 · 现状已正确** | |
| **贴图导入** | 32 PPU、点过滤、不压缩 | ✓ 无问题 像素风导入设置正确 | 保持<br>**保持 · 现状已正确** | |
| **图集** | 块渲染模式，157 张散图，没有 Sprite Atlas | P2 块渲染模式下多纹理瓦片无法正确排序、不能合批，瓦片间可能出现细缝（官方文档） | 每个地图一个图集（留间距、关闭旋转与紧凑打包）<br>**公开参考：Unity 文档要求块模式使用图集** | 排序一致、能合批、无接缝 |
| **分层** | 只有背景 + 地形两层，墙和地面同层 | 局限 “这面墙能不能滑” “是不是单向平台”无法单独配置：任何地形侧面都能滑墙 | 地形 / 单向平台 / 可滑墙 / 危险物 / 装饰分层，或瓦片携带表面数据<br>**设计方案 · 未实现** | 物理行为可按格子配置 |
| **资源与拼边** | 同一张图生成了两份 floor_tile_3；边角砖手工挑 | P3 两份同名 floor_tile_3 分别被 4 格和 14 格引用，改一份只影响一部分；改形状要手工重挑边角砖 | 瓦片资源集中目录 + 提交检查；Rule Tile 自动拼边<br>**设计方案 · 未实现** | 改形状不用重画 |
| **关卡度量** | 先画后定：沟 3 m、立柱 7 m，没有度量表 | P2 3 m 的沟对 5.6 m 的跳远几乎没有挑战；7 m 立柱能否上去取决于手速（F4 / F5） | 先定度量表（跳高 / 跳远 / 冲刺 / 蹬墙），再用关卡模块拼<br>**设计方案 · 未实现** | 难度可设计、可校验 |
| **实体** | 玩家、骷髅直接摆在场景里 | 局限 重开、检查点、刷怪都要改场景 | 关卡里只放标记，由加载器生成实体<br>**设计方案 · 未实现** | 重开、检查点、刷怪不改场景 |

现状问题：P2 ×2　P3 ×1　局限 ×2　✓ 无问题 ×2　（P0~P3 编号见附录 A，分级说明见 0.2）

## 19.2 原理：六个部件、一条管线

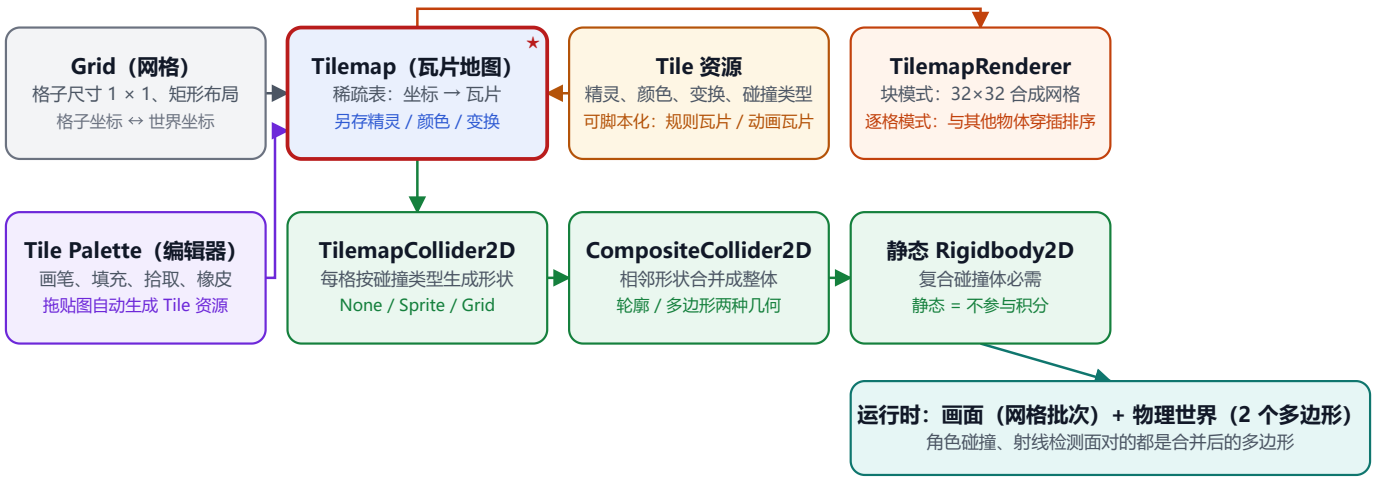



图 19-1　Tilemap 的部件与数据流：Tilemap 只存“哪个格子放了哪种瓦片”，画面由 TilemapRenderer 按块生成，碰撞由 TilemapCollider2D 生成后交给 CompositeCollider2D 合并

### 格子数据在场景文件里长什么样

把 SampleScene.unity 里地形层的 Tilemap 组件拆开看，就能看到它的存储方式 源码 （由 tools/scan_tilemap.py 解析）：

| 字段 | 内容 | 地形层的实际数据 |
| --- | --- | --- |
| 格子列表 | 只记录**有瓦片的格子**：坐标 → （瓦片索引、精灵索引、变换索引、颜色索引、标志位） | 79 条（地图包围盒是 64 × 13 = 832 格，空格不占空间） |
| 瓦片资源数组 | 本地图用到的瓦片资源 + 引用计数 | 6 项，引用计数 44 / 4 / 12 / 14 / 4 / 1，合计 79 |
| 精灵 / 变换 / 颜色数组 | 同样去重存储，格子只存下标 | 5 张精灵；变换和颜色各 1 项（没有旋转翻转、没有染色） |
| 原点与尺寸 | 包围盒 | 原点 (−23, 0)，尺寸 64 × 13 |

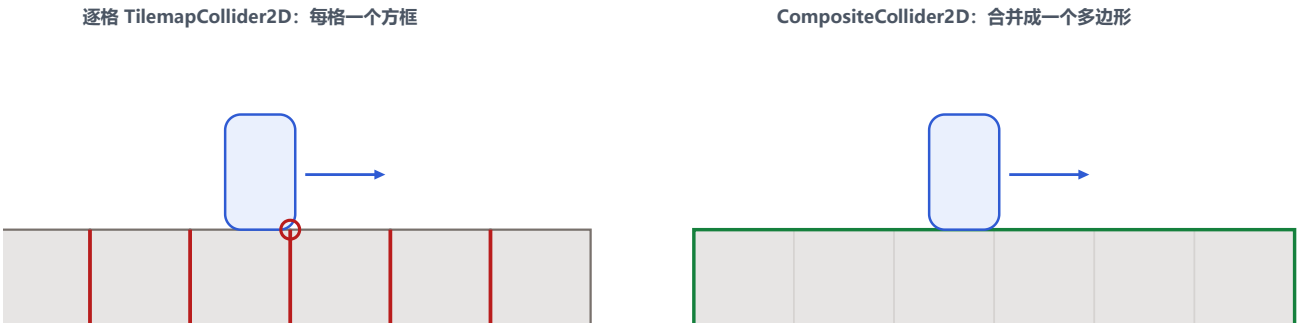
这个结构解释了 Tilemap 的几个性质：**稀疏**（大片空白不花钱）、**去重**（同一种砖铺一万格，资源只存一份）、**格子级查询便宜**（按坐标直接查到“这格是什么瓦片”，可以用来做地面类型、寻路网格）。瓦

片资源本身是一个可脚本化对象：渲染和碰撞时会问它“这一格应该显示什么、用什么碰撞”，规则瓦片就是根据邻居格子返回不同精灵来实现自动拼边角的。

### 渲染：块模式与逐格模式

TilemapRenderer 默认是**块模式**：把 32 × 32 格合成一个网格批量绘制，整张地图在排序时被当成一个整体，角色不能“穿插”到瓦片之间（适合横版的地形层）；**逐格模式**逐个瓦片参与排序，适合俯视角里“人走到树后面”的情况，但开销更大。官方文档特别说明：块模式下，来自多张纹理的瓦片无法被正确排序和一致地渲染，**应把同一张地图用到的精灵打进一个 Sprite Atlas** 公开资料 ——这正是 Demo 漏掉的一步（19.5 坑 ①）。

### 碰撞：逐格生成，再合并




现状 vs 理想 图 19-2　逐格碰撞体 vs 合并后的轮廓。Demo 的地形层 79 格：逐格会生成 316 条边，其中 168 条是两格之间的内部边；合并后只剩 2 个多边形、16 个顶点。内部边的角会让快速移动的角色或投射物被“绊一下”（俗称 ghost collision）

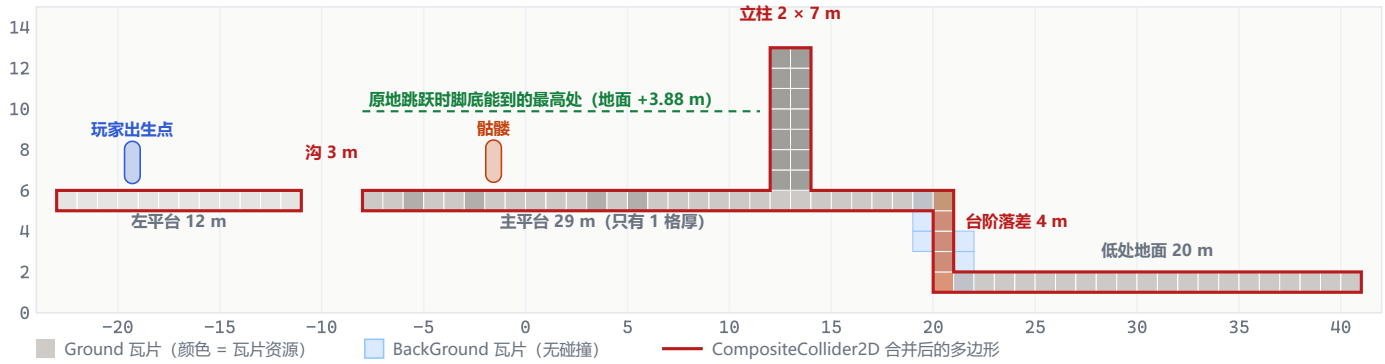
TilemapCollider2D 按每个瓦片资源的“碰撞类型”生成形状：**None**（无碰撞，装饰用）、**Sprite**（用精灵的物理轮廓，Demo 的 112 个瓦片资源全是这种）、**Grid**（整格方框）。勾选“由复合碰撞体使用”后，CompositeCollider2D 把相邻形状合并：几何类型选**多边形**（Demo 的选择，实心，可以做“点是否在地形内”的查询）或**轮廓**（只有边，更省，但物体整个陷进去时不会被推出）；生成方式选**同步**（每次修改立即重建，Demo 的选择）或**手动**（运行时大量改地形时，攒一批修改再重建一次）。

## 19.3 操作：十步搭出一个可玩关卡

| 步骤 | 做什么 | 关键设置 | Demo |
| --- | --- | --- | --- |
| 1 定度量 | 先写下角色的跳跃能力：最高能跳多高、最远能跳多远、蹬墙能爬多高 | 用“格”为单位，例如“原地跳 3 格高、助跑跳 5 格远” | **✗ 没有先定（19.4）** |
| 2 导入贴图 | 统一像素密度、过滤、压缩；切图；编辑物理轮廓 | 每米像素数一致（32）、点过滤、不压缩；多张小图用 Sprite Editor 切片 | **✓ 32 PPU / 点过滤 / 不压缩** |
| 3 打图集 | 把一个地图用到的精灵打进 Sprite Atlas | 留 2~4 像素间距，关闭旋转与紧凑打包（防止瓦片接缝） | **✗ 无图集，157 张散图** |
| 4 建调色板 | Tile Palette 里拖入贴图，自动生成 Tile 资源 | 瓦片资源集中放一个目录，命名规范 | **△ 有调色板，但同一张图生成了两份同名瓦片资源** |
| 5 建网格与分层 | 一个 Grid 下多层 Tilemap：背景 / 地形 / 单向平台 / 危险物 / 前景装饰 | 每层设排序层与顺序；地形层放 Ground 物理层 | **△ 只有背景和地形两层；墙也在地形层** |
| 6 加碰撞 | 地形层：TilemapCollider2D + CompositeCollider2D + 静态刚体 | 勾“由复合碰撞体使用”；装饰瓦片碰撞类型设 None | **✓ 合并成 2 个多边形** |
| 7 画地图 | 画笔、框选填充、拾取、橡皮、旋转 / 翻转 | 按度量画：沟宽、台阶高、墙高都应是“设计过”的数字 | **✓（但见 19.4 度量）** |
| 8 规则瓦片 | 用 Rule Tile 根据邻居自动选边角、表面、内部砖 | 2D Tilemap Extras 包（项目已安装） | **✗ 手动选每一块** |
| 9 特殊地形与实体 | 单向平台（PlatformEffector2D）、尖刺（触发器 + 伤害）、移动平台（独立物体）、出生点 / 刷怪点 / 检查点（实体层或 GameObject 画笔） | 移动 / 可交互的东西不要画进静态地形层 | **✗ 玩家与骷髅直接摆在场景里** |

| | | | |
|---|---|---|---|
| 10 验证 | 用度量检查每个跳跃点；摄像机像素对齐；接缝检查 | Pixel Perfect Camera（2D 功能包已包含） | △ **未见像素对齐相机** 推断 |

## 19.4 设计：平台尺寸由跳跃能力决定




**现状** 图 19-3　SampleScene 的瓦片地图（由场景文件解析绘制，1 格 = 1 m）。红线是 CompositeCollider2D 合并后的碰撞多边形，绿虚线是原地跳跃时脚底能到的最高处。立柱高 7 m，明显高于跳跃能力，所以它是关卡里唯一“必须借助墙交互”的设计点——而第 18 章说明这个墙交互恰恰不稳定

**现状问题** P2 立柱是唯一必须借助墙交互的设计点，而墙交互成败取决于手速（F5）

平台游戏的关卡设计从**度量（metrics）**开始：先由角色参数算出能力边界，再用这些边界设计“轻松 / 需要技巧 / 不可能”三类间距。按 Demo 的参数（实测 F4）：

| 能力 | 数值（Demo） | 关卡里的对应 | 评价 |
|---|---|---|---|
| 原地跳跃高度 | 3.88 m（连续公式 4.08 m，差值来自离散积分） | 台阶 4 m 只能往下走 | 台阶高度 ≈ 跳跃极限，从下往上走需要冲刺或墙 |
| 满速跳远 | 7 m/s × 0.80 s = 5.6 m | 沟宽 3 m | 非常宽松：可以在沟前 2.6 m（约 371 ms）就起跳 |
| 冲刺距离 | 约 5 m（0.2 s） | 无对应设计点 | 冲刺在空中可用，跳跃中途冲刺可再前进约 5 m 估算 |
| 蹬墙攀爬 | 取决于手速（第 18 章 F5） | 立柱 7 m | 唯一的“技巧”点，但规则不稳定 |

**度量驱动的做法**

把度量写成一张表交给关卡设计：例如“轻松跳 ≤ 3 格、需要助跑 4~5 格、必须冲刺 6~9 格、不可能 ≥ 11 格”，“一步台阶 ≤ 3 格、需要墙 4~7 格”。然后用规则瓦片和预制的“关卡模块”（一段沟、一段墙井、一个平台组）拼关卡，每个模块都有已知的难度。改了角色参数，度量表跟着重算，所有模块的难度一目了然。

## 19.5 Demo 里的坑与正确做法

**坑 ①　块渲染模式却没有图集**

**现状** 瓦片目录下 157 张独立贴图，项目里没有任何 Sprite Atlas；地形层用了 5 张来自不同纹理的精灵。按官方文档，块模式下多纹理瓦片无法被正确排序和一致地渲染，而且不同纹理之间无法合批。

**理想做法** 每个地图（或每个主题）一个图集，留间距、关闭旋转与紧凑打包；顺带解决瓦片之间偶尔出现的细缝（采样到相邻图块的像素）。

**坑 ②　墙和地面在同一层，Wall 物理层定义了却没用**

**现状** 地面检测和墙检测都检测 Ground 层，所以“墙顶”会被当地面，“地面块的侧面”会被当墙；无法单独配置“这面墙能不能滑”。

**理想做法** 不一定要拆层——用接触法线区分地面和墙（第 17 章）即可；但需要不同物理行为（可滑 / 不可滑、单向）时，按功能分层：地形、单向平台、可滑墙面、危险物各一层，或者用瓦片数据携带表面属性。

**坑 ③　同一张图生成了两份同名瓦片资源**

**现状** 地形层的瓦片资源数组里有两项都叫 floor_tile_3（一份在 Tile Assets 目录，一份在 Tiles 目录），分别被 4 格和 14 格引用。看起来一样，改其中一份的设置（比如碰撞类型）只会影响一部分格子。

**理想做法** 瓦片资源集中在一个目录、由调色板统一生成；提交前用脚本检查“同名不同 guid”的资源。

**坑 ④　背景用了和地形一模一样的砖块**

**现状** 背景层的 10 格用的是和地形相同的 brick_side_right_2，但没有碰撞——玩家看到的是“墙”，碰到的却是空气。

**理想做法** 可读性原则：**能碰的和不能碰的必须一眼区分。**背景层降低饱和度 / 亮度、加视差，或使用专门的背景瓦片。

**坑 ⑤　没有先定度量、没有自动拼边**

**现状** 沟宽 3 m 而跳远 5.6 m，几乎不需要技巧；立柱 7 m 远超跳跃能力、又依赖不稳定的墙交互。每一块边角砖手工挑选，改形状就要重画。

**理想做法** 度量表 + 关卡模块 + 规则瓦片（19.3 步骤 1、8）。

**坑 ⑥　实体直接摆在场景里**

**现状** 玩家出生点、骷髅都是场景里的物体，关卡数据和运行时对象混在一起；重开关卡、加检查点、刷怪都要改场景。

**理想做法** 关卡里只放“标记”（出生点、刷怪点、检查点、宝箱点），运行时由关卡加载器根据标记生成实体；标记可以放在单独的实体层（GameObject 画笔）或从外部关卡编辑器导入。

## 19.6 扩展：做成正式产品还要考虑什么

| 需求 | 做法 | 注意 |
| --- | --- | --- |
| 单向平台 | 单独一层，CompositeCollider2D + PlatformEffector2D（只接受上方接触）；下跳时临时忽略碰撞 | 着地判定要求竖直速度 ≤ 0（第 17 章） |
| 可破坏地形 | 运行时修改格子（清空瓦片）+ 生成碎片特效 | 复合碰撞体改为“手动生成”，一帧内的多次修改攒一起再重建一次 |
| 动画瓦片 / 机关 | 动画瓦片（水、火把）；会动的机关（移动平台、开合的门）做成独立物体 | 静态地形层保持静态，便于合批与碰撞缓存 |
| 地面类型 | 按格子坐标查瓦片资源，读它携带的表面数据（冰 / 泥 / 草，对应移动参数与脚步声） | 比物理材质表达力强，也便宜 |
| 寻路与 AI | 直接把瓦片格子转成导航数据（哪些格可站、可跳达关系） | 不用每帧做物理查询 |
| 大地图 | 按区域拆成多个场景或多个 Tilemap，按相机位置流式加载 / 卸载 | 块渲染本身就会剔除视野外的块；碰撞多边形过大时也要分块 |
| 外部关卡编辑器 | LDtk、Tiled 等编辑关卡，导入为 Tilemap + 实体标记 | 导入时校验度量（例如自动标出超过跳跃能力的间距） |

## 19.7 面试防守

追问树

```
关卡是怎么搭的？（Q19-1）
├─ Tilemap 的原理？和每格一个预制体有什么区别？（Q19-2）
├─ 为什么要复合碰撞体？（Q19-3 → 图 19-2）
├─ 块模式和逐格模式怎么选？为什么要图集？（Q19-4）
├─ 运行时破坏地形怎么做？会不会卡？（Q19-5）
└─ 平台间距是怎么定的？（Q19-6 → 度量）
```

### Q19-1 → Q19-2　关卡怎么搭的？Tilemap 原理是什么？

**L2**　用 Tilemap：一个网格下分背景层和地形层，用调色板画瓦片，地形层加瓦片碰撞并合并。原理上 Tilemap 是一张稀疏表，只记录有瓦片的格子，每格只存“瓦片 / 精灵 / 变换 / 颜色”的下标，实际资源去重存一份；渲染器按块合成网格批量绘制，碰撞体按瓦片的碰撞类型生成。

**L3** 和“每块地砖一个预制体”相比：不需要成千上万个 GameObject（没有 Transform 层级和组件的开销），渲染天然合批，碰撞可以合并成少量多边形，编辑效率也高得多；代价是格子之外的自由摆放、逐块的复杂行为要用独立物体实现。

### Q19-3　为什么要复合碰撞体?

**L2** 逐格碰撞体在相邻两格之间有内部边，角色或投射物高速滑过时会被内部边的角“绊住”，表现为走路时偶尔一顿、飞剑在平地上弹起来。合并成整体轮廓后没有内部边。Demo 的地形层 79 格，逐格有 316 条边、其中 168 条在内部，合并后只剩 2 个多边形 16 个顶点，物理查询也更便宜。

### Q19-4　块模式和逐格模式怎么选?

**L2** 横版的地形层用块模式：批量绘制、开销小，整层作为一个排序对象；需要角色穿插在瓦片之间（俯视角的树、柱子）时用逐格模式。块模式要把瓦片精灵打进同一个图集，否则多纹理的瓦片排序和渲染不一致——我的 Demo 就漏了这一步。

### Q19-5　运行时破坏地形?

**L2** 按世界坐标换算格子坐标，清掉瓦片并播碎块特效。性能关键在碰撞重建：复合碰撞体默认每次修改都同步重建，爆炸一次清掉几十格就会重建几十次；改成手动生成，一帧的修改攒在一起最后重建一次；地图大时按区域拆成多个复合碰撞体，只重建受影响的那块。

### Q19-6　平台间距怎么定?

**L2** 先由角色参数算度量：原地跳约 3.9 m 高、满速跳远约 5.6 m。然后定难度分级：轻松、需要助跑、需要冲刺、需要墙交互。回头看我的关卡，沟宽 3 m 太宽松，立柱 7 m 依赖不稳定的墙交互，这是“先画图后定度量”的结果。

**常见错误回答**

**错误**：“Tilemap 就是把小图拼起来，Unity 会自动处理碰撞。”

**问题**：没有讲出存储方式、渲染模式、碰撞合并这三个关键点；被问“为什么会卡接缝”“为什么要图集”就答不上。

**更好的回答**：按“数据（稀疏表）→ 画面（块渲染 + 图集）→ 碰撞（逐格生成 + 合并）→ 设计（度量）”四句话回答。

## 19.8 发散场景题

### 场景 19-1　策划要自己搭关卡，你会提供哪些工具?

考点：关卡管线与工具意识。

① 规则瓦片和主题调色板（地形、平台、危险物分开）；② 预制关卡模块（带难度标签）；③ 实体标记画笔（出生点、怪、宝箱、检查点）；④ 度量校验工具：自动标出超过跳跃能力的间距、不可达的平台；⑤ 一键试玩：从当前视图位置出生；⑥ 提交前检查：重复资源、缺失碰撞、未配置的刷怪点。

### 场景 19-2　做程序化生成的关卡（随机地牢），Tilemap 还适用吗?

考点：运行时生成与批量写入。

适用：生成算法输出一张格子网格（房间 + 走廊），一次性批量写入 Tilemap（批量设置比逐格设置快得多），规则瓦片自动拼边；碰撞体在全部写完后重建一次。可达性用格子图做验证（BFS 检查每个房间都能到达，平台类还要按跳跃度量判断“能不能跳过去”）。

### 场景 19-3　镜头移动时瓦片之间出现细缝 / 闪烁，怎么查?

考点：像素风渲染的常见问题。

三个常见原因：① 纹理采样到相邻图块（没有图集间距、开了双线性过滤或 mipmap）；② 相机位置不是像素的整数倍（用像素对齐相机，或把相机位置吸附到像素网格）；③ 瓦片精灵本身的边缘有半透明像素。Demo 的贴图设置（点过滤、不压缩）是对的，缺的是图集和像素对齐相机。

**本章速记**

- 六个部件：**Grid、Tilemap（稀疏表）、Tile 资源、Renderer（块 / 逐格）、TilemapCollider2D、CompositeCollider2D + 静态刚体。**
- Demo：地形 79 格 → 逐格 316 条边（168 条内部）→ 合并为 2 个多边形 16 个顶点；32 PPU、点过滤、不压缩。
- 十步流程：**度量 → 导入 → 图集 → 调色板 → 分层 → 碰撞 → 绘制 → 规则瓦片 → 特殊地形与实体标记 → 验证。**
- 坑：**无图集、墙地同层、重复瓦片资源、背景与地形难区分、没先定度量、实体直接摆场景。**

▶ **自测 1：为什么说 Tilemap 的格子级查询很便宜?**

▶ **自测 2：复合碰撞体选“多边形”和“轮廓”有什么区别?**

▶ **自测 3：为什么移动平台不要画进地形 Tilemap?**

CHAPTER 20

# 手感：输入缓冲、宽容窗口与反馈时机

问题 ④：“当前没有输入 / 输出缓冲，怎么优化手感？”——手感不是玄学，它可以拆成三条可以量化的链：**输入什么时候被接受**（宽容窗口：预输入、土狼时间）、**动作怎么响应**（跳跃曲线、取消窗口、强制时间）、**反馈什么时候出现**（预表现、动画、镜头、顿帧）。本章用“成功率曲线”量化 Demo 现状，再给出一套不写代码也能讲清楚的改造方法。

---

**本章目标**

- 把“手感好不好”拆成输入延迟、宽容窗口、动作曲线、反馈时机四个可调的部分
- 用 F2 / F3 讲清土狼时间和跳跃缓冲对“按了没反应”比例的影响
- 用 F4 讲清可变跳高、下落加重、顶点悬停分别改变了什么
- 说清“输出侧”的反馈设计：预表现、动画过渡、镜头缓冲、顿帧，以及它们为什么要由同一个事件驱动

## 20.1 口径：30 秒怎么讲 • 现状 vs 理想

**面试口播**

现状 **Demo 现状**：各状态在按键的那一帧直接读按键，只在“当前状态允许”时生效——跑出平台边缘的下一帧就切到下落、按跳无效；落地前按的跳会被丢掉；跳跃高度固定，轻按和长按一样高；动作之间没有预输入。

**量化**：我用模型算过，假设玩家在跑出边缘后平均 20 ms（标准差 40 ms）按跳，没有土狼时间时只有 24% 能跳起来，加 0.1 s 土狼时间是 96%；落地前平均 30 ms 按跳的玩家，没有缓冲只有 17% 能连跳，加 0.12 s 缓冲是 97%。

理想 **正确做法**：输入先变成带时间戳的“意图”进入缓冲；状态机在可以响应的时候消费最近 0.1~0.15 s 的意图；离开地面 / 墙后保留 0.1 s 的宽限；跳跃做成可变高度并加重下落；所有反馈（尘土、音效、镜头、震动）由同一个“起跳 / 落地 / 命中”事件在同一帧触发。

现状 → 问题 → 理想 **手感**

| 维度 | 当前做法（Demo 现状） | 存在的问题 | 理想重构方案 | 收益 / 代价 |
|---|---|---|---|---|
| **输入接收** | 按下那一帧直接读，当前状态不响应就丢弃 | P2 状态不响应的那一帧按键直接丢失：攻击收招时按跳、落地前按跳都无效 | 带时间戳的意图缓冲，由可响应的时刻消费<br>设计方案 · 未实现 | 按了就有用 |
| **早按（落地前）** | 没有跳跃缓冲 | P2 预判型玩家的连跳只有 17% 成功（F3） | 跳跃缓冲约 0.12 s：97%<br>模型验证：F3 | “按了没反应”大幅减少 |
| **晚按（离边后）** | 没有土狼时间 | P2 准时型玩家在平台边缘起跳只有 24% 成功（F2） | 土狼时间约 0.1 s：96%<br>模型验证：F2 | 代价：最多多跑出 0.7 m 才起跳 |
| **跳跃曲线** | 起跳时一次性设 vy = 20，松键不影响上升 | 局限 大跳、小跳不可控，精确平台跳无从设计；上升与下落对称（滞空 0.8 s） | 可变跳高（轻按约 1.82 m）+ 下落加重（滞空 0.80 → 0.72 s）<br>模型验证：F4 | 一个键表达大小跳；落地更干脆 |
| **窗口单位** | 没有窗口 | 隐患 现在还没有窗口；补窗口时最常见的坑是按帧数配置，高帧率玩家的窗口会变短 | 所有窗口按秒配置，不按帧数<br>设计方案 · 未实现 | 高帧率玩家的窗口不会变短 |
| **反馈** | 闪白 + 击退；动画过渡全为 0 | 局限 起跳、落地、命中缺少同帧的尘土 / 音效 / 震屏 / 顿帧，打击感弱 | 保持 0 过渡；增加由语义事件驱动的同帧反馈（尘土、音效、震屏、顿帧）<br>保持 · 过渡为 0 正确，补反馈 | 输入当帧可见可听 |
| **验证方式** | 靠感觉 | 局限 手感问题无法量化、无法回归：调完一个参数不知道是变好还是变坏 | 成功率曲线 + 输入回放 + “被丢弃的输入”日志<br>设计方案 · 未实现 | 手感问题可量化 |

现状问题：P2 ×3　隐患 ×1　局限 ×3　（P0~P3 编号见附录 A，分级说明见 0.2）

## 20.2 手感的四条链




现状 vs 理想 图 20-1　手感的四条链（上）与 Demo 现状（下）。①决定“多快”，②决定“多宽容”，③决定“动作长什么样”，④决定“玩家感觉到了没有”

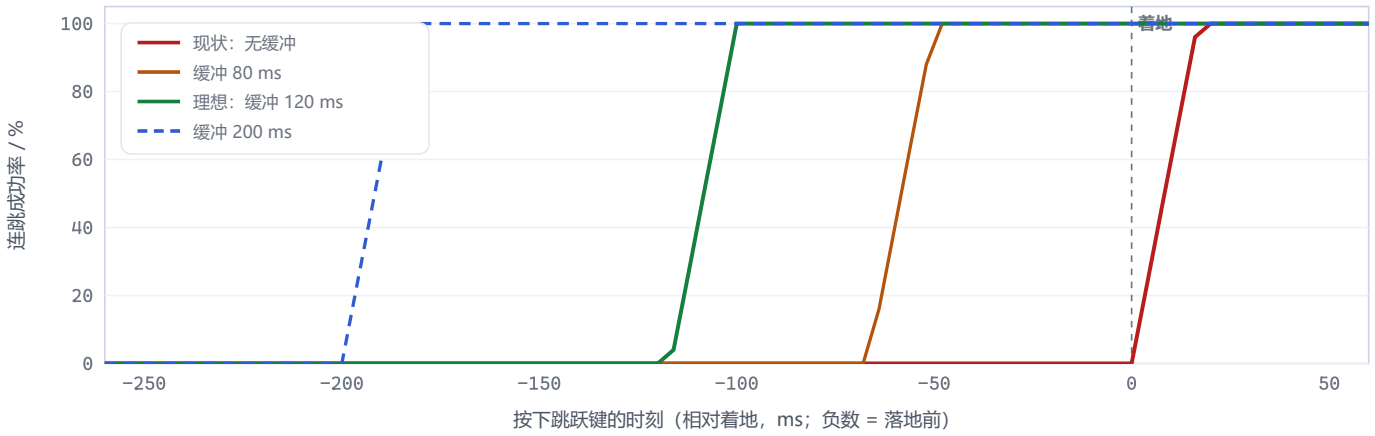
### ① 输入延迟从哪里来

| 环节 | Demo 中的情况（60 fps、物理 50 Hz） | 最大延迟 | 怎么降 |
| --- | --- | --- | --- |
| 按下 → 被读到 | 在下一帧的 Update 里读到 | 1 帧（16.7 ms） | 无法避免；高帧率降低 |
| 读到 → 逻辑生效 | 状态切换在当帧；但 Fall 落地帧切到 Idle 后，Idle 要下一帧才检查跳跃 | 0~1 帧 | 请求-提交后立即补一次本帧更新；状态检查顺序合理 |
| 逻辑 → 物理生效 | Update 里设速度，下一个物理步才产生位移 | 1 个物理步（20 ms） | 起跳这类一次性动作可以同时修正当前位置；或提高物理频率 |
| 物理 → 画面 | 刚体插值让画面落后物理最多一步 | ≤ 20 ms | 对玩家角色可以考虑外插或关闭插值 + 提高物理频率（取舍） |
| 画面 → 显示 | 垂直同步、渲染队列、显示器 | 1~3 帧 | 限制预渲染帧数、低延迟模式（平台相关） |

Demo 里“按跳 → 画面里角色开始上升”最坏要经过 1 帧 + 1 帧 + 1 个物理步 + 插值，约 50~70 ms，这还没算显示链路。所以**宽容窗口不是“放水”，而是在补偿玩家看不到的那几十毫秒。**

## 20.3 早按：跳跃缓冲（预输入）

场景：玩家在快落地时按跳，想落地即起跳。Demo 的规则是：落地那一帧下落状态切到待机，待机下一帧才检查“跳跃键是否在这一帧按下”——所以按键必须**恰好**落在落地之后的帧上，早一帧都不行。用模型扫按键时刻（相对着地时刻 0，60 fps，帧相位均匀取 400 个） 实测 F3 模型 ：




现状 vs 理想 图 20-2　按跳时刻与“落地即起跳”的成功率（F3）。没有缓冲时，只有着地之后才按下、并且在落地帧之后被读到的跳才有效——落地前早 1 ms 都会失败；缓冲时长从“读到按键”算到“可以起跳的那一帧”（落地帧的下一帧），所以 80 ms 缓冲实际覆盖落地前约 60 ms

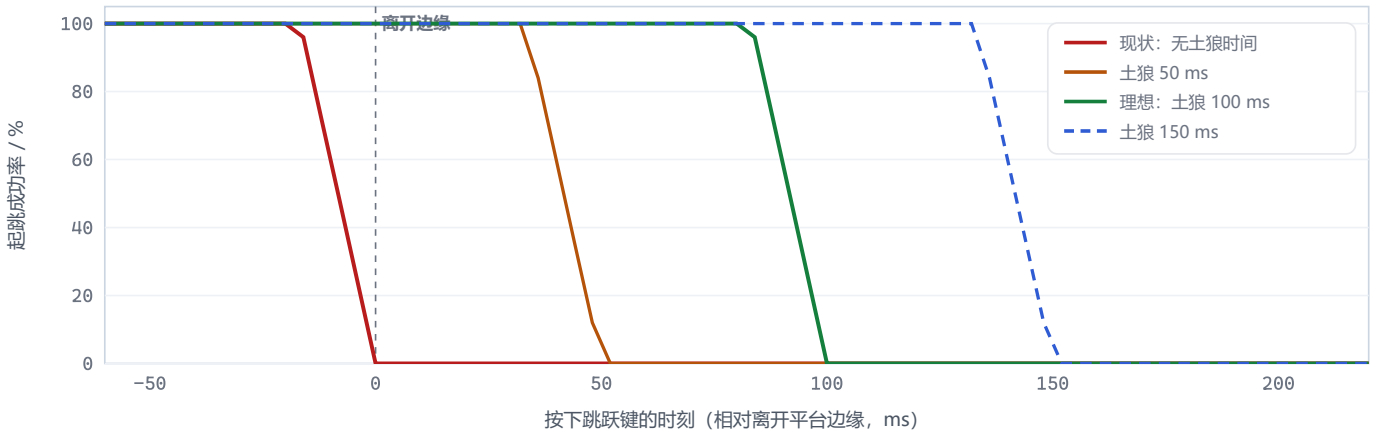
| 玩家类型（按键时刻分布） | 现状：无缓冲 / % | 80 ms / % | 理想：120 ms / % | 200 ms / % |
|---|---|---|---|---|
| 预判型 N(−30 ms, 40 ms) | 17.1 | 75.9 | 97.4 | 100.0 |
| 抢跳型 N(−70 ms, 50 ms) | 5.9 | 40.8 | 77.7 | 99.2 |

表 20-1　两类玩家的连跳成功率。“抢跳型”玩家（平均提前 70 ms）在无缓冲时几乎总是失败——他们会觉得“按了没反应”

**做法**：输入采样成“意图 + 时间戳”放进缓冲；每个可以起跳的时刻（着地、滑墙、二段跳可用）去缓冲里找“最近 N 秒内的跳跃意图”，找到就消费掉。缓冲长度常见 0.1~0.15 s：太短没效果，太长会出现“我早就不想跳了它还跳”。同一套缓冲也服务于攻击连段、冲刺、技能（第 3 章“取消窗口”）。

## 20.4 晚按：土狼时间

场景：玩家跑出平台边缘之后才按跳（人的反应 + 显示延迟让这非常常见）。Demo 在离地的那一帧就切到下落，下落状态不响应跳跃 实测 F2 模型：




现状 vs 理想 图 20-3　按跳时刻与起跳成功率（F2）。无土狼时间时，离开边缘之后的按键全部失败；100 ms 土狼时间覆盖了绝大多数“稍晚”的按键

| 玩家类型（按键时刻分布） | 现状：无 / % | 50 ms / % | 理想：100 ms / % | 150 ms / % |
|---|---|---|---|---|
| 准时型 N(+20 ms, 40 ms) | 24.1 | 70.5 | 96.2 | 99.9 |
| 偏晚型 N(+50 ms, 50 ms) | 12.3 | 43.4 | 79.7 | 96.6 |

表 20-2　两类玩家的起跳成功率。代价：土狼时间内角色以 7 m/s 继续跑出 0.35 / 0.70 / 1.05 m（50 / 100 / 150 ms）才起跳——太长会看起来像“从空中起跳”

**做法**：感知层记录“离开地面多久了”（第 17 章）；跳跃判定从“现在着地”改成“离地时长 < 土狼时间 且 本次离地不是因为起跳”。常见 0.08~0.12 s。同样的思路用于墙（墙宽限，第 18 章）和攀爬边缘。

**两个窗口一起用时的细节**

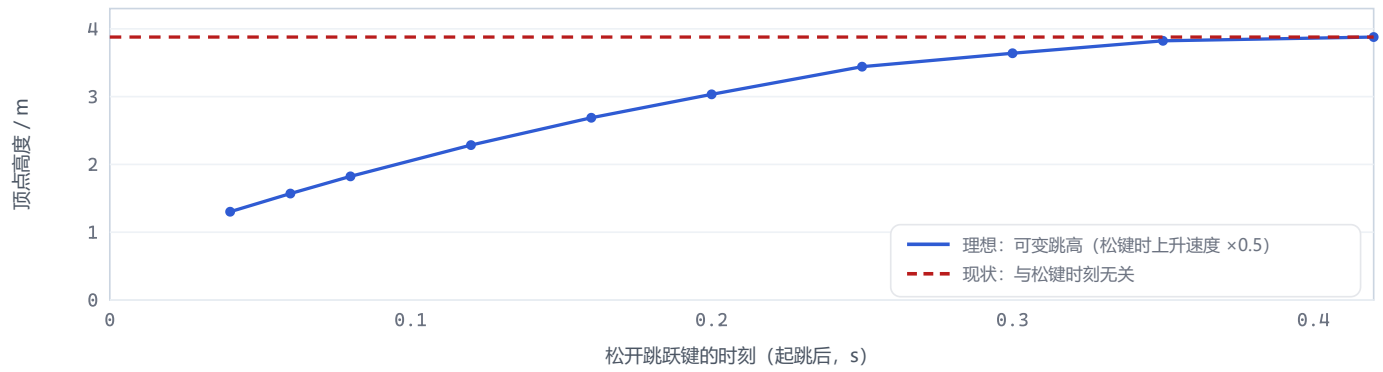
① 起跳后立即清空土狼计时，否则在土狼时间里可以“二段跳”；② 跳跃缓冲与土狼时间同时生效时，“落地前按的跳”在着地那一刻被消费，不要再等待下一帧；③ 窗口长度都按“时间”而不是“帧数”配置，否则高帧率玩家的窗口会变短。

## 20.5 动作曲线：同样的初速度，不同的“手感”

Demo 的跳跃是“起跳时一次性设定竖直速度 20，之后完全交给重力”，所以轻按和长按一样高，上升和下落对称。三种常见的曲线调整（按 Box2D 的离散积分计算，实测 F4）：

| 方案 | 顶点高度 / m | 到顶时间 / s | 总滞空 / s |
|---|---|---|---|
| 现状 · 按住与轻按相同 | 3.88 | 0.40 | 0.80 |
| 轻按（0.08 s 松键，速度 ×0.5） | 1.82 | 0.24 | 0.52 |
| 下落重力 ×1.6 | 3.88 | 0.40 | 0.72 |
| 下落 ×1.6 + 顶点悬停（\|vy\|<3 时重力 ×0.5） | 3.93 | 0.44 | 0.82 |
| 轻按 + 下落 ×1.6 | 1.82 | 0.24 | 0.46 |

表 20-3　跳跃曲线的三种调整。可变跳高让“轻按”成为小跳（顶点约为长按的一半）；下落加重缩短滞空、让落地更干脆；顶点悬停在最高点附近多给一点控制时间




现状 vs 理想　图 20-4　可变跳高：顶点高度随“按住时长”连续变化，0.4 s 之后松键等同于长按。玩家用按键时长控制跳多高——平台游戏里最基本、也最影响手感的一项

| 手段 | 解决什么 | 公开参考 公开资料 |
| --- | --- | --- |
| 可变跳高 | “小跳”与“大跳”用同一个键 | Celeste：VarJumpTime = 0.2 s（按住期间维持上升速度） |
| 顶点半重力 | 最高点附近更“飘”、更好调整落点 | Celeste：竖直速度绝对值 < 40（像素 / 秒）时重力减半 |
| 下落加重 / 终端速度 | 落地干脆、不“飘”；限制最大下落速度 | Celeste：MaxFall = 160，按下键快速下落 FastMaxFall = 240 |
| 土狼时间 | 跑出边缘后稍晚的起跳 | Celeste：JumpGraceTime = 0.1 s |
| 角落修正 | 上升时头部擦到平台角不被挡住 | Celeste：向上角落修正 4 像素 |

参考数值的单位是那款游戏自己的像素尺度，不能直接照搬；要搬的是**比例关系和机制**（例如土狼时间约 0.1 s、可变跳高窗口约 0.2 s）。另外两个和本 Demo 相关的动作曲线问题：冲刺的时长应按物理步计数（第 5 章 E10）；攻击的位移时长 0.15 s 写死在代码里，应跟随动画数据（第 4 章）。

## 20.6 输出侧：反馈的时机与“缓冲”

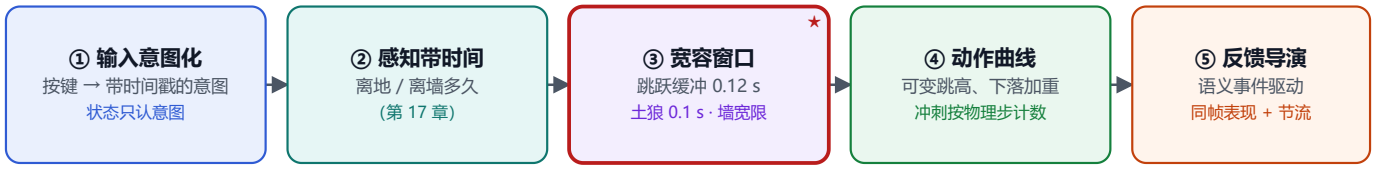
“输出缓冲”可以理解为：**逻辑结果与玩家感知之间的时间管理。**原则只有一条——**玩家的输入要在当帧得到可见、可听的回应，哪怕逻辑结果稍后才确定。**

| 手段 | 做法 | Demo 现状 |
| --- | --- | --- |
| 预表现 | 按下的那一帧就播起跳尘土、挥刀音效的起音、角色姿态的第一帧；联网游戏里这是“客户端预测”的表现部分 | 动画由状态切换驱动，基本是当帧；没有尘土 / 音效 |
| 动画过渡 | 逻辑状态立即切换，动画用极短的过渡（0~50 ms）或直接切；需要平滑时用“姿态匹配”而不是长混合 | 所有转移过渡时间为 0，干脆利落 ✓ |
| 挤压拉伸 | 起跳时纵向拉伸、落地时横向压扁 0.1 s | 无 |
| 镜头缓冲 | 死区、前瞻、竖直方向只在落地后跟随、平滑阻尼——让镜头“跟着意图”而不是“跟着每一帧的位置” | Cinemachine 默认跟随 |
| 顿帧与震屏 | 命中瞬间攻防双方减速 50~100 ms，镜头脉冲震动 | 无（且需要先有速度修正器栈，第 7 章） |
| 震动与音效 | 手柄短震、分层音效（起音 + 本体 + 尾音） | 无 |

**为什么所有反馈要由同一个事件驱动**

如果尘土由动画事件播、音效由状态 Enter 播、镜头由速度变化触发，它们会在不同的帧出现（第 4 章说过动画事件按帧量化、可能被打断）。正确的结构是：状态机产生语义事件（起跳、落地、命中、被弹反），一个“反馈导演”订阅这些事件，在同一帧触发所有表现，并统一做节流（同一帧多次命中只震一次）。这也让“关闭震屏”“降低特效”这类设置只改一处。

## 20.7 改造路线：不写代码也能讲清楚的五步




**理想方案** 图 20-5　手感改造的顺序：先有“意图”和“时间”，宽容窗口才有地方放；动作曲线和反馈是在此基础上的调参

**怎么验证**：① 调试 HUD 实时显示缓冲队列、离地时长、当前状态；② 录制输入回放，同一段操作在改动前后各跑一次；③ 像本章一样画“成功率曲线”——这比“感觉好多了”有说服力；④ 小范围试玩时记录“按了没反应”的次数（输入被丢弃的日志）。

## 20.8 面试防守

追问树

```
你怎么理解“手感”？（Q20-1）
├─ 什么是土狼时间 / 跳跃缓冲？为什么需要？（Q20-2）
│   └─ 会不会让游戏变简单？（Q20-3）
├─ 缓冲多长合适？用帧还是用秒？（Q20-4）
├─ 为什么跳跃要可变高度？怎么实现？（Q20-5）
├─ 输入延迟有哪些来源？（Q20-6 → 20.2 表）
└─ 动作游戏的输入在 Update 还是 FixedUpdate 读？（Q20-7）
```

### Q20-1　你怎么理解手感?

**L2**　我会拆成四件可调的事：输入延迟（多快生效）、宽容窗口（多宽容）、动作曲线（加速度、跳跃弧线、取消窗口）、反馈（玩家能不能立刻看到和听到）。“手感差”通常可以落到其中一项上，比如“按了没反应”多半是宽容窗口问题，“飘”多半是动作曲线问题。

### Q20-2 → Q20-3　土狼时间和跳跃缓冲是什么？会不会让游戏变简单?

**L2**　土狼时间：离开平台后一小段时间内仍允许起跳；跳跃缓冲：落地前按的跳在落地时生效。它们补偿的是人的反应和显示链路的几十毫秒延迟。我算过：按键平均晚 20 ms 的玩家，没有土狼时

间只有约四分之一能跳起来，加 0.1 s 后超过 95%。

**L3** 不会变简单，因为它们只修正“玩家意图正确、但时机差了几十毫秒”的情况，不改变跳跃能力本身；真正的难度由关卡度量决定。窗口过长才有问题：土狼时间 0.15 s 时角色已经跑出边缘 1 m，看起来像在空中起跳。

### Q20-4 缓冲多长合适？用帧还是秒？

**L2** 用秒配置，常见 0.1~0.15 s；用帧数会让 144 Hz 玩家的窗口只有 60 Hz 玩家的一半。具体长度按动作调：跳跃缓冲可以略长，攻击连段的预输入要配合取消窗口，太长会出现“不想出的招也出了”。最好用成功率曲线 + 试玩数据定。

### Q20-5 为什么跳跃要可变高度？怎么做？

**L2** 让玩家用一个键表达“小跳”和“大跳”，这是精确平台跳跃的基础。常见两种做法：松键时如果还在上升，把上升速度乘一个系数（如 0.5）；或者按住期间维持上升速度一段时间（约 0.2 s），松键后才开始受重力减速。再配合下落加重，让弧线“上升灵活、下落干脆”。Demo 是一次性给速度，所以轻按和长按都是约 3.9 m。

### Q20-7 输入在 Update 还是 FixedUpdate 读？

**L2** 在 Update 读（“按下”只在一帧为真，FixedUpdate 可能这一帧不执行而漏掉），读到后存成带时间戳的意图，由物理步或状态机在合适的时机消费。新输入系统也支持在固定步里处理事件，但同样要注意不能丢失只持续一帧的按下事件。

**常见错误回答**

**错误**：“手感就是调参数，多试几次就好了。”

**问题**：没有方法，面试官无法判断你能不能把手感做好。

**更好的回答**：“我先把手感拆成延迟、窗口、曲线、反馈四部分，先补结构（意图缓冲、带时间的感知、反馈事件），再用成功率曲线和录制回放去调参数。”

## 20.9 发散场景题

### 场景 20-1　格斗游戏的“搓招”（↓↘→ + 拳）怎么识别?

考点：输入缓冲的进阶——指令序列识别。

输入缓冲保存最近约 0.5 s 的方向与按键历史（带帧号）；每帧从最新输入往回匹配指令模板（允许中间夹杂其他方向、每一步有时间上限）；多个指令同时匹配时按优先级（必杀技 > 特殊技 > 普通技）；为了宽容，按键可以比最后一个方向稍早或稍晚几帧。

### 场景 20-2　移动端虚拟摇杆的平台游戏，手感怎么补?

考点：触屏的输入特性。

触屏没有触觉反馈、方向不精确：加大宽容窗口（土狼、缓冲适当加长）、方向输入加死区和吸附（八方向量化）、跳跃按钮区域大于视觉区域、关键操作给强烈的视觉 / 震动反馈；必要时增加辅助（自动抓边、自动冲刺）。

### 场景 20-3　联机对战的动作游戏，宽容窗口还能用吗?

考点：手感与网络的结合。

能，而且更重要：本地先预测执行（含土狼、缓冲），服务器用相同规则校验（服务器也要知道“离地多久”这类状态）；规则必须在两端一致且可确定，否则会频繁回滚纠正。窗口的时间基准要用逻辑帧 / 服务器时间，而不是本地渲染时间。

**本章速记**

- 手感四链：**延迟、宽容窗口、动作曲线、反馈时机**；Demo 的“按跳 → 画面上升”最坏约 50~70 ms。
- F3：早按 30 ms 的玩家，无缓冲 17% → 缓冲 120 ms 97%；F2：晚按 20 ms 的玩家，无土狼 24% → 100 ms 96%。
- F4：可变跳高让轻按顶点约 1.8 m（长按 3.9 m）；下落加重把滞空从 0.80 s 缩到 0.72 s。
- 输出侧：**输入当帧必须有可见 / 可听的回应**；所有反馈由同一个语义事件驱动。
- 窗口用**秒**配置，不用帧数。

▶ **自测 1：为什么 Demo 里“落地那一帧按跳”也会失败？**

▶ **自测 2：土狼时间为什么要在起跳后立即清零？**

▶ **自测 3：动画过渡时间设为 0 是好是坏？**

CHAPTER 21

# 系统设计：背包、装备与技能为什么这样设计

问题 ⑤：“讲一讲整个背包、装备乃至技能系统的设计思路，为什么这么设计？”——这是 RPG 方向最常见的系统设计题。好的回答不是罗列类名，而是先说清**分层原则**，再说三个系统各自的模型，最后说它们**通过哪几条共享通道协作**。Demo 里的问题在这里都变成了“为什么要分这一层”的论据。

---

**本章目标**

- 用“配置 / 实例 / 规则 / 表现”四层和“属性修正 / 效果 / 标签与事件”三条通道讲清整体设计
- 分别给出背包、装备、技能的数据模型和关键设计决策，并能说出每个决策的理由
- 用一个端到端例子（穿上火焰剑 → 普攻点燃敌人）演示三个系统如何协作
- 知道什么时候 Demo 式的简单实现就够了，什么时候必须升级——避免被问成“过度设计”

## 21.1 口径：30 秒怎么讲 • 现状 vs 理想

**面试口播**

现状 **Demo 现状**：物品是 ScriptableObject 配置，背包、材料仓库、装备栏各用“列表 + 以物品配置为键的字典”；穿装备时物品配置直接给玩家的各项属性加修正；技能是挂在技能管理器上的组件，基类只管冷却。

理想 **设计思路**：我会把三个系统统一成四层——**配置**（策划配的不变数据）、**实例**（运行时会变的状态，能存档、能同步）、**规则**（所有修改都通过有校验的操作完成）、**表现**（UI、特效只订阅变化）；三个系统之间只通过三条通道交流：**属性修正**（装备、Buff、技能都只登记修正，最终属性由修正推导）、**效果**（伤害、治疗、上 Buff 都是“效果”）、**标签与事件**（阻断、触发、刷新 UI）。

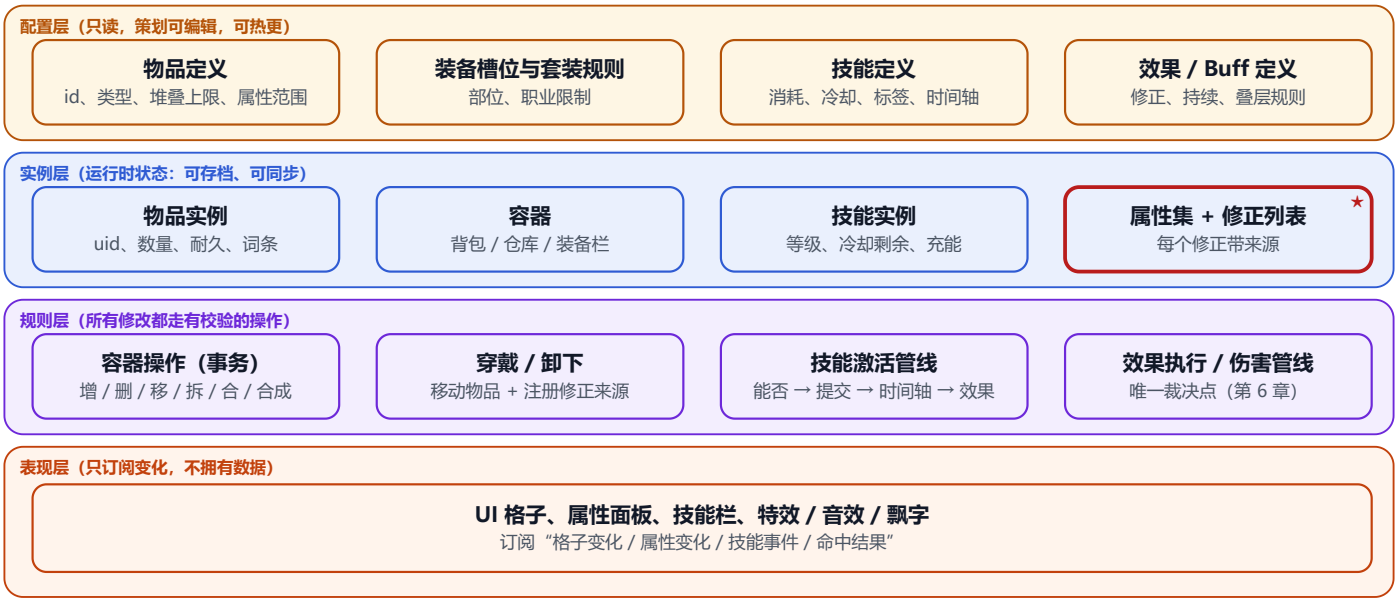
**为什么**：Demo 里的问题几乎都是缺了某一层——没有实例层，“+5 的剑”存不了；操作不是事务，合成少扣了材料；配置直接改玩家属性，没有来源、做不了属性预览；技能参数写在组件上，新增技能就要写新类。

现状 → 问题 → 理想 **背包、装备与技能系统**

| 维度 | 当前做法（Demo 现状） | 存在的问题 | 理想重构方案 | 收益 / 代价 |
|---|---|---|---|---|
| **物品** | 只有定义（ScriptableObject），没有实例 | 局限 个体差异（强化、耐久、词条）无处存放；存档、同步没有对象 | 定义 + 实例两层 设计方案 · 未实现 | 个体差异、存档、同步都有地方放 |
| **修改方式** | 直接改 List / Dictionary；合成不是事务 | P1 合成少扣料可以刷装备（E6，#6）；中途失败会留下半截状态 | 所有修改走操作：只读校验 → 一次提交 → 结果码 设计方案 · 未实现 | 不会出现半截状态 |
| **装备加属性** | 物品配置通过单例直接改 PlayerStats，按值删除 | P2 物品配置反向依赖玩家单例；上限变了当前血量不变（#30）；无法预览、无法与 Buff 统一撤销 | 装备栏注册带来源的修正；属性由修正推导 设计方案 · 未实现 | 可撤销、可预览、可与 Buff 叠加 |
| **技能** | 每个技能一个组件类；参数在字段；CanUseSkill 检查即扣冷却 | P0 没有统一的“能否释放”入口，死亡后仍能冲刺（#3）；被拒绝也扣冷却（#36） | 定义 + 实例 + 激活管线（能否 → 提交 → 时间轴 → 效果）+ 标签 设计方案 · 未实现 | 新技能靠配置；阻断规则统一 |
| **跨系统协作** | 直接调用 + 3 个单例 | P0 单例写反（#4）；系统之间互相直接调用，改一个牵动全部 | 属性修正、效果、标签与事件三条通道 设计方案 · 未实现 | 系统之间不互相引用 |
| **表现** | UI 格子直接调用 Inventory 并读键盘 | P1 UI 直接改数据、直接读键盘：点空格空引用（#13）；换手柄、换皮、联网都要改 UI | UI 只订阅事件，把点击转成意图 设计方案 · 未实现 | 可换皮、可联网 |
| **权威** | 全部在本地 | 局限 联网后客户端可以任意改背包、刷掉落 | 单机在本地；联网时规则层在服务器执行 设计方案 · 未实现 | 防刷、防改包 |

现状问题：P0 ×2　P1 ×2　P2 ×1　局限 ×2　（P0~P3 编号见附录 A，分级说明见 0.2）

## 21.2 总纲：四层与三条通道




**理想方案** 图 21-1　四层结构。纵向：上层不依赖下层的运行时对象（配置不知道玩家是谁），表现层只订阅不写入；横向：背包、装备、技能、效果四列在每一层都有对应部件。★ 属性集是三个系统交汇的地方

| 通道 | 谁写 | 谁读 | 解决什么 |
| --- | --- | --- | --- |
| 属性修正 | 装备（穿戴时注册）、Buff（生效时注册）、被动技能、等级成长 | 伤害公式、移动速度、UI 属性面板 | 多个系统影响同一属性时不互相覆盖（第 7 章 E8 的根因）；可按来源撤销；可做属性预览 |
| 效果 | 技能时间轴、武器命中、道具使用、环境（陷阱） | 伤害管线、Buff 系统 | “造成伤害、施加 Buff、治疗、召唤”统一成数据配置的效果，技能只是效果的编排 |
| 标签与事件 | 状态机（Dead、Stunned）、Buff（Frozen）、装备（Armed） | 技能能否释放、AI、任务成就、UI | 跨系统的条件判断不需要互相引用（“眩晕时不能放技能”由标签表达） |

## 21.3 为什么这样分：六个设计原则

| 原则 | 含义 | Demo 里的反例 |
| --- | --- | --- |
| 单一数据源 | 同一份状态只有一个拥有者，其他地方只读或订阅 | 属性被装备配置、冰冻、冻结、死亡多处直接修改 |
| 配置与状态分离 | 配置只读、可共享；会变化的东西放实例 | 以物品配置作字典键 → 无法表达同种物品的不同状态 |
| 修改走操作 | 每次修改都有校验、要么全成要么全败、有结果码 | 合成先校验后扣料，但扣料只扣一份；失败只打日志 |
| 依赖方向向下 | 表现依赖规则，规则依赖实例与配置，反之不行 | 物品配置（资源）通过单例直接改玩家组件 |
| 扩展靠数据，不靠子类 | 新增内容主要是配新数据，不写新类 | 每个技能一个组件类；飞剑四种模式写成标志位分支 |
| 可存档、可同步、可测试 | 实例层是纯数据，能序列化、能在服务器上跑、能单测 | 数据散在 MonoBehaviour 字段里，只能进 Play Mode 手测 |

## 21.4 背包：数据模型与关键决策

| 层 | 内容 | 设计要点 |
| --- | --- | --- |
| 物品定义 | 稳定 id、类型、堆叠上限、品质、图标、标签、可交易 / 可出售 | id 必须稳定（存档、网络、表格都用它），不能用资源路径或数组下标 |
| 物品实例 | 唯一 id、定义 id、数量、实例数据（耐久、强化等级、随机词条、绑定、获取时间） | 可堆叠的材料可以“只有数量没有实例数据”；装备每件一个实例 |
| 容器 | 容量、格子数组、过滤规则（只能放材料 / 只能放某部位装备） | 背包、仓库、装备栏、商店、邮件附件**都是容器**，一套代码 |
| 操作 | 添加、移除、移动、交换、拆分、合并、使用、丢弃、合成、整理 | 每个操作先只读校验、再一次性提交；返回结果码（成功 / 背包满 / 数量不足 / 不可放入） |
| 事件与视图 | “第 i 格变化了”事件；格子视图只持有（容器, 下标） | 增量刷新；格子多时用虚拟列表 |

| 关键决策 | 选项 | 怎么选 |
| --- | --- | --- |
| 格子制 or 列表制 | 格子制：固定格子、允许空洞、玩家可以摆放；列表制：按类型自动排序，没有“格子” | 需要玩家管理空间（生存、MMO）用格子制；以收集为主（动作 RPG 的材料）用列表制 |
| 堆叠规则 | 同定义即可堆叠 / 同定义且实例数据相同才堆叠 / 不可堆叠 | 按物品类型配置；有耐久、词条的物品不可堆叠 |
| 满了怎么办 | 拒绝 / 进溢出邮箱 / 掉在地上 | 写进规则，返回明确结果码，UI 给提示 |
| 谁是权威 | 单机：本地；联网：服务器 | 客户端只发“意图”（用第 3 格的药），服务器执行操作后回包；客户端可乐观预表现 |

**Demo 里对应的坑**

现状 以配置作键（没有实例）→ “+5 的剑”无处存放；三套“列表 + 字典”复制了三份代码 → 应抽象成容器；任何变化都全量刷新所有格子、一次装备刷新三次 → 应发格子级事件；合成每种材料只扣一份 → 操作不是事务；格子的物品字段被序列化，开局时是一个“空壳对象”导致点空格空引用 → 视图不该拥有数据（第 10 章）。

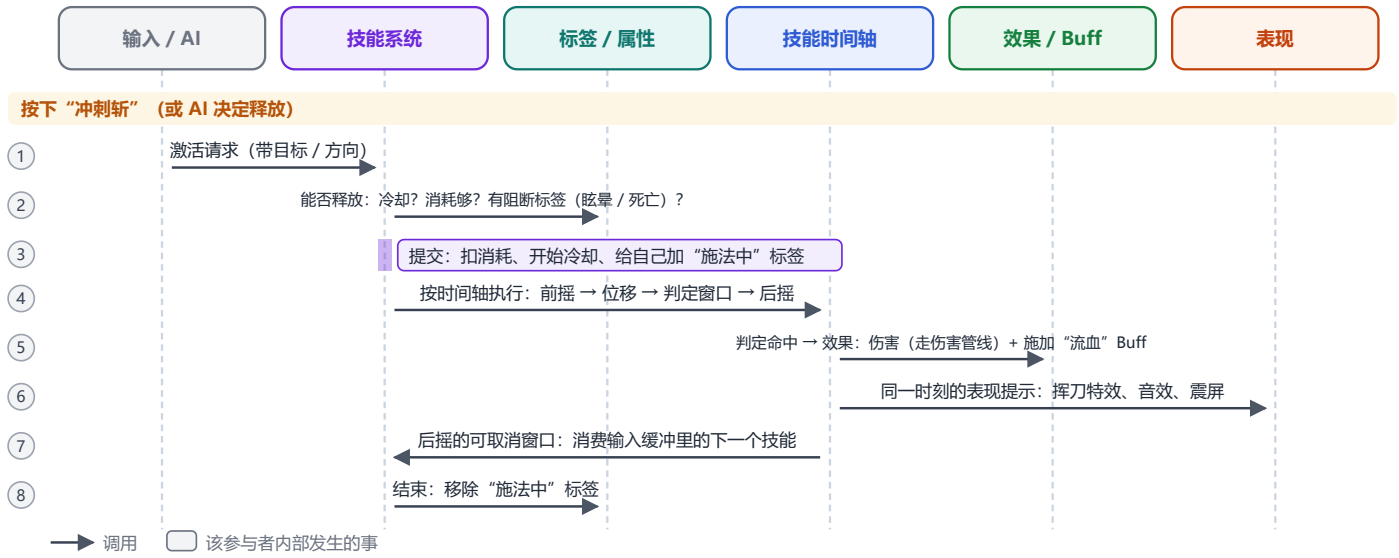
## 21.5 装备：有槽位规则的容器 + 修正来源




**理想方案** 图 21-2　穿戴流程。装备 = “在两个容器之间移动物品” + “按来源注册 / 移除属性修正”；任何一步校验失败，整个操作不生效

- **修正带来源、分层级**：每条修正记录“来自哪件装备实例 / 哪个 Buff”，并区分固定值、加算百分比、乘算百分比三类，按固定顺序合成（第 7 章）。卸下时按来源整体移除，不会误删别人的同值修正。
- **属性预览**：属性推导做成纯函数“基础值 + 一组修正 → 最终属性”。UI 想显示“换上这件后攻击 +12”，只需用“当前修正 − 旧装备 + 新装备”算一次，不需要真的穿上再脱下——Demo 的做法（配置直接改玩家属性）做不到这一点。
- **套装与条件属性**：作为“根据当前装备集合计算出的额外修正来源”，装备变化事件触发重算。
- **表现分离**：换外观、换武器动作集由装备变化事件驱动，属性系统不知道模型的存在。

## 21.6 技能：定义 + 实例 + 激活管线




**理想方案** 图 21-3　技能激活管线。技能本身不直接扣血、不直接改速度、不直接播特效——它只负责“什么时候发生什么”，具体做什么交给效果和表现

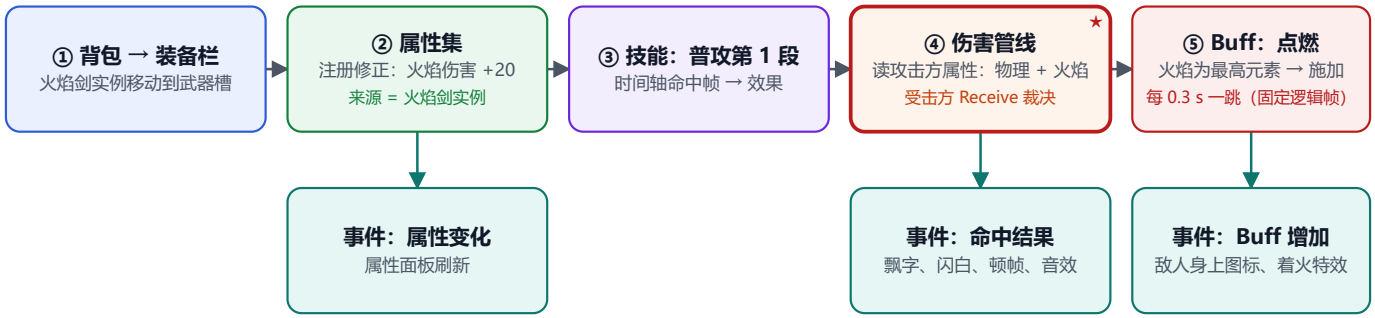
| 部件 | 内容 | 为什么 |
| --- | --- | --- |
| 技能定义 | 消耗、冷却、充能、需求标签、阻断标签、目标方式、时间轴、效果列表、表现提示 | 新增技能 = 新增一份配置；平衡调整不改代码 |
| 技能实例 | 等级、冷却剩余、充能层数、临时修正（冷却缩减） | 同一技能在不同角色身上状态不同；可存档、可同步 |
| 激活管线 | 能否释放 → 提交 → 执行 → 结束；查询和提交分开 | 避免“检查即消耗”（Demo 的 CanUseSkill）；被状态守卫拒绝时不扣冷却 |
| 时间轴 | 前摇、位移、判定窗口、霸体 / 无敌区间、可取消窗口、后摇 | 与动画对齐但不依赖动画播放（第 4 章）；可编辑、可预览 |
| 效果 | 伤害、治疗、施加 / 移除 Buff、位移、召唤、生成投射物 | 可组合：飞剑的“弹跳 / 穿刺 / 旋转”就是不同效果与投射物行为的组合（第 8 章） |
| 标签 | 技能给自己加、需要、被谁阻断、会取消哪些技能 | 跨系统条件不需要互相引用；和状态机的打断等级是同一个思想 |

**技能与状态机的关系**：角色状态机管“移动层”（地面、空中、滑墙），技能系统是“动作层”——技能释放时向状态机申请进入“施法”状态（带打断等级），时间轴结束再释放。这样冲刺、攻击、掷剑不再是状态机里的十几个特殊状态，而是同一种“施法”状态执行不同的技能数据。

**Demo 里对应的坑**

**现状** 冷却查询带副作用；技能参数（冲刺速度、时长、飞剑 20 个字段）在组件和角色上，无法复用；冲刺在角色每帧更新里全局检测，没有经过“能否释放”的统一入口，所以死亡后也能冲刺；分身的伤害来源靠层级查找、结果为空（第 8 章 P0）——技能产生的效果应该在创建时就带上“来源”。

## 21.7 三个系统怎么协作：一个端到端例子




**理想方案** 图 21-4 “穿上火焰剑 → 普攻点燃敌人”的完整链路。每一步只和相邻的通道打交道：背包不知道伤害公式，技能不知道装备，Buff 不知道 UI

这条链路在 Demo 里也“存在”，但每一步都是直接调用：物品配置直接改玩家属性、动画事件回调里直接算伤害、元素异常在伤害函数里直接施加（而且这条调用被注释掉了，第 7 章）、表现与结算分开调用（第 6 章）。分层之后，链路上任何一环都可以单独替换、单独测试、单独搬到服务器上。

## 21.8 取舍：什么时候简单实现就够了

| 信号 | 出现前（Demo 规模） | 出现后必须升级的部分 |
| --- | --- | --- |
| 物品需要“个体差异”（耐久、强化、随机词条） | 以配置作键的堆叠就够 | 实例层 + 唯一 id |
| 需要存档 / 跨场景 | 数据放组件上也能跑 | 实例层纯数据化 + 稳定 id + 版本迁移 |
| 多个来源影响同一属性（装备 + Buff + 被动） | 直接加减可以凑合 | 修正器栈 + 来源 |
| 技能超过 10 个 / 策划要自己配 | 每个技能一个类也能写 | 数据驱动定义 + 效果组合 + 时间轴编辑器 |
| 联网 | 本地即权威 | 操作改成请求 / 回包，随机与结算放服务器，实例层可序列化同步 |
| 多人协作开发 | 一个人记得所有约定 | 操作的结果码、事件契约、单元测试 |

**回答“会不会过度设计”**

“Demo 的规模下，直接实现是合理的，我也是这么做的；但我知道每一层是为哪个需求准备的——比如出现‘物品个体差异’就必须有实例层、出现‘多来源改属性’就必须有修正器。分层不是一开始全部做完，而是在信号出现时按这个方向演进，并且一开始就别做会挡路的事（比如以配置作字典键、让配置直接改玩家）。”

## 21.9 面试防守

系统设计题的答题模板（背包 / 装备 / 技能通用）

① 澄清需求：单机还是联网？要不要存档？物品有没有个体差异？技能多少个、谁来配？
② 先给分层：配置 / 实例 / 规则 / 表现，画图 21-1
③ 数据模型：每层有哪些字段，id 怎么定，谁是权威
④ 关键操作：以“穿装备 / 合成 / 释放技能”为例走一遍流程，强调校验与事务
⑤ 跨系统协作：属性修正、效果、标签与事件三条通道
⑥ 非功能：存档与版本、网络同步、性能（虚拟列表、脏标记）、可测试
⑦ 取舍：现阶段做到哪一层，哪些信号出现时再升级

### Q21-1　背包为什么要分配置和实例?

**L2** 配置是“这种物品是什么”，实例是“这一件物品现在怎样”。一把剑的名字、图标、基础攻击是配置；它被强化了几次、还剩多少耐久、随机出了什么词条是实例。只有配置时，同种物品永远完全相同，个体差异无处存放——我的 Demo 就是以配置作字典键，“+5 的剑”做不了。

**L3** 实例还带来两个好处：存档和网络只需要序列化实例（定义 id + 实例数据），配置随版本发布；同一份配置可以被成千上万个实例共享，内存也省。

### Q21-2 装备的属性怎么加？卸下怎么撤销？属性对比怎么做？

**L2** 装备不直接改属性，而是在属性集里注册一组“来自这件装备”的修正；卸下按来源整体移除；最终属性由“基础值 + 所有修正”按固定顺序推导，并用脏标记缓存。属性对比用同一个推导函数，在一组“假设的修正”上算一次即可，不需要真的穿脱。

### Q21-3 技能系统为什么要数据驱动？

**L2** 技能多了以后，差异主要在数据（数值、时间轴、效果组合），而不在逻辑结构。数据驱动后新增技能是配置工作，策划能自己调；逻辑只写一次（激活管线、时间轴执行器、效果执行器），测试也只测这几个通用部件。

**L4 • 攻击** “全部数据驱动，特殊技能怎么办？” → 留扩展点：效果类型可以注册自定义实现，时间轴可以插“脚本事件”节点；目标是 90% 的技能纯配置，10% 的特殊技能写少量自定义效果，而不是反过来。

### Q21-4 Buff 和技能是什么关系？

**L2** 技能是“什么时候发生什么”的编排，Buff 是“持续一段时间的状态”。技能通过效果施加 Buff；Buff 通过修正影响属性、通过标签影响技能能否释放（眩晕 Buff 带“禁止施法”标签）、也可以周期性地产生效果（点燃每 0.3 s 一次伤害效果）。两者通过效果和标签解耦，互相不直接引用。

### Q21-5 联网以后这三个系统怎么改？

**L2** 实例层由服务器持有和修改；客户端只发意图（装备第 3 格、释放技能 X 朝方向 d），服务器执行操作、结算效果，再把实例变化同步回来。客户端为了手感做预表现：技能动作和特效先播，伤害数字和背包变化以服务器为准；操作带请求 id 防止重发造成双扣。因为四层已经分开，改动集中在“规则层由谁执行”和“实例层怎么同步”，配置和表现几乎不用动。

**常见错误回答**

**错误**：“背包用 List，装备用 Dictionary，技能每个写一个类继承 Skill。”

**问题**：只讲了容器类型，没有讲分层、权威、事务、跨系统协作；面试官追问“+5 的剑”“多来源改属性”“新增 50 个技能”就会卡住。

**更好的回答**：按 21.9 的模板，从需求澄清讲到分层、模型、操作、协作、取舍。

## 21.10 发散场景题

**场景 21-1 做随机词条装备（暗黑类）：掉落时随机出 1~4 条属性。**

考点：实例数据与生成规则。

配置层存“词条池”（每条：属性、数值范围、权重、可出现的部位与品质）；掉落时由服务器（或单机时的权威逻辑）按品质决定条数、按权重无放回抽词条、在范围内随机数值，写进物品实例；穿戴时实例里的每条词条变成一条修正。随机过程用可记录种子的随机数，便于排查“为什么出了这件”。

**场景 21-2 加入天赋树：点了“冲刺后 2 秒暴击 +20%”。**

考点：被动效果如何接入事件与 Buff。

天赋是一种“被动技能”：解锁时注册一个事件监听（技能“冲刺”结束），触发时给自己施加一个 2 秒的 Buff，Buff 携带“暴击率 +20%”的修正。整个过程只用到技能事件、Buff、修正三样已有部件，不需要改冲刺技能本身。

**场景 21-3 商城、邮件、交易都要“给玩家物品”，怎么避免各写一套?**

考点：容器与操作的复用。

所有“给物品”都走同一个操作：向目标容器添加实例，返回结果码（成功 / 满了）；满了的处理（进邮箱、拒绝、掉地上）由调用方按场景决定。交易是两个容器之间的原子交换（双方物品先锁定，确认后一次性移动）。这样“背包满了怎么办”这类规则只写一次，日志与防刷也只做一处。

**本章速记**

- 四层：**配置（只读）/ 实例（可存档可同步）/ 规则（操作即事务）/ 表现（只订阅）**。
- 三条通道：**属性修正（带来源）、效果（统一伤害 / 治疗 / Buff）、标签与事件。**
- 背包：定义、实例、容器、操作、视图；装备 = 容器间移动 + 修正来源；技能 = 定义 + 实例 + 激活管线（能否 → 提交 → 时间轴 → 效果）。
- Demo 的问题都是“缺了一层”：无实例、非事务、配置直改属性、查询带副作用、参数在组件上。
- 取舍：**不必一开始全做，但别做会挡路的事**；按信号演进。

▶ **自测 1：为什么说背包、仓库、装备栏、邮件附件都是“容器”？**

▶ **自测 2：属性预览为什么要求属性推导是纯函数?**

▶ **自测 3：技能“能否释放”和“提交”为什么要分开?**

APPENDIX · A

# 问题总表（36 条）

按严重度排序。P0 = 无法构建 / 功能完全失效 / 状态机核心约束被破坏；P1 = 明显错误的玩法结果；P2 = 边界、体验、隐患；P3 = 代码质量与工程规范。行号对应 822270c。

| # | 级 | 模块 | 位置 | 问题 | 修复方向 |
|---|---|---|---|---|---|
| 1 | P0 | 构建 | Inventory.cs:4 · Enemy_SkeletonBattleState.cs:3 | 运行时脚本 using UnityEditor 命名空间 → 打包编译失败 | 删除；asr 真打包 |
| 2 | P0 | 技能 | SkillController_Clone.cs:52, 66 | 分身用 GetComponentInParent 找玩家，独立生成的分身上为 null → 有敌人时空引用，分身攻击从未生效 | 创建时注入（快照） |
| 3 | P0 | 状态机 | player.cs:105-119, 138-142 | 全局冲刺与接剑强切不检查当前状态 → 死亡后冲刺 / 接剑即“复活”（E1-S3/S4） | 统一 Requ 级；Deatl |
| 4 | P0 | 生命周期 | PlayerManager.cs:10-16 · SkillManager.cs:21-31 | 单例写反：已有实例时销毁旧的、不登记新的（推断：冲刺只能用一次） | 后来者自毁 OnDestro |
| 5 | P1 | 战斗 | PlayerAnimationTriggers.cs:21-22 · Enemy_SkeletonAnimationTrigger.cs:22-23 | 表现（DamageEffect）与结算（DoDamage）分离 → 闪避也闪白击退、弹反成功仍掉血 | DamageI → HitRes |
| 6 | P1 | 物品 | Inventory.cs:214-217 | 合成每种材料只扣 1 份（E6：10 皮 10 铁做 9 件，应 5） | TryRemov count)；事 |
| 7 | P1 | 数值 | CharacterStats.cs:115-121 · EnemyStats.cs:50-56 | TakeDamage 无死亡保护 → 弹跳剑 / 雷击让尸体重复 Die、重复掉落 | isDead 守 销目标 |
| 8 | P1 | AI | Enemy_SkeletonGroundState.cs:17, 28-33 | Battle 继承 GroundState → 看见玩家每帧自转移 + GameObject.Find（E7：10 s 252 次） | 拒绝自转移 用 |
| 9 | P1 | AI | Enemy_SkeletonBattleState.cs:32, 56 · Enemy.cs:114 | 战斗计时每帧重置永不超时；追击无边缘检测；射线只检测 | 只在看见时 线；可达性 |

| | | | | | |
|---|---|---|---|---|---|
| | | | | Player 层（穿墙）→ 平台下抽搐（E7-C） | |
| 10 | P1 | 数值 | CharacterStats.cs:145 | 冰冻破甲判断攻击者的 isChilled（E3） | `_targetS` `d` |
| 11 | P1 | 数值 | Enemy.cs:58-80, 126-139 · SkillController_ThrowSword.cs:219 | 冻结 / 减速 / 死亡都直接写 moveSpeed、animator.speed；重叠冻结只冻住 26%~85%（E8） | 修正器栈；大 |
| 12 | P1 | 动画 / 表现 | Enemy_SkeletonStunState.cs:17, 24 | InvokeRepeating 调用不存在的 RedColorBlink；CancelInvoke() 误伤异常状态变色 | nameof / 取消自己的 |
| 13 | P1 | UI | UI_EquipmentSlot.cs:16 · UI_ItemSlot.cs:11, 41 | 点击空装备格空引用；序列化器给格子造默认 InventoryItem → 开局点空背包格空引用 | 判空 `iter` NonSeria |
| 14 | P1 | 状态机 | PlayerWallSlideState.cs:21-44 | 没有“墙没了 → Fall”出口 → 半空慢落（E1-S2） | 出口覆盖这 |
| 15 | P1 | 状态机 | PlayerFallState.cs:22-35 · Eenmy_SkeletonIdleState / MoveState | 切换后继续执行旧状态代码 → 一帧两次转移、Enter 的初始化被覆盖（E1-S1） | 切换后 ret 交模式 |
| 16 | P2 | 数值 | CharacterStats.cs:136 | 暴击 `≤` → 实际 (c+1)%（E3） | 统一 Char |
| 17 | P2 | 物品 | ItemDrop.cs:21, 46 | 掉落 `≤` +1%、小数无效、0 也掉；“最多 3 件”改变概率；int 随机使掉落偏左（E5） | Random.v 表；float |
| 18 | P2 | 数值 | EnemyStats.cs:35-43 | 每级 +25% 当前值 → 复利（E4） | 曲线表驱动 |

| | | | | | |
|---|---|---|---|---|---|
| 19 | P2 | 数值 | CharacterStats.cs:110, 226 · Controller_ThunderStrike.cs:29, 63 | DoMagicalDamage 无调用方；平局分支用 damages[0] 判感电；雷击伤害写死 1；目标销毁后雷击悬停不销毁 | 接线 + 修 damage |
| 20 | P2 | 时间 | PlayerDashState.cs:16-45 | Update 计时决定位移时长 → 距离随帧率 / 卡顿变化（E10） | 按物理步 |
| 21 | P2 | 时间 | CharacterStats.cs:79 · SkillController_ThrowSword.cs:144-147 | 到点重置的周期计时漂移；场景 hitCoolDown = 0 → 命中数 = 帧数（E10） | 累加计时； |
| 22 | P2 | 时间 / 物理 | Skill_ThrowSword.cs:145-151 · Sword.prefab | 瞄准虚线用连续抛物线，与 Box2D 差 ½·g·s·Δt·t（E9）；飞剑 Discrete 碰撞、无插值 | 离散闭式 Continuo Interpola |
| 23 | P2 | 技能 | SkillController_Clone.cs:33-49, 72-96 | 朝向判断写进 foreach（E11）；分身不继承玩家朝向 | 循环外判 rotation |
| 24 | P2 | 技能 | Skill_ThrowSword.cs:57-89, 91-115 | Update 未调 base（冷却不减）；SetupGravity 覆盖序列化字段；Setup 调用顺序依赖；7 个 bool 标志 | 行为组合 |
| 25 | P2 | 动画 | PlayerAimSwordState.cs:18-21 · Animator_Player | 瞄准被打断仍播掷剑并掷出、方向陈旧；瞄准点不收起（推断） | 显式 Thro 退出路径 |
| 26 | P2 | 战斗 | Entity.cs:66-73 | 击退方向用受击者朝向；玩家 knockBack (0,0) 导致受击清零速度（推断） | 方向来自 DamageI |
| 27 | P2 | 战斗 / 性能 | 9 处 OverlapCircleAll | 无 LayerMask、每次分配、按碰撞体不去重 | ContactFi 缓冲 + Ha |

| | | | | | |
|---|---|---|---|---|---|
| 28 | P2 | UI | UI_HealthBar.cs:23-40 · Entity.cs:21 · CharacterStats.cs:49 | Start 订阅 / OnDisable 退订不对称；回调是 public Action 字段 | OnEnable 成对；eve |
| 29 | P2 | 表现 | EntityFX.cs:26, 36-59 | sr.material 克隆材质；颜色三写者（推断残留异常色） | MPB + Sp 由效果推导 |
| 30 | P2 | 数值 / UI | CharacterStats.cs:55, 331-341 | 当前血量不随上限（装备）变化；血条上限不刷新 | 上限变化 |
| 31 | P2 | 生命周期 | CharacterStats.cs.meta / ItemData.cs.meta / ItemDrop.cs.meta | 执行顺序配在父类（对子类无效）、SO 和无事件函数的脚本上 | [DefaultE 或显式初始 |
| 32 | P2 | AI | Enemy_SkeletonDeathState.cs:13-36 | 尸体不销毁不回收，穿地坠落后 Update 照跑 | 死亡动画后 |
| 33 | P3 | 质量 | StartCoroutine("...") 5 · Invoke 10 · InvokeRepeating 4 | 字符串反射调用，改名不报错 | 直接调用 / 时器 |
| 34 | P3 | 性能 | GameObject.Find 2 · GetComponentInParent 属性 3 · Debug.Log 8 | 热路径查找与日志 | 注入 / 缓存 志 |
| 35 | P3 | 工程 | .gitignore | `/[Logs]/` 等字符集写法；Logs / UserSettings 12 个文件入库；美术资源无 LFS | `/[Ll]ogs` cached； |
| 36 | P3 | 质量 | 多处 | 拼写（facinDir、Eenmy_、CreatClone、attaclMovement、Iventory）；字段隐藏（PlayerDeathState.player、Battle.playerTransform）；Can / Has 函数有副作用；未使用代码（FindClosestEnemy、SpriteStatusFX.shader、 | 重命名、C |

ItemDrop.item、Skill_Dash
空实现）

APPENDIX · B

# Play Mode 验证清单

书中标 `推断` 的结论，以及依赖引擎运行时、无法用 C++ 移植验证的结论，在 Unity 里各用 1~3 分钟即可确认。面试前把结果补到自己的笔记里，就可以把“推断”改成“我验证过”。

---

| 编号 | 结论（章节） | 验证步骤 | 预期现象 |
| --- | --- | --- | --- |
| B1 | 单例写反 → 冲刺只能用一次（图 2-2） | 复制场景里的 SkillManager 物体 → Play → 按两次 Shift（间隔 > 1 s） | 开局第一个 SkillManager 从 Hierarchy 消失；第二次起 Console 打印 “Skill [Dash] is on CoolDown” 且再也冲不出去 |
| B2 | 瞄准被冲刺打断仍掷剑（图 4-2） | 按住右键瞄准 → 不松键直接按 Shift | 播放掷剑动画并生成飞剑；首次时飞剑速度为 0 直接下落，之后方向为上一次松键时的方向 |
| B3 | 落地贴墙闪现墙滑（表 3-1） | Animator 窗口 + 暂停逐帧（Ctrl+Alt+P），斜跳到墙脚落地 | 某一帧 WallSlide 为 true、Idle 与 Air 为 false，下一帧回到 Idle |
| B4 | 墙到头慢落（表 3-2） | 摆一个悬空平台，贴着其侧面下滑并按住朝墙方向 | 越过平台下沿后仍播墙滑动画，以 2 m/s 下落直到着地 |
| B5 | 死亡后复活（表 3-3） | 把骷髅 EnemyStats.damage 改为 2000 → 被打死后按 Shift，再按方向键 | 离开死亡动画并可正常移动 |
| B6 | 分身攻击空引用（Ch8.3） | 贴近骷髅冲刺 | Console：NullReferenceException at SkillController_Clone.AttackTrigger；骷髅不掉血 |
| B7 | 开局点空格空引用（Ch10.3） | 开局不拾取任何物品，点击背包空格；再点装备栏空格 | 两次都报 NullReferenceException（前者只在首次 UpdateSlotUI 之前出现） |
| B8 | 受击清零速度（Ch6.3） | 跳跃上升途中被骷髅命中（或在 Inspector 里调用 DamageEffect） | 竖直速度瞬间归零，“顿一下再掉” |
| B9 | 颜色残留（图 11-2） | 临时解开 DoDamage 里 DoMagicalDamage 的注释并给玩家 fireDamage；点燃骷髅后在第 1.9 s 左右再命中一次 | 点燃结束后骷髅仍保持点燃颜色 |

| | | | |
|---|---|---|---|
| B10 | 执行顺序对子类无效（Ch2.5） | Project Settings → Script Execution Order 查看列表；在 PlayerStats、EnemyStats 的 Start 打印日志 | 列表中只有 CharacterStats（300），子类按默认顺序执行 |
| B11 | 打包失败（Ch2.6） | File → Build Settings → Build（任意平台） | 编译错误：找不到 UnityEditor 命名空间（CS0234 / CS0246） |
| B12 | 骷髅每帧自转移（E7） | 在 Enemy_SkeletonBattleState.Enter 里打印 Time.frameCount | 玩家站在攻击距离内时几乎每帧一条 |
| B13 | 插值位置导致落地判定晚一个物理步（第 17 章坑 ④） | 从高处下落，在 Update 里同时打印 transform.position.y、rb.position.y 和地面射线结果 | 下落时 transform 比 rb.position 高出约“速度 × 0.02 s”；着地后的第一帧射线可能仍未命中 |
| B14 | 单墙攀爬取决于手速（F5） | 在 7 m 立柱左侧起跳并一直按住 →，分别“滑墙一出现就按跳”和“稍等约 0.1 s 再按跳” | 前者能翻上立柱；后者在墙顶下方约 1.3 m 处反复滑落 |
| B15 | 跳跃缓冲缺失（F3） | 下落中提前约 50 ms 按跳 | 落地后不起跳 |

**复现本书的 C++ 实测**

在 Git Bash 中（需要 VS2022 的 MSVC 与 Windows SDK）

```
$ source ../Network_TCP_UDP/demos/build_env.sh        # 设置 INCLUDE / LIB / PATH
$ cd RPG_Demo/demos
$ MSYS2_ARG_CONV_EXCL='*' cl.exe /nologo /std:c++latest /EHsc /O2 /utf-8 /D_CRT_SECURE_NO
$ ./rpg_verify.exe > run_output.txt                    # 全部实验，写 figdata.json
$ ./rpg_verify.exe E5 E9                               # 只跑指定实验，写 figdata_partial.j
```

APPENDIX · C

# 源码索引

从文件找章节：面试官指着某个文件问时，知道去哪一章找答案。

| 文件 | 主要内容 | 章 |
| --- | --- | --- |
| Entity.cs · EntityFX.cs | 组件缓存、检测、翻转、击退；闪白与异常色 | 1 · 2 · 6 · 11 |
| player/player.cs · PlayerStateMachine.cs · PlayerState.cs | 状态创建、全局冲刺、接剑、死亡、减速 | 3 · 5 |
| player/PlayerGroundedState.cs · Idle · Move | 地面输入分支、currentState 守卫、HasNoSword 副作用 | 3 |
| player/PlayerFallState.cs · Jump · Air · WallSlide · WallJump | 一帧两次转移、墙滑出口 | 3 |
| player/PlayerDashState.cs | 冲刺计时与帧率 | 5 |
| player/PlayerPrimaryAttackState.cs | 连招计数、攻击位移、isBusy | 3 · 4 |
| player/PlayerCounterAttackState.cs | 弹反判定（受击时）与注释掉的原写法 | 6 |
| player/PlayerAimSwordState.cs · CatchSword · Death · PlayerAnimationTriggers.cs | 瞄准、接剑、死亡、动画事件转发与命中 | 3 · 4 · 6 |
| Enemy/Enemy.cs · EnemyState.cs · EnemyStateMachine.cs | 弹反窗口、冻结、减速、感知射线 | 6 · 7 · 9 |
| Enemy/Enemy_Skeleton/*.cs | 6 个状态、自转移、永不超时、死亡 | 9 |
| Skill/Skill.cs · SkillManager.cs · PlayerManager.cs | 冷却、两个单例 | 2 · 8 |
| Skill/Skill_Clone.cs · SkillController_Clone.cs | 分身、空引用、朝向 | 8 |
| Skill/Skill_ThrowSword.cs · SkillController_ThrowSword.cs | 瞄准虚线、四种模式、召回、冻结 | 5 · 7 · 8 |
| Stats/Stat.cs · CharacterStats.cs · PlayerStats.cs · EnemyStats.cs | 属性、伤害、异常状态、等级成长、死亡 | 7 |
| Controller_ThunderStrike.cs | 感电连锁雷击 | 7 |

| | | |
|---|---|---|
| Items and Iventory/*.cs | 背包、装备、合成、掉落、拾取 | 10 |
| UI/*.cs | 格子、装备格、合成格、血条 | 10 · 11 |
| ParalleBackGround.cs · SpriteStatusFX.shader | 视差背景；未使用的状态 Shader | 11 |
| Animation/Controller/*.controller · *.anim | Animator 结构与全部动画事件 | 4 · 6 |
| Scenes/SampleScene.unity · Prefabs/*.prefab · ProjectSettings | 所有序列化数值、物理与时间设置、执行顺序 | 0 · 1 · 2 · 5 |
| Scenes/SampleScene.unity（Grid / Ground / BackGround）· Graphics/…/Tiles | 瓦片地图、复合碰撞、瓦片资源与贴图导入设置 | 19 |
| Material/forPlayer.physicsMaterial2D | 玩家零摩擦材质（避免挂墙） | 17 |

APPENDIX · D

# 现状 vs 理想：全书总表

把各章 x.1 的对照表收拢到一处：每个模块现在在哪一档、理想要到哪一档、哪些理想做法有实验支撑、什么情况下不必上理想方案、从现状走到理想按什么顺序。

---

## D.1 成熟度阶梯：Demo 在哪，理想在哪

每个模块从左到右四档。**现状** 是 Demo 822270c 的真实位置；**修正 + 重构**两档是“重做这个 Demo 就该做到”的**理想**；**上线级**只在规模化或联网时才值得做，用来回答“上线 / 联网怎么办”的追问，不要在介绍 Demo 时把它当成目标。

| | Demo 在这里（现状 · 主要问题） | 重做这个 Demo 就该做到（理想） | | 追问时再展开（扩展） |
|---|---|---|---|---|
| | 现状 · Demo 822270c | 修正 · 只修错 | 重构 · 改结构 | 上线级 · 规模化 / 联网 |
| 状态机 · 3 | 三处入口 · 不检查当前状态 | 死亡拒绝切换 · 补“墙没了”出口 | Request + 守卫 + 帧末提交 | 分层状态机 · 输入回放 |
| 状态判定 · 17 | 单射线 · 各状态自己查 | 射线加长 · 加脚部射线 | 感知层 · 形状投射 + 法线 | 单向 / 移动平台 · 滞回 |
| 时间与帧率 · 5 | Update 计时 · 到点重置 | 累加计时 · 单帧 dt 上限 | 固定逻辑步计数 | 确定性 · 可回放 |
| 战斗判定 · 6 | 表现与结算分两次调用 | 弹反成功不再结算伤害 | DamageInfo → 唯一裁决 | Hitbox 时间轴 · 服务器裁决 |
| 属性与数值 · 7 | <= 多 1% · 冰冻判错对象 | 统一 Roll · 判目标 · 死亡守卫 | 修正器栈（带来源） | 属性集 + 标签 · 曲线表 |
| 技能 · 8 | 参数在组件 · 检查即扣冷却 | 分身注入来源 · 修冷却 | 定义 + 实例 + 激活管线 | 技能编辑器 · 时间轴 |
| 敌人 AI · 9 | 每帧自转移 · 永不脱战 | 拒绝自转移 · 计时修正 | 感知 → 黑板 → 决策 | 行为树 · 攻击令牌 · LOD |
| 物品 · 10 / 21 | SO 作键 · 合成少扣料 | 合成按数量扣 · 判空 | 定义 + 实例 + 事务 | 存档迁移 · 服务器执行 |
| UI 与表现 · 11 | 全量刷新 · 多方写颜色 | 订阅成对 · 共享材质 | 增量刷新 · 反馈导演 | 虚拟列表 · 屏幕空间血条 |
| 手感 · 20 | 无缓冲 · 固定跳高 | （不是 bug，没有修正档） | 缓冲 + 土狼 + 可变跳高 | 输入回放 · 成功率回归 |
| 关卡 · 19 | 先画后定 · 无图集 | 补图集 · 清重复瓦片 | 度量表 + 分层 + Rule Tile | 关卡模块 · 加载器生成 |
| 工程 · 2 / 12 | Editor 引用 · 字符串反射 | asmdef · nameof · 判空 | 对象池 · 复用缓冲 · 单测 | CI 真打包 · 性能预算 |

现状 vs 理想 图 D-1　成熟度阶梯：12 个模块 × 4 档（模块名后是章号）。橙列是 Demo 的真实位置；“修正”只在现有结构里改错（C++ 实验里的“修正版”多在这一档）；“重构”是各章对照表“理想重构方案”列的主体；“上线级”只在追问时展开。灰格表示这一档不存在：手感问题不是 bug，没有“只修错”的版本，只能按需求补机制

## D.2 各章对照表索引

下表由构建脚本扫描全书 17 张对照表自动统计：共 107 个维度。**现状一侧**：缺陷 66 个（P0 9、P1 18、P2 36、P3 3），隐患 9 个，局限 26 个，做法正确 6 个（按对照表维度计数：同一个缺陷会在相关的几章各出现一次，例如“死亡后冲刺复活”同时出现在状态机、技能、系统设计三章；去重后的 36 条见附录 A）——缺陷之外的一半多是“现在能用、规模上来才成问题”的局限，讲的时候不要把它们说成 bug。**理想一侧**：理想做法有 C++ 模型支撑的 13 个、现状已正确只需保持的 7 个、参数来自公开资料的 2 个，其余 85 个是**纯设计方案。**也就是说，**理想列的大部分内容没有任何代码实现**——讲的时候一律用条件句，只有“模型验证”那几项可以报对比数字。

| 节 | 对照表 | 维度 | 现状问题分级 | 设计方案 · 未实现 | 模型验证 | 保持现状 | 公开参考 | 有 C++ 标 |
|---|---|---|---|---|---|---|---|---|
| 1.1 | 架构总览 | 6 | P0 ×1<br>P1 ×1<br>P2 ×1<br>局限 ×2<br>✓ 无问题 ×1 | 5 | — | 1 | — | — |
| 2.1 | 生命周期与对象存亡 | 6 | P0 ×2<br>P2 ×1<br>隐患 ×2<br>局限 ×1 | 6 | — | — | — | — |
| 3.1 | 玩家状态机 | 6 | P0 ×1<br>P1 ×2<br>局限 ×3 | 6 | — | — | — | — |
| 4.1 | Animator 与动画事件 | 6 | P1 ×1<br>P2 ×1<br>隐患 ×2<br>局限 ×1<br>✓ 无问题 ×1 | 5 | — | 1 | — | — |
| 5.1 | 时间与帧率 | 6 | P2 ×5<br>✓ 无问题 ×1 | 4 | 1 | 1 | — | 轨迹预测 |
| 6.1 | 战斗判定 | 6 | P1 ×1<br>P2 ×3<br>隐患 ×2 | 6 | — | — | — | — |
| 7.1 | 属性、伤害与异常状态 | 7 | P1 ×3<br>P2 ×3<br>局限 ×1 | 4 | 3 | — | — | 概率判定者 |
| 8.1 | 技能系统 | 6 | P0 ×2<br>P2 ×2<br>局限 ×2 | 6 | — | — | — | — |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 9.1 | 敌人 AI | 6 | P1 ×4<br>P2 ×1<br>局限 ×1 | 4 | 2 | — | — | 状态复用 |
| 10.1 | 背包、装备、合成与掉落 | 7 | P1 ×2<br>P2 ×1<br>局限 ×4 | 6 | 1 | — | — | 合成 |
| 11.1 | UI 与表现层 | 6 | P2 ×4<br>隐患 ×1<br>局限 ×1 | 6 | — | — | — | — |
| 12.1 | 性能与工程 | 6 | P0 ×1<br>P2 ×1<br>P3 ×2<br>局限 ×2 | 6 | — | — | — | — |
| 17.1 | 状态判定 | 6 | P2 ×3<br>隐患 ×1<br>局限 ×1<br>✓ 无问题 ×1 | 4 | 1 | 1 | — | 检测形状 |
| 18.1 | 滑墙与蹬墙跳 | 6 | P1 ×2<br>P2 ×4 | 3 | 2 | — | 1 | 玩法规则 |
| 19.1 | 关卡搭建与 Tilemap | 7 | P2 ×2<br>P3 ×1<br>局限 ×2<br>✓ 无问题 ×2 | 4 | — | 2 | 1 | — |
| 20.1 | 手感 | 7 | P2 ×3<br>隐患 ×1<br>局限 ×3 | 3 | 3 | 1 | — | 早按（落<br>边后）、 |
| 21.1 | 背包、装备与技能系统 | 7 | P0 ×2<br>P1 ×2<br>P2 ×1<br>局限 ×2 | 7 | — | — | — | — |

| | | | | | | |
|---|---|---|---|---|---|---|
| **合计（17 张表）** | **107** | **P0** ×9<br>**P1** ×18<br>**P2** ×36<br>**P3** ×3<br>隐患 ×9<br>局限 ×26<br>✓ 无问题 ×6 | **85** | **13** | **7** | **2** |

## D.3 什么时候不必上理想方案

理想方案都有代价。下面这些情况成立时，**保持现状本身就是正确的工程判断**；面试时能说出“为什么这里我不会上行为树”，和说出“行为树怎么做”一样加分。

| 理想做法 | 可以不做（现状够用） | 必须做 | 章 |
| --- | --- | --- | --- |
| 逻辑时间轴取代动画事件命中 | 单机、动作少、动画和逻辑由同一个人维护 | 联网、回放、严格判定（格斗、精确弹反） | 4 |
| 固定逻辑步驱动计时 | 只影响表现的计时（闪白、UI 动画） | 影响结果的计时（冲刺距离、伤害跳数、冷却） | 5 |
| 行为树 / 黑板取代状态机 | 怪物种类少、行为简单（巡逻 → 追击 → 攻击） | 怪物种类多、要复用行为片段、策划要配 AI | 9 |
| 物品实例（uid） | 物品没有个体差异（材料、消耗品） | 有耐久 / 词条 / 强化，要存档或交易 | 10 · 21 |
| 屏幕空间统一绘制血条 | 同屏角色很少（Demo 只有两个） | 同屏几十个以上 | 11 |
| 对象池 | 生成频率低（掉落物） | 每秒多次生成（飞剑、分身、子弹、特效） | 12 |
| 复合碰撞改为手动生成 | 地形不在运行时改 | 可破坏地形、运行时放置平台 | 19 |
| 服务器裁决 | 单机 | 联网，或有经济系统（掉落、合成、交易） | 6 · 10 · 21 |
| 技能编辑器 / 时间轴 | 技能少、程序自己配 | 技能多、策划要配、需要预览 | 8 · 21 |

## D.4 从现状走到理想：顺序比方案更重要

第 12 章图 12-2 给出了两周路线：前 5 天只修错（能打包 → 状态正确 → 数值正确 → 帧率无关），后 5 天才改结构（输入层 → 伤害管线 → 修正器与 Buff → 物品实例与对象池）。背后是三条原则：

- **先修错，再改结构**：结构重构会同时动很多文件，错误没修完就分不清新问题是重构引入的还是原来就有的。
- **有实验的先做**：E1~E11、F1~F5 已经给出“现状 vs 修正”的对拍结果，改完立刻能用同一组用例回归（第 12 章：把实验写成 EditMode 测试）。

- **理想不是待办清单**：D.3 里“可以不做”的条件成立时就不做；阶梯图最右一列只在面试官追问规模化 / 联网时才展开。

---